Towards Agentic Robots：逐段中文解析与引用导航

更新于 2026-11-06。对应随包原文 arXiv:2610.20442v1，2026-10-14，79 页；物理页 1-48 为正文，49-79 为参考文献。本版按该固定 PDF 解析，不声称核验了每篇引用的最新版。

使用方式

先读逐段解析：每段有中文原意、机制提示和自测。遇到工作名，打开同编号工作卡；同一工作在多个段落重复引用时合并一张卡，列出不同出现位置。不要按编号从头读完工作辞典。

全部 469 条参考文献均有查询卡；其中表2-18有307个不同条目，合并重复后保留386次表格出现。其余条目包含背景方法、模型、软件与综述，按实际角色区分。关键代表方法有进一步的人工编写机制说明，其余以本综述证据和接口解释为主。

每卡包含“是什么、核心功能、核心实现、与其他工作的对比、教学伪代码、注意事项”。当综述只给名称或并列引用时，会明确说实现信息不足；通用接口骨架不是该方法的官方实现。

中文原意段落以已有机器译稿为底稿，进行了结构恢复、术语和典型误译修正；新增机制提示、比较问题、工作卡和自测由导读编写。它不是未经标记的纯翻译，也不宣称逐句学术审校已完成。

原始七幅总图恢复为完整截图，配中文读图；所有原表保留原页图片核对，原文79页一并保留在解析PDF末尾，以保证图标、公式与复杂表格可查。正文不展示失效替换字符，图标类字段标为“见原表”，不将丢失图标猜成是/否。

数字与模型能力均是该综述的转述。预印本、游戏、仿真、离线视频或理想真值实验不等于真机部署；工作卡中的比较通常比较设计轴，不是公平性能排名。

> 阅读提示：PDF中的每段保留工作编号，并标注工作辞典页码。网页可点击引用前后跳转。正文之后附全部79页原文，复杂图表以附录原页为准。

原文第 1 页 · 段落 p01-002

中文原意：基础模型（Foundation Model） 正在推动机器人智能的范式转变，超越低层动作预测，转向目标导向的数据收集、模型训练、任务规划和实际执行的编排。我们将这种新兴范例描述为 智能体式机器人（Agentic Robots），其中智能体对人类指令进行推理，与机器人环境交互，自主纠正执行错误，并动态调整操作策略。尽管这一领域发展很快，但目前的研究还没有很好的联系。对于如何将不同的智能体功能组合成一个完整的机器人系统，研究人员没有一致的看法。

> 解析：把本段看作研究地图的说明：作者希望追踪智能体做了什么决定、改变了什么对象、怎样获得反馈。四域是功能分类；同一系统可以同时出现在数据、训练和技能条目中。
>
> 解析：贯穿例子是开抽屉取杯子：训练决定动作能力，数据提供经验，环境提供世界与判据，Harness组织步骤与失败恢复。后续工作卡都可放回这条链上理解。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 1 页 · 段落 p01-003

中文原意：本文对 智能体式机器人（Agentic Robots） 的现有研究进行了统一概述。我们首先追踪从机器人策略到智能体系统的发展，涵盖视觉语言动作（Vision-Language-Action）模型、世界模型（World Models）和世界动作模型（World-Action Models）。然后，我们阐明了 智能体式机器人（Agentic Robots） 的定义和范围，并将现有研究组织为跨四个功能域的 17 个类别：模型、数据、环境和 运行支撑框架（Harness）。在这些领域中，我们研究智能体如何塑造模型学习和机器人决策，收集和管理机器人数据，构建用于学习和评估的世界、任务和测试，以及组织机器人使用的能力。最后，我们讨论了开放的挑战并概述了未来研究的方向。展望未来，我们希望这项工作有助于更清楚地了解 智能体式机器人（Agentic Robots），并支持新的方法，使它们更加强大、可靠和适应性更强。

> 解析：把本段看作研究地图的说明：作者希望追踪智能体做了什么决定、改变了什么对象、怎样获得反馈。四域是功能分类；同一系统可以同时出现在数据、训练和技能条目中。
>
> 解析：贯穿例子是开抽屉取杯子：训练决定动作能力，数据提供经验，环境提供世界与判据，Harness组织步骤与失败恢复。后续工作卡都可放回这条链上理解。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

图1：四种相互重叠的机器人智能范式

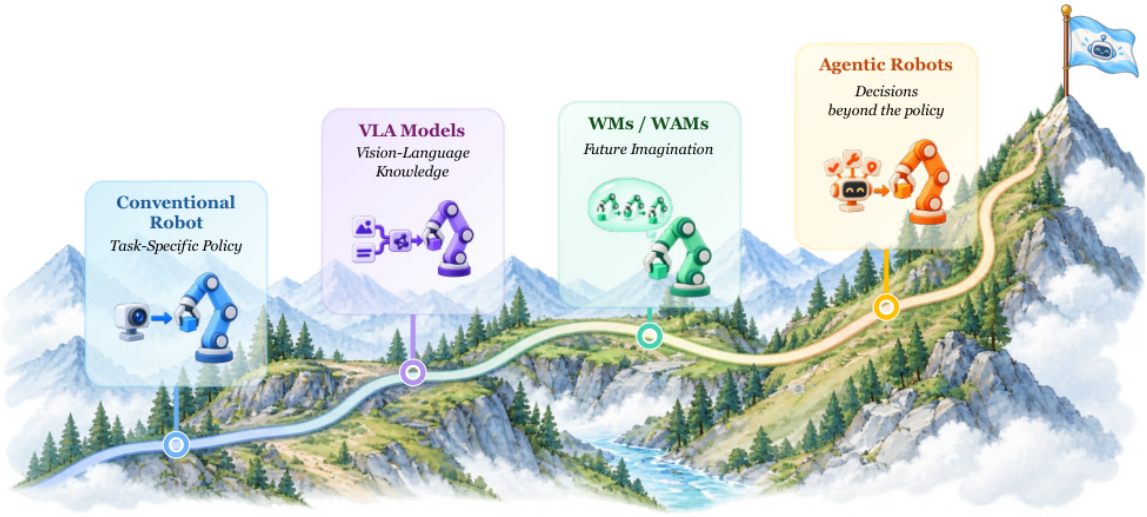


Figure 1. Robot intelligence across four paradigms: task-specific policies for direct control, vision-language-action models grounded in language model knowledge, imagination-centric world models or world-action models, and agentic decision-making that reasons beyond learned policies.

中文读图：从任务策略，到利用视觉语言知识的VLA，再到预测未来的WM/WAM，最后到组织能力与反馈的Agentic系统。箭头是发展线索，不是四种互斥架构。

图2：四个功能域的闭环

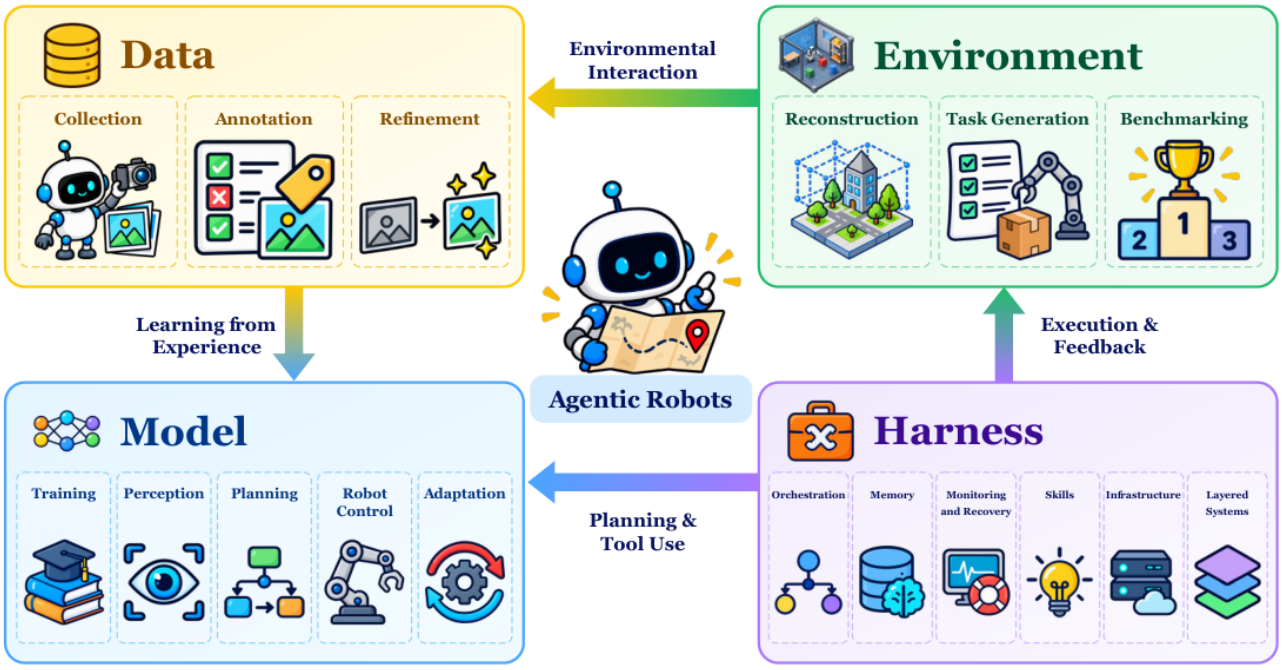


Figure 2. Overview of agentic robots across Model, Data, Environment, and Harness. The four domains connect model decisions, robot execution, environmental interaction, and learning from experience.

中文读图：Model决定怎样学习和行动，Data形成经验，Environment提供世界与测试，Harness组织执行与反馈；同一失败可能跨多个域。

原文第 3 页 · 段落 p03-004

中文原意：在过去的几年里，包括大语言模型（LLM）和多模态大语言模型（MLLM）在内的基础模型（Foundation Model）极大地推进了机器人策略学习。它们通过扩展机器人数据集和模型大小，赋予机器人传统特定任务模型所缺乏的新功能，特别是在场景和任务理解和跨场景泛化方面[20, 256]。作为该领域的主导技术解决方案，视觉语言动作（Vision-Language-Action）（VLA）模型利用离散动作标记或基于扩散或流匹配的连续动作预测策略来连接机器人动作预测和预训练LLM中嵌入的语义知识[13,19,269]。然而，仅改进动作预测并不能保证 长时程（Long-Horizon） 任务的可靠执行。这些任务要求机器人跟踪任务进度、根据环境变化调整计划并从故障中恢复。 VLA 策略本身无法始终如一地提供这些功能。

> 解析：动作预测解决“眼下怎样动”，长时程执行还需要“接下来做哪一步、上一动作是否成功、失败后怎样继续”。例如杯子没抓住时，即使策略能生成合理轨迹，系统仍要发现抓空并决定重看还是重抓。
>
> 解析：VLA提供动作能力，Agentic系统组织这些能力。两者可以在一个模型内实现，也可以分成多个模块；不能用是否调用大语言模型这一点来判断系统已经具备可靠恢复。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

本段引用：π0 [13]（辞典第14页）；RT-2 [19]（辞典第20页）；RT-1 [20]（辞典第21页）；Octo [256]（辞典第257页）；π0.5 [269]（辞典第270页）

原文第 3 页 · 段落 p03-005

中文原意：为什么智能体式机器人（Agentic Robots）很重要？最近，智能体式机器人（Agentic Robots）在研究界引起了极大的关注，它代表了将基础模型（Foundation Model）推理与具身智能体和环境交互能力相结合的新一代具身智能系统。 GPT-6 Astra [258] 和 Gemini 4 Argon [87] 等 通用（General-Purpose） 模型为此类系统开辟了新的可能性。例如，GPT-6 Astra 展示了作为机器人操纵智能体的有前途的功能，使用视觉观察来生成末端执行器命令并根据执行反馈调整其动作无需针对特定任务的微调[439]。在 RoboDojo 上，它在 42 个模拟任务中实现了 22.48% 的平均成功率，优于经过训练的机器人策略 DM0.5 (19.34%) [59]、GalaxeaVLA G0.5 (14.88%) [76]、Xiaomi-Robotics-1 (13.93%) [384]、OpenWAM-□ (11.92%) [363]和π 0 . 5 (6.91%) [269]。尽管取得了这些进展，推理延迟和资源成本凸显了 基础模型（Foundation Model） 如何集成到机器人系统中的重要性。除了优化和缩放模型参数之外，智能体式机器人（Agentic Robots） 还使用 基础模型（Foundation Model） 来规划操作、监控任务进度并根据执行反馈修改决策。因此，它们的能力取决于模型和环境中可用的工具、观察结果和反馈。智能体机器人范例解决了先前机器人解决方案的“缸中之脑”限制，将机器人智能从静态模式匹配转向自适应自主操作。

> 解析：这段数字用于说明通用模型参与机器人执行的可能性。机器人系统的分数还受到工具、执行器、可见信息和预算影响；同一模拟任务集上的转述结果，不能推出它在真实机器人或所有任务上更强。
>
> 解析：把案例画成“相机观察 → 模型决策 → 机器人命令 → 执行反馈 → 再决策”。理解这里的变化是模型开始组织闭环，再查各引用卡确认具体执行接口。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

本段引用：OpenDM / DM0.5 [59]（辞典第60页）；Galaxea G0.5 [76]（辞典第77页）；Gemini 4 Argon [87]（辞典第88页）；GPT-6 Astra [258]（辞典第259页）；π0.5 [269]（辞典第270页）；OpenWAM [363]（辞典第364页）；Xiaomi-Robotics-1 [384]（辞典第385页）；GPT-6 Astra Robot Evaluation [439]（辞典第440页）

原文第 3 页 · 段落 p03-006

中文原意：智能体式机器人（Agentic Robots） 需要分类。 尽管人们的兴趣日益浓厚，但对 智能体式机器人（Agentic Robots） 的研究仍然分散在模型学习、数据采集、环境构建和运行时系统设计方面。

> 解析：把本段看作研究地图的说明：作者希望追踪智能体做了什么决定、改变了什么对象、怎样获得反馈。四域是功能分类；同一系统可以同时出现在数据、训练和技能条目中。

解析：贯穿例子是开抽屉取杯子：训练决定动作能力，数据提供经验，环境提供世界与判据，Harness组织步骤与失败恢复。后续工作卡都可放回这条链上理解。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 4 页 · 段落 p04-007

中文原意：这些方向通常是单独研究的，尽管它们的贡献在机器人系统内相互作用。诸如体现智能体、自主机器人和自我改进（Self-Improving）机器人系统等相关术语强调不同的功能，而没有一致地澄清其边界，从而进一步模糊了该领域的范围。总之，这些问题使得很难确定智能体在哪里做出决策以及这些决策如何对整个系统做出贡献。因此，统一的分类法应该区分各个组件的角色，同时解释它们如何协同工作。为此，我们将 智能体式机器人学（Agentic Robotics） 组织为四个功能域：模型、数据、环境和 运行支撑框架（Harness），跟踪每个域内智能体的决策及其在整个系统中的影响。图 2 显示了这四个领域如何在智能体机器人系统中协同工作。 关键问题

解析：把本段看作研究地图的说明：作者希望追踪智能体做了什么决定、改变了什么对象、怎样获得反馈。四域是功能分类；同一系统可以同时出现在数据、训练和技能条目中。

解析：贯穿例子是开抽屉取杯子：训练决定动作能力，数据提供经验，环境提供世界与判据，Harness组织步骤与失败恢复。后续工作卡都可放回这条链上理解。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 4 页 · 段落 p04-008

中文原意：- 1 模型：智能体应该学习、理解、决定和控制什么？

解析：Model（模型）关注决策与学习机制。例如开抽屉失败，可以改奖励、重新辨认把手、改变动作顺序或更新策略；这些修复作用于不同对象。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 4 页 · 段落 p04-009

中文原意：- 2 数据：智能体应该如何收集、标注和提炼机器人数据？

解析：Data（数据）关注经验怎样变成可学的材料。例如一次失败抓取是采集记录；标出“抓空”是标注；删掉错位帧或保留恢复片段是数据改进。三者的输出不同。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 4 页 · 段落 p04-010

中文原意：- 3 环境：智能体应该构建哪些世界、任务和测试？

解析：Environment（环境）关注世界、任务与测试。例如生成可开合的抽屉是构造世界；规定取蓝块是构造任务；定义物体确实到桌面才算成功是构造评测。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 4 页 · 段落 p04-011

中文原意：- 4 运行支撑框架（Harness）：机器人能力应该如何编排、恢复和发展？

解析：Harness（运行支撑框架）把能力接成可执行闭环。例如维护“门已开”的进度，调用抓取技能，检查杯子是否随夹爪移动，失败后重看并重试。它影响执行组织，未必训练新的动作模型。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 4 页 · 段落 p04-012

中文原意：上述四个问题引导我们讨论智能体如何为机器人系统做出贡献。第 4 节研究智能体如何支持模型学习和机器人决策，第 5 节解释它们如何收集、标注和提炼机器人数据。第 6 节考虑了智能体为学习和评估而构建的世界、任务和测试。第 7 节研究智能体如何协调可执行功能、监视执行、从故障中恢复以及为后续任务保留有用的知识。

> 解析：把本段看作研究地图的说明：作者希望追踪智能体做了什么决定、改变了什么对象、怎样获得反馈。四域是功能分类；同一系统可以同时出现在数据、训练和技能条目中。
>
> 解析：贯穿例子是开抽屉取杯子：训练决定动作能力，数据提供经验，环境提供世界与判据，Harness组织步骤与失败恢复。后续工作卡都可放回这条链上理解。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 4 页 · 段落 p04-013

中文原意：- 智能体式机器人学（Agentic Robotics）的统一插图。我们追踪从机器人策略到智能体系统的演变，并阐明智能体在机器人任务执行和能力开发中的作用。我们引入了涵盖模型、数据、环境和 运行支撑框架（Harness）的分类法，将这些研究方向纳入一个通用框架中。

> 解析：把本段看作研究地图的说明：作者希望追踪智能体做了什么决定、改变了什么对象、怎样获得反馈。四域是功能分类；同一系统可以同时出现在数据、训练和技能条目中。
>
> 解析：贯穿例子是开抽屉取杯子：训练决定动作能力，数据提供经验，环境提供世界与判据，Harness组织步骤与失败恢复。后续工作卡都可放回这条链上理解。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 4 页 · 段落 p04-014

中文原意：- 智能体方法的系统分析。我们综合了智能体获取、协调和提高机器人能力的机制。通过比较整个分类法中的代表性方法，我们解释了智能体决策如何依赖于可用工具、机器人执行器和交互反馈，并检查这些方法的优点和局限性。

> 解析：把本段看作研究地图的说明：作者希望追踪智能体做了什么决定、改变了什么对象、怎样获得反馈。四域是功能分类；同一系统可以同时出现在数据、训练和技能条目中。
>
> 解析：贯穿例子是开抽屉取杯子：训练决定动作能力，数据提供经验，环境提供世界与判据，Harness组织步骤与失败恢复。后续工作卡都可放回这条链上理解。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 4 页 · 段落 p04-015

中文原意：- 实用 智能体式机器人（Agentic Robots）的研究议程。根据这一分析，我们概述了确定智能体贡献并将当前进展转化为可靠、高效的机器人操作的研究方向。该议程将公平评估与跨机器人平台的物理部署和泛化的挑战联系起来。

> 解析：把本段看作研究地图的说明：作者希望追踪智能体做了什么决定、改变了什么对象、怎样获得反馈。四域是功能分类；同一系统可以同时出现在数据、训练和技能条目中。
>
> 解析：贯穿例子是开抽屉取杯子：训练决定动作能力，数据提供经验，环境提供世界与判据，Harness组织步骤与失败恢复。后续工作卡都可放回这条链上理解。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

2 机器人学习的演变

原文第 4、5 页 · 段落 p04-016

中文原意：机器人学习跨越四个相互关联的阶段：学习可执行行为、整合多模态知识、预测身体结果以及协调执行和改进。 (1)传统机器人策略：学习可执行行为。强化和 模仿学习（Imitation Learning） 将观察结果映射到行动，而通才策略则跨任务和实施例共享经验。他们的能力很大程度上取决于机器人在训练期间的经验。 (2) 视觉语言动作（Vision-Language-Action） 模型：将知识与行动联系起来。 VLA 将预训练的 基础模型（Foundation Model） 的视觉和语言知识纳入控制，将场景、指令和语义关系的解释与动作生成联系起来。 (3) 世界模型（World Models） 和 世界动作模型（World-Action Models）：预期身体结果。这些模型使用未来状态、观察及其与行动的关系的表示，将行动选择和生成与环境变化的预测联系起来。 (4)智能体式机器人（Agentic Robots）：组织执行和改进。智能体系统将模型介导的决策扩展到选择和协调能力、获取经验以及使用反馈来指导后续执行和能力改进。这些阶段描述了可以在一个系统中共存的重叠研究传统和互补能力。

> 解析：这部分串起能力的发展：动作策略提供行为，VLA加入语义，世界模型加入未来动态，Agentic系统组织目标与反馈。它们可以组合使用，区别在于各模块的输入、产物和实际决策角色。

自问：各能力在同一系统里可以怎样组合？

2.1 传统机器人策略

> 本节先抓住：这里至少有三条独立设计轴：观测表示是图像还是几何；学习目标是模仿还是强化学习；动作头是回归、分块还是生成式。不要把“使用点云”与“使用扩散”当成互斥方法。
>
> 贯穿例子：同样预测末端位姿，PerAct选择体素位置，RVT汇合多个虚拟视角，DP3用点云特征条件化动作扩散。

原文第 5 页 · 段落 p05-018

中文原意：传统的机器人策略从演示或基于奖励的经验中学习。我们通过场景表示对视觉策略进行分组：基于图像的策略使用图像特征，而 3D 感知策略则引入用于动作预测的几何结构。这种区别涉及场景表示，而不是动作维度。

> 解析：先分清观察表示与动作分布。视觉编码器/点云改变模型看见什么；ACT或扩散策略改变动作怎样生成。一个模型完全可以同时使用点云输入与扩散动作头，两条轴不互斥。
>
> 解析：预测动作块只是生成未来一段动作，实际可以执行前段后重新观察。物体被移动时，长块若不刷新会继续使用旧证据；长度、执行步数与重规划频率要一起理解。

自问：我能分开说明观测表示、动作生成方式和动作块执行策略吗？

原文第 5 页 · 段落 p05-019

中文原意：基于图像的策略。早期的策略从相机图像中学习动作。引导策略搜索[168]将图像和机器人状态映射到电机扭矩，而QT-Opt[143]学习用于抓取的图像条件动作值函数。 RoboMimic [238] 通过离线演示研究视觉 行为克隆（Behavior Cloning）。视觉预训练提供可重用的策略功能：R3M [244] 在下游 模仿学习（Imitation Learning） 的人类视频上预训练图像编码器。除了视觉编码之外，ACT [449] 和 扩散策略（Diffusion Policy） [55] 分别使用条件变分自编码器和条件扩散对动作序列进行建模。任务条件化将这些策略扩展到多种行为：BC-Z [128] 和 RT-1 [20] 使用任务描述，而 VIMA [138] 将多模式提示与以对象为中心的图像标记相结合。跨机器人数据集拓宽了训练范围：RT-1-X [260] 利用来自 Open X-Embodiment 的跨机器人数据，而 Octo [256] 将异构图像观察与Transformer（变换器）和扩散动作头结合起来。

> 解析：先分清观察表示与动作分布。视觉编码器/点云改变模型看见什么；ACT或扩散策略改变动作怎样生成。一个模型完全可以同时使用点云输入与扩散动作头，两条轴不互斥。

解析：预测动作块只是生成未来一段动作，实际可以执行前段后重新观察。物体被移动时，长块若不刷新会继续使用旧证据；长度、执行步数与重规划频率要一起理解。

自问：我能分开说明观测表示、动作生成方式和动作块执行策略吗？

本段引用：RT-1 [20]（辞典第21页）；Diffusion Policy [55]（辞典第56页）；BC-Z [128]（辞典第129页）；VIMA [138]（辞典第139页）；QT-Opt [143]（辞典第144页）；Guided Policy Search [168]（辞典第169页）；RoboMimic [238]（辞典第239页）；R3M [244]（辞典第245页）；Octo [256]（辞典第257页）；RT-1-X / Open X-Embodiment [260]（辞典第261页）；ACT [449]（辞典第450页）

原文第 5 页 · 段落 p05-020

中文原意：3D 感知策略。 这些策略使用场景几何形状来预测操作。 Transporter Networks [427] 使用 2.5D 场景表示，从 RGB-D 观测结果构建正交颜色高度图来预测空间拾取和放置动作。 PerAct [298] 用 3D 体素表示观察结果和动作位置，而 RVT [88] 渲染重建点云的虚拟视图并在 3D 中组合它们的预测。 Act3D [83] 将图像特征提升到 3D 特征场中，并对动作位置进行从粗到细的采样。这些几何表示还支持生成动作模型：DP3 [426] 在从稀疏点云编码的紧凑特征上条件动作扩散，而 3D Diffuser Actor [146] 使用空间定位的场景标记和 3D 注意力来对末端效应器轨迹进行降噪。 Mamba Policy [25] 保留了点云调节，并将选择性状态空间模型与去噪网络中的注意力相结合。其他方法学习下游策略的几何表示。 GSRL [341] 预训练并冻结可泛化高斯预测器，然后将其预测的 3D 高斯场景表示提供给离线强化学习算法。结构潜在点预训练[137]使用逐点潜在变分自动编码器规范点坐标和特征，并使用高斯渲染监督；然后将其潜在表示提供给 ACT 策略。场景表示、学习目标和动作解码器仍然是不同的、可组合的设计选择。

解析：先分清观察表示与动作分布。视觉编码器/点云改变模型看见什么；ACT或扩散策略改变动作怎样生成。一个模型完全可以同时使用点云输入与扩散动作头，两条轴不互斥。

解析：预测动作块只是生成未来一段动作，实际可以执行前段后重新观察。物体被移动时，长块若不刷新会继续使用旧证据；长度、执行步数与重规划频率要一起理解。

自问：我能分开说明观测表示、动作生成方式和动作块执行策略吗？

本段引用：Mamba Policy [25]（辞典第26页）；Act3D [83]（辞典第84页）；RVT [88]（辞典第89页）；Structural Latent Point Pretraining [137]（辞典第138页）；3D Diffuser Actor [146]（辞典第147页）；PerAct [298]（辞典第299页）；GSRL [341]（辞典第342页）；DP3 [426]（辞典第427页）；Transporter Networks [427]（辞典第428页）

2.2 VLA：知识怎样接到动作

本节先抓住：视觉语言主干提供语义与表征，动作头负责可执行输出。离散token与连续扩散/流匹配改变生成方式；记忆、语言子任务与未来预测改变条件信息。条件更多不保证控制更可靠。

贯穿例子：“把杯子立起来”的语义需要转换成动作单位、坐标和控制频率；语言回答正确只是链条的一部分。

原文第 5 页 · 段落 p05-021

中文原意：通用 VLA 模型。 VLA 采用大规模预训练视觉语言主干来生成机器人动作，如 RT-2 [19]、OpenVLA [149] 和 RoboFlamingo [182] 中所示。早期的策略已经使用了语言调节和预训练编码器； VLA 通过机器人轨迹训练将更广泛的对象、关系和指令知识与动作预测联系起来。 τ0-VLA [24] 将高级子任务策略与流程匹配动作专家相结合，使用执行记忆和选择性世界模型引导的测试时搜索来进行子任务选择。

解析：VLA可能把过去、未来预测或语言推理作为条件，再用动作头生成运动。辅助信息怎样进入动作头才是机制关键；生成了一段推理文字或未来画面本身不证明动作变正确。

解析：训练时有专家未来标签，部署时没有。因此区分教师监督、内部预测和真实观测；若部署需要额外未来生成器，还要计入它的误差与耗时。

自问：语言视觉主干提供什么，动作头怎样把它变成机器人命令？

本段引用：RT-2 [19]（辞典第20页）；τ0-VLA [24]（辞典第25页）；OpenVLA [149]（辞典第150页）；RoboFlamingo [182]（辞典第183页）

原文第 5、6 页 · 段落 p05-022

中文原意：动作表示。 一个核心选择是如何解码机器人动作。 RT-2 [19] 和 OpenVLA [149] 将连续动作离散化为用于序列生成的标记。基于扩散和流的方法，包括 RDT-1B [217]、π0 [13]、π0.5 [269]、CogACT [177]、TinyVLA [371]、GR00T N1 [254]、SmolVLA [299]、DexVLA [372]、InternVLA-M1 [43]、LingBot-VLA [378] 和 Qwen-VLA [349]，根据视觉和语言特征为连续动作解码器提供条件以生成动作序列。 FAST [265] 将动作序列压缩为紧凑的令牌，而 OpenVLA-OFT [150] 结合并行解码和 动作分块（Action Chunking） 在一次前向传递中预测多个未来动作。 Vela [187] 用三次 B 样条表示未来运动，使用流匹配专家联合预测控制点和运动相关的时间跨度。 StarVLA [305] 提供了一个模块化的开源框架，用于训练和评估视觉语言主干和动作解码器的组合。

解析：VLA可能把过去、未来预测或语言推理作为条件，再用动作头生成运动。辅助信息怎样进入动作头才是机制关键；生成了一段推理文字或未来画面本身不证明动作变正确。

解析：训练时有专家未来标签，部署时没有。因此区分教师监督、内部预测和真实观测；若部署需要额外未来生成器，还要计入它的误差与耗时。

自问：语言视觉主干提供什么，动作头怎样把它变成机器人命令？

本段引用：π0 [13]（辞典第14页）；RT-2 [19]（辞典第20页）；InternVLA-M1 [43]（辞典第44页）；OpenVLA [149]（辞典第150页）；OpenVLA-OFT [150]（辞典第151页）；CogACT [177]（辞典第178页）；Vela [187]（辞典第188页）；RDT-1B [217]（辞典第218页）；GR00T N1 [254]（辞典第255页）；FAST [265]（辞典第266页）；π0.5 [269]（辞典第270页）；SmolVLA [299]（辞典第300页）；StarVLA [305]（辞典第306页）；Qwen-VLA [349]（辞典第350页）；TinyVLA [371]（辞典第372页）；DexVLA [372]（辞典第373页）；LingBot-VLA [378]（辞典第379页）

原文第 6 页 · 段落 p06-024

中文原意：VLA 与记忆。 当单个观察未揭示早期操作或任务进度时，历史和记忆提供时间背景。 RoboFlamingo [182] 通过 LSTM 策略头聚合连续的视觉语言特征。 MemoryVLA [295] 从记忆库中检索感知和认知特征，并将它们与当前表征融合以条件化扩散动作专家。 MEM [324]将低层策略中用于短期观察历史的视频编码器与其高层策略中过去事件的语言摘要相结合。 NativeMEM [365] 重用预训练的 VLA 视觉编码器，将每个摄像机视图中的每个历史帧压缩到单个记忆令牌中，从而允许策略关注长期视觉历史和当前观察。

解析：把VLA拆成“视觉/语言主干 + 动作表示/动作头”。RT-2、OpenVLA把动作离散化后生成词元；扩散或流匹配路线生成连续动作；有些工作并行预测动作块或压缩动作编码。它们是动作接口的不同选择。

解析：语言知识可以帮助识别“杯子”“把手”等概念，真实控制还需要坐标、量纲、动作频率与机器人数据对齐。工作卡按这些设计轴比较，不能只按主干大小推断效果。

自问：语言视觉主干提供什么，动作头怎样把它变成机器人命令？

本段引用：RoboFlamingo [182]（辞典第183页）；MemoryVLA [295]（辞典第296页）；MEM [324]（辞典第325页）；NativeMEM [365]（辞典第366页）

原文第 6 页 · 段落 p06-025

中文原意：VLA 与推理。 中间推理和语义预测在任务指令和机器人动作之间提供明确的指导。 RT-H [10] 和 π0.5 [269] 引入了运动的语言描述或影响后续动作的语义子任务。 ECoT [425]进一步将任务推理与空间信息（包括对象位置和末端执行器位置）结合起来，以指导动作预测。 OneTwoVLA [203] 和 Gemini Robotics 1.5 [320] 将推理与动作生成相结合，允许任务解释告知机器人的下一步行为。视觉子目标提供了一种补充表示：CoT-VLA [447] 预测中间目标图像并根据所需的场景配置调整后续动作。这些方法共同提供了不同级别的明确中间指导，从任务分解和运动描述到空间推理和视觉目标，将机器人对任务的理解与执行任务所需的动作联系起来。

> 解析：把VLA拆成“视觉/语言主干 + 动作表示/动作头”。RT-2、OpenVLA把动作离散化后生成词元；扩散或流匹配路线生成连续动作；有些工作并行预测动作块或压缩动作编码。它们是动作接口的不同选择。
>
> 解析：语言知识可以帮助识别“杯子”“把手”等概念，真实控制还需要坐标、量纲、动作频率与机器人数据对齐。工作卡按这些设计轴比较，不能只按主干大小推断效果。

自问：语言视觉主干提供什么，动作头怎样把它变成机器人命令？

本段引用：RT-H [10]（辞典第11页）；OneTwoVLA [203]（辞典第204页）；π0.5 [269]（辞典第270页）；Gemini Robotics 1.5 [320]（辞典第321页）；ECoT [425]（辞典第426页）；CoT-VLA [447]（辞典第448页）

原文第 6 页 · 段落 p06-026

中文原意：VLA 与想象力。 未来预测将 VLA 动作学习与机器人交互过程中场景的演变联系起来。 GR-1 [375]、GR-2 [32]、UP-VLA [434]、WorldVLA [29] 和 RynnVLA-002 [28] 将动作学习与未来图像预测结合起来，使用视觉变化作为机器人控制的附加监督。 FLARE [456]、VLA-JEPA [309] 和 World Guidance [307] 相反，对潜空间中的未来观测进行建模，学习预测特征或状态转换，而无需重建图像。这些目标将未来信息纳入策略表征中，尽管在执行过程中不需要生成未来图像。 BagelVLA [111]进一步交织语言推理、视觉预测和动作生成，将语义任务指导与动作执行循环内的视觉预期相结合。

> 解析：VLA可能把过去、未来预测或语言推理作为条件，再用动作头生成运动。辅助信息怎样进入动作头才是机制关键；生成了一段推理文字或未来画面本身不证明动作变正确。
>
> 解析：训练时有专家未来标签，部署时没有。因此区分教师监督、内部预测和真实观测；若部署需要额外未来生成器，还要计入它的误差与耗时。

自问：语言视觉主干提供什么，动作头怎样把它变成机器人命令？

本段引用：RynnVLA-002 [28]（辞典第29页）；WorldVLA [29]（辞典第30页）；GR-2 [32]（辞典第33页）；BagelVLA [111]（辞典第112页）；World Guidance [307]（辞典第308页）；VLA-JEPA [309]（辞典第310页）；GR-1 [375]（辞典第376页）；UP-VLA [434]（辞典第435页）；FLARE（未来表征）[456]（辞典第457页）

2.3 世界模型与世界动作模型

> 本节先抓住：要问预测什么、是否以动作作条件、预测怎样参与动作选择。未来图像可能作为监督、规划目标或推理条件；这些用途不同。只有训练时用了视频，并不表示部署时必须生成未来视频。
>
> 贯穿例子：比较两个候选推杯动作：世界模型预测结果，再用目标代价选动作；逆动力学则由当前与期望未来推断动作。

原文第 6 页 · 段落 p06-027

中文原意：世界模型（World Models） 了解环境如何响应机器人动作而变化。预测未来状态或观察结果支持对候选行动进行规划，并从想象的交互中学习策略[73,96,98,377]。 DINO-WM [459] 和 V-JEPA 2-AC [5] 通过模型预测控制预测动作条件视觉特征并优化动作序列以实现视觉目标。

解析：世界模型学习“给定当前状态和可能动作，未来会怎样”。规划需要比较候选动作的后果；训练也可利用未来表示作监督。三维重建、视频生成与可用于控制的动态预测各有侧重。

解析：例子：规划器考虑推杯子，却被漂亮视频误导，预测杯子穿过障碍物。视觉质量不足以支持规划；还要看动作条件、接触动态、多步误差以及闭环任务结果。

自问：未来预测在这里用于训练监督、规划还是环境模拟？证据支持哪一种？

本段引用：V-JEPA 2-AC [5]（辞典第6页）；World Models（Ha / Schmidhuber）[73]（辞典第74页）；DreamerV3 [96]（辞典第97页）；Dreamer [98]（辞典第99页）；DreamerV2 [377]（辞典第378页）；DINO-WM [459]（辞典第460页）

原文第 6 页 · 段落 p06-028

中文原意：世界模型（World Model）作为环境。 动作条件视频模型模拟策略学习和评估的交互。 UniSim [400] 模拟指令和低层控制的视觉结果，以训练具身规划器和策略。 Cosmos [251] 提供预训练的扩散和自回归视频模型，以适应机器人特定的预测。 Cosmos-Predict2.5 [252]统一了文本、图像和视频条件生成，并支持适应后的动作条件机器人 执行轨迹（rollout）。 Genie Envisioner [201]和DreamDojo [78]使用动作条件视频预测进行策略评估；后者还在适应目标机器人之前，通过推断潜在动作从人类视频中学习交互动态。 Cosmos 3 [253] 将系列扩展到共享多模式理解、世界生成和机器人动作建模。

解析：世界模型学习“给定当前状态和可能动作，未来会怎样”。规划需要比较候选动作的后果；训练也可利用未来表示作监督。三维重建、视频生成与可用于控制的动态预测各有侧重。

解析：例子：规划器考虑推杯子，却被漂亮视频误导，预测杯子穿过障碍物。视觉质量不足以支持规划；还要看动作条件、接触动态、多步误差以及闭环任务结果。

自问：未来预测在这里用于训练监督、规划还是环境模拟？证据支持哪一种？

本段引用：DreamDojo [78]（辞典第79页）；Genie Envisioner [201]（辞典第202页）；Cosmos [251]（辞典第252页）；Cosmos-Predict2.5 [252]（辞典第253页）；Cosmos 3 [253]（辞典第254页）；UniSim [400]（辞典第401页）

原文第 7 页 · 段落 p07-029

中文原意：4D 世界模型（World Model）. 4D 世界建模捕捉 3D 场景几何形状如何随时间演变，支持几何规划和交互模拟。 PointWorld [121] 将机器人动作表示为 3D 点流，并预测在时间范围内产生的场景点位移，支持基于采样的轨迹优化。 TesserAct [454] 联合生成 RGB、深度和表面法线视频，重建时间一致的 3D 点云，并使用点云逆动力学模型来推断动作。 4DGen [224] 使用跨视图点映射（pointmap）监督来生成几何一致的多视图 RGB-D 未来，位姿跟踪器从中恢复末端执行器轨迹。 MVISTA-4D [342] 从单视图观察中生成同步的、视图一致的 RGB-D 未来。它优化了测试时的潜在轨迹，以匹配想象的未来，并通过残余逆动力学细化解码的动作。 Kinema4D [394] 将机器人命令转换为运动学计算的 4D 轨迹，并使用其投影点映射（pointmap）来调节联合 RGB 和点映射（pointmap）生成，模拟周围环境的响应。

解析：世界模型学习“给定当前状态和可能动作，未来会怎样”。规划需要比较候选动作的后果；训练也可利用未来表示作监督。三维重建、视频生成与可用于控制的动态预测各有侧重。

解析：例子：规划器考虑推杯子，却被漂亮视频误导，预测杯子穿过障碍物。视觉质量不足以支持规划；还要看动作条件、接触动态、多步误差以及闭环任务结果。

自问：未来预测在这里用于训练监督、规划还是环境模拟？证据支持哪一种？

本段引用：PointWorld [121]（辞典第122页）；4DGen [224]（辞典第225页）；MVISTA-4D [342]（辞典第343页）；Kinema4D [394]（辞典第395页）；TesserAct [454]（辞典第455页）

原文第 7 页 · 段落 p07-030

中文原意：世界模型（World Model）作为规划师。 世界模型（World Models）提供视觉计划。逆动力学将预测的观察结果或潜表示映射到机器人动作。 UniPi [62] 和 RoboDreamer [463] 使用逆动力学将语言条件的视觉计划转化为行动。 Seer [323] 在端到端模型中将视觉预见和逆动力学结合起来，根据预测视觉状态的潜在表示来为动作预测提供条件。 GE-Act 2.0 [2] 在单个生成器过程中预测未来的视觉潜表示，然后将它们与逆动力学模型中的当前视觉潜表示和本体感觉相结合以产生动作块。这种联系也扩展到几何计划：TesserAct [454] 从重建的点云序列推断动作，而 MVISTA-4D [342] 将测试时轨迹潜在优化与残差逆动力学相结合，以细化针对想象的未来的动作。这些方法取决于预测未来的可行性以及将其转化为机器人特定控制的逆向能力。

> 解析：世界模型学习“给定当前状态和可能动作，未来会怎样”。规划需要比较候选动作的后果；训练也可利用未来表示作监督。三维重建、视频生成与可用于控制的动态预测各有侧重。
>
> 解析：例子：规划器考虑推杯子，却被漂亮视频误导，预测杯子穿过障碍物。视觉质量不足以支持规划；还要看动作条件、接触动态、多步误差以及闭环任务结果。

自问：未来预测在这里用于训练监督、规划还是环境模拟？证据支持哪一种？

本段引用：GE-Act 2.0 [2]（辞典第3页）；UniPi [62]（辞典第63页）；Seer [323]（辞典第324页）；MVISTA-4D [342]（辞典第343页）；TesserAct [454]（辞典第455页）；RoboDreamer [463]（辞典第464页）

原文第 7 页 · 段落 p07-031

中文原意：联合视频-动作建模。 世界动作模型（World-Action Models）(WAM) 将未来视觉预测与机器人动作学习相结合，将行为与其环境后果联系起来。 UVA [179]、UWM [468] 和 Motus [12] 在共享框架内联合学习视频预测和动作生成。 τ0-WM [462] 统一了视频动作生成和动作条件模拟。在测试时，预测的任务进度对候选者的未来进行评分；当最初的候选者不可靠时，选定的未来将指导行动的细化。 DreamZero [408]、LingBot-VA [176]、Cosmos Policy [151]、GigaWorld-Policy [406] 和 Being-H0.7 [230] 通过观察预测或潜在对齐将动作学习与未来视觉信息连接起来。 Fast-WAM [422] 通过视频协同训练学习高效的动作生成，无需未来的推理想象。 ImageWAM [442] 使用预先训练的图像编辑功能来指导动作生成，而无需解码未来的图像。 MaskWAM [418] 将可选的初始掩模提示与联合未来掩模、RGB 和动作预测相结合，以将控制集中在任务相关区域。最近的一个方向是联合建模语言、视频和动作：UniWAM [303] 通过联合多模态注意力、学习语言输出以及未来的视觉预测和动作，将物理推理器、世界生成器和动作预测器连接起来。

> 解析：世界模型学习“给定当前状态和可能动作，未来会怎样”。规划需要比较候选动作的后果；训练也可利用未来表示作监督。三维重建、视频生成与可用于控制的动态预测各有侧重。
>
> 解析：例子：规划器考虑推杯子，却被漂亮视频误导，预测杯子穿过障碍物。视觉质量不足以支持规划；还要看动作条件、接触动态、多步误差以及闭环任务结果。

自问：未来预测在这里用于训练监督、规划还是环境模拟？证据支持哪一种？

本段引用：Motus [12]（辞典第13页）；Cosmos Policy [151]（辞典第152页）；LingBot-VA [176]（辞典第177页）；UVA [179]（辞典第180页）；Being-H0.7 [230]（辞典第231页）；UniWAM [303]（辞典第304页）；GigaWorld-Policy [406]（辞典第407页）；DreamZero [408]（辞典第409页）；MaskWAM [418]（辞典第419页）；Fast-WAM [422]（辞典第423页）；ImageWAM [442]（辞典第443页）；τ0-WM [462]（辞典第463页）；UWM [468]（辞典第469页）

2.4 Agentic Robots

> 本节先抓住：作者强调模型介导的决策跨越策略执行、能力组织和经验利用。Agentic不是“换成更大LLM”的同义词，也不是对所有传统机器人系统的替代。
>
> 贯穿例子：动作失败时，系统可以补看、改计划、换技能或收集新数据；改变哪一层决定研究问题。

原文第 7 页 · 段落 p07-032

中文原意：智能体式机器人（Agentic Robots）组织机器人执行力和能力提升。给定一个目标，他们协调能力，解释执行反馈，并调整后续操作[119,125,196,297]。学习策略（包括 VLA 和 世界动作模型（World-Action Models））在此流程中提供可执行功能，将注意力转移到执行过程中如何选择、协调和修改行为。

> 解析：这部分串起能力的发展：动作策略提供行为，VLA加入语义，世界模型加入未来动态，Agentic系统组织目标与反馈。它们可以组合使用，区别在于各模块的输入、产物和实际决策角色。

自问：各能力在同一系统里可以怎样组合？

本段引用：Inner Monologue [119]（辞典第120页）；SayCan [125]（辞典第126页）；Code as Policies [196]（辞典第197页）；Hi Robot [297]（辞典第298页）

原文第 7 页 · 段落 p07-033

中文原意：这种观点还延伸到数据收集、模型训练和实验部署。智能体决策组织经验获取、指导学习实验并利用评估反馈来提高能力 [233, 383]。这些决策将当前的执行与未来任务的改进联系起来。以下部分检查四个功能域：模型、数据、环境和 运行支撑框架（Harness），它们描述了这些决策的作用以及如何获取、使用和改进机器人功能。

> 解析：这部分串起能力的发展：动作策略提供行为，VLA加入语义，世界模型加入未来动态，Agentic系统组织目标与反馈。它们可以组合使用，区别在于各模块的输入、产物和实际决策角色。

自问：各能力在同一系统里可以怎样组合？

本段引用：Eureka [233]（辞典第234页）；ENPIRE [383]（辞典第384页）

图3：十七类工作的位置地图

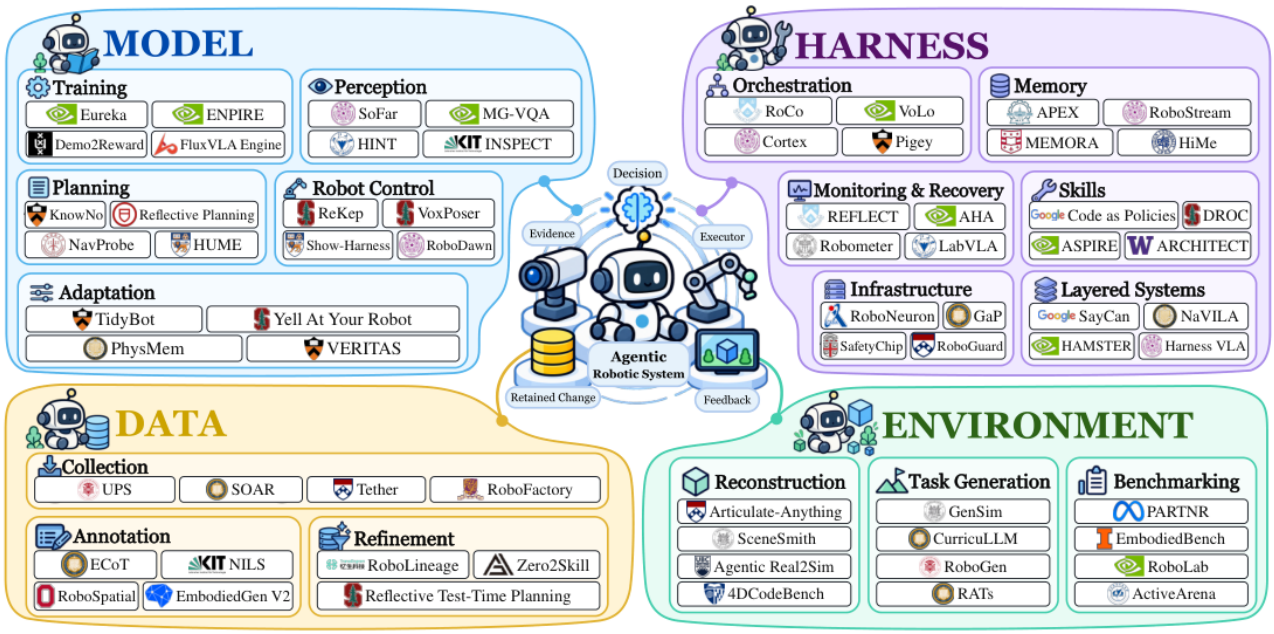


Figure 3. Taxonomy and literature landscape of agentic robotic systems. Four functional domains cover seventeen non-exclusive categories: Model covers learning and robot-facing decisions. Data covers the acquisition and curation of data. Environment covers worlds, tasks, and evaluation. Harness covers how robotic capabilities are coordinated, reused, and improved to support agentic task execution.

> 中文读图：图中的型号和项目是例子。先用四域定位作用，再查对应工作卡；不要把型号大小或名称数量当作质量证据。

3 智能体式机器人（Agentic Robots）的基础

原文第 8 页 · 段落 p08-034

中文原意：第 2 节中讨论的方法提供了感知、推理、动作预测和世界建模的功能。构建 智能体式机器人（Agentic Robots）需要通过任务规划、工具使用和执行反馈来协调这些功能，同时还支持新机器人功能的开发。为了描述这些系统的特征，本节首先定义 智能体式机器人（Agentic Robots）及其范围，然后介绍涵盖模型、数据、环境和 运行支撑框架（Harness）的分类法，最后将 智能体式机器人学（Agentic Robotics）与相关概念进行比较。

解析：这里预测未来的用途要单独确认：作为训练监督、测试时规划条件，还是模拟环境。前两者不等于可交互物理仿真；部署时是否生成未来也要看本段具体说明。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

3.1 定义与范围

本节先抓住：观察智能体介入的位置：它修改模型学习、处理数据、构造环境，还是组织运行。运行时调整计划与离线更新权重是不同时间尺度。

贯穿例子：一次试验中重试叫恢复；把失败轨迹加入数据并重新训练叫能力更新。

原文第 8 页 · 段落 p08-035

中文原意：我们将 智能体式机器人（Agentic Robots）定义为机器人系统，其中智能体使用环境观察和执行反馈自主计划和协调任务执行。给定一个目标，这些智能体确定实现该目标所需的步骤，选择和协调行动、技能或工具，并监控进度，而无需人类指定每个步骤。随着执行的进行，他们使用新的观察结果和结果来决定是否继续、收集更多信息、修改计划或从失败中恢复。我们在物理机器人和模拟中都考虑这样的系统。

解析：这里的定义强调目标、选择和反馈形成闭环：先选择打开抽屉，再检查开没开，根据结果继续取物或修改尝试。仅有低层位置误差反馈，还没有说明谁组织整个任务。

解析：执行中的临时重规划与任务结束后的能力改进是两种变化。前者改变本次计划；后者可能改变数据、技能库或模型，需要看是否保留到后续任务。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 8 页 · 段落 p08-036

中文原意：这些功能可以在集成模型内或通过模型、策略、程序和工具的组合来实现。例如，当从抽屉中取回物体时，智能体可以选择打开技能，检查抽屉是否已打开，并决定是否取回物体或修改打开尝试。定义性的考虑因素是智能体如何组织和调整任务执行以实现目标。仅靠行动预测或低层反馈控制并不能建立这些能力。

解析：这里的定义强调目标、选择和反馈形成闭环：先选择打开抽屉，再检查开没开，根据结果继续取物或修改尝试。仅有低层位置误差反馈，还没有说明谁组织整个任务。

解析：执行中的临时重规划与任务结束后的能力改进是两种变化。前者改变本次计划；后者可能改变数据、技能库或模型，需要看是否保留到后续任务。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 8 页 · 段落 p08-037

中文原意：除了任务执行之外，智能体还可以为开发机器人功能做出贡献。他们可以收集和完善机器人数据、构建任务和环境、修改训练程序或提高机器人技能和执行软件。来自交互和实验的反馈可以指导这些变化并帮助提高后续任务的性能。我们的论文研究了模型、数据、环境和 运行支撑框架（Harness） 中的这些贡献，涵盖了智能体如何支持机器人操作和能力开发。

> 解析：这里的定义强调目标、选择和反馈形成闭环：先选择打开抽屉，再检查开没开，根据结果继续取物或修改尝试。仅有低层位置误差反馈，还没有说明谁组织整个任务。
>
> 解析：执行中的临时重规划与任务结束后的能力改进是两种变化。前者改变本次计划；后者可能改变数据、技能库或模型，需要看是否保留到后续任务。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 9 页 · 段落 p09-038

中文原意：与当前作品的比较。 PAPAV [47]通过感知、预期、计划、行动和验证来比较计算机的使用和机器人的使用，强调物理体现的后果。 [287]通过协议接口、编排和嵌入式策略检查模型与机器人的集成。 [129]调查了涵盖动作预测权重和可执行技能的机器人学习方法。 [250]研究组织智能体执行的可执行软件。 [331]检查验证信号来自何处以及它们是否可用且值得信赖。我们的重点是这些组件在机器人使用和能力建设过程中如何协同工作。

> 解析：这里的定义强调目标、选择和反馈形成闭环：先选择打开抽屉，再检查开没开，根据结果继续取物或修改尝试。仅有低层位置误差反馈，还没有说明谁组织整个任务。
>
> 解析：执行中的临时重规划与任务结束后的能力改进是两种变化。前者改变本次计划；后者可能改变数据、技能库或模型，需要看是否保留到后续任务。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

本段引用：PAPAV [47]（辞典第48页）；Weights or Skills? [129]（辞典第130页）；Code as Agent Harness [250]（辞典第251页）；Embodied Agentic AI Review [287]（辞典第288页）；No Free Checker [331]（辞典第332页）

3.2 四域十七类

> 本节先抓住：四域是功能归类，十七类不是互斥标签。同一工作在多个表出现，只是同一机制影响多个功能环节，本包合并卡片并保留所有出现位置。
>
> 贯穿例子：RoboLineage既参与数据审查，也维护训练与部署记录，所以不能简单只归为一个动作模型。

原文第 9 页 · 段落 p09-039

中文原意：我们根据智能体在开发和操作机器人系统中扮演的角色将现有工作组织为四个领域：模型、数据、环境和运行支撑框架（Harness）。 模型涉及智能体如何使用和改进机器人感知、规划、控制和适应的模型。数据涉及智能体如何收集、标注和提炼机器人学习的数据。 Environment关注智能体如何构建场景、生成任务以及设计机器人训练和评估的评估。 运行支撑框架（Harness） 关注智能体如何协调模型、工具和机器人技能来执行任务，使用记忆和执行反馈来监控进度并从故障中恢复。这些领域共同描述了智能体在机器人能力开发和任务执行中的角色。图 3 展示了总体分类法，表 1 定义了这些领域内的 17 个类别。

> 解析：分类依据是贡献的功能。例如Zero2Skill的采集与后续训练可以共享同一条流水线；重复引用表示作者在另一段强调了它的另一种作用，不是出现了另一套同名方法。
>
> 解析：读工作卡时先看修改对象与产物，再看反馈和执行器。这样既能合并重复引用，也能比较相同类别内不同方法的接口。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 9 页 · 段落 p09-040

中文原意：该分类法区分功能贡献，而不是将每个机器人系统分配到单个类别。这些贡献可能跨越多个领域：例如，Zero2Skill [334] 将智能体引导的数据收集和准备与后续的机器人策略训练联系起来。它们还可以在一个域内一起操作，如 HarnessVLA [441] 中所示，它围绕固定执行器集成了记忆、编排、监控和恢复。ENPIRE [383] 说明了研究智能体如何协调实验以改进机器人策略，将实验管理与能力开发联系起来。因此，我们在与其相关功能相对应的类别下讨论每项工作，允许一篇论文在对多个类别做出贡献时出现在多个类别中。

解析：分类依据是贡献的功能。例如Zero2Skill的采集与后续训练可以共享同一条流水线；重复引用表示作者在另一段强调了它的另一种作用，不是出现了另一套同名方法。

解析：读工作卡时先看修改对象与产物，再看反馈和执行器。这样既能合并重复引用，也能比较相同类别内不同方法的接口。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

本段引用：Zero2Skill [334]（辞典第335页）；ENPIRE [383]（辞典第384页）；Harness VLA [441]（辞典第442页）

3.3 与相关概念比较

本节先抓住：具身、自治、基础模型与Agentic分别强调交互条件、自治能力、训练基础和决策组织。不要用一个关键词推断系统是否有记忆、重训练或真机闭环。

贯穿例子：一个能自动避障的传统导航器有自治能力，但未必用基础模型来选择工具或修改工作流。

原文第 9 页 · 段落 p09-041

中文原意：智能体式机器人学（Agentic Robotics）与 具身智能（Embodied AI）、自主机器人和机器人 基础模型（Foundation Model）密切相关。这些概念在方法和应用方面有所重叠，但强调机器人智能的不同方面。我们使用本文采用的范围来阐明它们的关系。

解析：具身智能是通过环境交互研究智能的广泛领域；自主机器人强调无需持续操作者指挥；机器人基础模型强调训练出的通用能力；本文的Agentic视角强调目标、能力选择与反馈怎样组织。它们重叠，不是四个互斥赛道。

解析：例如“抓杯子”的VLA与“抓空后先重看、再换抓法”的任务组织可以同属一个系统。要辨认的是模型能力和运行时决策各自贡献了什么。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 9 页 · 段落 p09-042

中文原意：智能体式机器人学（Agentic Robotics）与 具身智能（Embodied AI）. 具身智能（Embodied AI）通过智能体与其环境的交互来研究智能，包括探索、导航、操作和具身问答等任务。 具身智能（Embodied AI）和 智能体式机器人学（Agentic Robotics）都将物理或模拟环境中的感知、推理和行动联系起来 [63, 396]。区别在于强调这些能力的组织方式。 具身智能（Embodied AI）广泛研究通过环境交互进行的学习和行动;我们的智能体视角侧重于智能体如何选择能力、协调任务执行以及使用反馈修改决策。这一重点与检查基础模型（Foundation Model）作为调用和协调机器人功能的智能体的工作是一致的[287]。它还扩展到能力开发：例如，AutoRT 使用 基础模型（Foundation Model）来提议和过滤机器人数据收集的任务 [3]。

解析：具身智能是通过环境交互研究智能的广泛领域；自主机器人强调无需持续操作者指挥；机器人基础模型强调训练出的通用能力；本文的Agentic视角强调目标、能力选择与反馈怎样组织。它们重叠，不是四个互斥赛道。

> 解析：例如“抓杯子”的VLA与“抓空后先重看、再换抓法”的任务组织可以同属一个系统。要辨认的是模型能力和运行时决策各自贡献了什么。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

本段引用：AutoRT [3]（辞典第4页）；Embodied AI Survey [63]（辞典第64页）；Embodied Agentic AI Review [287]（辞典第288页）；Robotics Foundation Models Survey [396]（辞典第397页）

原文第 9、10 页 · 段落 p09-043

中文原意：智能体式机器人学（Agentic Robotics）与自主机器人技术。自主机器人技术涉及在没有持续人类控制的情况下执行任务的机器人。其既定研究已经整合了规划、行动、监控、观察和学习，以支持不断变化的环境中的刻意行为。因此，这些功能构成了共享基础，而不是 智能体式机器人学（Agentic Robotics）[126] 独有的功能。在这个更广泛的领域内，我们的论文研究了 基础模型（Foundation Model） 智能体如何通过解释目标、协调技能或工具以及将执行反馈纳入后续决策来参与这些功能。Inner Monologue（内部反馈独白）通过将观察结果和技能结果返回到语言模型以进行持续规划来说明这种联系[119]。因此，“智能体”确定了决策是如何进行的系统内组织；它并不意味着完全独立于人类指导或更高水平的自主权。

> 解析：具身智能是通过环境交互研究智能的广泛领域；自主机器人强调无需持续操作者指挥；机器人基础模型强调训练出的通用能力；本文的Agentic视角强调目标、能力选择与反馈怎样组织。它们重叠，不是四个互斥赛道。
>
> 解析：例如“抓杯子”的VLA与“抓空后先重看、再换抓法”的任务组织可以同属一个系统。要辨认的是模型能力和运行时决策各自贡献了什么。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

本段引用：Inner Monologue [119]（辞典第120页）；Deliberation Survey [126]（辞典第127页）

原文第 10 页 · 段落 p10-045

中文原意：智能体式机器人学（Agentic Robotics）与机器人 基础模型（Foundation Model）. 机器人 基础模型（Foundation Model） 的目标是通过广泛的训练和适应来获得机器人任务的可重用能力，支持从感知、规划到控制的功能[110]。它们的能力包括推理和决策：PaLM-E 将多模态观察整合到具身规划中，而 RT-2 将视觉和语言知识与行动生成联系起来，并探索行动之前的中间推理 [19, 61]。因此，区别在于模型学习的能力和这些能力如何参与正在进行的任务。 智能体式机器人学（Agentic Robotics）检查目标、可用操作和执行反馈如何控制后续决策，无论是在一个模型内还是跨多个组件实施。 Hi Robot 通过将更新中间指令的 VLM 与执行它们的 VLA 相结合来说明它们的兼容性[297]。

> 解析：具身智能是通过环境交互研究智能的广泛领域；自主机器人强调无需持续操作者指挥；机器人基础模型强调训练出的通用能力；本文的Agentic视角强调目标、能力选择与反馈怎样组织。它们重叠，不是四个互斥赛道。
>
> 解析：例如“抓杯子”的VLA与“抓空后先重看、再换抓法”的任务组织可以同属一个系统。要辨认的是模型能力和运行时决策各自贡献了什么。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

本段引用：RT-2 [19]（辞典第20页）；PaLM-E [61]（辞典第62页）；General-Purpose Robots Survey [110]（辞典第111页）；Hi Robot [297]（辞典第298页）

原文第 10 页 · 段落 p10-046

中文原意：以下四个部分依次回顾这些领域。在每个领域内，我们根据方法所服务的功能对方法进行分组，并通过代表性工作解释其主要方法。

> 解析：具身智能是通过环境交互研究智能的广泛领域；自主机器人强调无需持续操作者指挥；机器人基础模型强调训练出的通用能力；本文的Agentic视角强调目标、能力选择与反馈怎样组织。它们重叠，不是四个互斥赛道。

解析：例如“抓杯子”的VLA与“抓空后先重看、再换抓法”的任务组织可以同属一个系统。要辨认的是模型能力和运行时决策各自贡献了什么。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原表1 · 原文第10页

表 1. 智能体式机器人（ Agentic Robots ）的操作分类。四个功能域按照由模型介导的决策形成的机器人系统的一部分组织了十七个非排他性类别。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

图4：模型侧的五种介入

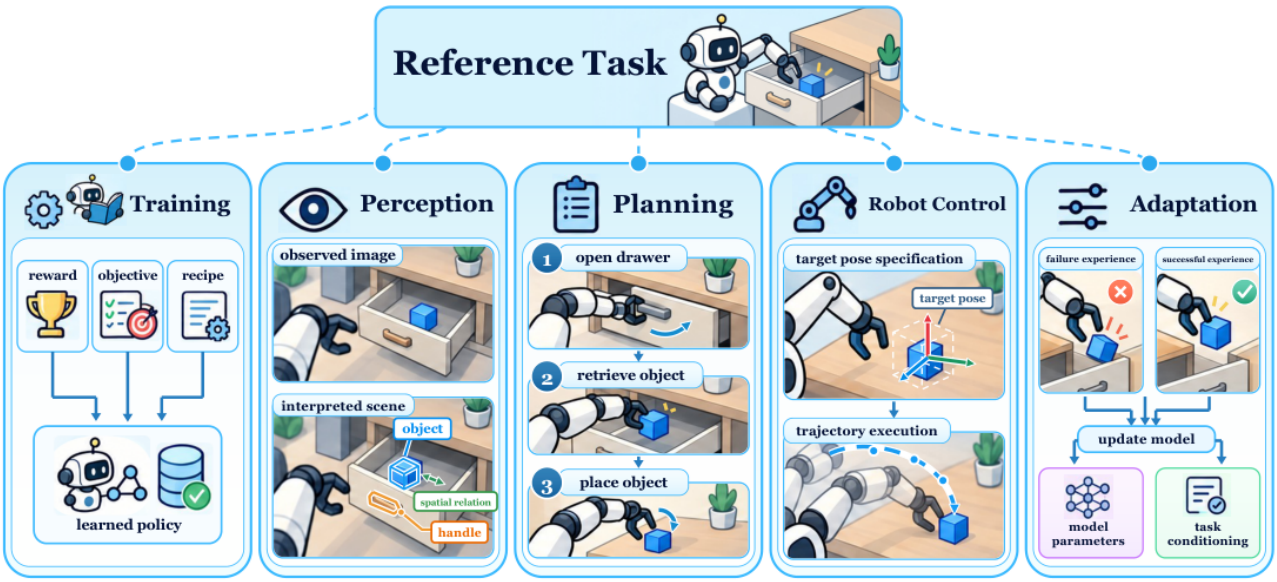


Figure 4. Five model-side interventions illustrated by a drawer-retrieval task. Training shapes the learning process; Perception interprets task-relevant evidence; Planning organizes task structure; Robot Control specifies robot-facing commands; and Adaptation changes model parameters or task-specific conditioning using experience.

中文读图：用开抽屉取物说明训练、感知、规划、控制和适应。五者分别改变学习过程、证据、步骤、动作实现和后续行为。

4 模型：智能体应该学习、理解、决策和控制什么？

原文第 11 页 · 段落 p11-048

中文原意：图 4 显示抽屉任务可能因不同原因而失败，并且每种原因都需要不同的修复：有关句柄的更好证据、修改的操作顺序、可行的抓取命令或在重复失败后改进的策略。这些是不同的模型端干预措施。训练询问谁选择学习过程；感知、规划和机器人控制追踪证据如何转化为行动；适应询问哪些参数或特定任务条件会随着经验而变化。它们的有效性取决于将每个决策与可以执行该决策的执行者相匹配。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

4.1 Model / Training：修改什么训练对象

本节先抓住：奖励代码、训练训练配方、模型权重与数据记录是不同的可编辑对象。智能体提出修改，训练器产生证据，固定评测判断修改是否有效；三者不应混为一个自我打分过程。

贯穿例子：Eureka搜索奖励程序；其他方法可能调配置、残差策略或数据。比较时先固定学习算法、交互预算和测试任务。

原文第 11 页 · 段落 p11-049

中文原意：智能体作用于学习过程而不是环境。它指定奖励、目标、配置或实验。在此过程中，优化器会消耗生成的工件，机器人 执行轨迹（rollout） 会揭示策略是否学习了所需的行为。在迭代工作流程中，智能体使用这些结果来决定下一步要修改的内容。现有系统的不同之处主要在于可编辑工件的范围：1) 奖励设计方法保持优化器固定并仅修改其消耗的信号，而 2) 程序和实验级方法将修订扩展到策略代码、训练管道，以及 3) 实验工作流程本身。

解析：把“智能体修改的目标”与“判断好坏的标准”分开：奖励代码可变，任务成功判据和测试任务应保持可比。奖励变高不能直接推出物理任务做得更好。

解析：问可修改的东西究竟是什么：一次程序、持久代码库、运行上下文还是模型权重。代码保存下来与训练模型不同；修改监测代码还可能改变评判标准。

自问：我能说清被修改的是奖励、训练流程、数据还是权重，以及评价它的独立信号吗？

原文第 11 页 · 段落 p11-050

中文原意：奖励设计。 奖励将行为目标转化为优化器可以使用的信号。 Eureka [233] 建立了基本循环：语言模型编写奖励代码并使用训练反馈对其进行修改。然而，这样的反馈可以确认终端谓词得到满足，而不揭示执行是否实际上遵循指令。 RDA [161] 通过对结果行为进行语义分析进一步解决了这一差距。它将指令分解为子任务，生成可执行奖励候选，训练策略，并评估 展开轨迹（Rollout） 视频以及子任务奖励日志。

解析：把“智能体修改的目标”与“判断好坏的标准”分开：奖励代码可变，任务成功判据和测试任务应保持可比。奖励变高不能直接推出物理任务做得更好。

解析：本段并列了 RDA、Eureka。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：我能说清被修改的是奖励、训练流程、数据还是权重，以及评价它的独立信号吗？

本段引用：RDA [161]（辞典第162页）；Eureka [233]（辞典第234页）

原文第 11、12 页 · 段落 p11-051

中文原意：Eureka 和 RDA 通过完整的策略训练来评估候选奖励，这使得每次评估的计算成本都很高。后来的方法通过两种方式降低了这种成本：它们用更便宜的证据来验证奖励，或者缩小搜索范围。 EvoNav [453] 通过规则和简短的智能体训练来筛选奖励计划，假设廉价的测试保留了有用候选方案的排名。然后，CRWM [401] 根据离线奖励干预得出的因果先验条件设置一次性奖励代码生成器。它预先支付先前的费用，而强化学习的结果永远不会反馈到奖励中。 Demo2Reward [92] 使用 VLM 元批评家在策略之前调整带标签的演示前缀上的 VLM 奖励提示学习。在 ReCoVLA [106] 中，VLM 将故障观察映射到描述符，确定性编译器将它们绑定到预定义的基于状态的奖励组件，RL 训练特定于故障的剩余策略，同时基本 VLA 保持冻结。一般来说，每种方法都会用全面训练反馈的通用性来换取更便宜但更受限制的可靠性来源：智能体 执行轨迹（rollout）、离线因果结构、标记演示或预定义的奖励组件。

解析：把“智能体修改的目标”与“判断好坏的标准”分开：奖励代码可变，任务成功判据和测试任务应保持可比。奖励变高不能直接推出物理任务做得更好。

解析：本段并列了 Demo2Reward、ReCoVLA、CRWM、EvoNav。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：我能说清被修改的是奖励、训练流程、数据还是权重，以及评价它的独立信号吗？

本段引用：Demo2Reward [92]（辞典第93页）；ReCoVLA [106]（辞典第107页）；CRWM [401]（辞典第402页）；EvoNav [453]（辞典第454页）

原文第 12 页 · 段落 p12-053

中文原意：策略训练。除了奖励设计之外，智能体还通过实施策略、组织训练实验和修改学习过程来参与机器人策略的制定。 ENPIRE [383] 首先利用人类反馈构建重置、验证和安全接口，然后将这些接口暴露给编码智能体，以检查真实的 执行轨迹（rollout） 并修改策略或训练代码。 行为克隆（Behavior Cloning）、强化学习（Reinforcement Learning） 和启发式策略合成是研究过程中的替代方案，而不是每个结果控制器的同时属性。 HARBOR [191] 将模拟 强化学习（Reinforcement Learning） 组织为类型化工件和门检查阶段。失败的接口检查返回可操作的诊断信息；并行调优试验保留代码、视频和日志，并检索重复的课程以供以后的实验使用。 ENPIRE 依赖于可靠的物理重置； HARBOR 取决于生成的任务、奖励和算法接口之间的协议。每一个都会给实验智能体带来不同的瓶颈。

解析：把“智能体修改的目标”与“判断好坏的标准”分开：奖励代码可变，任务成功判据和测试任务应保持可比。奖励变高不能直接推出物理任务做得更好。

解析：本段并列了 HARBOR、ENPIRE。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：我能说清被修改的是奖励、训练流程、数据还是权重，以及评价它的独立信号吗？

本段引用：HARBOR [191]（辞典第192页）；ENPIRE [383]（辞典第384页）

原文第 12 页 · 段落 p12-054

中文原意：这种对策略训练的更广泛的探索使得研究议程的可重复执行和人为控制变得尤为重要。这项工作[148]使用编码智能体编辑、实验启动和基于日志的分类来在人类主导的研究浪潮中进行四足策略改进。 Nautilus [139] 相反，标准化了策略、机器人和基准合约，并隔离了它们的执行环境，允许编码智能体组装和验证学习工作流程。总的来说，这些工作不仅扩大了人类议程下的实验搜索，而且减少了实施这些实验所需的集成工作。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

解析：本段并列了 Nautilus、ADARLR。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：我能说清被修改的是奖励、训练流程、数据还是权重，以及评价它的独立信号吗？

本段引用：Nautilus [139]（辞典第140页）；ADARLR [148]（辞典第149页）

原文第 12 页 · 段落 p12-055

中文原意：工作流程. 工作流程包含使智能体保持正确方向的结构和测量。 [417]将每个实验视为具有不可变目标、控制器拥有的测量、受工件限制的工具质量和崩溃恢复的持久事务。在 LWD [359] 中，车队的成功、失败和人为干预通过离线到在线的后训练来更新预训练的通才 VLA。这两个工作流程在不同层面进行干预：根据车队经验进行策略后训练，以及用于评估修订的实验管理。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

解析：本段并列了 LWD、AgenticRobotics。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：我能说清被修改的是奖励、训练流程、数据还是权重，以及评价它的独立信号吗？

本段引用：LWD [359]（辞典第360页）；AgenticRobotics [417]（辞典第418页）

原文第 12 页 · 段落 p12-056

中文原意：当智能体编辑的内容不仅仅是训练训练训练配方时，决策和测量的分离也很重要。奖励、提示、工具和数据编辑可能会改变干预及其判断。 RoboLineage [231] 通过将 执行轨迹（rollout）、评论、数据集决策、训练运行、策略元数据和部署建议存储为类型化工件，使这种转换变得明确。从基础设施的角度来看，FluxVLA Engine [189]提供了连接数据、训练、评估和部署的配置解析合约。然后，EmbodiedSkills [351]提供单独的接口，通过这些接口可以训练或调整规划、验证和执行组件。有用的设计原则是公开目标、方案和实验以供修改，同时保持用于判断它们的测量结果的稳定。学习选择在“训练”下进行分析；支持这些选择的执行合同和测量接口在第 7.5 节中作为能力建设基础设施进行讨论。相关工作对比如表2所示。

解析：把“智能体修改的目标”与“判断好坏的标准”分开：奖励代码可变，任务成功判据和测试任务应保持可比。奖励变高不能直接推出物理任务做得更好。

解析：本段并列了 FluxVLA Engine、RoboLineage、EmbodiedSkills。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：我能说清被修改的是奖励、训练流程、数据还是权重，以及评价它的独立信号吗？

本段引用：FluxVLA Engine [189]（辞典第190页）；RoboLineage [231]（辞典第232页）；EmbodiedSkills [351]（辞典第352页）

原表2 · 原文第12页

表 2. 训练工作比较（第 4.1 节）。在策略学习期间收集训练指标；评估指标评估由此产生的策略。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：Nautilus [139]；ADARLR [148]；Eureka [233]；LWD [359]；EvoNav [453]。

原表2 · 原文第13页

表 2（续）

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：ReCoVLA [106]；RDA [161]；FluxVLA Engine [189]；HARBOR [191]；RoboLineage [231]；EmbodiedSkills [351]；ENPIRE [383]；CRWM [401]；AgenticRobotics [417]。

4.2 Model / Perception：为下一步提供证据

本节先抓住：感知输出可能是答案、场景图、目标像素、物体句柄或相机动作。主动感知会改变可见证据；语义定位后仍需深度、坐标与可达性检查。

贯穿例子：移开遮挡物能帮助回答“里面有什么”，但动作风险也变了；旧物体ID不代表旧位姿仍可用于抓取。

原文第 13 页 · 段落 p13-057

中文原意：感知将观察结果转化为与任务相关的概念、关系、几何或信念。主体干预可能是一种解释、要求另一次观察或揭示缺失证据的身体动作。该表示必须支持下一个决定：例如，识别手柄还需要足够精确的几何形状以实现预期的抓握。

解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。

自问：这段方法最终输出给下一模块的是什么？额外观察怎样改变证据？

原文第 13 页 · 段落 p13-058

中文原意：观察。 观察过程选择当前观点需要更新时寻求哪些证据。定位人机对话系统[160]将多模态交互与任务级注意力和工具调用相结合，而HINT[239]在操作模式转换时更新意图并路由摄像机视图，并在跟踪置信度较低时重新定位相同的已经选定的目标。 SCOPE [103] 使相机定向和缩放显式动作，其返回的观察结果决定下一个感知请求，说明相机智能体而不是固定检测器。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。
>
> 解析：本段并列了 SCOPE、MSR、HINT。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这段方法最终输出给下一模块的是什么？额外观察怎样改变证据？

本段引用：SCOPE [103]（辞典第104页）；MSR [160]（辞典第161页）；HINT [239]（辞典第240页）

原文第 13 页 · 段落 p13-059

中文原意：主动感知改变了证据本身。 MG-VQA（以前称为 Probe）[11] 让 VLM 在回答视觉问题之前调用感知、抓取和移除以及推送工具来暴露被遮挡的信息。 [257]使用视觉谓词检查和附加视点来纠正提供给规划域定义语言（PDDL）规划器的状态。因此，操纵可以收集信息，而额外的观察可能会使象征性的规划前提失效。移动物体或摄像机的选择会牺牲新证据的价值来对抗运动成本和风险。

> 解析：先把作者的分类判断与实验结论分开。分类说明工作在系统里起什么作用；提升有多大、适用哪些条件，仍要查看该工作给出的实验协议。
>
> 解析：本段并列了 MG-VQA、VAP-TAMP。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这段方法最终输出给下一模块的是什么？额外观察怎样改变证据？

本段引用：MG-VQA [11]（辞典第12页）；VAP-TAMP [257]（辞典第258页）

原文第 13 页 · 段落 p13-060

中文原意：Decisions. 根据之前的观察，决策是执行者可以使用的最终产品。 ABot-N1 [86] 使用循环空间推理器来更新语义目标，同时独特的动作专家提供连续的导航行为；可操纵的 VLA 策略 [42] 类似地公开了语义子任务、原子运动和接地点或跟踪操纵模式。分布式会话操作框架 [142] 通过每个命令的深度和机器人坐标进行像素预测，而 RoboVista 的 [386] 机器人 VQA 抽象评估模型是否可以回答跨不同应用模块的感知问题。无论输出是更新的视图、几何目标还是象征性的答案，其质量取决于它必须支持的下游决策。

> 解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。
>
> 解析：本段并列了 Steerable Policies、ABot-N1、CFHRCM、RoboVista。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这段方法最终输出给下一模块的是什么？额外观察怎样改变证据？

本段引用：Steerable Policies [42]（辞典第43页）；ABot-N1 [86]（辞典第87页）；CFHRCM [142]（辞典第143页）；RoboVista [386]（辞典第387页）

原文第 13 页 · 段落 p13-061

中文原意：导航需要空间证据才能在机器人移动时保持可访问性。 AgenticNav [188] 提供航向索引视图、选择性召回、紧凑轨迹图和几何工具。 VLM可以请求证据并选择导航目标；基于深度的有效性检查和移动仍然低于该选择。 SoFar [271] 提供了早期的空间 落地关联（Grounding） 比较：对象感知构建了一个包含位置和语义方向向量的场景图，并且 VLM 指定了确定性拟合和运动规划实现的所需对象几何形状。仅当绑定到执行者理解的方向表示时，诸如“直立”之类的文字描述才变得有用。

> 解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。
>
> 解析：本段并列了 AgenticNav、SoFar。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这段方法最终输出给下一模块的是什么？额外观察怎样改变证据？

本段引用：AgenticNav [188]（辞典第189页）；SoFar [271]（辞典第272页）

原文第 14 页 · 段落 p14-062

中文原意：反射. 机器人需要对环境或行为的变化进行及时的反射。 [193]首先通过接地对象锚点和执行收据公开感知和运动工具；模型接收图像和收据，而校准的深度几何图形由后端处理。存档的身份不会使其旧位姿成为有效的运动目标：丢失或无效的锚点需要更新 落地关联（Grounding）。然后，[100]在接受候选者之前使用任务条件关系和共享实例证据来请求补充视图，但他们的模拟使用预言机实例掩码和 ID。 INSPECT [370]通过带标注的视频重放和固定的机器人图像网格而不是闭环物理检查试验进一步评估学习的视图效用和承诺决策。

> 解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。
>
> 解析：本段并列了 CoRelNav、Robo-Harness K1、INSPECT。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这段方法最终输出给下一模块的是什么？额外观察怎样改变证据？

本段引用：CoRelNav [100]（辞典第101页）；Robo-Harness K1 [193]（辞典第194页）；INSPECT [370]（辞典第371页）

原文第 14 页 · 段落 p14-063

中文原意：导航总结了所有动态和静态空间信息。 OpenFrontier [261] 对视觉边界进行评分并验证候选目标，当候选目标被拒绝时返回搜索； GoalVLM [127] 将前沿评分与跨智能体分配和共享地图结合起来。 FARE [200] 通过不断演化的置信图推理来选择全局路径点，而 AnchorVLN [330] 通过 ROS 服务器将模型命令绑定到类型化对象句柄。前沿分数有助于选择探索的位置，置信图保留更长的空间依赖性，并且类型句柄限制命令引用。它们的常见漏洞是陈旧的 落地关联（Grounding）：当其位置不再支持建议的操作时，必须重新观察记住的对象标识。相关工作对比如表3所示。

> 解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。
>
> 解析：本段并列了 GoalVLM、FARE、OpenFrontier、AnchorVLN。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这段方法最终输出给下一模块的是什么？额外观察怎样改变证据？

本段引用：GoalVLM [127]（辞典第128页）；FARE [200]（辞典第201页）；OpenFrontier [261]（辞典第262页）；AnchorVLN [330]（辞典第331页）

原表3 · 原文第14页

表 3. 感知方面的研究比较（第 4.2 节）。输入列出观察结果，不包括任务指令和查询。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：MG-VQA [11]；Steerable Policies [42]；ABot-N1 [86]；CoRelNav [100]；SCOPE [103]；GoalVLM [127]；CFHRCM [142]；MSR [160]；AgenticNav [188]；Robo-Harness K1 [193]；FARE [200]；HINT [239]；VAP-TAMP [257]；OpenFrontier [261]；SoFar [271]；AnchorVLN [330]；INSPECT [370]；RoboVista [386]。

4.3 Model / Planning：提出、验证、更新

本节先抓住：规划可以输出下一操作、语言子目标、依赖图或代码。验证依据可能是符号规则、场景图、预测画面或真实反馈。预测可靠性限制可规划的时程。

贯穿例子："先开抽屉再拿杯子"是依赖；抽屉没打开时应更新剩余计划，而不是继续执行旧序列。

原文第 14 页 · 段落 p14-064

中文原意：根据感知，机器人计划一系列动作来完成任务。在执行计划之前，需要检查这些行动是否可行。计划在执行过程中也可能需要更改，例如当对象移动或操作失败时。因此，当机器人执行任务并了解其决策是否有效时，规划会继续进行。我们分三个阶段讨论这个过程：生成计划、执行前检查以及执行期间更新。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

自问：谁提出下一步，谁检查，遇到新证据后改计划还是改规划模型？

原文第 15 页 · 段落 p15-065

中文原意：生成计划。一些计划者一次选择一个操作，并使用其结果来决定下一步做什么。当后续步骤依赖于机器人尚不具备的信息时，这非常有用。例如，一个系统提供了用于环顾四周、移动到某个位置、抓取对象和访问记忆的工具[290]。规划器在选择下一个操作之前读取这些工具返回的文本。 AgentRob [218]遵循类似的方法，VLM 控制器在选择下一个操作之前观察动作基础动作（primitive）的结果。在这两个系统中，规划者选择操作，机器人控制器处理物理运动。

解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。

解析：本段并列了 AgentRob、FLA。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：谁提出下一步，谁检查，遇到新证据后改计划还是改规划模型？

本段引用：AgentRob [218]（辞典第219页）；FLA [290]（辞典第291页）

原文第 15 页 · 段落 p15-066

中文原意：规划者还可以指定一个子目标，并让训练有素的策略产生实现该子目标所需的行动。 Hi Robot [297] 使用高层 VLM 向 VLA 发出简短的语言命令。这些命令会定期更新或在用户干预后更新。 Gemini Robotics-ER 1.5 [320] 类似地处理高级推理并向 Gemini Robotics 1.5 动作模型提供语言指令。 LocalNav [9] 将这种划分应用于导航。其经过精炼的场景图推理器决定是走向某个物体、探索一个房间、执行边界搜索还是停止。然后，PointGoal 策略将执行导航。

解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。

> 解析：本段并列了 LocalNav、Hi Robot、Gemini Robotics 1.5。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：谁提出下一步，谁检查，遇到新证据后改计划还是改规划模型？

本段引用：LocalNav [9]（辞典第10页）；Hi Robot [297]（辞典第298页）；Gemini Robotics 1.5 [320]（辞典第321页）

原文第 15 页 · 段落 p15-067

中文原意：除了选择下一个操作或子目标之外，规划人员还可以指定多个操作之间的相互依赖程度。例如，[145]将多机器人任务分解为经典规划器的 PDDL 子问题，并使用规划失败来改进智能体的提示。在工业环境中，IMR-LLM [306] 使用 LLM 来帮助构建析取图，确定性求解器使用该析取图来调度操作。然后，进程树指导可执行机器人程序的生成。为了将符号规划与执行联系起来，EmboTeam [428] 将语义处理与基于 PDDL 的搜索相结合，并将结果计划转换为包含条件、回退和同步的行为树。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

> 解析：本段并列了 HLMF、IMR-LLM、EmboTeam。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：谁提出下一步，谁检查，遇到新证据后改计划还是改规划模型？

本段引用：HLMF [145]（辞典第146页）；IMR-LLM [306]（辞典第307页）；EmboTeam [428]（辞典第429页）

原文第 15 页 · 段落 p15-068

中文原意：执行前检查计划。一旦提出行动，规划者就可以检查它们是否与其场景模型和任务要求一致。例如，SayPlan [274] 识别分层 3D 场景图的相关部分，并使用来自场景图模拟器的反馈来修改其计划。在此过程中，经典路径规划器提供位置之间的路线。 GAVEL [350] 同样使用图 世界模型（World Model） 在执行之前验证和修复候选计划。对于从工业过程中得出的约束，MaCoPlanner [389]将手册转换为类型化的中间表示，并根据过程和状态转换约束检查操作序列。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

> 解析：本段并列了 SayPlan、GAVEL、MaCoPlanner。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：谁提出下一步，谁检查，遇到新证据后改计划还是改规划模型？

本段引用：SayPlan [274]（辞典第275页）；GAVEL [350]（辞典第351页）；MaCoPlanner [389]（辞典第390页）

原文第 15 页 · 段落 p15-069

中文原意：在规划行动之前，系统还可以检查目标是否明确且适合机器人。例如，STARS [153] 生成候选目标描述，并在底层任务学习智能体搜索操作以实现所选目标之前对其进行分析、修复和选择。在这里，检查关注的是所提出的目标是否可以理解并且适合智能体及其情况，而不是是否可以执行特定的动作序列。

> 解析：先把作者的分类判断与实验结论分开。分类说明工作在系统里起什么作用；提升有多大、适用哪些条件，仍要查看该工作给出的实验协议。

自问：谁提出下一步，谁检查，遇到新证据后改计划还是改规划模型？

本段引用：STARS [153]（辞典第154页）

原文第 15 页 · 段落 p15-070

中文原意：除了基于图表和规则的检查之外，规划人员还可以使用学习模型来评估候选方案的行动。例如，反思式测试时规划[105]通过内部反思模型对提出的高级行动进行评分，而外部和回顾性反思则随后借鉴实际结果。在执行之前评估行动的另一种方法是想象它们的视觉后果。 ReflectVLM [71] 预测提议的动作序列的未来图像，并将这些预测与当前和目标图像一起考虑以选择下一个动作基础动作（primitive）。然后，脚本控制器执行选定的基础动作（primitive）。

> 解析：这里预测未来的用途要单独确认：作为训练监督、测试时规划条件，还是模拟环境。前两者不等于可交互物理仿真；部署时是否生成未来也要看本段具体说明。
>
> 解析：本段并列了 ReflectVLM、Reflective Test-Time Planning。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：谁提出下一步，谁检查，遇到新证据后改计划还是改规划模型？

本段引用：ReflectVLM [71]（辞典第72页）；Reflective Test-Time Planning [105]（辞典第106页）

原文第 15 页 · 段落 p15-071

中文原意：在执行期间更新计划。即使在这些检查之后，操作也可能会产生意外的结果，或者场景的更改可能会使剩余的计划不再适用。因此，规划者需要执行反馈来决定是继续还是修改后续步骤。例如，当实际结果与其预测不同、用户更改指令或场景发生变化时，WCM [51] 会修改其下一个决策。必要时，还可以要求用户作出澄清。

> 解析：这里预测未来的用途要单独确认：作为训练监督、测试时规划条件，还是模拟环境。前两者不等于可交互物理仿真；部署时是否生成未来也要看本段具体说明。

自问：谁提出下一步，谁检查，遇到新证据后改计划还是改规划模型？

本段引用：WCM [51]（辞典第52页）

原文第 15 页 · 段落 p15-072

中文原意：为了支持这些决策，系统需要报告执行结果并确定何时需要重新计划。例如，[202]为此提供了状态、事件和反射工具。物理检查失败可以作为直接触发因素。当动作验证失败时，ActionReasoning 2.0 [335] 会丢弃剩余的动作序列，并根据更新的场景状态生成新的序列，保留先前验证的任务进度。类似地，ATP-VPDSG [232] 检查每个基础动作（primitive）的结果，并使用执行反馈来诊断故障并修改其余操作。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。
>
> 解析：本段并列了 AARC、ATP-VPDSG、ActionReasoning 2.0。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：谁提出下一步，谁检查，遇到新证据后改计划还是改规划模型？

本段引用：AARC [202]（辞典第203页）；ATP-VPDSG [232]（辞典第233页）；ActionReasoning 2.0 [335]（辞典第336页）

原文第 16 页 · 段落 p16-073

中文原意：在选择下一步行动时，规划者还可以借鉴之前的行动和观察结果，而不仅仅是依赖最新的结果。MaP-WAM [448]使用已完成的行动段的记录作为计划上下文，并检查执行进度以确定何时更新计划及其上下文。对于视觉历史，ActiveArena-Plan [186] 预测当前的子任务并选择历史帧保留在记忆中。单独训练的动作策略使用子任务、选定的帧、当前观察和机器人状态来生成动作。

解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。

解析：本段并列了 ActiveArena (Plan)、MaP-WAM。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：谁提出下一步，谁检查，遇到新证据后改计划还是改规划模型？

本段引用：ActiveArena (Plan) [186]（辞典第187页）；MaP-WAM [448]（辞典第449页）

原文第 16 页 · 段落 p16-074

中文原意：对于具体问题回答，HFLEX-EQA [270] 将过去的视觉观察与先验平面图和增量构建的场景图相结合，以在房间探索、对象检查和房间发现之间进行选择。专门的低层规划人员执行这些操作。同样，NavProbe [210] 检索历史图像和几何记录以修改中间目标并在选择下一个导航操作之前评估它们是否已得到满足。

解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。

解析：本段并列了 NavProbe、HFLEX-EQA。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：谁提出下一步，谁检查，遇到新证据后改计划还是改规划模型？

本段引用：NavProbe [210]（辞典第211页）；HFLEX-EQA [270]（辞典第271页）

原文第 16 页 · 段落 p16-075

中文原意：然而，如果规划模型没有描述机器人遇到的情况，修改动作序列可能是不够的。为了解决这个问题，休谟[382]提出了对领域模型的可能补充，将它们视为不确定的假设，并指定了检查它们的条件。然后，感知会确认或拒绝这些假设，而被拒绝的假设将指导后续的重新规划。在这种情况下，执行反馈会更改计划和用于生成计划的模型。

解析：先把作者的分类判断与实验结论分开。分类说明工作在系统里起什么作用；提升有多大、适用哪些条件，仍要查看该工作给出的实验协议。

自问：谁提出下一步，谁检查，遇到新证据后改计划还是改规划模型？

本段引用：HUME [382]（辞典第383页）

原文第 16 页 · 段落 p16-076

中文原意：总的来说，机器人的计划应该取决于它预测其行动结果的可靠性。当这些结果是可预测的时，更长的计划可以组织多个操作之间的依赖关系。当结果不确定或场景发生变化时，提前计划更少的步骤可以让新的观察结果指导后续决策。无论哪种情况，执行前的检查和执行期间的反馈都有助于确定是否继续执行计划、修改其操作或更新用于生成计划的模型。相关工作对比如表4所示。

解析：这里预测未来的用途要单独确认：作为训练监督、测试时规划条件，还是模拟环境。前两者不等于可交互物理仿真；部署时是否生成未来也要看本段具体说明。

自问：谁提出下一步，谁检查，遇到新证据后改计划还是改规划模型？

原表4 · 原文第16页

表 4. 规划工作比较（第 4.3 节）。操控包括移动操控；模型列出了规划、重新规划或验证主干。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：LocalNav [9]；WCM [51]；ReflectVLM [71]；Reflective Test-Time Planning [105]；HLMF [145]；STARS [153]；NavMCP [167]；SERP [171]；SafeMem [180]；DV-VLN [194]；AARC [202]；AgentRob [218]；ATP-VPDSG [232]；RoCo [237]；SayPlan [274]；KnowNo [279]；FLA [290]；Hi Robot [297]；IMR-LLM [306]；Gemini Robotics 1.5 [320]；ActionReasoning 2.0 [335]；EEAgent [340]；HUME [382]；MaCoPlanner [389]；EmboTeam [428]；MaP-WAM [448]。

原表4 · 原文第17页

表 4（续）

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：CoRelNav [100]；ActiveArena (Plan) [186]；NavProbe [210]；BTA [235]；HFLEX-EQA [270]；GAVEL [350]。

4.4 Model / Robot Control：语义怎样落到运动

本节先抓住：明确控制输出与执行器：像素目标、关键点约束、代价图、轨迹、技能调用都可能在低层转成动作。模型提出的动作意图仍需运动学、接触或限制检查。

贯穿例子：VoxPoser构造空间价值信息，再交给优化器；ReKep写关键点关系约束，数值求解器完成运动选择。

原文第 17 页 · 段落 p17-077

中文原意：机器人控制将规划过程中选择的步骤转化为物理运动。模型需要提供多少细节取决于执行层已经可以做什么。该层可以处理逆运动学、碰撞检查和低层控制。下面，我们讨论模型引导机器人运动的三种方式：动作和语言命令、目标位姿和控制程序以及运动约束和成本函数。然后我们讨论动作模型、语言模型和固定控制器如何处理执行反馈。

解析：问可修改的东西究竟是什么：一次程序、持久代码库、运行上下文还是模型权重。代码保存下来与训练模型不同；修改监测代码还可能改变评判标准。

自问：文本/像素/约束如何经过执行器变成真实运动？

原文第 17 页 · 段落 p17-078

中文原意：动作和语言命令。模型可以通过从一组预定义的动作中进行选择来控制机器人。例如，[460]为导航智能体提供基本的移动命令并评估航航航路点工具作为替代或附加接口。固定的执行界面执行选定的动作。Show-运行支撑框架（Harness） [46]提供平移、旋转和夹具控制命令，使用固定规则将它们转换为目标位置、方向和夹具设置。 ETA [50]将跨机器人类型共享的命令转换为每个机器人支持的动作。它还需要在每次工具调用后进行新的观察，从而改变场景并在缺少所需的抓握或附件证据时阻止后续操作。演示还可以向模型展示这些命令如何影响机器人。 RoboDawn [93] 使用任务演示来指导 VLM 选择平移、旋转和夹具命令。即使没有提供任务演示，模型仍然会收到显示基本命令效果的示例。

解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。

解析：本段并列了 Show-Harness、ETA、RoboDawn、Minimal-Interface Navigation Agents。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：文本/像素/约束如何经过执行器变成真实运动？

本段引用：Show-Harness [46]（辞典第47页）；ETA [50]（辞典第51页）；RoboDawn [93]（辞典第94页）；Minimal-Interface Navigation Agents [460]（辞典第461页）

原文第 17 页 · 段落 p17-079

中文原意：执行层还可以接受语言指令。例如，[130] 搜索能够提高冻结 VLA 性能的指令序列，并将它们提炼成语言反馈策略。在部署过程中，只有当经过校准的预测器判断该策略可能有帮助时，该策略才会提供新的指令；否则，VLA保留原来的指令。相反，ART [412] 为 VLA 本身添加了推理和工具使用，同时为连续的机器人动作保留了单独的输出路径。

> 解析：这里预测未来的用途要单独确认：作为训练监督、测试时规划条件，还是模拟环境。前两者不等于可交互物理仿真；部署时是否生成未来也要看本段具体说明。
>
> 解析：本段并列了 LWSV、ART。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：文本/像素/约束如何经过执行器变成真实运动？

本段引用：LWSV [130]（辞典第131页）；ART [412]（辞典第413页）

原文第 17 页 · 段落 p17-080

中文原意：目标位姿和控制程序。当预定义的动作或语言指令没有指定足够的空间细节时，模型可以直接提供目标位置和方向。例如，VIA [107] 让智能体调整虚拟夹具，预览其位姿，并将其作为路径点提交。屏幕截图、悬停时显示的坐标以及位姿读数可帮助智能体选择目标，而无需处理 3D 点的原始数组。 GPT-Policy [53] 使用 VLM 以结构化格式请求机器人运动和夹具操作。上下文编译器为 VLM 组装任务引用和交互历史记录，而演示操作则为其推理提供示例。执行软件在位姿之间进行插值、求解逆运动学、设置运动时序并检查执行情况。这些检查确定请求的运动在运动学上是否有效以及机器人是否已稳定在目标位姿附近。避免碰撞和任务完成需要单独检查。

> 解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。
>
> 解析：本段并列了 GPT-Policy、VIA。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：文本/像素/约束如何经过执行器变成真实运动？

本段引用：GPT-Policy [53]（辞典第54页）；VIA [107]（辞典第108页）

原文第 17 页 · 段落 p17-081

中文原意：其他系统帮助模型选择和检查运动目标。世界动作智能体（WAA）[440]允许智能体在执行之前使用场景几何和运动规划器反馈在视觉工作空间中预览和编辑建议的运动。作为可重用技能存储的文本过程和参考图像可以指导后续操作。 Robo-运行支撑框架（Harness） K1 [193] 提供校准的空间测量和持久的视觉锚点来指导运动请求。其工具支持有界增量运动和朝向目标的直接运动，在执行后返回已实现的运动和残余误差。

> 解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。
>
> 解析：本段并列了 Robo-Harness K1、WAA。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：文本/像素/约束如何经过执行器变成真实运动？

本段引用：Robo-Harness K1 [193]（辞典第194页）；WAA [440]（辞典第441页）

原文第 18 页 · 段落 p18-082

中文原意：另一种方法是从预先准备的模板或关键帧生成运动。例如，ManiSkillFormer [419] 使用人类定义的技能模式来指导智能体在部署之前生成几何要求和运动模板。在执行过程中，感知提供所需的几何形状，模板使用该几何形状来生成机器人路径点。对于全身运动，KINO [39] 使用 VLM 从预定义库中选择与任务相关的关键帧。重定向器将选定的关键帧调整到当前场景，并且关键帧条件策略生成联合级动作。

> 解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。
>
> 解析：本段并列了 KINO、ManiSkillFormer。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：文本/像素/约束如何经过执行器变成真实运动？

本段引用：KINO [39]（辞典第40页）；ManiSkillFormer [419]（辞典第420页）

原文第 18 页 · 段落 p18-083

中文原意：模型还可以编写一个程序来协调多个动作并检查其结果。 MMENTO [314]使用 展开轨迹（Rollout）反馈和记忆引导的程序修订来发展可执行的机器人策略。其演示的策略使用有限状态控制、抓取和放置检查以及重试来处理执行失败。 FAEA [325] 生成 Python 程序，使用 API 来采取操作、读取观察结果、重置环境并检查是否成功。它使用模拟器 执行轨迹（rollout） 期间观察到的状态和错误来修改这些程序。它的评估提供了真实的物体和夹爪状态，因此对相机图像应用相同的方法将需要感知来提供 API 期望的状态信息。在制造中，A-RAM [79] 使用法学硕士来解释制造目标并确定要探索的规划变量。然后，专用工具生成 G 代码，并在执行前检查逆运动学、轨迹定时和关节运动。在执行过程中，数字影子会记录机器人和挤压状态，并将其与选定的计划进行比较。

> 解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。
>
> 解析：本段并列了 A-RAM、MEMENTO、FAEA。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：文本/像素/约束如何经过执行器变成真实运动？

本段引用：A-RAM [79]（辞典第80页）；MEMENTO [314]（辞典第315页）；FAEA [325]（辞典第326页）

原文第 18 页 · 段落 p18-084

中文原意：运动约束和成本函数。模型可以定义运动要求并让求解器找到轨迹，而不是指定目标位姿或命令序列。 ReKep [120] 对子目标和路径上的关键点之间的关系生成约束。它会随着场景的变化跟踪这些关键点，反复优化运动，并在任务阶段失败时支持恢复。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

自问：文本/像素/约束如何经过执行器变成真实运动？

本段引用：ReKep [120]（辞典第121页）

原文第 18 页 · 段落 p18-085

中文原意：VoxPoser [118] 编写的程序可以构建空间值图，包括移动位置、要避开哪些区域、如何定位夹具、移动速度以及何时打开或关闭夹具。这些地图根据模型预测控制期间的新观察结果进行更新。在 ReKep 和 VoxPoser 中，运动解算器可以响应几何形状的变化，而无需在每个控制步骤调用语言模型。

> 解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。

> 解析：这里预测未来的用途要单独确认：作为训练监督、测试时规划条件，还是模拟环境。前两者不等于可交互物理仿真；部署时是否生成未来也要看本段具体说明。

自问：文本/像素/约束如何经过执行器变成真实运动？

本段引用：VoxPoser [118]（辞典第119页）

原文第 18 页 · 段落 p18-086

中文原意：对于涉及大量物理接触的操作，CoRAL [469] 使用 LLM 生成成本函数和接触策略以进行运动优化，而 VLM 提供物理参数的初始估计。交互反馈支持物理模型的更新以及成本函数或接触策略的修订。这区分了两种调整：求解器可以在当前要求下找到不同的运动，或者模型可以在证明不合适时更改这些要求。

> 解析：先把作者的分类判断与实验结论分开。分类说明工作在系统里起什么作用；提升有多大、适用哪些条件，仍要查看该工作给出的实验协议。

自问：文本/像素/约束如何经过执行器变成真实运动？

本段引用：CoRAL [469]（辞典第470页）

原文第 18 页 · 段落 p18-087

中文原意：执行反馈。当机器人移动时，系统需要监视发生的情况并决定如何响应。一些系统在动作模型中处理此反馈，而另一些系统则将其发送到语言模型。例如，ARMS [411] 向 VLA 添加了视觉触发器、机器人状态预测和过去动作的压缩历史记录。这些组件帮助 VLA 决定何时采取行动以及如何做出反应，同时感知和记忆在运动过程中不断更新。然而，记住过去的操作并不能保证存储的对象位置仍然正确。如果物体在没有被观察到的情况下移动，机器人需要新的观察来更新其位置。

> 解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。

> 解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。

自问：文本/像素/约束如何经过执行器变成真实运动？

本段引用：ARMS [411]（辞典第412页）

原文第 18 页 · 段落 p18-088

中文原意：反馈也可以从运动控制器传递到语言模型。 COMPASS [329] 在模拟多智能体集群中使用这种方法，将较慢的语言模型推理与较快的运动控制分开。空间转换器控制器生成动作并将学习到的反馈标记发送到语言模型。然后，语言模型使用这些标记来更新其推理。

> 解析：构造对象、构造场景、构造任务是不同产物。视觉一致性只能支持外观，接触、运动轴、可达性和成功判据还需要各自检查。

自问：文本/像素/约束如何经过执行器变成真实运动？

本段引用：COMPASS [329]（辞典第330页）

原文第 18 页 · 段落 p18-089

中文原意：其他系统在执行之前赋予语言模型更有限的作用。在这样一个系统中，LLM 解释指令并解决歧义，然后确定性有限状态控制器执行选定的操作 [104]。执行层将经过验证的工具调用映射到预定义的运动规划模板。在移动机器人之前，它会检查运动可行性和可能的碰撞。这里，所描述的检查发生在移动之前，而固定控制器管理执行。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

自问：文本/像素/约束如何经过执行器变成真实运动？

本段引用：BSP [104]（辞典第105页）

原文第 18、19 页 · 段落 p18-090

中文原意：总的来说，模型的输出应该与执行层的功能相匹配。例如，如果所需的技能已经可用，则模型可能只需要命名动作或提供语言指令。当这些命令不够时，它可以指定目标位姿，编写控制程序，或定义运动约束和成本函数。在每种情况下，系统都可以在执行之前检查机器人是否可以执行请求的运动。然后，它可以在执行期间和执行之后使用反馈来检查操作是否有效并决定下一步要做什么。相关工作比较如表5所示。

> 解析：问可修改的东西究竟是什么：一次程序、持久代码库、运行上下文还是模型权重。代码保存下来与训练模型不同；修改监测代码还可能改变评判标准。

自问：文本/像素/约束如何经过执行器变成真实运动？

原表5 · 原文第19页

表 5. 机器人控制工作的比较（第 4.4 节）。控制输出列出了传递到执行模块的命令或表示。执行器识别将这些输出转化为机器人动作的模块。 Feedback列出了执行过程中返回的信息。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：KINO [39]；Show-Harness [46]；ETA [50]；GPT-Policy [53]；A-RAM [79]；ABot-N1 [86]；RoboDawn [93]；SCOPE [103]；BSP [104]；VIA [107]；VoxPoser [118]；ReKep [120]；LWSV [130]；Robo-Harness K1 [193]；MEMENTO [314]；FAEA [325]；COMPASS [329]；ARMS [411]；ART [412]；ManiSkillFormer [419]；WAA [440]；CoRAL [469]。

4.5 Model / Adaptation：失败后改哪一层

> 本节先抓住：上下文偏好、记忆、计划、残差策略与整个模型权重代表不同适应方式。一个回合内纠错不等于跨任务学习；更新后应测试旧能力是否遗忘。
>
> 贯穿例子：机器人把“我的杯子”理解错了，修正偏好记忆可能比重新训练整套动作模型更合适。

原文第 19 页 · 段落 p19-092

中文原意：适应使机器人系统能够通过修改推理时间条件、规划器配置或学习的模型参数来根据经验进行改进。这些机制的范围和持久性各不相同，但可以结合起来以支持立即恢复和长期改善。

> 解析：问可修改的东西究竟是什么：一次程序、持久代码库、运行上下文还是模型权重。代码保存下来与训练模型不同；修改监测代码还可能改变评判标准。
>
> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

自问：适应改变了哪个持久对象，还是仅在当前回合换一种做法？

原文第 19 页 · 段落 p19-093

中文原意：以经验为条件的决策。先前的经验可以指导决策，而无需更新模型参数。 TidyBot [376] 从示例中推断用户偏好，而 DROC [430] 从语言更正中提取可重用的知识。 CMMR-VLN [173] 将检索到的导航经验转换为优先规则，而 PhysMem [172] 维护和修改有关物理交互的假设。这些方法通过根据存储的经验做出后续决策，将适应与记忆联系起来（第 7 节）。类似地，GPT-Policy [53] 根据演示、动作参考和交互历史记录冻结的 VLM，以支持任务内的适应。然而，依赖于保留上下文的改进并不一定表明持久的能力获取。

> 解析：问可修改的东西究竟是什么：一次程序、持久代码库、运行上下文还是模型权重。代码保存下来与训练模型不同；修改监测代码还可能改变评判标准。
>
> 解析：本段并列了 GPT-Policy、PhysMem、CMMR-VLN、TidyBot等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：适应改变了哪个持久对象，还是仅在当前回合换一种做法？

本段引用：GPT-Policy [53]（辞典第54页）；PhysMem [172]（辞典第173页）；CMMR-VLN [173]（辞典第174页）；TidyBot [376]（辞典第377页）；DROC [430]（辞典第431页）

原文第 19 页 · 段落 p19-094

中文原意：规划者适应。经验也可以在不修改底层策略的情况下改进规划。 SERP [171] 将基于梯度的规划器参数优化与优化停滞时的 VLM 引导调整相结合。其 MARS 扩展进一步检索先前的恢复经验以通知后续配置。此类方法主要解决规划和搜索中的失败问题，而不是解决策略学习能力的限制。

> 解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。
>
> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

自问：适应改变了哪个持久对象，还是仅在当前回合换一种做法？

本段引用：SERP [171]（辞典第172页）

原文第 19、20 页 · 段落 p19-095

中文原意：策略和模型更新。参数更新可以实现更持久的行为变化。 OLAF [209]将口头纠正转换为修订后的动作标签，用于视觉运动策略训练。Yell At Your Robot[296]改为更新高层策略，同时保持低层控制器固定。不确定性感知策略指导[421]使用纠正性演示将指令澄清与剩余策略学习分开。这些方法针对不同的错误来源，包括任务解释、操作选择和执行。

> 解析：问可修改的东西究竟是什么：一次程序、持久代码库、运行上下文还是模型权重。代码保存下来与训练模型不同；修改监测代码还可能改变评判标准。
>
> 解析：本段并列了 OLAF、Yell At Your Robot、UPS。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：适应改变了哪个持久对象，还是仅在当前回合换一种做法？

本段引用：OLAF [209]（辞典第210页）；Yell At Your Robot [296]（辞典第297页）；UPS [421]（辞典第422页）

原文第 20 页 · 段落 p20-097

中文原意：自主交互提供了额外的学习信号。 自我改进（Self-Improving） 体现的 基础模型（Foundation Model） [289] 使用冻结的监督检查点来进行进度奖励和成功检测，同时使用 强化学习（Reinforcement Learning） 优化单独的策略。其物理实验使用 LanguageTable，而 ALOHA 自我改进（Self-Improvement） 在模拟中进行评估。语言模型预测控制[197]通过模拟教学交互微调代码生成LLM。反思式测试时规划[105]更新了高层行动选择和回顾性评估的反思，同时保持外部评估者固定。这些方法说明了学习如何在不同的监督机制下针对不同的组成部分。

> 解析：问可修改的东西究竟是什么：一次程序、持久代码库、运行上下文还是模型权重。代码保存下来与训练模型不同；修改监测代码还可能改变评判标准。
>
> 解析：本段并列了 Reflective Test-Time Planning、LMPC、SIEFM。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：适应改变了哪个持久对象，还是仅在当前回合换一种做法？

本段引用：Reflective Test-Time Planning [105]（辞典第106页）；LMPC [197]（辞典第198页）；SIEFM [289]（辞典第290页）

原文第 20 页 · 段落 p20-098

中文原意：跨任务和部署学习。执行过程中收集的经验可以支持后续训练。 PAFF [80] 用语言指令重新标记探索轨迹，而 RoboCat [18] 将任务适应策略生成的数据合并到连续的通才训练迭代中。 VERITAS [437] 选择经过验证的成功执行，而 HERO [174] 在策略训练之前结合了启发式执行和样本重用。 VLAW [95]使用真实的交互和过滤器生成的策略更新轨迹迭代地改进了 世界模型（World Model）。在部署规模上，RECAP [268] 和边学习边部署 [359] 利用自主经验和人工修正来改进策略。这些方法将适应与数据收集和选择（第 5 节）联系起来，其性能取决于监督质量、经验多样性和交互成本。

> 解析：这里预测未来的用途要单独确认：作为训练监督、测试时规划条件，还是模拟环境。前两者不等于可交互物理仿真；部署时是否生成未来也要看本段具体说明。
>
> 解析：本段并列了 RoboCat、PAFF、VLAW、HERO等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：适应改变了哪个持久对象，还是仅在当前回合换一种做法？

本段引用：RoboCat [18]（辞典第19页）；PAFF [80]（辞典第81页）；VLAW [95]（辞典第96页）；HERO [174]（辞典第175页）；RECAP [268]（辞典第269页）；LWD [359]（辞典第360页）；VERITAS [437]（辞典第438页）

原文第 20 页 · 段落 p20-099

中文原意：适应机制的选择取决于失败的根源。感知错误可能需要额外的观察，规划错误可能需要重新规划，持续的执行失败可能会促使策略更新。评估应区分片段内恢复、跨任务泛化和再训练后的改进，同时评估是否保留了先前获得的能力。相关工作比较见表6。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

自问：适应改变了哪个持久对象，还是仅在当前回合换一种做法？

原表6 · 原文第20页

表 6. 适应工作的比较（第 4.5 节）。演示包括任务示例和动作轨迹；纠正措施包括纠正指示和干预措施。执行结果包括成功、失败和执行反馈。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：RoboCat [18]；PAFF [80]；VLAW [95]；Reflective Test-Time Planning [105]；SERP [171]；PhysMem [172]；CMMR-VLN [173]；LMPC [197]；OLAF [209]；RECAP [268]；SIEFM [289]；Yell At Your Robot [296]；TidyBot [376]；UPS [421]；DROC [430]。

原表6 · 原文第21页

表 6（续）

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：GPT-Policy [53]；RoboDawn [93]；HERO [174]；EmbodiedSkills [351]；LWD [359]；VERITAS [437]。

5 数据：智能体应如何收集、标注和提炼机器人数据？

原文第 21 页 · 段落 p21-100

中文原意：智能体式机器人（Agentic Robots）的进展很大程度上是由机器人学习的数据处理管道驱动的。现代以智能体为中心的方法主动收集交互数据，解释机器人演示行为，并从现实世界的执行反馈中提炼监督信号。本节将面向智能体的数据操作分解为三个关键阶段：collection、annotation和refinement。数据收集从机器人任务 执行轨迹（rollout）中获取智能体探索的交互轨迹和机器人自我演示数据集。数据标注通过结构化监督来增强原始机器人数据，以实现任务理解、空间 落地关联（Grounding）等。数据细化通过去除噪声样本和纠正不准确的监督来提高数据质量。这三个阶段共同构成了智能体机器人学习的完整数据优化工作流程。以下部分详细阐述了每个阶段并回顾了代表性的技术方法。

解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。

自问：得到的是新经验、新标签还是修订后的旧数据？

图5：数据侧的三种介入

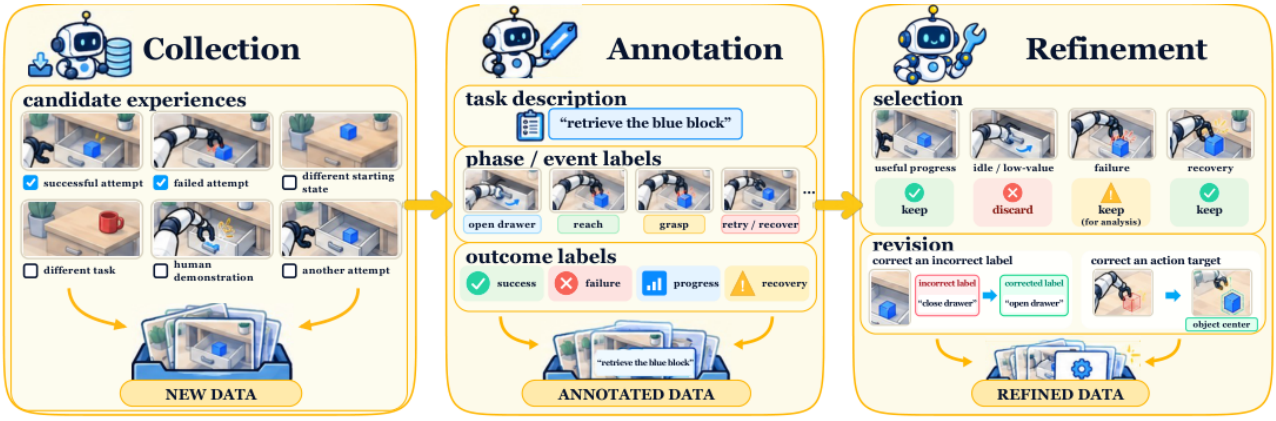


Figure 5. Three data-side interventions over shared robot experience, illustrated by a drawer interaction with failure and recovery. Collection acquires or generates experience; Annotation adds task meaning, temporal structure, and supervision; and Refinement selects useful records or segments and revises unreliable supervision.

中文读图：采集取得经验；标注赋予语义、阶段与结果；改进选择或修正经验。失败片段可以保留用于恢复研究。

5.1 Data / Collection：经验从哪里来

本节先抓住：区分真实交互、仿真执行和生成式合成；合成画面不自动拥有可信动作标签。任务选择、重置、成功判定与人工干预共同决定采集效率。

贯穿例子：一次成功抓取后，必须把杯子放回可采集状态；没有可靠重置，第二次数据可能不再是同一个任务。

原文第 21 页 · 段落 p21-101

中文原意：基于智能体的数据收集旨在通过自主智能体生成新的机器人轨迹或演示。我们将这些方法分为三类。物理数据收集从真实的机器人交互和人类演示中收集数据。 模拟数据生成通过在模拟器中运行任务、策略或程序来生成轨迹。 生成数据合成使用生成模型来创建人工观察和运动。

> 解析：构造对象、构造场景、构造任务是不同产物。视觉一致性只能支持外观，接触、运动轴、可达性和成功判据还需要各自检查。
>
> 解析：问可修改的东西究竟是什么：一次程序、持久代码库、运行上下文还是模型权重。代码保存下来与训练模型不同；修改监测代码还可能改变评判标准。

自问：得到的是新经验、新标签还是修订后的旧数据？

原文第 21、22 页 · 段落 p21-102

中文原意：物理数据收集。 一个早期的例子，AutoRT [3]，使用语言模型根据现有的视觉场景描述和可用收集策略的功能来提出收集任务。然后通过脚本化挑选、RT-2 [19] 或人工远程操作来尝试选定的任务收集真实世界的轨迹。 SOAR [467] 使用 CogVLM [352] 来识别可以在当前场景中执行的任务，然后优先考虑较少尝试的任务以收集更广泛任务中的数据。基于智能体的持续数据收集涉及任务执行、场景重置和人工干预的关键设计选择。 Zero2Skill [334] 利用 Seed1.8 [22] 处理数据收集和场景重置阶段后捕获的图像。根据试验结果评估结果，数据收集循环要么继续进行，要么重试正在进行的试验，一旦重试达到预定义的限制，就会调用人工协助。 RoboClaw [178] 将视觉观察与 VLM 控制器中的任务进度相结合，在确认任务完成后调用配对重置策略。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。
>
> 解析：本段并列了 AutoRT、RT-2、Seed1.8、RoboClaw等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：得到的是新经验、新标签还是修订后的旧数据？

本段引用：AutoRT [3]（辞典第4页）；RT-2 [19]（辞典第20页）；Seed1.8 [22]（辞典第23页）；RoboClaw [178]（辞典第179页）；Zero2Skill [334]（辞典第335页）；CogVLM [352]（辞典第353页）；SOAR [467]（辞典第468页）

原文第 22 页 · 段落 p22-104

中文原意：智能体还可以针对机器人执行所揭示的代表性不足的任务和弱点进行额外的数据收集。 Tether [199] 优先考虑缺乏足够成功演示的任务，并生成适合场景的准备行动。在其早期的收集实验中，管道使用 Gemini 2.5 Flash [56]来规划任务序列，以进行场景准备，从而实现目标收集任务，而GPT-4.1则判断任务是否成功。成功评估比较了执行前后的相机视图，并进行了额外检查，将源演示的最终关键点与夹具的最终图像位置相匹配。该论文随后采用 Gemini Robotics-ER 1.5 进行任务选择和成功评估，并报告事后评估，而无需额外的对应检查。 RoboLineage [231] 审查录制的视频片段和关键帧，以及操作员标注和机器人日志，以识别任务执行失败的位置。它将这些发现与数据覆盖范围和策略评估摘要相结合，以建议针对薄弱任务阶段进行额外的收集。 FIND [68]还使用VLM根据任务指令和前后场景图像来判断每次尝试是否成功。当 UPS [421] 验证者确定当前策略无法完成任务时，会请求人工更正。

> 解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。
>
> 解析：本段并列了 Gemini 2.5、FIND、Tether、RoboLineage等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：得到的是新经验、新标签还是修订后的旧数据？

本段引用：Gemini 2.5 [56]（辞典第57页）；FIND [68]（辞典第69页）；Tether [199]（辞典第200页）；RoboLineage [231]（辞典第232页）；UPS [421]（辞典第422页）

原文第 22 页 · 段落 p22-105

中文原意：模拟数据生成. RoboGen [362] 是一个早期的例子，它引入了一个机器人智能体，该智能体提出任务、构建模拟环境并生成训练监督。具体来说，智能体将任务分解为子任务，在Genesis[82]中选择和排列所需的对象，并使用运动规划等方法在模拟中生成机器人行为。为了生成基于日常人类活动的任务，Video2Policy [409] 使用互联网视频作为任务规范的来源。它重建被操纵的对象并估计它们的位姿，然后使用视觉语言模型为 Isaac Gym [236] 生成可执行任务代码。 GenSim2 [113] 使用智能体创建操作任务并指导运动规划，然后在 SAPIEN [381] 中执行运动以收集机器人轨迹。 RoboFactory [273] 专注于多个机器人之间的协作，使用语言模型规划器来分配子任务并协调 ManiSkill [318] 中的执行。

> 解析：构造对象、构造场景、构造任务是不同产物。视觉一致性只能支持外观，接触、运动轴、可达性和成功判据还需要各自检查。
>
> 解析：本段并列了 Genesis、GenSim2、Isaac Gym、RoboFactory等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：得到的是新经验、新标签还是修订后的旧数据？

本段引用：Genesis [82]（辞典第83页）；GenSim2 [113]（辞典第114页）；Isaac Gym [236]（辞典第237页）；RoboFactory [273]（辞典第274页）；ManiSkill3 [318]（辞典第319页）；RoboGen [362]（辞典第363页）；SAPIEN [381]（辞典第382页）；Video2Policy [409]（辞典第410页）

原文第 22 页 · 段落 p22-106

中文原意：最近的工作拓宽了收集的模拟数据的类型，并赋予智能体对收集过程更大的责任。 RoboTalk [84] 与关节机器人动作一起生成通信消息，验证其序列，并在 RoboCasa365 [246] 中重播它们以收集相应的视觉观察结果。 ARSTAG [169] 和 EmbodiedSWE-Gen [294] 自动收集程序的准备和扩展。策略失败还可以指导智能体接下来收集的内容。 EmbodiRSI [158] 监视 Isaac Sim 中的策略执行，并使用观察到的故障来生成恢复轨迹并收集困难场景配置中的附加数据。然后，该策略会被再次更新和评估，从而允许智能体根据机器人的剩余弱点调整后续收集。

> 解析：构造对象、构造场景、构造任务是不同产物。视觉一致性只能支持外观，接触、运动轴、可达性和成功判据还需要各自检查。
>
> 解析：本段并列了 RoboTalk、EmbodiRSI、ARSTAG、RoboCasa365等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：得到的是新经验、新标签还是修订后的旧数据？

本段引用：RoboTalk [84]（辞典第85页）；EmbodiRSI [158]（辞典第159页）；ARSTAG [169]（辞典第170页）；RoboCasa365 [246]（辞典第247页）；EmbodiedSWE [294]（辞典第295页）

原文第 22、23 页 · 段落 p22-107

中文原意：生成数据合成。与之前通过预定义的生成和轨迹处理管道合成机器人数据的方法（如 RoboTransfer [214]、Real2Edit2Real [451]、RoboVIP [333] 和 RoboDream [407]）不同，基于智能体的方法使用大语言模型或多模态大语言模型来决定生成哪些数据并协调生成过程。 RoboCurate [152] 利用 Gemini 3 Pro 从视觉场景观察中导出操作任务，并生成视频生成提示。此外，它还检查模拟器重播的预测动作是否与相应生成视频中的运动一致。 V-Dreamer [101] 使用 Qwen-Max [4] 将场景描述转换为对象描述列表，从而指导 3D 资产生成和场景构建。对于长指令，基础模型（Foundation Model） 将任务划分为按顺序处理的子任务。视频模型根据模拟场景的图像和当前子任务指令生成操作视频。该管道跨视频帧跟踪目标对象上的点并重建其三维运动。它将这个运动与生成的抓取位姿结合起来计算机器人轨迹，然后执行 Genesis [82] 中的轨迹来收集策略训练的演示。 Wh0 [45]使用两个智能体来生成操作指令：一个扩展对象和属性词汇，另一个根据使用计数选择较少使用的单词。 Qwen3-VL 描述了指导从编辑的工作空间图像生成人手视频所需的手部和物体运动。从这些视频中恢复的手腕位姿和指关节运动提供了动作标签，这些标签与真实的机器人演示相结合来训练灵巧的策略。这些收集路线涉及经验记录。 RAT [435] 和 Teach and Grow [249] 主要保留项目、课程或行为库条目。因此，他们的实践到技能工作流程将在第 7.6 节中讨论，该部分检查程序工件的构建、验证和重用。相关工作比较见表7。

> 解析：构造对象、构造场景、构造任务是不同产物。视觉一致性只能支持外观，接触、运动轴、可达性和成功判据还需要各自检查。
>
> 解析：本段并列了 Qwen-Max、Wh0、Genesis、V-Dreamer等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：得到的是新经验、新标签还是修订后的旧数据？

本段引用：Qwen-Max [4]（辞典第5页）；Wh0 [45]（辞典第46页）；Genesis [82]（辞典第83页）；V-Dreamer [101]（辞典第102页）；RoboCurate [152]（辞典第153页）；RoboTransfer [214]（辞典第215页）；Teach and Grow [249]（辞典第250页）；RoboVIP [333]（辞典第334页）；RoboDream [407]（辞典第408页）；RATs [435]（辞典第436页）；Real2Edit2Real [451]（辞典第452页）

原表7 · 原文第23页

表 7. 数据收集工作比较（第 5.1 节）。类型：物理交互；模拟器执行；生成模型综合。计数保留每篇论文的单位和范围。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：Wh0 [45]；FIND [68]；RoboTalk [84]；V-Dreamer [101]；HumanoidGen [140]；RoboCurate [152]；EmbodiRSI [158]；ARSTAG [169]；Tether [199]；RoboLineage [231]；RoboFactory [273]；Zero2Skill [334]；RoboGen [362]；Video2Policy [409]；UPS [421]；SOAR [467]。

5.2 Data / Annotation：把经验变成监督

> 本节先抓住：语义标签、时序阶段、空间几何和结果标签监督的是不同对象。模型生成的解释可能只是事后叙述，需要检查它是否和实际轨迹、物体坐标及成功标准一致。
>
> 贯穿例子：“抓到杯子”的文字应与夹爪和杯子运动证据吻合；接近杯子不等于完成抓取。

原文第 23 页 · 段落 p23-109

中文原意：虽然数据收集产生与机器人状态序列配对的视频，但此类记录通常缺乏文本描述、空间标记、交互目标和结果标注。为了弥补这一差距，智能体驱动的标注采用智能智能体来解释原始记录，编排标注实用程序，并生成和验证丢失的标签。我们根据各自的标注目标，将现有的基于智能体的标注方法分为四组：文本标注、空间标注、交互标注和结果标注。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

解析：问可修改的东西究竟是什么：一次程序、持久代码库、运行上下文还是模型权重。代码保存下来与训练模型不同；修改监测代码还可能改变评判标准。

自问：得到的是新经验、新标签还是修订后的旧数据？

原文第 23 页 · 段落 p23-110

中文原意：文本标注。最常见的基于智能体的文本标注是生成分别标记任务阶段和可观察事件的子任务描述和视频字幕。例如，NILS [16] 为机器人视频片段生成任务级语言指令。具体来说，它从对象运动、状态变化和夹具交互中检测分段边界，然后使用 Gemini Pro 描述每个分段中执行的操作阶段。 IVLR [211] 生成子任务级视频字幕。它将机器人轨迹分为多个阶段，然后查询 Qwen3-VL 来描述每个阶段中详细的夹具-物体交互或空间关系。 EgoTools [322] 采用 Gemini 3 Flash 为连续视频片段添加字幕，将先前的字幕传播到后续片段以提高时间一致性。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

解析：本段并列了 NILS、IVLR、EgoTools。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：得到的是新经验、新标签还是修订后的旧数据？

本段引用：NILS [16]（辞典第17页）；IVLR [211]（辞典第212页）；EgoTools [322]（辞典第323页）

原文第 24 页 · 段落 p24-111

中文原意：另一种形式的文本标注采用问答（QA）对来捕获场景属性、动作序列和细粒度的执行细节。例如，RoboAnnotatorX [154] 使用 GPT-4o 生成有关当前对象、它们的空间关系以及机器人动作顺序的 QA，使用场景描述和带有时间戳的动作标注作为源信息。同时，AnnotateAnything [227] 从模拟轨迹日志创建面向机器人的 QA 对。 VLM 针对目标对象、交互部分、方法方向和执行结果改变问题措辞，答案来自模拟器记录。 REAL [241] 和 RoboTalk [84] 使用 Gemini 模型在机器人动作的同时生成对话标注，涵盖模糊指令的澄清以及协作机器人之间的通信。 SpatialVLM [33]、SpaceVista [311] 和 Exemplar2VQA [415] 等方法将视觉模型或编码智能体与 3D 几何相结合，生成有关对象位置、距离和空间关系的问答对。 RoboSpatial [301] 从 3D 几何、投影和光线投射中得出空间答案，并使用确定性模板构建问答对。

解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。

解析：本段并列了 SpatialVLM、RoboTalk、RoboAnnotatorX、AnnotateAnything等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：得到的是新经验、新标签还是修订后的旧数据？

本段引用：SpatialVLM [33]（辞典第34页）；RoboTalk [84]（辞典第85页）；RoboAnnotatorX [154]（辞典第155页）；AnnotateAnything [227]（辞典第228页）；REAL [241]（辞典第242页）；RoboSpatial [301]（辞典第302页）；SpaceVista [311]（辞典第312页）；Exemplar2VQA [415]（辞典第416页）

原文第 24 页 · 段落 p24-112

中文原意：除了问答对之外，推理跟踪标注还通过高级计划和解释性原理来增强记录的行为。具体来说，ECoT [425] 和 TIC-VLA [124] 分别使用 Gemini 1.0 和 GPT-5 来生成推理轨迹，解释机器人当前的子任务以及操纵和导航任务中的下一步动作。沿着这条线，ERVLA [310] 利用 Qwen3.5-397B 来导出每个回合的共享目标和子任务序列，以及针对选定时间步的细粒度推理。

解析：先把作者的分类判断与实验结论分开。分类说明工作在系统里起什么作用；提升有多大、适用哪些条件，仍要查看该工作给出的实验协议。

解析：本段并列了 TIC-VLA、ERVLA、ECoT。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：得到的是新经验、新标签还是修订后的旧数据？

本段引用：TIC-VLA [124]（辞典第125页）；ERVLA [310]（辞典第311页）；ECoT [425]（辞典第426页）

原文第 24 页 · 段落 p24-113

中文原意：空间标注。 机器人智能体通过解释演示、调用感知工具和验证空间标签来执行空间标注。 NILS [16] 和 FSD [424] 使用语言模型引导的管道来识别要标注的对象。 NILS 使用 落地关联（Grounding） DINO [216] 来获取对象边界框，并使用 DEVA [54] 来跟踪视频帧中的对象蒙版。 FSD 使用 Grounded SAM [281] 来获取边界框和分割掩模。 AnnotateAnything [227] 和 EmbodiedGen V2 [357] 将 3D 分割与基于 VLM 的解释相结合来标注对象部分，并使用 EmbodiedGen V2 额外检查和细化生成的标签。 SPARC [17] 和 Keypose Exploration [229] 通过语言模型解释交互事件，并将它们与视觉轨迹或机器人状态对齐以标注轨迹和关键位姿。 AutoFocus-IL [85] 和工作空间模型 [57] 通过 VLM 推理识别与任务相关的对象和事件，并将它们本地化以生成空间显着性标注。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

解析：本段并列了 NILS、SPARC、DEVA、Workspace Models等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：得到的是新经验、新标签还是修订后的旧数据？

本段引用：NILS [16]（辞典第17页）；SPARC [17]（辞典第18页）；DEVA [54]（辞典第55页）；Workspace Models [57]（辞典第58页）；AutoFocus-IL [85]（辞典第86页）；Grounding DINO [216]（辞典第217页）；AnnotateAnything [227]（辞典第228页）；Keypose Exploration [229]（辞典第230页）；Grounded SAM [281]（辞典第282页）；EmbodiedGen V2 [357]（辞典第358页）；FSD [424]（辞典第425页）

原文第 24 页 · 段落 p24-114

中文原意：交互标注。交互标注捕获手、机器人抓手、工具和目标对象之间的物理交互，涵盖触摸、抓握、推动和按压等典型行为。标注管道通常包括对交互意图的推理和通过外部工具检索与接触相关的证据。对于可供性（Affordance）标注，一些代表性的作品，如AFUN [364]、EgoAfford [91]、可供性（Affordance）2Action [213]、A4-Agent [444]和可供性（Affordance） Agent 运行支撑框架（Harness） [114]将模型推理与工具辅助定位相结合。在这些工作中，智能体或 LLM 驱动的工作流程首先识别适合所需操作的对象区域，然后调用检测和分割模块来查明这些功能区域。对于灵巧的手，DextER [162] 和 OmniDexVLG [436] 将模型推理与工具衍生的接触证据相结合，以标注手指和链接级与物体部分的接触并解释它们的抓取功能。

解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。

解析：本段并列了 EgoAfford、Affordance Agent Harness、DextER、Affordance2Action等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：得到的是新经验、新标签还是修订后的旧数据？

本段引用：EgoAfford [91]（辞典第92页）；Affordance Agent Harness [114]（辞典第115页）；DextER [162]（辞典第163页）；Affordance2Action [213]（辞典第214页）；AFUN [364]（辞典第365页）；OmniDexVLG [436]（辞典第437页）；A4-Agent [444]（辞典第445页）

原文第 24 页 · 段落 p24-115

中文原意：结果标注。结果标注标记机器人执行中的任务完成情况，包括中间进度、最终成功或失败以及安全违规。智能体或模型引导的工作流程解释评估标准并检查执行证据以生成这些标签。对于任务完成情况，RoboReward [163] 和 Zero2Skill [334] 根据明确的任务标准使用视觉语言判断来分配完成分数或成功标签。对于中间进度，ROVER [288] 和 ProcVLM [70] 围绕子任务组织视觉推理，以标注其完成状态并估计任务进度。ARS [109] 进一步使用智能体来调用视频检查工具，验证建议的任务事件，并修改帧级进度标注。为了安全起见，ShieldVLA [319] 使用智能体工作流程从错误排名的示例中细化安全标准，并生成校准的框架级风险标签。相关工作比较见表8。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

解析：本段并列了 ProcVLM、ARS、RoboReward、ROVER等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：得到的是新经验、新标签还是修订后的旧数据？

本段引用：ProcVLM [70]（辞典第71页）；ARS [109]（辞典第110页）；RoboReward [163]（辞典第164页）；ROVER [288]（辞典第289页）；ShieldVLA [319]（辞典第320页）；Zero2Skill [334]（辞典第335页）

原表8 · 原文第25页

表 8. 数据标注工作比较（第 5.2 节）。输入：图像；视频;机器人数据；点云。机器人数据包括演示元数据和日志。模型列列出了主要的语义或推理支柱；工具栏列出了支持的感知模型、算法和模拟接口。显示了代表性标注配置。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：NILS [16]；SPARC [17]；SpatialVLM [33]；ARS [109]；Affordance Agent Harness [114]；DextER [162]；IVLR [211]；Affordance2Action [213]；AnnotateAnything [227]；REAL [241]；ROVER [288]；RoboSpatial [301]；EgoTools [322]；Zero2Skill [334]；EmbodiedGen V2 [357]；Exemplar2VQA [415]；FSD [424]；ECoT [425]；A4-Agent [444]。

5.3 Data / Refinement：保留什么、修正什么

本节先抓住：选择数据与修正标签是两个操作。失败轨迹中的有效阶段、恢复片段也可能有价值；过滤器不应只保留容易成功的样本。

贯穿例子：删除所有失败会丢掉恢复经验；应按任务阶段和用途决定保留，而非仅按整条轨迹成功与否。

原文第 25 页 · 段落 p25-116

中文原意：数据细化通过数据选择和监督修订提高了记录的机器人演示的适用性。基于智能体的系统将模型推理与执行证据相结合，以确定要保留哪些轨迹或片段以及早期标注是否需要纠正。

解析：先把作者的分类判断与实验结论分开。分类说明工作在系统里起什么作用；提升有多大、适用哪些条件，仍要查看该工作给出的实验协议。

自问：得到的是新经验、新标签还是修订后的旧数据？

原文第 25、26 页 · 段落 p25-117

中文原意：RoboLineage [231] 和 Zero2Skill [334] 使用模型辅助审查来确定记录的轨迹是否符合训练标准。选择也可以在轨迹内进行：HELP [431] 和 ARS [109] 使用估计的任务进度来保留有用的部分，同时排除无效的操作，允许部分不成功的尝试支持学习。 HELP 依靠其 VLAC-CUT 批评者区分进度和恢复与空闲或故障诱导行为，而 ARS 使用智能体调用视频检查工具并在进度估计指导段选择之前验证任务事件。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

解析：本段并列了 ARS、RoboLineage、Zero2Skill、HELP。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：得到的是新经验、新标签还是修订后的旧数据？

本段引用：ARS [109]（辞典第110页）；RoboLineage [231]（辞典第232页）；Zero2Skill [334]（辞典第335页）；HELP [431]（辞典第432页）

图6：环境侧的三种介入

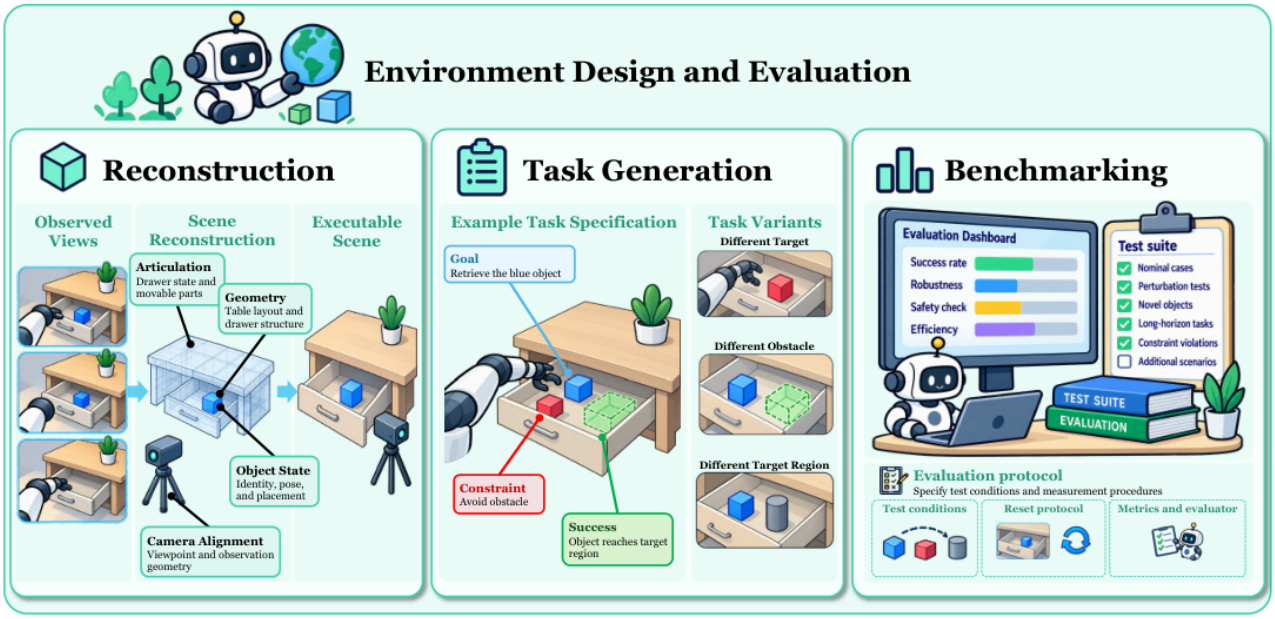


Figure 6. Three ways to build and test an environment for one robot task. Reconstruction recovers an executable scene from observations. Task Generation defines goals, constraints, and success conditions. Benchmarking fixes test conditions, reset protocols, metrics, and evaluators for success, safety, efficiency, and robustness.

中文读图：重建解决世界能否被交互；任务生成解决要完成什么；评测解决如何证明完成。外观、任务可行与判据正确要分开测试。

原文第 26 页 · 段落 p26-119

中文原意：当反馈改变如何判断执行时，Zero2Skill [334]和反思式测试时规划[105]会重新审视早期的监督。Zero2Skill 将人工修正的成功标准应用于之前收集的演示并更新其标签。反思式测试时规划使用后来的执行结果来修改早期的行动评估，保留更新的反思和分数作为后续学习的监督。相关工作比较见表9。

解析：先把作者的分类判断与实验结论分开。分类说明工作在系统里起什么作用；提升有多大、适用哪些条件，仍要查看该工作给出的实验协议。

解析：本段并列了 Reflective Test-Time Planning、Zero2Skill。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：得到的是新经验、新标签还是修订后的旧数据？

本段引用：Reflective Test-Time Planning [105]（辞典第106页）；Zero2Skill [334]（辞典第335页）

原表9 · 原文第26页

表 9. 数据细化工作的比较（第 5.3 节）。模型列出了用于审查数据或修改监管的主干。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：Reflective Test-Time Planning [105]；ARS [109]；RoboLineage [231]；Zero2Skill [334]。

6 环境：智能体应该构建哪些世界、任务和测试？

原文第 26 页 · 段落 p26-120

中文原意：环境分支涵盖了塑造机器人交互的世界、任务和评估器。重建、任务生成和基准测试解决了这三个目标（图 6）。它们可能会独立失败：场景可能不稳定，任务可能不可行，或者评估者可能很少谈论被测试的能力。在抽屉任务中，场景必须允许关节运动，任务必须识别要检索的对象，评估器必须检查最终关系。构建循环应将每个失败的检查链接到需要修订的对象。

> 解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。
>
> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

自问：生成的是世界、任务还是判据？哪种验证支持它？

6.1 Environment / Reconstruction：构造可交互世界

> 本节先抓住：外观、几何、关节、质量、摩擦与碰撞是不同属性。图像漂亮或物体静态稳定，只证明部分条件，不能证明抓取、开门或接触真实。
>
> 贯穿例子：把柜门画出来不够，还要建立铰链、运动轴、碰撞体与可达的把手。

原文第 26 页 · 段落 p26-121

中文原意：重建将观察、演示或请求的布局转变为可执行的场景。它必须恢复任务使用的结构、交互引起的变化以及任务所通过的接口。

> 解析：构造对象、构造场景、构造任务是不同产物。视觉一致性只能支持外观，接触、运动轴、可达性和成功判据还需要各自检查。
>
> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

自问：生成的是世界、任务还是判据？哪种验证支持它？

原文第 27 页 · 段落 p27-122

中文原意：智能体机器人可以检查和修改结果。检查应该揭露对执行很重要的错误，而不是仅仅奖励视觉相似性。

> 解析：把“智能体修改的目标”与“判断好坏的标准”分开：奖励代码可变，任务成功判据和测试任务应保持可比。奖励变高不能直接推出物理任务做得更好。

自问：生成的是世界、任务还是判据？哪种验证支持它？

原文第 27 页 · 段落 p27-123

中文原意：静态结构。静态结构包括对象、位姿、支撑以及使场景稳定的属性。 ARSTAG [169] 重建与指令相关的对象和支持关系，将身份链接到分割掩模，并在收集演示之前修复位姿。 SimFoundry [275] 将视频分解为网格和铰接部分，稳定场景，并创建数字表兄弟变体； Genie Sim 3.0 [414] 从资产库中组装场景。 RoboLab [398] 将稳定性模拟中的位移误差返回到生成器，而 RoboPlayground [360] 确定目标配置并在其成功谓词失败时修复任务代码。 GIF [393]在生成演示之前在解决对象之后检查所请求的关系。这些检查将重建与支持、联系和任务可行性联系起来。

解析：构造对象、构造场景、构造任务是不同产物。视觉一致性只能支持外观，接触、运动轴、可达性和成功判据还需要各自检查。

解析：本段并列了 ARSTAG、SimFoundry、RoboPlayground、GIF等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：生成的是世界、任务还是判据？哪种验证支持它？

本段引用：ARSTAG [169]（辞典第170页）；SimFoundry [275]（辞典第276页）；RoboPlayground [360]（辞典第361页）；GIF [393]（辞典第394页）；RoboLab [398]（辞典第399页）；Genie Sim 3.0 [414]（辞典第415页）

原文第 27 页 · 段落 p27-124

中文原意：动态和铰接结构。可重播的场景还必须正确响应新动作。 Agentic Real2Sim [36] 识别与任务相关的对象，重建其网格，并跟踪 6-DoF 位姿，并具有可选的 VLM 细化。 Video2World [316] 评估从具体视频中恢复场景和控制程序的编码智能体，而 4DCodeBench [292] 通过分析运动、关键帧或模拟来测试基于代码的动态重建。匹配观察到的运动并不能建立对新动作的正确响应，因此这些是单独的检查。

解析：构造对象、构造场景、构造任务是不同产物。视觉一致性只能支持外观，接触、运动轴、可达性和成功判据还需要各自检查。

解析：本段并列了 Agentic Real2Sim、4DCodeBench、Video2World。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：生成的是世界、任务还是判据？哪种验证支持它？

本段引用：Agentic Real2Sim [36]（辞典第37页）；4DCodeBench [292]（辞典第293页）；Video2World [316]（辞典第317页）

原文第 27 页 · 段落 p27-125

中文原意：铰接资产需要运动学测试。 Articulate-Anything [159] 渲染生成的链接和联合代码，将其与输入进行比较，并在修改后导出 URDF。 Articraft [461] 在代码中指定零件、接头和测试，使用编译和几何诊断来指导修复。可变形资产需要不同的表示：CoDimRecon [387]通过模拟测试曲线、曲面和体积； DiagGen [35] 将诊断故障路由到分割、材料或网格修复； DexAgent [361] 在生成轨迹和训练数据之前为刚性、铰接或可变形物体选择重建技能和验证器。这些程序建立了模拟行为和任务的可行性，而不是现实世界的有效性。

解析：构造对象、构造场景、构造任务是不同产物。视觉一致性只能支持外观，接触、运动轴、可达性和成功判据还需要各自检查。

解析：本段并列了 DiagGen、Articulate-Anything、DexAgent、CoDimRecon等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：生成的是世界、任务还是判据？哪种验证支持它？

本段引用：DiagGen [35]（辞典第36页）；Articulate-Anything [159]（辞典第160页）；DexAgent [361]（辞典第362页）；CoDimRecon [387]（辞典第388页）；Articraft [461]（辞典第462页）

原文第 27 页 · 段落 p27-126

中文原意：构造接口和修订。接口确定智能体可以进行哪些场景更改。 SceneSmith [266] 通过设计者、评论家和编排者角色通过回滚来编辑结构化场景对象。 SAGE [379]公开放置、移动和删除操作，并拒绝或修复不稳定的排列。 EmbodiedGen V2 [357] 在持久的世界状态上提供类型化的场景图操作。代码接口允许更具表现力的更改：ChronoAgentic [338] 生成并修复 PyChrono 代码； SimWorld Studio 的 SimCoder [144] 使用编译、碰撞、物理和视觉反馈； LEGO-Anything [183] 通过重复执行代码在 Blender 中重建图像； Code4Scene [410] 检查虚幻场景编辑的支持、渗透和导航实用程序。受约束的 API 缩小了错误空间，而生成的代码需要软件、几何和物理检查。

> 解析：构造对象、构造场景、构造任务是不同产物。视觉一致性只能支持外观，接触、运动轴、可达性和成功判据还需要各自检查。
>
> 解析：本段并列了 SimWorld Studio、LEGO-Anything、SceneSmith、ChronoAgentic等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：生成的是世界、任务还是判据？哪种验证支持它？

本段引用：SimWorld Studio [144]（辞典第145页）；LEGO-Anything [183]（辞典第184页）；SceneSmith [266]（辞典第267页）；ChronoAgentic [338]（辞典第339页）；EmbodiedGen V2 [357]（辞典第358页）；SAGE [379]（辞典第380页）；Code4Scene [410]（辞典第411页）

原文第 27 页 · 段落 p27-127

中文原意：修订应符合预期的下游用途。 RAPID [222] 使用智能体编写的可行性过滤器来筛选受干扰的模拟器变体，并在部署时保持生成的基础动作（primitive）和策略代码固定。 Awomo-SimDataEngine [6] 通过验收检查和依赖性感知修复分阶段构建世界。 DexSim2Real [429] 使用 VLM 现实主义批评家和 LLM 派生的技能课程来调整 仿真到现实（Sim-to-Real）传输的模拟随机化。外观和稳定性检查不能取代接触、摩擦、可达性或下游执行测试。相关工作比较见表10。

> 解析：构造对象、构造场景、构造任务是不同产物。视觉一致性只能支持外观，接触、运动轴、可达性和成功判据还需要各自检查。
>
> 解析：本段并列了 Awomo-SimDataEngine、RAPID、DexSim2Real。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：生成的是世界、任务还是判据？哪种验证支持它？

本段引用：Awomo-SimDataEngine [6]（辞典第7页）；RAPID [222]（辞典第223页）；DexSim2Real [429]（辞典第430页）

原表10 · 原文第27页

表 10. 重建工作的比较（第 6.1 节）。选定的输入和表示可以同时出现。场景包括图形和布局；参数：参数。引擎列出了构建或验证软件，不包括仅供出口的目标。输入：[图标见原表] 文本；图像;视频; 3D 资产。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：Articulate-Anything [159]；Articraft [461]。

原表10 · 原文第28页

表 10（续）

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：Awomo-SimDataEngine [6]；DiagGen [35]；Agentic Real2Sim [36]；SimWorld Studio [144]；ARSTAG [169]；LEGO-Anything [183]；RAPID [222]；SceneSmith [266]；SimFoundry [275]；4DCodeBench [292]；Video2World [316]；ChronoAgentic [338]；EmbodiedGen V2 [357]；RoboPlayground [360]；DexAgent [361]；SAGE [379]；CoDimRecon [387]；GIF [393]；RoboLab [398]；Video2Policy [409]；Code4Scene [410]；Genie Sim 3.0 [414]；DexSim2Real [429]。

6.2 Environment / Task Generation：生成可检验任务

本节先抓住：任务目标、初始化、成功函数和可执行解需要相互一致。用学习算法表现调整课程时，应保留独立任务测试，以防只是生成器把任务改容易了。

贯穿例子："把杯子放进柜子"必须有合适空间、可打开柜门、明确成功区域，而不只是自然语言一句话。

原文第 28 页 · 段落 p28-128

中文原意：任务生成指定机器人必须执行的目标、初始状态、约束和成功条件。它涵盖了可执行任务的构建以及随着学习者的变化调整任务分配。

解析：先把作者的分类判断与实验结论分开。分类说明工作在系统里起什么作用；提升有多大、适用哪些条件，仍要查看该工作给出的实验协议。

自问：生成的是世界、任务还是判据？哪种验证支持它？

原文第 28 页 · 段落 p28-129

中文原意：可执行任务构建. GenSim [345]生成任务库作为模拟代码。 RoboGen [362] 将任务建议与场景和监督结合起来，而 Video2Policy [409] 从互联网视频中获取任务代码和奖励。 RoboLab [398] 根据能力轴、谓词和资产约束组成任务，然后将语法和资产错误反馈给生成器。 HARBOR [191] 检查场景阶段、初始化、终止、动作和观察。 Scene2Demo [219]通过对象动作图链接子任务并生成可执行的初始条件和目标条件。 PARTNR [31] 过滤生成的指令和初始状态，并为命题、依赖性和时间约束构建评估器。

解析：构造对象、构造场景、构造任务是不同产物。视觉一致性只能支持外观，接触、运动轴、可达性和成功判据还需要各自检查。

解析：本段并列了 PARTNR、HARBOR、Scene2Demo、GenSim等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：生成的是世界、任务还是判据？哪种验证支持它？

本段引用：PARTNR [31]（辞典第32页）；HARBOR [191]（辞典第192页）；Scene2Demo [219]（辞典第220页）；GenSim [345]（辞典第346页）；RoboGen [362]（辞典第363页）；RoboLab [398]（辞典第399页）；Video2Policy [409]（辞典第410页）

原文第 28 页 · 段落 p28-130

中文原意：检查还可以控制人类互动和物理可行性的生成。 FATE [366]将静态语义、几何和前提条件检查与动态执行检查分开，并修复故障组件。 GenPHRI [343] 使用 text2仿真到现实（Sim2Real）管道来生成人物、场景和辅助任务；碰撞和运动学检查限制了最终的放置和运动。这些修复可能会改变任务难度或意义，因此应同时检查可行性和客观保留。生成解决方案并不一定会生成新任务。 EmbodiedSmith [272] 将技能组合到任务图中，并修改计划和初始放置，同时保持资产、布局和成功标准固定。 Scene2Demo 同样修改了 展开轨迹（Rollout）评论的动作流，并从保存的状态重新启动当前子任务。由此产生的演示属于数据生成（第 5.1 节），而不是新任务定义。

解析：构造对象、构造场景、构造任务是不同产物。视觉一致性只能支持外观，接触、运动轴、可达性和成功判据还需要各自检查。

解析：本段并列了 EmbodiedSmith、GenPHRI、FATE。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：生成的是世界、任务还是判据？哪种验证支持它？

本段引用：EmbodiedSmith [272]（辞典第273页）；GenPHRI [343]（辞典第344页）；FATE [366]（辞典第367页）

原文第 28、29 页 · 段落 p28-131

中文原意：自适应任务分配。 CurricuLLM [286]生成子任务序列、候选奖励和目标分配代码，并将所选策略传递到下一个子任务。 RAT [435] 选择练习来自场景信息、代码技能库和故障记忆的任务，平衡新颖性和难度。 SimWorld Studio [144] 相反，将具体智能体性能提供给环境生成器，以改变环境和任务难度。 RAT 改变了下一个目标； SimWorld Studio 改变了任务分配。需要对单独指定的任务进行评估，因为根据学习者自身的成功信号优化的课程可能会变得更容易，而不会提高目标能力。相关工作比较见表11。

解析：把“智能体修改的目标”与“判断好坏的标准”分开：奖励代码可变，任务成功判据和测试任务应保持可比。奖励变高不能直接推出物理任务做得更好。

解析：本段并列了 SimWorld Studio、CurricuLLM、RATs。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：生成的是世界、任务还是判据？哪种验证支持它？

本段引用：SimWorld Studio [144]（辞典第145页）；CurricuLLM [286]（辞典第287页）；RATs [435]（辞典第436页）

原表11 · 原文第29页

表 11. 任务生成工作的比较（第 6.2 节）。输出列出生成或修订的工件。 Eval.：成功/评估代码；配置：初始化/随机化设置；计划：可执行的解决方案，可以保留任务。 UE5：虚幻引擎5； MolmoSp.：MolmoSpaces。多个（HARBOR）：Isaac Lab、ManiSkill、Genesis、MJLab。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：PARTNR [31]；SimWorld Studio [144]；ARSTAG [169]；HARBOR [191]；Scene2Demo [219]；EmbodiedSmith [272]；CurricuLLM [286]；GenPHRI [343]；GenSim [345]；RoboPlayground [360]；RoboGen [362]；FATE [366]；RoboLab [398]；Video2Policy [409]；RATs [435]。

6.3 Environment / Benchmarking：能力怎样被测量

本节先抓住：答案、动作、工具调用和代码是不同评测接口；离线问答、模拟交互和真机任务也不同。接口、重置、提示、预算与成功函数必须一起报告。

贯穿例子：同一模型一个版本有碰撞规划器，另一个只能直接输出位姿，其成功率差异不能全归因于模型。

原文第 29 页 · 段落 p29-133

中文原意：基准测试修复了用于衡量能力的任务、界面、重置协议和评分规则。智能体可以是被测试的系统，或者他们可以构建和修改评估本身。当观察访问、执行协助、反馈时间或交互预算发生变化时，同一任务可以衡量不同的能力。

解析：比较时记录任务、执行接口、工具、预算与重置条件。动作接口、工具调用、代码生成和离线问答测的是不同能力，不能只按一个平均数混排。

解析：把模型生成时间、网络等待和机器人执行时间分开。核对新输出是否针对当前状态，控制权由什么事件交回；本段频率或时延只描述相应配置。

自问：生成的是世界、任务还是判据？哪种验证支持它？

原文第 29 页 · 段落 p29-134

中文原意：评估机器人控制中的智能体。规划基准测试智能体是否将指令转换为机器人操作。 EmbodiedBench [399] 改变动作抽象并返回环境反馈。 Embodied Agent Arena [115] 在其规划和操作领域使用多轮交互。 MissionBench [247] 评估为空中任务选择运动基元、幅度和停止决策的 MLLM。 KinDER [122] 评估参数化技能的开环组合，因此其分数不衡量反馈驱动的重新规划。

> 解析：比较时记录任务、执行接口、工具、预算与重置条件。动作接口、工具调用、代码生成和离线问答测的是不同能力，不能只按一个平均数混排。
>
> 解析：本段并列了 Embodied Agent Arena、KinDER、MissionBench、EmbodiedBench。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：生成的是世界、任务还是判据？哪种验证支持它？

本段引用：Embodied Agent Arena [115]（辞典第116页）；KinDER [122]（辞典第123页）；MissionBench [247]（辞典第248页）；EmbodiedBench [399]（辞典第400页）

原文第 29 页 · 段落 p29-135

中文原意：工具和编程基准评估可执行决策。 EmbodiedToolBench [466] 将工具识别、选择、执行、输出使用和工具链组成分开。 CaP-X [74] 在执行生成的控制代码时改变状态访问、API 抽象、示例和可视化 落地关联（Grounding）。 EmbodiedSWE [294] 提供编码智能体模拟器状态、渲染和执行反馈，然后在单独的环境中对生成的程序进行评分。

> 解析：比较时记录任务、执行接口、工具、预算与重置条件。动作接口、工具调用、代码生成和离线问答测的是不同能力，不能只按一个平均数混排。
>
> 解析：本段并列了 CaP-X、EmbodiedSWE、EmbodiedTool。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：生成的是世界、任务还是判据？哪种验证支持它？

本段引用：CaP-X [74]（辞典第75页）；EmbodiedSWE [294]（辞典第295页）；EmbodiedTool [466]（辞典第467页）

原文第 29、30 页 · 段落 p29-136

中文原意：现有的机器人套件可以进行这些评估。 LIBERO [206] 提供了一个标准的终身学习套件，LIBERO-PRO [465] 改变了协议以减少记忆并提高鲁棒性。 CaP-X 在 LIBERO-PRO 上通过位置和指令扰动评估 CaP-Agent0，而 ASPIRE [228] 使用 LIBERO-PRO、robosuite 和 BEHAVIOR-1K，并为固定程序和程序提供单独的协议基于跟踪的代码修订。因此，智能体分数取决于程序是在评估之前固定的还是根据当前跟踪进行修改的。

> 解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。
>
> 解析：本段并列了 LIBERO、ASPIRE、LIBERO-PRO。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：生成的是世界、任务还是判据？哪种验证支持它？

本段引用：LIBERO [206]（辞典第207页）；ASPIRE [228]（辞典第229页）；LIBERO-PRO [465]（辞典第466页）

原文第 30 页 · 段落 p30-138

中文原意：运行支撑框架（Harness） VLA [441] 使用 LIBERO、LIBEROPRO、RoboCasa365 和 RoboTwin C2R 上的冻结 VLA 测试分析基元的智能体选择。 RoboHarness [116] 在 LIBERO 变体上测试异构策略编排、扰动鲁棒性和 长时程（Long-Horizon） 执行，并添加策略切换任务。在这两种情况下，任务套件本身并不能确定所测量的能力；执行和适应协议也是如此。 RoboDojo [41] 将 42 个模拟任务与 18 个现实世界任务相结合，并评估泛化、记忆、精度、长时程（Long-Horizon） 执行和开放词汇指令遵循。 RobotWorld [404]将此设置扩展到操纵、移动操纵、运动、驾驶和空中控制。它的 84 个模拟任务指定了交互预算和可执行的成功检查，其跟踪分析将结果与智能体的感知、计算、反馈使用和完成决策联系起来。

解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。

解析：本段并列了 RoboDojo、RoboHarness、RobotWorld、Harness VLA。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：生成的是世界、任务还是判据？哪种验证支持它？

本段引用：RoboDojo [41]（辞典第42页）；RoboHarness [116]（辞典第117页）；RobotWorld [404]（辞典第405页）；Harness VLA [441]（辞典第442页）

原文第 30 页 · 段落 p30-139

中文原意：主动感知测试评估信息收集。 VABench [443] 要求 MLLM 检查场景，发出相机和笛卡尔命令，并在新观察后修改决策。 ActiveArenaPlan [186] 使用规划器为 VLA 执行器选择子任务和历史记录。前者直接测试观测采集；后者还测试选定的历史记录如何指导任务进度。

解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。

解析：本段并列了 ActiveArena (Plan)、VABench。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：生成的是世界、任务还是判据？哪种验证支持它？

本段引用：ActiveArena (Plan) [186]（辞典第187页）；VABench [443]（辞典第444页）

原文第 30 页 · 段落 p30-140

中文原意：记忆测试询问早期的观察是否仍然有用。 RoboGraph [346] 控制依赖关系、观察访问和外部事件来测试状态维护和更新。 RoboMemArena [164] 使用 VLM 规划器来选择子任务并为低层执行器保留观察结果。EvoNav-Bench [355]引入了隐藏的场景变化，这可能会使保留的证据变得陈旧。 MEMOBench [308] 为记忆存储、更新和压缩添加了可执行检查点，将记忆保真度与最终操作成功分开。尽管其报告的主题是VLA策略，但相同的进程级别检查可以应用于智能体控制的执行轨迹（rollout）；记忆机制将在第 7 节中讨论。

解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。

解析：本段并列了 RoboMemArena、MEMOBench、RoboGraph、EvoNav-Bench。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：生成的是世界、任务还是判据？哪种验证支持它？

本段引用：RoboMemArena [164]（辞典第165页）；MEMOBench [308]（辞典第309页）；RoboGraph [346]（辞典第347页）；EvoNav-Bench [355]（辞典第356页）

原文第 30 页 · 段落 p30-141

中文原意：安全测试将危险避免与任务完成分开。 ReactHuman [190] 根据模拟器的真实情况对突发事件的 MLLM 运动计划进行评分。 ManiGuard [264] 使用基于规范的运行时监视器并将安全性与任务成功分开报告。其发布的评估使用 VLA 策略，但监视器还可以在同一任务协议下对智能体控制的 执行轨迹（rollout）进行评分。 ReactHuman 在推理过程中暂停模拟，因此它测量危险响应决策而不是实时反应。

解析：构造对象、构造场景、构造任务是不同产物。视觉一致性只能支持外观，接触、运动轴、可达性和成功判据还需要各自检查。

解析：本段并列了 ReactHuman、ManiGuard。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：生成的是世界、任务还是判据？哪种验证支持它？

本段引用：ReactHuman [190]（辞典第191页）；ManiGuard [264]（辞典第265页）

原文第 30 页 · 段落 p30-142

中文原意：因此，基准分数取决于完整的执行合约：智能体决策、执行者、反馈路径、观察访问、控制器协助和交互预算。

> 解析：比较时记录任务、执行接口、工具、预算与重置条件。动作接口、工具调用、代码生成和离线问答测的是不同能力，不能只按一个平均数混排。
>
> 解析：把模型生成时间、网络等待和机器人执行时间分开。核对新输出是否针对当前状态，控制权由什么事件交回；本段频率或时延只描述相应配置。

自问：生成的是世界、任务还是判据？哪种验证支持它？

原文第 30 页 · 段落 p30-143

中文原意：智能体导向的评估构建。构建评估的智能体必须指定测试的能力、任务条件以及用于评分的证据。ManipEvalAgent [49]分解评估请求，生成任务和评估工具，并使用展开轨迹（Rollout）结果提出进一步的测试或总结测试的策略。 Embodied-BenchForge [133] 生成任务模板、数据和可执行验证器；交互式任务需要可恢复的状态和终端检查，而离线任务需要有依据的证据和可执行的评分。其依赖图支持有针对性的回滚和修复。 A2Eval [438] 按能力对体现推理问题进行分组，生成推理和评分代码，并根据执行反馈修复该代码。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。
>
> 解析：本段并列了 ManipEvalAgent、Embodied-BenchForge、A2Eval。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：生成的是世界、任务还是判据？哪种验证支持它？

本段引用：ManipEvalAgent [49]（辞典第50页）；Embodied-BenchForge [133]（辞典第134页）；A2Eval [438]（辞典第439页）

原文第 30 页 · 段落 p30-144

中文原意：适应性评估可以揭示弱点，但其结论取决于测试选择、先前结果的获取和分数验证。独立于评估系统确定测试条件支持更广泛的主张。改进评估器和改进机器人是不同的结果。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

自问：生成的是世界、任务还是判据？哪种验证支持它？

原文第 30 页 · 段落 p30-145

中文原意：世界构建、任务生成和基准测试诊断不同的故障：不稳定的几何体需要场景修复，不可行的目标需要任务修订，无效的成功信号需要评估器修复。将这些检查分开可以更容易地定位故障，并且更容易在硬件上验证模拟条件。相关工作比较见表12。

> 解析：比较时记录任务、执行接口、工具、预算与重置条件。动作接口、工具调用、代码生成和离线问答测的是不同能力，不能只按一个平均数混排。
>
> 解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。

自问：生成的是世界、任务还是判据？哪种验证支持它？

原表12 · 原文第31页

表 12. 基准测试工作比较（第 6.3 节）。界面描述了测试系统的输出：答案（预测）、动作（电机命令）、调用（工具/技能）、代码（程序）；评估者生成的代码被排除在外。 RobotWorld 的代码是可选的。引擎列出了模拟后端；多个表示混合套件。 NR：未报道；N/A：离线评估，无需模拟器执行。协作：协作； UE5：虚幻引擎5。设置：真实；模拟； [图标见原表] 离线数据。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：PARTNR [31]；RoboDojo [41]；ManipEvalAgent [49]；CaP-X [74]；Embodied Agent Arena [115]；RoboHarness [116]；KinDER [122]；Embodied-BenchForge [133]；RoboMemArena [164]；ActiveArena (Plan) [186]；ReactHuman [190]；LIBERO [206]；ASPIRE [228]；MissionBench [247]；ManiGuard [264]；EmbodiedSWE [294]；MEMOBench [308]；RoboGraph [346]；EvoNav-Bench [355]；RoboPlayground [360]；RoboLab [398]；EmbodiedBench [399]；RobotWorld [404]；AGOS-Agent [416]；A2Eval [438]；Harness VLA [441]；VABench [443]；LIBERO-PRO [465]；EmbodiedTool [466]。

7 运行支撑框架（Harness）：机器人能力应该如何编排、恢复和进化？

原文第 31 页 · 段落 p31-146

中文原意：机器人可能拥有强大的模型和策略，但如果它们围绕执行组织得不好，仍然会失败。我们使用 运行支撑框架（Harness） 来表示通过管理组件的调用方式、接收的信息、检查执行的方式以及以后可以重用的内容，使智能体功能可供机器人使用的软件。本节将 运行支撑框架（Harness） 函数组织为六个组件，如图 7 所示。Orchestration 管理组件之间的调用和切换。 记忆提供先前步骤中保留的信息。 监控和恢复评估执行情况并对故障做出响应。 Skills 封装可重用的行为。 基础设施公开并控制可执行功能。 分层系统将权力划分给规划者、学习策略和分析控制。前三个组件在一次尝试中发挥作用，技能在尝试之间传递行为，最后两个组件描述了它们下面的软件和系统结构。

解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

图7：运行支撑框架的六种介入

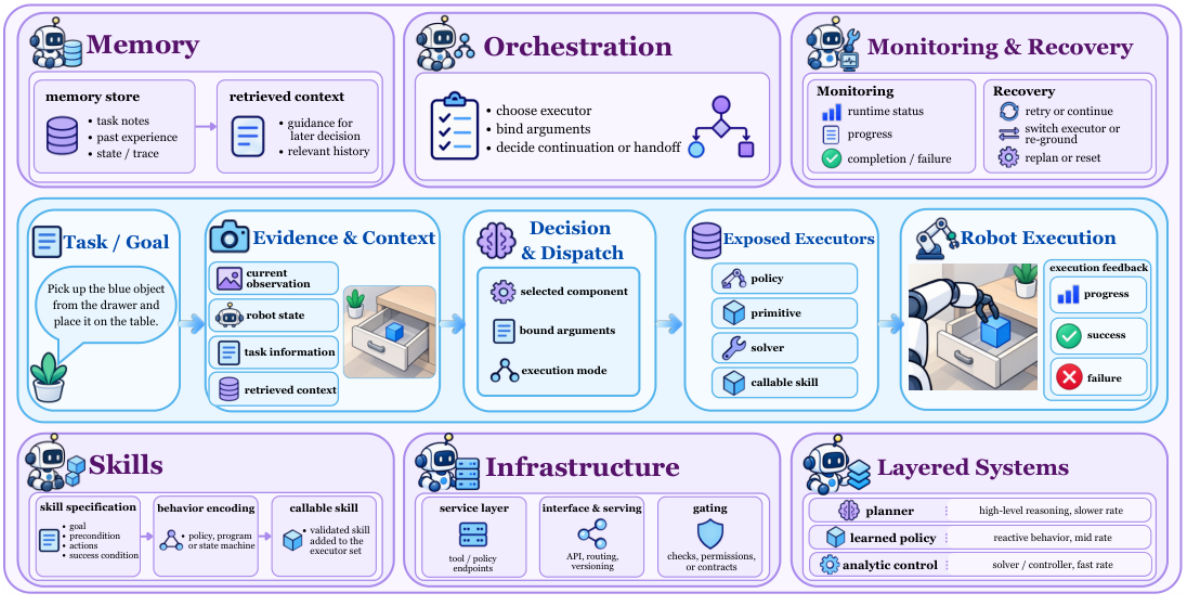


Figure 7. Harness functions that organize agentic capabilities around robot execution. The central row connects task goals and evidence to decision and dispatch, exposed executors, robot execution, and returned feedback. Orchestration governs calls and handoffs; Memory supplies retained information; Monitoring and Recovery assess execution and respond to failure; Skills package reusable behavior; Infrastructure exposes and gates callable capabilities; and Layered Systems divide authority and operating rates across planners, learned policies, and analytic control.

中文读图：编排决定谁执行，记忆提供历史，监测与恢复处理异常，技能积累可复用产物，基础设施保证接口与时间，分层定义高低层交接。

7.1 Harness / Orchestration：谁得到控制权

本节先抓住：关注路由依据、执行预算与接管触发。模型选择能力，规则层可限制调用；异步组件需要处理不同速率和新旧状态的因果一致性。

贯穿例子：高层十秒前决定抓取，目标却已移动：低层再快也不能直接执行旧目标，必须重新绑定证据。

原文第 32 页 · 段落 p32-148

中文原意：编排决定运行哪个模型、策略、技能或工具、保持控制的时间以及控制如何传递到下一个组件。我们通过四个步骤遵循单个切换顺序来说明编排，其中 运行支撑框架（Harness） 选择下一个执行器，定义函数调用接受和返回的内容，确保下一个执行器可以从结果状态开始，并决定何时返回控制权。

解析：先把作者的分类判断与实验结论分开。分类说明工作在系统里起什么作用；提升有多大、适用哪些条件，仍要查看该工作给出的实验协议。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

原文第 32 页 · 段落 p32-149

中文原意：路由和调度。 在智能体 运行支撑框架（Harness） 中，规划器很少直接对机器人进行操作。相反，它决定哪个执行者应该处理下一步，例如学习策略、分析工具或人员，并将任务传递给它。关键的设计选择是何时做出的。 VoLo [40] 在执行过程中实现了这一点。它监视正在运行的 VLA，当操作出错时，会停止 VLA 并切换到由感知引导的手工设计技能。 MistyPilot [354] 提前做到了。它在传感器事件和社交机器人行为技能之间生成持久绑定，以便后续事件无需新的规划步骤即可触发技能。 AeroWeaver [226] 在团队层面上做到了这一点，将特定于身体的空中技能分配给模拟群体中的分布式角色。 M2HRI [99] 进一步将人类纳入路由各方，添加人类参与协调员和以记忆为条件的交互规划器。这些系统共同将路由决策置于不同的点，从运行操作到长期规则再到团队级角色。

> 解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。
>
> 解析：本段并列了 VoLo、M2HRI、AeroWeaver、MistyPilot。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：VoLo [40]（辞典第41页）；M2HRI [99]（辞典第100页）；AeroWeaver [226]（辞典第227页）；MistyPilot [354]（辞典第355页）

原文第 32 页 · 段落 p32-150

中文原意：同时，只有当调度程序知道每个执行程序可以做什么和不能做什么时，路由才是可靠的。这些知识可以来自对机器人身体的描述或早期调用的结果。 EMOS [37] 对不同机器人的团队采用第一种方法。它根据 URDF 模型和正向运动学工具为每个机器人构建机器人简历，每个机器人的智能体使用此简历来评估分配的子任务，并在调用其技能库之前报告它是否可以执行该子任务。 运行支撑框架（Harness） VLA [441] 和 Physical Agency [75] 在单个机器人上采用第二种方法，将分析工具和 VLA 放置在一个调度接口后面，并根据每次调用返回的证据（例如抓取检查和键入的失败消息）选择下一个执行器。 Guava [208] 在控制层面应用相同的想法，在语义操作（例如抓取和对齐）和基于视觉观察、机器人状态和早期工具结果的数值运动工具之间进行选择。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。
>
> 解析：本段并列了 EMOS、Physical Agency、Guava、Harness VLA。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：EMOS [37]（辞典第38页）；Physical Agency [75]（辞典第76页）；Guava [208]（辞典第209页）；Harness VLA [441]（辞典第442页）

原文第 33 页 · 段落 p33-151

中文原意：函数协议。机器人功能通常作为函数公开给智能体，智能体使用名称和一些参数调用这些函数，而无需了解它们内部的工作原理。函数协议通过说明函数应该如何调用、函数报告什么以及调用何时算成功来填补这一空白。其中第一个定义了什么算作有效调用，并且可以为智能体编写以遵循或由 运行支撑框架（Harness） 强制执行。 EmbodiedTool [466]为智能体编写它，为每个工具提供其输入、输出和智能体调用必须匹配的使用条件的类型化描述。 ROSClaw [26] 在 运行支撑框架（Harness） 中强制执行，检查对物理 TurtleBot3 或 Go2 机器人的每次调用是否符合速度限制和允许的接口列表，以便即使智能体产生无效调用也会停止。

解析：比较时记录任务、执行接口、工具、预算与重置条件。动作接口、工具调用、代码生成和离线问答测的是不同能力，不能只按一个平均数混排。

解析：本段并列了 ROSClaw、EmbodiedTool。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：ROSClaw [26]（辞典第27页）；EmbodiedTool [466]（辞典第467页）

原文第 33 页 · 段落 p33-152

中文原意：一旦调用运行，协议就会定义返回给智能体的内容。一个简单的成功标志几乎无法说明机器人的情况，因此某些协议会返回更多信息。物理智能体 AI [220] 返回每个技能结果以及当时的机器人状态，以后的调度可以使用该结果，而无需再次调用 基础模型（Foundation Model）。物理智能体[75]返回类型化的失败，以便智能体可以对每种类型做出不同的响应，例如通过再次设置操作或切换到另一个执行器。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

解析：本段并列了 Physical Agency、Physical Agentic AI。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：Physical Agency [75]（辞典第76页）；Physical Agentic AI [220]（辞典第221页）

原文第 33 页 · 段落 p33-153

中文原意：即使是详细的结果，仍然是执行者自己的报告。成功条件通过说明调用成功后应该为 true 的内容来添加独立检查。 Thea [348]将这些条件附加到每个工具上，并且与规划器分开的任务评估器根据当前观察结果检查它们。

解析：先把作者的分类判断与实验结论分开。分类说明工作在系统里起什么作用；提升有多大、适用哪些条件，仍要查看该工作给出的实验协议。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：Thea [348]（辞典第349页）

原文第 33 页 · 段落 p33-154

中文原意：状态连接。 每个执行器仅在某些起始状态下可靠地工作，因此如果上一步将机器人留在其他地方，正确选择的执行器仍然可能失败。状态连接确保一个执行器留下的状态连接到下一个执行器需要的状态。现有系统通过四种方式建立这种连接。前两种方法调整状态或执行器。 RoboHarness [116] 调整状态以适应执行器。它从记忆中检索过去的轨迹，并使用运动计划记忆桥将机器人移动到下一个策略预期的起始状态。RoboRouter [48] 调整执行器以适应状态。它从多个候选策略中选择适合当前状态的策略，并使用过去运行的摘要来改进以后的选择。

解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。

解析：本段并列了 RoboRouter、RoboHarness。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：RoboRouter [48]（辞典第49页）；RoboHarness [116]（辞典第117页）

原文第 33 页 · 段落 p33-155

中文原意：其他两种方式在切换之前建立连接，这在多个机器人共享任务时很常见。在这种情况下，下一个机器人也必须知道其他机器人做了什么以及它可能使用哪些资源。有些系统会检查继任者是否准备好。有些系统在每次切换之前都会检查这一点。 RoCo[237]让机器人通过对话商定一个计划，然后在移动之前检查该计划是否可行。 RoboTalk [84] 根据符号前提条件和资源所有权检查每条消息和联合操作，以便机器人仅在被告知的内容和可能使用的内容一致时才采取行动。其他系统提前固定好状态连接结构。 AGOS-Agent [416]将任务分为搜索候选者、地面路线规划和地面视图验证，以便每个阶段接收的输入在设计时设置。总之，这些方法要么在出现不匹配时进行修复，要么通过提前检查或修复切换来防止不匹配。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

解析：本段并列了 RoboTalk、RoCo、AGOS-Agent。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：RoboTalk [84]（辞典第85页）；RoCo [237]（辞典第238页）；AGOS-Agent [416]（辞典第417页）

原文第 33 页 · 段落 p33-156

中文原意：切换计时。切换计时决定控制权何时从执行器返回到决策过程。太晚返回控制权会使失败的操作继续进行，而太早返回控制权则会中断本来会成功的操作。现有系统的不同之处主要在于触发此返回的信号，该信号可以是任务进度、执行预算或者定期或事件驱动的计划。当观察到的任务状态发生变化时，基于进度的触发器会返回控制权。 HarnessWAM [89] 使用快速进度估计器来检测此类变化，然后 VLM 任务管理器决定是否移动到下一个子任务、收集新的观察结果、重新计划或重置机器人。 Cortex [263] 在机器人移动时连续检查视觉进度，并保持运行的语义记忆来决定当前阶段是否应该继续。

解析：把模型生成时间、网络等待和机器人执行时间分开。核对新输出是否针对当前状态，控制权由什么事件交回；本段频率或时延只描述相应配置。

解析：本段并列了 HarnessWAM、Cortex。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：HarnessWAM [89]（辞典第90页）；Cortex [263]（辞典第264页）

原文第 33 页 · 段落 p33-157

中文原意：其他系统限制每次调用的时间或协调以不同速率运行的组件。 VLAs-asTools [166] 将 VLA 作为执行预算有限的工具来运行，在此期间，VLA 将进度报告给可以继续或重新计划的外部监视器。 Retriever [446] 异步运行感知、信念更新、规划、VLA 和控制，具有单独的周期性或事件驱动时钟以及保留因果关系的显式输入同步和提交规则。一般来说，这些切换时序的触发器的不同之处在于 运行支撑框架（Harness） 在收回控制权之前等待的内容，无论是观察到的正在进行的更改、预算结束还是下一个时钟滴答。相关工作比较见表13。

解析：把模型生成时间、网络等待和机器人执行时间分开。核对新输出是否针对当前状态，控制权由什么事件交回；本段频率或时延只描述相应配置。

解析：本段并列了 VLAs-as-Tools、Retriever。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：VLAs-as-Tools [166]（辞典第167页）；Retriever [446]（辞典第447页）

原表13 · 原文第34页

表 13.编排工作的比较（第 7.1 节），按文本中的四个类别分组。执行器列标记哪些类型的组件接收控制，即学习策略，例如 VLA 和 WAM（策略）、可调用实用程序，例如运动规划器和控制器（工具）、技能库中的可重用行为（技能）以及其他机器人或人类（其他智能体）。 Selection 显示决定哪个执行器运行的因素，即 LLM 或 VLM 智能体（基于模型）、其调用由规则层检查或触发的模型（混合）或预设计划（基于规则）。图标链接到论文和代码（如果有）。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：ROSClaw [26]；EMOS [37]；VoLo [40]；RoboRouter [48]；Physical Agency [75]；RoboTalk [84]；HarnessWAM [89]；M2HRI [99]；RoboHarness [116]；VLAs-as-Tools [166]；Guava [208]；Physical Agentic AI [220]；AeroWeaver [226]；RoCo [237]；Cortex [263]；ABot-AgentOS [321]；Thea [348]；EmbodiedSkills [351]；MistyPilot [354]；AGOS-Agent [416]；Retriever [446]；EmbodiedTool [466]。

7.2 Harness / Memory：什么证据可重用

> 本节先抓住：原图、场景图、摘要、技能和错误经验不是同一种记忆。需要同时说明写入、检索、验证、过期与跨回合保留策略。更多历史不等于更有效信息。
>
> 贯穿例子：记住“杯子在左边”可以减少搜索，但人移动杯子后，旧记录必须更新或标为不可信。

原文第 34 页 · 段落 p34-158

中文原意：记忆在不改变模型权重的情况下提供从早期步骤到后期决策的信息。与规则或程序等可重复使用的行为不同，记忆记录发生的事情，包括机器人尝试过的事情、观察到的事情以及任务的进展情况。我们根据现有方法所处理的记忆阶段将其分为三类。 记忆写入决定记录什么以及何时记录。 记忆更新使记录与不断变化的场景保持一致。 记忆检索为下一个决策选择记录并重用早期的解决方案。

> 解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。
>
> 解析：问可修改的东西究竟是什么：一次程序、持久代码库、运行上下文还是模型权重。代码保存下来与训练模型不同；修改监测代码还可能改变评判标准。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

原文第 34 页 · 段落 p34-159

中文原意：记忆写作。记忆写作决定了剧集的哪些部分值得记录。常见的记录包括失败的选择、任务进度和观察到的经验。有些系统会记录故障，以便机器人稍后返回。 HAM-VLN [205]将每个故障及其发生的位置存储在一起，这支持导航期间的回溯和验证停止。 Robo-Cortex [30] 根据新的真实观察结果检查想象的候选动作，并将结果保存在具有两个细节级别的记忆中。其他系统记录任务进度。 HiMe [131] 保留一个子任务列表，其中包含每个项目的状态以指导计划，与剧集的可编辑记录分开，并使用单独的检查器来判断最近的视觉进度。第三组记录了更广泛的经验。 MEMORA [420] 将经验存储为地点、物体、活动和推断知识的记录，这些记录在任务期间进行编辑并在之后合并。 ABot-AgentOS [321] 将此类记录存储为链接到其来源的图表，而不是原始历史记录。

> 解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。
>
> 解析：本段并列了 Robo-Cortex、HiMe、HAM-VLN、ABot-AgentOS等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：Robo-Cortex [30]（辞典第31页）；HiMe [131]（辞典第132页）；HAM-VLN [205]（辞典第206页）；ABot-AgentOS [321]（辞典第322页）；MEMORA [420]（辞典第421页）

原文第 34、35 页 · 段落 p34-160

中文原意：除了记录什么之外，运行支撑框架（Harness）还必须决定何时写入以及以何种形式写入。每一步之后都写用冗余记录填充记忆，而写入很少会错过后续决策所依赖的事件。 MessyMem [8] 在每次物理交互后写入。 VLM 会审查操作，记录结果（例如错过抓握或锁抽屉），并使用这些标注和关键图像更新匹配的对象条目。权重技能，代码记忆 [452] 相反，将记忆策略本身视为代码，从 展开轨迹（Rollout）跟踪开发它并将其保留为可运行的程序。因此，这些系统的不同之处在于书写是由事件触发还是由学习的程序控制。

解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。

解析：本段并列了 MessyMem、HyMeS。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：MessyMem [8]（辞典第9页）；HyMeS [452]（辞典第453页）

原文第 35 页 · 段落 p35-162

中文原意：记忆更新。记忆更新使存储的记录与不断变化的场景保持一致。随着物体的移动以及早期的观察结果变得过时，写入时正确的记忆条目后来可能是错误的。现有系统通过三种方式处理此问题：将不同类型的记录分开保存、定期刷新记录或在依赖记录之前检查记录。 Mimir [392] 将任务记忆与世界记忆分开，并使用不同的算子更新执行进度、失败的假设、对象关系和证据。 RoboStream [123] 在每个规划步骤中都会刷新有关机器人和场景的证据。 OVAL [262] 根据目标检查每个记住的候选者，拒绝错误的目标并恢复探索。

解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。

解析：本段并列了 RoboStream、OVAL、Mimir。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：RoboStream [123]（辞典第124页）；OVAL [262]（辞典第263页）；Mimir [392]（辞典第393页）

原文第 35 页 · 段落 p35-163

中文原意：更新还决定记录的保存时间。保留每条记录会使记忆变大并且搜索速度变慢，而过早删除记录会丢失机器人可能仍然需要的信息。 MetaTool [235] 将计划和工具结果保存在仅在当前情节中使用的暂存器中。其他记录竞争固定的空间量。 EEAgent [340] 将成功和失败交互的反映压缩到固定大小的文本记忆中，ActiveArena [186] 比较规则以选择保留在此类记忆中的内容。第三种记录持续存在，超出了机器人目前所能看到的范围。 SafeMem [180] 将视图之外的对象保留在场景图中，在观察到变化时修改它们的关系，并在预测风险较高时请求重新规划。 CoRelNav [100] 保留有关对象实例的共享证据。世界动作智能体[440]保留可重复使用的关系技能、程序和参考图像，并根据当前场景进行检查；当相关的观察或控制状态发生变化时，场景特定的建议就会过期。

解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。

解析：本段并列了 CoRelNav、SafeMem、ActiveArena (Plan)、BTA等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：CoRelNav [100]（辞典第101页）；SafeMem [180]（辞典第181页）；ActiveArena (Plan) [186]（辞典第187页）；BTA [235]（辞典第236页）；EEAgent [340]（辞典第341页）；WAA [440]（辞典第441页）

原文第 35 页 · 段落 p35-164

中文原意：记忆检索。记忆检索决定哪些存储的记录到达下一个决策。随着记忆的增长，它所保存的大部分内容并不影响手头的决定，而通过完整的历史记录可能会掩盖一些重要的细节。现有系统检索三种内容：当前决策的证据、继续工作所需的任务状态以及可以替代新计算的早期结果。第一种是当前决定的证据。通常根据与当前目标、位置或问题的相关性来选择此类证据。 NavMCP [167] 通过迄今为止基于关键帧的路线摘要来回答冻结导航模型的问题，并在调用中保留未解决目标的列表。 HFLEX-EQA [270] 和 NavProbe [210] 选择与每个导航决策最相关的图表或过去的视图证据。 EmbodiedLGR [282] 在机器人构建的场景图中搜索相关条目，CMMR-VLN [173] 检索存储的图像和文本体验以进行重复导航。 HoloAgent-0 [464]从3D空间记忆中检索对象、房间、视图和任务历史记录，并通过视觉验证和主动探索来确认它们。在这些系统中，检索将漫长的历史范围缩小到与一个决策有关的少数记录。

解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。

解析：本段并列了 NavMCP、CMMR-VLN、NavProbe、HFLEX-EQA等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：NavMCP [167]（辞典第168页）；CMMR-VLN [173]（辞典第174页）；NavProbe [210]（辞典第211页）；HFLEX-EQA [270]（辞典第271页）；EmbodiedLGR [282]（辞典第283页）；HoloAgent-0 [464]（辞典第465页）

原文第 35 页 · 段落 p35-165

中文原意：第二种是继续工作所需的任务状态。当执行者无法跟踪任务本身时，这种情况很重要，例如每次调用时都重新开始的技能或没有记忆的动作模型。 BATON [391] 在技能转换时检索此状态。它不是检索整个情节，而是检索为下一个技能记录的设置和开始条件，并根据实时手腕摄像头视图检查它们。凌晨 2 点 [112] 在一项长时间任务中检索它。它将历史浓缩为目标、目的地、某个步骤重复了多少次以及当前阶段，并将此摘要作为语言和可选的 2D 方向提示传递给没有记忆的动作模型。在这两个系统中，运行支撑框架（Harness）都保留自己的完整历史记录，并仅向执行器传递将当前步骤链接到任务其余部分的状态。

解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。

解析：本段并列了 2AM、BATON。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：2AM [112]（辞典第113页）；BATON [391]（辞典第392页）

原文第 35 页 · 段落 p35-166

中文原意：第三种是较早的结果，可以替代新的计算。当以前处理过类似的情况时，重用早期的结果可能比计算新的结果更快、更便宜，前提是结果仍然适用。 MeCo [332]检索类似成功的多机器人任务的解决方案并重用其航航路点结构，如果验证拒绝传输，则从失败的步骤回退到 LLM。

解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：MeCo [332]（辞典第333页）

原文第 36 页 · 段落 p36-167

中文原意：APEX [432] 重用早期 VLM 更新产生的空间证据，以便无人机可以继续探索，而无需等待每个新更新完成。这两个系统以不同的方式处理早期结果是否仍然适用的问题，MeCo 通过验证重用的解决方案，而 APEX 通过对可能比当前场景稍旧的证据采取行动。相关工作比较见表14。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：APEX [432]（辞典第433页）

原表14 · 原文第36页

表 14. 记忆工作比较（第 7.2 节）。结构化包括记录、索引、计划和代码；持续意味着记忆在各个情节中被保留。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：MessyMem [8]；Robo-Cortex [30]；CoRelNav [100]；2AM [112]；RoboStream [123]；HiMe [131]；NavMCP [167]；CMMR-VLN [173]；SafeMem [180]；ActiveArena (Plan) [186]；Robo-Harness K1 [193]；HAM-VLN [205]；NavProbe [210]；BTA [235]；OVAL [262]；HFLEX-EQA [270]；EmbodiedLGR [282]；MeCo [332]；EEAgent [340]；BATON [391]；Mimir [392]；ARMS [411]；MEMORA [420]；APEX [432]；WAA [440]；HyMeS [452]；HoloAgent-0 [464]。

7.3 Harness / Monitoring and Recovery：何时检查、怎样恢复

本节先抓住：执行前验证、执行中监测、执行后判定解决不同问题。恢复可改变子目标、动作、策略或软件；重试预算和人工接管是闭环的一部分。

贯穿例子：杯子抓空与机器人驱动没启动不能用同一种恢复：前者补看/重抓，后者修执行基础设施。

原文第 36 页 · 段落 p36-168

中文原意：监控判断执行是否正在进行、已完成、失败或违反约束，发现问题后恢复会更改执行。我们将现有方法分为三类，遵循从检测到修复的过程。 执行监控检查操作之前、期间或之后的执行情况。 响应选择 将判决转换为响应，并在运行之前检查该响应。 恢复操作执行响应，包括恢复开始的位置以及何时停止尝试。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

解析：沿着语义到执行的接口看：物体名称/关系 → 几何证据或句柄 → 目标/约束 → 执行器。若输入依赖理想真值或旧位姿，就要额外问真实感知时是否仍成立。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

原文第 36 页 · 段落 p36-169

中文原意：执行监控。执行监控可以在操作运行之前、运行期间或结束之后检查操作。检查的时间决定了它可以发现哪些问题以及生成什么样的报告。执行前的检查针对尚未产生任何实际效果的拟议操作或路线进行操作，通常会接受、拒绝或对它们进行排名。 SafeHarness [390]测试建议的路线是否有障碍物并返回拒绝的原因，以便可以修改路线。 DV-VLN [194] 回答有关候选导航操作的正确或错误问题，并相应地重新排序。 CoVer [157] 提前准备指令的改写，并在每个动作运行之前使用验证器对候选动作和指令进行评分。由于这些检查只需要在选项中进行选择，因此它们的输出是简单的决策或分数。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。
>
> 解析：本段并列了 CoVer、DV-VLN、SafeHarness。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：CoVer [157]（辞典第158页）；DV-VLN [194]（辞典第195页）；SafeHarness [390]（辞典第391页）

原文第 36、37 页 · 段落 p36-170

中文原意：执行过程中的检查会在操作仍在进行时发出信号。他们主要是跟踪约束违规行为和任务进度，有些系统会同时检查多个级别。对于约束，DoReMi [94] 使用 LLM 为下一个技能编写约束，并使用 VLM 定期检查它们，一旦技能被破坏就停止该技能。代码即监视器[458]将视觉约束编写为可以在故障发生时或发生之前检测到故障的代码。同时，任务级运行时保证[147]检查任务事件的约束，并在证据缺失时报告“未知”。对于进度，OnFly [455] 在飞行的同时检查任务阶段，并在检测到停止或丢失情况时做出反应，而 Robometer [195] 通过从专家标签以及成功和失败运行之间的比较中学习的分数来跟踪进度。 ABot-AgentOS [321] 结合了多个级别，在运行时、技能级别进行检查，并在允许任务完成之前再次进行检查。这些检查主要输出信号来停止、调整或报告进度，因为它们必须在运动结束之前起作用。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。
>
> 解析：本段并列了 DoReMi、MLRA、Robometer、ABot-AgentOS等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：DoReMi [94]（辞典第95页）；MLRA [147]（辞典第148页）；Robometer [195]（辞典第196页）；ABot-AgentOS [321]（辞典第322页）；OnFly [455]（辞典第456页）；Code-as-Monitor [458]（辞典第459页）

原文第 37 页 · 段落 p37-172

中文原意：执行后的检查可以使用所发生事件的完整记录，这使他们能够判断结果并解释其原因。 RoboLab [398]，首先检查任务资产和稳定性测试结果后，使用生成的任务代码来判断结果，该代码定义任务何时成功或应该结束。 REFLECT [223]总结了传感器、事件和子目标级别的执行，以向纠正计划者解释失败。 AHA [64] 根据关键图像序列以语言提供特定于故障的反馈。 DQAF [245] 在每一集之后为下一个操作员编写反馈。总体而言，检查的时机决定了其输出。在执行前检查在候选者中进行选择，在执行期间检查信号是否停止或调整，并在执行后检查解释发生了什么。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。
>
> 解析：本段并列了 AHA、REFLECT、DQAF、RoboLab。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：AHA [64]（辞典第65页）；REFLECT [223]（辞典第224页）；DQAF [245]（辞典第246页）；RoboLab [398]（辞典第399页）

原文第 37 页 · 段落 p37-173

中文原意：响应选择。响应选择决定系统在检测到问题后执行的操作。它涵盖三个问题：检测到的问题如何到达智能体、智能体如何选择响应以及是否在运行之前检查该响应。仅当选择下一个操作的智能体可以看到检测到的问题时，它才能改变行为。现有的工作要么将此链接构建到智能体的工具中，要么测试该链接是否影响以后的决策。机器人控制的智能体机器人工智能[202]采用第一种方法，为智能体提供查询机器人状态和事件的工具。诊断语义切换失败 [285] 是第二步，检查视觉可达性检查是否会改变智能体的下一个技能或路线。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。
>
> 解析：本段并列了 AARC、DSHF。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：AARC [202]（辞典第203页）；DSHF [285]（辞典第286页）

原文第 37 页 · 段落 p37-174

中文原意：一旦智能体看到问题，它必须选择响应。现有系统要么从一组固定的响应中进行选择，要么让规划者修改计划。固定响应简单且可预测，而规划者可以生成未提前列出的响应。 Goal2Skill [225] 在重试、调整和重新规划之间进行选择，而用于鲁棒多机器人操纵的闭环多智能体框架 [102] 将局部位姿修复与完全重新规划分开。在基于规划器的系统中，规划器是由对 REFLECT [223] 中失败的解释、WCM [51] 中预测结果与实际结果之间的差距以及 Stop Wandering [184] 中缺乏进展而触发的。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。
>
> 解析：本段并列了 WCM、CLMAF、MetaNav、REFLECT等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：WCM [51]（辞典第52页）；CLMAF [102]（辞典第103页）；MetaNav [184]（辞典第185页）；REFLECT [223]（辞典第224页）；Goal2Skill [225]（辞典第226页）

原文第 37 页 · 段落 p37-175

中文原意：选择的响应也可能是错误的，因此系统会验证建议的操作并使用执行证据来决定是否重试。这些检查依赖于感知、子目标证据或工具输入的有效性。 ETA [50]在无法定位目标时拒绝采取行动。 ACE [165] 使用跟踪的拾放掩模、执行报告和新观察来评估子目标完成情况并指导重试。 HarnessVLN [44] 在机器人移动或停止之前检查工具输入和证据。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。
>
> 解析：本段并列了 HarnessVLN、ETA、ACE。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：HarnessVLN [44]（辞典第45页）；ETA [50]（辞典第51页）；ACE [165]（辞典第166页）

原文第 37 页 · 段落 p37-176

中文原意：恢复操作。 一旦检测到故障，机器人就需要采取恢复操作来修复它。一般来说，它们的不同之处在于所改变的内容，即当前策略的输入、执行者、任务流或策略之外的东西。 VLA-Corrector [302] 改变输入，将阶段特定的恢复提示提供给冻结的 VLA。 FLARE [445]、VoLo [40] 和 ReCoVLA [106] 改变了执行器，切换到训练有素的重置适配器、由感知引导的手工设计技能，或针对每种已知故障类型的模拟训练校正策略。 AgentChord [395] 改变了任务流程，转移到编译到其任务图中的恢复分支，而无需再次调用 LLM。 SERP [171] 和 Dream2Fix [170] 改变了策略之外的一些东西，调整运动成本参数并应用小的学习物理修正。在所有情况下，恢复仅限于系统提供的操作。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。
>
> 解析：本段并列了 VoLo、ReCoVLA、Dream2Fix、SERP等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：VoLo [40]（辞典第41页）；ReCoVLA [106]（辞典第107页）；Dream2Fix [170]（辞典第171页）；SERP [171]（辞典第172页）；VLA-Corrector [302]（辞典第303页）；AgentChord [395]（辞典第396页）；FLARE [445]（辞典第446页）

原文第 37 页 · 段落 p37-177

中文原意：Recovery还需要选择从哪个状态恢复。失败的尝试通常会导致场景部分改变，物体移动或步骤完成一半。机器人可以从此状态继续并保持已经取得的进度，或者首先将场景恢复到已知状态并重新开始。 PhyAgentOS [221]从当前状态继续，诊断失败的尝试并使用新指令恢复。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：PhyAgentOS [221]（辞典第222页）

原文第 38 页 · 段落 p38-178

中文原意：HALTER [134] 首先恢复场景，使用场景图历史来查找已完成的内容，规划重置技能，并检查重置是否有效。 LIBERO-RECOVER [212] 分别评估这两种情况，比较从受干扰场景中的恢复与在恢复场景中开始的尝试。

> 解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。
>
> 解析：本段并列了 HALTER、LIBERO-RECOVER。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：HALTER [134]（辞典第135页）；LIBERO-RECOVER [212]（辞典第213页）

原文第 38 页 · 段落 p38-179

中文原意：最后，重复的尝试可能会再次失败，因此系统需要一个规则来确定何时停止。现有系统在固定预算后停止，将任务交给人工，仅在特定条件下重试，或者瞄准任务完成之外的终点。 PEACE [328] 允许规划者在无人机工具发生故障后仅在固定预算内重试。 LabVLA [280] 允许在离线工作流程 YAML 生成期间进行一次经过验证的重试，然后再返回工作流程进行人工审核。物理智能体抓取循环[353]仅在夹具为空时重试，否则停止或要求澄清。 SPINE [97] 修复机器人软件和设置，直到机器人准备好进行远程操作，而不是直到任务完成。相关工作比较见表15。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

解析：本段并列了 SPINE、LabVLA、PEACE、PAL。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：SPINE [97]（辞典第98页）；LabVLA [280]（辞典第281页）；PEACE [328]（辞典第329页）；PAL [353]（辞典第354页）

原表15 · 原文第38页

表 15. 监测和恢复工作的比较（第 7.3 节）。检查时间与检查的步骤相关：执行之前、期间或之后。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：HarnessVLN [44]；WCM [51]；AHA [64]；DoReMi [94]；SPINE [97]；CLMAF [102]；ReCoVLA [106]；HALTER [134]；MLRA [147]；CoVer [157]；ACE [165]；Dream2Fix [170]；SERP [171]；MetaNav [184]；DV-VLN [194]；Robometer [195]；AARC [202]；LIBERO-RECOVER [212]；PhyAgentOS [221]；REFLECT [223]；Goal2Skill [225]；DQAF [245]；LabVLA [280]；DSHF [285]；VLA-Corrector [302]；PEACE [328]；PAL [353]；SafeHarness [390]；AgentChord [395]；RoboLab [398]；FLARE [445]；OnFly [455]；Code-as-Monitor [458]。

7.4 Harness / Skills：什么成为持久能力

本节先抓住：生成一次代码、保存可复用技能、迭代技能库和训练模型权重是不同层次。入库前要检查前后置条件、接口、失败处理和独立任务表现。

贯穿例子：把抓杯代码换个名字存下来不算已证明通用技能；要能在新杯子位置和不同初始状态调用。

原文第 38、39 页 · 段落 p38-180

中文原意：编排、记忆和监控决定 运行支撑框架（Harness） 如何处理当前尝试;技能决定了它会给下一个带来什么。技能是 运行支撑框架（Harness） 可以加载或调用的可插入程序知识包。加载到上下文中后，文本技能将决定语言模型如何推理任务并使用 运行支撑框架（Harness） 的工具；作为函数调用，可执行技能为模型提供了可重用的操作在机器人上。在抽屉示例中，相同的体验可以成为从下面接近句柄的标注或可调用的 opendrawer 程序，并且可以在不重新训练模型的情况下进行交换。因此，技能不同于记录发生的事情的记忆，也不同于技能保持不变的模型权重。最近的一项调查根据是否使用执行反馈、持久技能记忆和程序搜索对代码即策略系统进行排名 [129]；相反，我们关注技能如何进入、在内部变化以及离开 运行支撑框架（Harness）。

解析：问可修改的东西究竟是什么：一次程序、持久代码库、运行上下文还是模型权重。代码保存下来与训练模型不同；修改监测代码还可能改变评判标准。

解析：记忆不是一个统一模块。辨认存的是图像、特征、语言摘要、物体状态还是代码，再追写入、检索、更新与过期；过去证据能否继续用，需要当前观测支持。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：Weights or Skills? [129]（辞典第130页）

原文第 39 页 · 段落 p39-182

中文原意：从技能到代码即策略。早期语言模型 运行支撑框架（Harnesses） 将技能视为一组固定的预训练基础动作（primitive），模型的作用只是在其中进行选择，如 SayCan [125] 中所示。代码生成改变了这个角色。在“代码即策略”[196]中，模型将感知和控制 API 组合到程序中，并动态定义缺失的功能，因此技能集成为词汇表而不是菜单。 ProgPrompt [300] 和 VoxPoser [118] 遵循相同的想法，将计划表达为带有断言的程序或为运动规划器构建值图的代码。因为这些程序是为一项任务编写的，然后被丢弃，所以后来的工作让程序在任务仍在运行时得到改进：Act-Observe-Rewrite [155] 在关键帧和日志的试验之间重写控制器，MMENTO [314] 搜索单个代码策略，Agent as Policy [132] 让编码智能体直接在物理机器人上编写和修改程序。因此，生成的代码成为任务中的有效策略，这就提出了跨任务保留什么的问题。

> 解析：问可修改的东西究竟是什么：一次程序、持久代码库、运行上下文还是模型权重。代码保存下来与训练模型不同；修改监测代码还可能改变评判标准。
>
> 解析：本段并列了 VoxPoser、SayCan、AGP、Act-Observe-Rewrite等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：VoxPoser [118]（辞典第119页）；SayCan [125]（辞典第126页）；AGP [132]（辞典第133页）；Act-Observe-Rewrite [155]（辞典第156页）；Code as Policies [196]（辞典第197页）；ProgPrompt [300]（辞典第301页）；MEMENTO [314]（辞典第315页）

原文第 39 页 · 段落 p39-183

中文原意：技能库和重用。重用是自然的答案，它需要组织技能，以便以后的任务可以找到它们。 Voyager [336] 在 Minecraft 智能体中设置了模式：程序一旦通过自我验证就加入库，新任务通过嵌入相似性来检索条目。技能库现在在其他智能体领域很常见，其中技能是从智能体经验中提炼出来的，并与 强化学习（Reinforcement Learning） [380] 一起发展，或者根据科学资源 [293] 和人工创建的多模式材料（例如教程、存储库和文章 [66]）构建。机器人系统采用了这种库结构，主要区别在于它们存储的内容以及如何接纳条目。RoboCoder [175] 存储视觉语言评估器接受的联合级动作代码，而 DROC [430] 和 ARCHITECT [34] 存储从人类校正中提取的知识，因此不需要两次进行相同的校正。 Uni-Skill [385] 通过实施视频衍生存储库中缺失的技能来填补其库中的空白，而 Teach and Grow [249] 仅在技能通过了对保留状态的范围结果测试后才承认该技能。库甚至可以构建一次并冻结，就像 CaP-X [74] 那样，使用从成功的 执行轨迹（rollout） 中提取的九个辅助函数，或者压缩为宏，如 SkillComposer [374] 中。由于这些图书馆中的条目在入馆时都会受到检查，并且很少重新访问，因此有缺陷的技能会一直存在，直到有人对其进行修改为止。

> 解析：问可修改的东西究竟是什么：一次程序、持久代码库、运行上下文还是模型权重。代码保存下来与训练模型不同；修改监测代码还可能改变评判标准。
>
> 解析：本段并列了 ARCHITECT、Resource2Skill、CaP-X、RoboCoder等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：ARCHITECT [34]（辞典第35页）；Resource2Skill [66]（辞典第67页）；CaP-X [74]（辞典第75页）；RoboCoder [175]（辞典第176页）；Teach and Grow [249]（辞典第250页）；SkillFoundry [293]（辞典第294页）；Voyager [336]（辞典第337页）；SkillComposer [374]（辞典第375页）；SkillRL [380]（辞典第381页）；Uni-Skill [385]（辞典第386页）；DROC [430]（辞典第431页）

原文第 39 页 · 段落 p39-184

中文原意：技能进化。技能进化通过将执行结果反馈回库来提供修订。早期系统通过简单的循环修改了技能：LRLL [327] 定期将积累的经验抽象为新功能，而 LYRA [240] 依赖于人类修正，同时重新测试早期的行为。自动化修订器提出了一个新问题，因为编辑自己技能的智能体也会使它们变得更糟，因此最近的系统主要在修订必须通过的大门上有所不同。 ASPIRE [228] 让编码智能体从每个基元跟踪中调试程序，并仅促进验证和泛化的修复； RAT[435]在多次成功后提升一项技能，并在多次失败后贬低它； RoboRSI [373] 将每次失败追溯到任务技能层次结构中最早的失败节点，并在通过审查、功能测试和最近运行的重播（不回归）后保留补丁。 SkillOpt [402] 与训练进行了明确的类比，将技能文档视为冻结智能体的可训练状态，并且仅当它提高了保留的验证分数时才接受编辑，而 Reconstruct、Practice、Go Real [358] 在机器人练习任务中应用了类似的规则，VASO [403] 用模型检查器反例替换了分数。相同的循环从技能延伸到它们周围的 运行支撑框架（Harness）代码，如 RHO [65] 和 Zetta [60]，它会改变整个策略存储库，而 Zetta [60] 使用智能体来诊断失败的 执行轨迹（rollout） 并发展经过验证的批评者恢复代码，该代码监视执行并触发本地恢复，同时保持基本 VLA 冻结。如果没有这样的门，更新就不需要累积：探索、执行、进化[388]报告连续更新的非单调进度，即使有重播、配对和保留检查，一项123轮的研究发现接受的更改并不能保证其目标任务的成功[339]。因此，有用的技能库既取决于修订版本的测试和停用方式，也取决于修订版本的生成方式。

> 解析：问可修改的东西究竟是什么：一次程序、持久代码库、运行上下文还是模型权重。代码保存下来与训练模型不同；修改监测代码还可能改变评判标准。
>
> 解析：本段并列了 Zettaζ、RHO、ASPIRE、LYRA等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：Zettaζ [60]（辞典第61页）；RHO [65]（辞典第66页）；ASPIRE [228]（辞典第229页）；LYRA [240]（辞典第241页）；LRLL [327]（辞典第328页）；WSRSI [339]（辞典第340页）；RPG [358]（辞典第359页）；RoboRSI [373]（辞典第374页）；RoboSkill [388]（辞典第389页）；SkillOpt [402]（辞典第403页）；VASO [403]（辞典第404页）；RATs [435]（辞典第436页）

原文第 40 页 · 段落 p40-185

中文原意：从技能进化到训练。所有这些机制都使写入技能的模型保持不变，这就提出了技能经验是否可以流入权重的问题。在实践中，大多数这样做的系统都是训练执行者而不是编码者。 Agentic Skill Discovery [450] 为每个提出的技能训练强化学习策略，RoboClaw [178] 和 ENPIRE [383] 从收集的或智能体设计的训练运行中微调机器人策略，HarnessPAI [356] 和 RACaP [192] 将 运行支撑框架（Harness） 轨迹提取为 VLA 或更小的决策模型。对编码员本身进行训练的情况较少。 RoboCodeX [242] 和 Uni-Skill [385] 对策划或视频衍生程序的代码生成模型进行微调，而 CaP-X [74] 将带有环境奖励的 强化学习（Reinforcement Learning）应用于三个模拟任务的编码模型，然后将其转移到物理手臂上。在自己的 执行轨迹（rollout） 上持续训练多模态编码器，或者将进化的技能库折叠到其权重中，将关闭技能进化和模型学习之间的循环。相关工作比较见表16。

> 解析：问可修改的东西究竟是什么：一次程序、持久代码库、运行上下文还是模型权重。代码保存下来与训练模型不同；修改监测代码还可能改变评判标准。
>
> 解析：本段并列了 CaP-X、RoboClaw、RACaP、RoboCodeX等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：CaP-X [74]（辞典第75页）；RoboClaw [178]（辞典第179页）；RACaP [192]（辞典第193页）；RoboCodeX [242]（辞典第243页）；HarnessPAI [356]（辞典第357页）；ENPIRE [383]（辞典第384页）；Uni-Skill [385]（辞典第386页）；Agentic Skill Discovery [450]（辞典第451页）

原表16 · 原文第40页

表 16. 技能著作比较（第 7.4 节），按文本的四个部分分组。更新将代码/库更改与策略训练区分开来。 NR：没有报道。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：ARCHITECT [34]；CaP-X [74]；VoxPoser [118]；AGP [132]；Act-Observe-Rewrite [155]；LTCLRS [156]；RoboCoder [175]；Code as Policies [196]；RAPID [222]；LYRA [240]；Teach and Grow [249]；EmbodiedSWE [294]；ProgPrompt [300]；IMR-LLM [306]；MEMENTO [314]；LRLL [327]；Voyager [336]；SkillComposer [374]；Uni-Skill [385]；VASO [403]；DROC [430]。

原表16 · 原文第41页

表 16（续）

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：Zettaζ [60]；RHO [65]；RoboClaw [178]；RACaP [192]；ASPIRE [228]；RoboCodeX [242]；WSRSI [339]；SHAPER [347]；HarnessPAI [356]；RPG [358]；ENPIRE [383]；RoboSkill [388]；AgenticRobotics [417]；RATs [435]；Agentic Skill Discovery [450]。

7.5 Harness / Infrastructure：接口、时间与边界

本节先抓住：工具协议不提供自动安全保证；模型调用需要可执行契约、观测时间戳和运行约束。加速模型与保证系统动作及时性也不是一回事。

贯穿例子：一个动作块按旧帧生成，网络延迟后到达机器人；需要调度或实时分块处理，而不是单看每秒token数。

原文第 41 页 · 段落 p41-186

中文原意：在技能之下，每个 运行支撑框架（Harness） 功能都依赖于软件基础设施：实现机器人功能的接口、该接口背后的感知和操作工具、运行学习策略的服务以及在提议的操作和硬件之间进行的检查。在抽屉示例中，opendrawer 可以扩展为查找句柄的分段调用、提出位姿的抓取生成器、到达句柄的运动规划器以及在执行拉动的远程 GPU 上提供服务的 VLA。这些组件中的每一个都可能发生故障、增加延迟或向智能体隐藏信息。我们首先讨论接口，然后讨论主导实际系统的工具链和策略服务，最后讨论运行时门控。

解析：把模型生成时间、网络等待和机器人执行时间分开。核对新输出是否针对当前状态，控制权由什么事件交回；本段频率或时延只描述相应配置。

解析：比较时记录任务、执行接口、工具、预算与重置条件。动作接口、工具调用、代码生成和离线问答测的是不同能力，不能只按一个平均数混排。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

原文第 41 页 · 段落 p41-187

中文原意：能力接口。 该基础设施的第一层是接口，它确定智能体可以表达的内容以及它学到的内容。EmbodiedTool [466] 使用功能卡注册了一百多个工具，这些功能卡说明了输入、输出和触发条件；座席几乎总是会形成有效的呼叫，但选择正确工具的频率要低得多。返回通道与调用一样重要：NavMCP [167] 结合了结构化意图、基于关键帧的旅程摘要和持久记忆；用情景界面替换这个完整的三通道协议会将问答准确率从 74.0% 降低到 59.1%。粒度是进一步的选择。 Show-运行支撑框架（Harness） [46]公开了解释器投影到安全设定点的语义笛卡尔步骤，并且将步长减半将零样本成功率从 60% 提高到 82%。模型上下文协议正在成为一种常见的传输：RoboNeuron [90] 从 ROS 2 消息模式派生类型化 MCP 工具，ROS MCP 服务器 [284] 通过 rosbridge 公开主题、服务和操作。在机器人方面，Cardenas 等人的 ROSClaw [26] 在调度之前检查每个调用是否符合速度范围和接口许可名单，而 EMOS [37] 从机器人的 URDF 和正向运动学而不是手写文本中得出功能配置文件。

解析：把模型生成时间、网络等待和机器人执行时间分开。核对新输出是否针对当前状态，控制权由什么事件交回；本段频率或时延只描述相应配置。

解析：本段并列了 ROSClaw、EMOS、Show-Harness、RoboNeuron等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：ROSClaw [26]（辞典第27页）；EMOS [37]（辞典第38页）；Show-Harness [46]（辞典第47页）；RoboNeuron [90]（辞典第91页）；NavMCP [167]（辞典第168页）；ROS MCP Server [284]（辞典第285页）；EmbodiedTool [466]（辞典第467页）

原文第 42 页 · 段落 p42-188

中文原意：感知和操作工具链。 在界面背后，代码即策略 运行支撑框架（Harnesses） 通过一系列预训练工具起作用：开放词汇检测和分割，例如 落地关联（Grounding） DINO [216]、SAM 2 [276] 和 SAM 3 [27]；指向模型，例如 Molmo [58]；姿态和深度估计，例如 FoundationPose [368] 和 FoundationStereo [369]；掌握生成器，例如 AnyGrasp [67]、ContactGraspNet [313]、M2T2 [423] 和 GraspGen [243]；以及 GPU 运动规划器，例如 cuRobo [312]。 CaP-X [74] 将此类堆栈公开为 Python API 背后的无状态服务，其基准测试将接收特权模拟器掩模和位姿的层与必须从 RGB-D 感知的层分开。 GaP [38] 在单个 gRPC 方法背后组织了 51 种感知、抓取、运动和验证技能，具有位姿、掩模、点云和轨迹的共享类型，并将它们转换为 ROS 进行物理执行。工具也开始提供面向智能体的端点：GraspGen 提供了一个 GPU 服务器和一个 MCP 包装器，以便 LLM 智能体可以在没有本地模型的情况下请求抓取。物理智能体[75]说明了完整的物理部署：检测、分割、立体深度、抓取生成以及任务和运动规划作为服务器在一个工作站上运行，而实时控制计算机驱动手臂。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

解析：本段并列了 SAM 3、GaP、Molmo / PixMo、AnyGrasp等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：SAM 3 [27]（辞典第28页）；GaP [38]（辞典第39页）；Molmo / PixMo [58]（辞典第59页）；AnyGrasp [67]（辞典第68页）；CaP-X [74]（辞典第75页）；Physical Agency [75]（辞典第76页）；Grounding DINO [216]（辞典第217页）；GraspGen [243]（辞典第244页）；SAM 2 [276]（辞典第277页）；cuRobo [312]（辞典第313页）；Contact-GraspNet [313]（辞典第314页）；FoundationPose [368]（辞典第369页）；FoundationStereo [369]（辞典第370页）；M2T2 [423]（辞典第424页）

原文第 42 页 · 段落 p42-189

中文原意：这些工具链决定了 运行支撑框架（Harness） 可以实现的大部分功能。感知是几个系统中物理故障的主要来源：VoxPoser [118] 的大多数真实机器人错误来自感知，点跟踪是 ReKep [120] 中最大的错误源，OK-Robot [215] 展示了当现成的检测、分割、映射和抓取被链接而无法恢复时，错误如何复合。特权国家隐藏了这一成本。 Kuroki 和 Tang [156] 报告称，完整的模拟器观察成功率为 65.0%，但单独使用 RGB 成功率为 22.5%，并且 CaP-X 在特权层上训练其编码模型，以避免在转移到感知输入之前产生感知噪声。当工具链报告故障时，智能体可以修复工具链。 ASPIRE [228] 提供了一个具有原始级别跟踪、调试和修复验证的执行引擎；添加该引擎将评估的 LIBERO-Pro 扰动的宏观平均成功率从 14% 提高到 62%。当夹具宽度传感器显示它是空的时，物理机构返回键入的失败并覆盖报告的抓取。在整个系统中，编码智能体重复地在工具周围添加相同的强化，包括从文本提示到点提示分割的后备链、抓取方向的过滤器、抓取前的反向运动学检查以及传感器检查的抓取； RHO [65] 和 ASPIRE 将此类逻辑移至工具包装器本身中。因此，报告不确定性和故障的工具接口，以及将特权与感知输入分开的评估，使工具链结果在 运行支撑框架（Harnesses） 之间具有可比性。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

解析：本段并列了 RHO、VoxPoser、ReKep、LTCLRS等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：RHO [65]（辞典第66页）；VoxPoser [118]（辞典第119页）；ReKep [120]（辞典第121页）；LTCLRS [156]（辞典第157页）；OK-Robot [215]（辞典第216页）；ASPIRE [228]（辞典第229页）

原文第 42 页 · 段落 p42-190

中文原意：策略推理和服务. 学习的策略形成了该堆栈的另一半：当 运行支撑框架（Harness） 调用 VLA 时，该策略成为具有自己的延迟和容量的服务。 openpi 远程推理接口 [267] 通过 WebSocket 提供检查点，返回操作块，使机器人代码不受模型依赖性的影响，而 LeRobot [23] 将策略服务器与机器人客户端分开，当操作队列不足时，机器人客户端会请求新块。异步执行隐藏了推理延迟：SmolVLA [299] 使用这种设计以较小的成功成本将任务时间缩短约 30%，实时分块 [14, 15] 冻结推理期间提交的操作，而 VLASH [315] 条件是这些操作将达到的状态。布局是进一步的设计维度。 VLA-Perf [136] 对跨设备和网络的延迟进行建模，发现对于 π0，通过 WiFi 的边缘服务器可以胜过设备上的推理； Gemini Robotics [81] 将云主干网与机器人解码器相结合，而 RoboECC [457] 根据预测带宽在边缘和云之间划分 VLA。对于车队，ROSA [135] 在延迟目标下在共享 GPU 池上调度 VLA 和 VLM 组件，Robion [1] 在视觉语言和动作阶段之间划分一个 GPU，为大约 20 个机器人提供服务，而 Armoury [7] 在以不同速率运行的机器人之间调度批量动作块推理。运行时工程缩小了剩余的差距：内核级优化在消费级 GPU [234] 上将π 0 缩短到 27 毫秒，OxyGen [181] 在动作头和语言头之间共享 VLA 缓存，vla.cpp [248] 在嵌入式板上运行量化的 VLA，BitVLA [337] 将 4 位基线与三元权重进行匹配。这些系统主要根据策略吞吐量进行评估；将它们与智能体 运行支撑框架（Harnesses） 结合将使服务延迟成为规划预算的一部分。

解析：把模型生成时间、网络等待和机器人执行时间分开。核对新输出是否针对当前状态，控制权由什么事件交回；本段频率或时延只描述相应配置。

解析：本段并列了 Robion、Armory、RTC、Training-time RTC等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：Robion [1]（辞典第2页）；Armory [7]（辞典第8页）；RTC [14]（辞典第15页）；Training-time RTC [15]（辞典第16页）；LeRobot [23]（辞典第24页）；Gemini Robotics [81]（辞典第82页）；ROSA [135]（辞典第136页）；VLA-Perf [136]（辞典第137页）；OxyGen [181]（辞典第182页）；Running VLAs at Real-time Speed [234]（辞典第235页）；vla.cpp [248]（辞典第249页）；openpi Remote Inference [267]（辞典第268页）；SmolVLA [299]（辞典第300页）；VLASH [315]（辞典第316页）；BitVLA [337]（辞典第338页）；RoboECC [457]（辞典第458页）

原文第 42 页 · 段落 p42-191

中文原意：运行时门控和版本控制。一旦可以访问工具和策略，运行支撑框架（Harness） 还必须决定哪些建议的命令可以到达硬件，并且此检查最好放置在模型之外。

解析：先把作者的分类判断与实验结论分开。分类说明工作在系统里起什么作用；提升有多大、适用哪些条件，仍要查看该工作给出的实验协议。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

原文第 43 页 · 段落 p43-192

中文原意：RoboPAIR [283] 通过在每个尝试场景中越狱自动驾驶语言模型、GPT-4o 计划的地面机器人和商业四足机器人来激发这种布局。Safety Chip [405] 将经过专家验证的时间逻辑约束编译成自动机，在执行前修剪不安全的操作，而 RoboGuard [277] 根据隔离模型导出的规范对计划进行模型检查，将越狱成功率从 92.3% 降低到 2.3%。只要经过验证的模板未涵盖危险，SafeGate [255] 就会在未覆盖的危险情况下拒绝继续并交给用户处理（fail closed），并且 RoboSafe [344] 根据生成的可执行谓词检查每个操作。接口处的其他系统门：ETA [50] 将智能体想要做的事情与接口允许的事情分开，SafeHarness [390] 拒绝违反障碍物间隙的路线，以及 Hong 等人的约束 LLM 设计。 [104]让模型仅陈述象征性意图。相反，CORE [278]通过控制屏障函数在控制器中强制推断约束。当 运行支撑框架（Harness） 本身发生变化时，同样的门也适用。 ETA 要求修订后的 运行支撑框架（Harness） 通过接口契约检查、单独审查、重播和留出评测，但在其报告的运行中没有候选方案通过； Harness Robotic OS [397] 提出版本化、可回滚的更新，而无需演示修改后的可执行文件。相关工作比较见表17。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

解析：本段并列了 ETA、BSP、SafeGate、RoboGuard等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：ETA [50]（辞典第51页）；BSP [104]（辞典第105页）；SafeGate [255]（辞典第256页）；RoboGuard [277]（辞典第278页）；CORE [278]（辞典第279页）；RoboPAIR [283]（辞典第284页）；RoboSafe [344]（辞典第345页）；SafeHarness [390]（辞典第391页）；Harness Robotic OS [397]（辞典第398页）；Safety Chip [405]（辞典第406页）

原表17 · 原文第43页

表 17. 基础设施工程比较（第 7.5 节）。包括支持工具和部署系统，它们不必是自主智能体。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：RTC [14]；Training-time RTC [15]；LeRobot [23]；GaP [38]；Gemini Robotics [81]；RoboNeuron [90]；ROSA [135]；VLA-Perf [136]；OxyGen [181]；OK-Robot [215]；Running VLAs at Real-time Speed [234]；vla.cpp [248]；SafeGate [255]；RoboGuard [277]；CORE [278]；RoboPAIR [283]；SmolVLA [299]；VLASH [315]；BitVLA [337]；RoboSafe [344]；Safety Chip [405]；RoboECC [457]。

原表17 · 原文第44页

表 17（续）

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：Robion [1]；Armory [7]；Show-Harness [46]；ETA [50]；BSP [104]；Harness Robotic OS [397]。

7.6 Harness / Layered Systems：分层接口是什么

本节先抓住：本综述的System 0/1/2是作者采用的分析标签，重点在显式控制、快速学习策略和慢速推理的接口。分层既带来模块优势，也引入延迟、子目标偏差和交接问题。

贯穿例子：高层说“抓红杯”，中层产生动作块，底层执行关节控制；每层频率、失败信号和验证职责都应明确。

原文第 44 页 · 段落 p44-193

中文原意：分层系统将上面的工具和策略组织成以不同速率运行的层。在抽屉示例中，规划器决定打开抽屉，VLA 伸手去拿把手，逆运动学解算器以各自的速率跟踪拉力。我们将这个堆栈描述为三层：系统 2 规划器（VLM、LLM 或生成的代码），对任务和子目标进行推理；系统 1 学习视觉运动策略，通常是 VLA，以控制速率做出反应；以及系统 0 分析控制器，例如逆运动学、运动规划、模型预测控制和全身控制。系统 1 和 2 遵循双进程理论的快慢区别 [141, 304]。系统 0 采用 Taniguchi 等人的标签。 [317]用于形态学计算；这里它表示显式分析控制，我们避免使用系统 3，因为它已经指的是集体智慧和运行时批评家 [413]。现有的 VLA 调查将整体模型与分层模型进行对比 [291]，但将分析控制排除在比较之外。 运行支撑框架（Harness）是连接这三个层的层，本小节将研究它在它们之间传递的内容、频率以及哪一层拥有权限。

> 解析：把模型生成时间、网络等待和机器人执行时间分开。核对新输出是否针对当前状态，控制权由什么事件交回；本段频率或时延只描述相应配置。
>
> 解析：本段并列了 Thinking, Fast and Slow、VLA Survey、Dual-Process Theory、System 0/1/2/3等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：Thinking, Fast and Slow [141]（辞典第142页）；VLA Survey [291]（辞典第292页）；Dual-Process Theory [304]（辞典第305页）；System 0/1/2/3 [317]（辞典第318页）；Critic in the Loop [413]（辞典第414页）

原文第 44 页 · 段落 p44-194

中文原意：层接口和速率。第一个设计选择是每层向下传递的内容和频率。 Helix [72] 将单个潜在向量从 7-9 Hz 的视觉语言模型写入 200 Hz 策略，GR00T N1 [254] 将中间嵌入从 10 Hz 推理模块传递到产生 120 Hz 块的动作模块，由全身逆运动学控制器跟踪。 Ground Slow, Move Fast [367] 使所有三层都变得明确：2 Hz 规划器输出像素目标，30 Hz 策略产生航航路点，模型预测控制跟踪它们。其他系统传递可解释的信号。 Hi Robot [297] 大约每秒约一次或当用户插入时发送语言子任务，π0.5 [269] 在一个模型内的动作之前解码子任务文本，NaVILA [52] 将参数化语言命令发送到运动策略，而 HAMSTER [185] 每个回合发送一次几个点映射（pointmap）像路径。在受控比较中，VLA-OS [77] 发现低层策略比语言计划更忠实地遵循视觉计划。由于慢速层的输出在使用时已过时，RoboDual [21] 和 HiRT [433] 在延迟调节上训练快速层，而 HarnessWAM [89] 触发对进度事件的审议，这优于以固定间隔调用规划器。潜在接口携带更多信息，但对监控不透明；文本、路径和航点的检查可以牺牲它们所能表达的内容。

> 解析：把模型生成时间、网络等待和机器人执行时间分开。核对新输出是否针对当前状态，控制权由什么事件交回；本段频率或时延只描述相应配置。
>
> 解析：本段并列了 RoboDual、NaVILA、Helix、VLA-OS等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：RoboDual [21]（辞典第22页）；NaVILA [52]（辞典第53页）；Helix [72]（辞典第73页）；VLA-OS [77]（辞典第78页）；HarnessWAM [89]（辞典第90页）；HAMSTER [185]（辞典第186页）；GR00T N1 [254]（辞典第255页）；π0.5 [269]（辞典第270页）；Hi Robot [297]（辞典第298页）；DualVLN [367]（辞典第368页）；HiRT [433]（辞典第434页）

原文第 44、45 页 · 段落 p44-195

中文原意：智能体将 VLA 称为工具。 除了集成的双系统模型之外，分层系统还可以由单独的模型组装而成，其中 VLA 作为工具之一公开。 Gemini Robotics 1.5 [320]通过语言指令使其动作模型可用于编排推理模型，并且用具体推理编排器替换通用模型使总失败率减半。 运行支撑框架（Harness） VLA [441]将冻结的VLA放置在分析运动基元旁边的vlaact基元后面并验证每个调用； VLA 占其操纵基准的少数呼吁。物理智能体[75]让智能体在任务和运动规划和VLA之间进行选择，并覆盖与抓取感知相矛盾的报告成功。 VLAs-as-Tools [166]返回进度信号，让监视器通过每个回合的几个规划器调用来推进或重新规划。对分层 VLA 智能体的系统研究[108] 发现终止规则和 VLA 可操纵性与规划器大小一样重要。与集成式双系统模型相比，这种安排使每一层都可替换，每个呼叫都可检查，但每次切换都必须重新建立状态和意图，如编排中所讨论的。

解析：分开看发现异常、解释原因和实施修复。检测到失败不等于有正确修复；重试预算、停止条件及人工接管都影响完整任务成本。

解析：本段并列了 Physical Agency、WMORP、VLAs-as-Tools、Gemini Robotics 1.5等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：Physical Agency [75]（辞典第76页）；WMORP [108]（辞典第109页）；VLAs-as-Tools [166]（辞典第167页）；Gemini Robotics 1.5 [320]（辞典第321页）；Harness VLA [441]（辞典第442页）

原文第 45 页 · 段落 p45-197

中文原意：决策 API。 在此类协调器及其执行器之间，最近的一项开发添加了快速决策层。决策模型针对提供的一组选项回答有界问题，而不是生成文本。 TypeSafe 的 Jev [326] 被描述为 System One 模型，接受文本状态和一批问题，要求它在选项中进行选择，将案例按顺序排列，或判断条件是否成立，并返回每个答案的概率，而不产生输出标记。 Liquid AI 的 d1 [204] 在一次前向传递中提供了相同的三种关于文本和图像的问题类型，文本决策在 200-300 毫秒内完成，OpenAI 也宣布了一个类似的决策界面 [259]。在机器人 运行支撑框架（Harness） 中，这样的模型位于较慢的规划器之下：代码提出合法候选者，决策模型以概率选择一个，而运行支撑框架（Harness） 仅在高于置信阈值时起作用。 EmbodiedJev [69] 是一个开源仿真工作台，将此协议应用于操作，让 Jev 在技能、阶段或笛卡尔步骤中进行选择，而逆向运动学、安全检查和成功判断仍保留在代码中。将语言模型限制为对候选者进行评分的想法早在这些产品出现之前就出现了。 SayCan [125] 将语言模型可能性与在固定技能库上学习的 可供性（Affordances） 相结合，MOKA [207] 要求 VLM 在标记的关键点中进行选择，KnowNo [279] 和 IntroPlan [198] 将选项可能性转换为触发帮助请求的保形预测集。专用的决策模型使该模式足够便宜，可以在每一步中运行。一旦根据机器人自己的决定进行校准，它们的阈值就有意义，因为候选方案的概率表达了对所提供选项的偏好，而不是实际成功的机会。

解析：把模型生成时间、网络等待和机器人执行时间分开。核对新输出是否针对当前状态，控制权由什么事件交回；本段频率或时延只描述相应配置。

解析：本段并列了 EmbodiedJev、SayCan、IntroPlan、d1等。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：EmbodiedJev [69]（辞典第70页）；SayCan [125]（辞典第126页）；IntroPlan [198]（辞典第199页）；d1 [204]（辞典第205页）；MOKA [207]（辞典第208页）；Decisions, 2026 [259]（辞典第260页）；KnowNo [279]（辞典第280页）；Jev [326]（辞典第327页）

原文第 45 页 · 段落 p45-198

中文原意：代码作为上层和 Outlook。在堆栈的顶部，代码本身可以充当系统 2。技能下讨论的 codeas-policies 行在生成的程序中组成系统 0 基础动作（primitive）； ReKep [120] 通过让 VLM 每个任务编写一次关键点约束函数，其求解器以大约 10 Hz 运行，关键点跟踪和位置控制以 20 Hz 运行，从而强化了分工； Instruct2Act [117] 在生成的代码中组成感知模型和运动基元。 AgentChord [395] 使用预编译的恢复分支和约束求解器来编译受监控的任务图，以便在不调用新的语言模型的情况下处理检测到的违规行为。调用 VLA 和分析基础动作（primitive）的程序（如 运行支撑框架（Harness） VLA 中）将两者结合起来：代码保持可检查和可验证，而 VLA 涵盖难以指定的接触丰富的段。一个自然的方向是堆栈，其中 VLA 作为可调用的下层，程序提供上层，智能体通过技能进化离线修改两者。本章中的每个选择都会在该堆栈中产生成本。执行者之间的桥梁扩大了后继状态，但会消耗动作时间；检索到的历史保留了承诺，但可能会传播陈旧的证据;可编辑的代码可以修复重复出现的故障，但也可以更改判断它的监视器。最强烈的比较会隔离已更改的层或接口，保持其他层或接口固定，并测试超出用于优化的反馈的后续行为。相关工作比较见表18。

解析：把模型生成时间、网络等待和机器人执行时间分开。核对新输出是否针对当前状态，控制权由什么事件交回；本段频率或时延只描述相应配置。

解析：本段并列了 Instruct2Act、ReKep、AgentChord。并列引用首先表示它们支持同一讨论主题，不能推断内部实现一样或已有直接对照实验；工作卡会标明本综述实际提供了哪些区别。

自问：这一变化放在运行系统哪一层，谁接收它，使用什么反馈更新？

本段引用：Instruct2Act [117]（辞典第118页）；ReKep [120]（辞典第121页）；AgentChord [395]（辞典第396页）

原表18 · 原文第45页

表 18. 分层系统上的工作比较（第 7.6 节）。更新时序标识刷新的模块；费率参考报告的配置。建筑研究列出了替代方案。 NR：没有报道。

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：Instruct2Act [117]；ReKep [120]；SayCan [125]；IntroPlan [198]；MOKA [207]；KnowNo [279]；HiRT [433]。

原表18 · 原文第46页

表 18（续）

整页原表可在本PDF末尾原文附录或网页版打开；图标不据提取空白推断。

本页条目：RoboDual [21]；NaVILA [52]；VLA-OS [77]；WMORP [108]；HAMSTER [185]；GR00T N1 [254]；π0.5 [269]；Hi Robot [297]；Gemini Robotics 1.5 [320]；DualVLN [367]；Critic in the Loop [413]；Harness VLA [441]。

8 展望：智能体式机器人（Agentic Robots）的下一步是什么？

原文第 46 页 · 段落 p46-199

中文原意：智能体式机器人（Agentic Robots） 将 通用（General-Purpose） 推理模型与机器人策略、工具和物理执行连接起来。然而，集成这些组件并不一定会让机器人系统变得更强大。仍然很难确定智能体的贡献，其决策是否可以在物理机器人上可靠地执行，以及系统在有限资源或不熟悉的平台上的表现如何。这些挑战激发了四个研究方向。

解析：这部分提出研究问题，不能当成已解决的结论。把建议转成可比较实验：固定任务与资源，明确要改变的模块，再记录收益和新增成本。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

8.1 展望 / 公平评测

本节先抓住：固定任务、资源与可用工具，分别消融模型、提示、执行器、记忆和验证器。来自同一闭环的自评信号不能替代独立测试。

贯穿例子：先保持VLA不变，只添加恢复模块，才能解释提升是否来自恢复。

原文第 46 页 · 段落 p46-200

中文原意：关于智能体机器人，任务成功率实际上告诉我们什么？考虑在相同的操作基准上评估的两个系统。一种可能依赖于更强大的 VLA 执行器，而另一种则使用智能体来协调多个较弱的技能、收集额外的观察结果并从故障中恢复。相似的分数可以反映出截然不同的能力。 CaP-X [74] 评估生成机器人控制程序的智能体，而 运行支撑框架（Harness） VLA [441] 和 RoboHarness [116] 研究智能体如何协调机器人策略和技能。他们的结果不仅取决于智能体，还取决于底层执行者、可用工具、观察访问和允许的重试。因此，任务的成功本身并不能表明系统的哪一部分发挥了作用。

解析：同分不等于同能力。把模型、执行器、工具、额外观察、重试和时间预算记录下来；固定这些条件做模块消融，再看完整系统是否值得额外成本。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

本段引用：CaP-X [74]（辞典第75页）；RoboHarness [116]（辞典第117页）；Harness VLA [441]（辞典第442页）

原文第 46 页 · 段落 p46-201

中文原意：为了公平地比较智能体机器人，我们需要在可比较的条件下检查他们的决定。使用相同的执行器、观察访问和交互预算可以更轻松地评估规划、工具选择和恢复。比较有和没有记忆或重新规划的系统可以进一步显示每个组件的贡献。然而，仅进行受控比较是不够的，因为选择和组合不同的模型和工具本身就是一项重要的能力。因此，端到端评估应报告每个系统可用的资源，包括额外的观察结果、计算成本、执行时间和人工协助。这种比较可以揭示额外推理和工具使用的好处是否值得其成本。

解析：同分不等于同能力。把模型、执行器、工具、额外观察、重试和时间预算记录下来；固定这些条件做模块消融，再看完整系统是否值得额外成本。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 46、47 页 · 段落 p46-202

中文原意：即使经过仔细比较，如果智能体的任务可以通过单个固定策略完成，基准测试也可能无法告诉我们有关该智能体的信息。一些机器人基准测试专注于任务完成，而不要求智能体处理丢失的信息、不合适的执行器或失败的操作。提供更多信息的基准可以引入意外的环境变化、不完整的观察以及需要跨不同技能协调的任务。对于收集数据、更新模型或开发新技能的智能体，评估还应该检查这些变化是否提高了看不见的任务的性能。这将有助于确定智能体何时有用以及其改进是否普遍超出原始设置。

解析：同分不等于同能力。把模型、执行器、工具、额外观察、重试和时间预算记录下来；固定这些条件做模块消融，再看完整系统是否值得额外成本。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

8.2 展望 / 真机部署

本节先抓住：模拟结果提供线索，真机还涉及标定、传感噪声、接触、延迟与人工干预。只展示成功视频不能估计稳定性。

贯穿例子：应报告失败、重置成本与干预率，而不仅是一条任务完成演示。

原文第 47 页 · 段落 p47-204

中文原意：对智能体有意义的计划可能并不总是可以由机器人执行。推理模型可能会要求机器人掌握超出机器人能力范围的东西，或者根据过时的观察结果选择一项技能。 智能体式机器人（Agentic Robots） 可以结合推理模型、VLA、世界动作模型（World-Action Models） 和专门的机器人技能，每种技能都有不同的能力和执行要求。 ReKep [120] 使用模型生成的约束来指导机器人运动，而 运行支撑框架（Harness） VLA [441] 允许智能体与其他机器人操作一起调用 VLA。然而，选择合适的执行器只是问题的一部分。有效的命令在物理执行过程中仍然可能会失败，并且在智能体收到反馈之前场景可能会发生变化。

> 解析：语言上合理的步骤不保证物理可行。例如目标位姿可能过期，停止请求也可能尚未完成。接口应反馈当前状态与执行状态，真机感知误差还需要闭环纠正。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

本段引用：ReKep [120]（辞典第121页）；Harness VLA [441]（辞典第442页）

原文第 47 页 · 段落 p47-205

中文原意：因此，机器人界面需要描述的不仅仅是可用技能的名称和输入。智能体应该知道何时可以使用技能、需要什么条件以及如何判断执行是否已完成。例如，失败的抓取可能需要另一次观察、修改抓取命令或不同的技能。正确的响应取决于实际发生的情况：对象可能已移动，或者上一个命令可能仍在运行。发送停止请求并不能保证机器人已停止移动。在重新计划之前，智能体需要反馈来确认当前状态以及已采取的操作的效果。

> 解析：语言上合理的步骤不保证物理可行。例如目标位姿可能过期，停止请求也可能尚未完成。接口应反馈当前状态与执行状态，真机感知误差还需要闭环纠正。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 47 页 · 段落 p47-206

中文原意：当从模拟机器人转向物理机器人时，这些困难变得更难解决。模拟器可以提供准确的物体位置和明确定义的成功条件，而真实的机器人通常必须根据嘈杂的相机图像和传感器读数来估计这些。微小的感知错误、校准差异和意外接触都可能导致原本合理的计划失败。未来的系统应该认识到他们的观察何时不足，在执行之前检查重要的假设，并使用物理反馈来纠正他们的决策。真实的机器人评估还应该明确智能体接收到哪些信息，低层的控制器处理什么，以及智能体在其假设被证明是错误的情况下是否可以恢复。

> 解析：语言上合理的步骤不保证物理可行。例如目标位姿可能过期，停止请求也可能尚未完成。接口应反馈当前状态与执行状态，真机感知误差还需要闭环纠正。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

8.3 展望 / 效率

> 本节先抓住：计算成本需要按完整任务测量，包括多次观察、推理、重试、规划与控制等待。快慢模型协作的收益要放在成功率与时延共同指标下。
>
> 贯穿例子：模型单次更快但重试更多，总任务耗时仍可能更长。

原文第 47 页 · 段落 p47-207

中文原意：机器人可能会在几秒钟内完成一个动作，但要花更长的时间等待智能体决定下一步该做什么。重复的模型调用、视觉观察和工具交互可能会导致严重的延迟，尤其是当任务需要许多小操作时。最近的工作已经开始通过减少不必要的高级推理来解决这个问题。 HarnessWAM [89] 使用轻量级进度估计器来监视执行情况，并在重要事件发生时调用其 VLM 任务管理器。类似地，VLAs-as-Tools [166] 提供执行反馈，允许智能体在需要时重新计划，而不是不断检查机器人的进度。因此，效率不仅取决于模型运行的速度，还取决于调用模型的时间。

> 解析：推理延迟是系统问题：模型生成、网络、工具与运动分别耗时。高层可以在异常事件出现时再调用，低层持续控制；调用少是否更好，应以总完成时间和失败成本判断。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

本段引用：HarnessWAM [89]（辞典第90页）；VLAs-as-Tools [166]（辞典第167页）

原文第 47 页 · 段落 p47-208

中文原意：主要的挑战是决定何时值得延迟另一个模型调用。遵循熟悉的操作顺序的机器人可能会继续进行轻量级监控，而意外的障碍或失败的动作可能需要推理模型做出新的决策。过于频繁地调用模型会浪费时间，但等待太久可能会让机器人遵循不再有效的计划。时间安排应取决于任务进度、不确定性以及延迟响应可能产生的后果。

> 解析：推理延迟是系统问题：模型生成、网络、工具与运动分别耗时。高层可以在异常事件出现时再调用，低层持续控制；调用少是否更好，应以总完成时间和失败成本判断。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 47 页 · 段落 p47-209

中文原意：效率还取决于计算发生的位置。在云中运行的大型模型可能会提供更强的推理能力，但会引入通信延迟，而较小的本地模型可以利用有限的计算资源更快地做出响应。未来的系统可以结合这些选项，使用本地组件进行时间敏感的决策，并在需要额外推理时使用更强大的模型。因此，评估应考虑完整的执行过程，包括推理时间、工具调用、通信开销和物理任务完成时间。如果更少的模型调用或更快的推理导致更多的失败、重试或更长的任务执行时间，则它们不一定会提高效率。

> 解析：推理延迟是系统问题：模型生成、网络、工具与运动分别耗时。高层可以在异常事件出现时再调用，低层持续控制；调用少是否更好，应以总完成时间和失败成本判断。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

8.4 展望 / 跨本体

> 本节先抓住：机器人外形、传感器、动作空间和低层控制器改变时，语义相同不代表动作接口相同。跨本体迁移需区分数据归一化、适配器与真正共享能力。
>
> 贯穿例子：“拿杯子”可跨机器人共享任务含义，六轴臂和灵巧手的控制向量不能直接互换。

原文第 48 页 · 段落 p48-210

中文原意：与一个机器人配合良好的智能体在提供不同平台时可能会遇到困难。即使是一项简单的任务，例如到达目标位置，也需要轮式机器人、人形机器人或无人机采取不同的动作。高层目标可能保持相似，但可用的传感器、控制接口和物理约束不同。 RobotWorld [404] 在 84 个模拟任务上评估 通用（General-Purpose）智能体，这些任务涵盖操纵、移动操纵、运动、驾驶和空中控制。目前尚不清楚的是，同一智能体是否可以在不对其规划和工具使用设置进行重大更改的情况下适应新机器人。

> 解析：跨形态泛化需要重组可行计划。例如无人机能从上方查看，轮式机器人可能必须绕行；高层知识可以复用，坐标、运动约束和控制命令不能直接照搬。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

本段引用：RobotWorld [404]（辞典第405页）

原文第 48 页 · 段落 p48-211

中文原意：一个有希望的方向是让智能体在使用不熟悉的机器人之前了解它们的功能。智能体可以识别机器人的可用传感器、运动能力和支持的技能，然后围绕这些能力进行规划。例如，轮式机器人可能需要绕过障碍物来检查某个区域，而无人机则可以从上方接近它。目标相似，但行动和物理限制不同。因此，智能体必须调整其计划，而不是简单地将相同的命令翻译到另一个机器人的控制界面中。

> 解析：跨形态泛化需要重组可行计划。例如无人机能从上方查看，轮式机器人可能必须绕行；高层知识可以复用，坐标、运动约束和控制命令不能直接照搬。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 48 页 · 段落 p48-212

中文原意：另一个重要的问题是，智能体机器人以前的经验有多少可以被重用。高层策略，例如在信息丢失时收集额外的观察结果或检查操作失败的原因，可能在不同的机器人中仍然有用。相反，特定的运动命令和控制参数通常取决于机器人的形态与配置（embodiment）。未来的系统应该保留一般任务知识，而不是假设相同的执行细节适用于每个平台。评估可以在以前未见过的机器人上测试智能体，衡量任务的成功、所需的额外配置或训练的数量，以及其他机器人的经验是否提高了性能。

> 解析：跨形态泛化需要重组可行计划。例如无人机能从上方查看，轮式机器人可能必须绕行；高层知识可以复用，坐标、运动约束和控制命令不能直接照搬。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

9 结论

> 本节先抓住：用四域定位智能体决策，以执行证据连接任务组织与能力改进。分类图用于整理问题；证明一个系统有效仍需要方法细节与独立实验。
>
> 贯穿例子：读完应能指出一个工作修改了什么、使用什么反馈，以及如何证实提升来自该修改。

原文第 48 页 · 段落 p48-213

中文原意：本文研究了智能体如何支持机器人执行和学习。我们追踪从机器人策略到智能体系统的演变，并阐明 智能体式机器人（Agentic Robots） 的范围。我们将现有工作分为四个功能域的 17 个重叠类别：模型、数据、环境和 运行支撑框架（Harness）。这些类别涵盖模型学习和决策、数据收集和准备、世界构建、任务和测试以及协调机器人功能的软件。

> 解析：读完后应能沿“目标 → 当前证据 → 选择执行器 → 动作 → 反馈 → 下一步”解释一个系统，并指出经验如何改变以后可用的能力。选择研究点时，只改一个明确对象，再用固定条件检验。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 48 页 · 段落 p48-214

中文原意：在这些方法中，智能体的决策取决于它可以观察到什么、机器人可以做什么以及它收到什么反馈。在执行过程中，智能体协调工具和技能，检查进度，并使用反馈来决定是否继续、收集更多信息或从失败中恢复。除了当前任务之外，存储的经验还可以指导以后的决策以及对数据、训练程序、技能、模型和软件的更改。这些变化需要在新的任务和条件下进行测试，同时检查之前学习的功能是否得到保留。

> 解析：读完后应能沿“目标 → 当前证据 → 选择执行器 → 动作 → 反馈 → 下一步”解释一个系统，并指出经验如何改变以后可用的能力。选择研究点时，只改一个明确对象，再用固定条件检验。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

原文第 48 页 · 段落 p48-215

中文原意：在此分析的基础上，我们讨论了未来研究的四个方向：公平基准测试、真实机器人部署、高效机器人使用和跨实体泛化。未来的工作需要确定智能体在可比条件下何时提高性能，以及它们如何处理物理机器人的不确定性和故障。它还应该检查这些改进是否证明所使用的时间和资源是合理的，以及智能体如何适应不熟悉的机器人。我们希望这项工作能够帮助研究人员连接这些问题并开发出更强大的 智能体式机器人（Agentic Robots）。 智能体式机器人学（Agentic Robotics） 的进展可以让我们更接近将知识转化为行动、从经验中学习并适应不断变化的物理世界的智能系统。

> 解析：读完后应能沿“目标 → 当前证据 → 选择执行器 → 动作 → 反馈 → 下一步”解释一个系统，并指出经验如何改变以后可用的能力。选择研究点时，只改一个明确对象，再用固定条件检验。

自问：本段是在定义概念、解释接口还是汇报实验？证据支持到哪一层？

缩写与核心术语

VLA · Vision-Language-Action：视觉-语言-动作模型，把观测和语言接到动作生成。

VLM · Vision-Language Model：视觉语言模型，通常输出语义、文本或视觉表征；不默认直接控制机器人。

LLM · Large Language Model：大语言模型，在这里常用于目标、程序、工具和反馈解释。

WM / WAM · World Model / World-Action Model：世界模型 / 世界动作模型，重点区分预测动态和联合建模动作。

BC / RL · Behavior Cloning / Reinforcement Learning：行为克隆 / 强化学习，分别侧重专家标签与交互回报。

ACT / CVAE · Action Chunking with Transformers / Conditional Variational Autoencoder：Transformer动作分块 / 条件变分自编码器。

MPC · Model Predictive Control：模型预测控制，反复规划未来、只执行前段、再观察。

PDDL · Planning Domain Definition Language：规划领域定义语言，描述符号对象、前置条件和作用。

TAMP · Task and Motion Planning：任务与运动规划，连接逻辑步骤和几何运动可行性。

ROS / MCP · Robot Operating System / Model Context Protocol：机器人操作系统 / 模型上下文协议，提供接口和通信，不自动保证安全。

RGB-D / LiDAR · Red-Green-Blue plus Depth / Light Detection and Ranging：彩色加深度 / 激光雷达。

VQA / EQA · Visual Question Answering / Embodied Question Answering：视觉问答 / 具身问答，后者可能需要行动来获得证据。

SFT / CoT · Supervised Fine-Tuning / Chain of Thought：监督微调 / 思维链；生成推理文本不证明物理判断正确。

CBF · Control Barrier Function：控制障碍函数，通过控制约束维护安全集合。

PTZ / BEV · Pan-Tilt-Zoom / Bird's-Eye View：云台转动与变焦 / 鸟瞰视图。

IK / FK · Inverse Kinematics / Forward Kinematics：逆运动学 / 正运动学，连接关节与末端位姿。

NR / N/A · Not Reported / Not Applicable：未报告 / 不适用；空白图标不能直接翻译为“没有”。

Token / Latent / Rollout · 词元 / 潜表示 / 轨迹展开：表示单元、压缩内部表征与执行或想象的一串状态动作。

Harness / Grounding / Affordance · 运行支撑框架 / 落地关联 / 可供性：组织能力、将语义绑定证据、判断物体支持什么动作。

原文核对附录

后续完整保留原文79页。原文页码与页上印刷号一致；解析前文的“原文第N页”指下面原始PDF的第N页。原页保留所有图、公式、表格和引用。此附录是核对依据，不是另外的中文正文。

arXiv:2610.20442v1 [cs.RO] 14 Oct 2026

Towards Agentic Robots:

Model, Data, Environment, and Harness

Chonghan Qin, Dongming Wu, Kaituo Feng, Jiaxu Wang, Xinlei Yu, Peiwen Sun
Yijia Fan, Xilin He, Bingao Chen, Weitong Lian, Ryan Ji, Wenqi Ren, Xiaobin Hu
Dahua Lin†, Yu-Gang Jiang†, Xiangyu Yue†

MMLab, The Chinese University of Hong Kong Fudan University NovaXBot Tsinghua University
Zhejiang University Sun Yat-sen University

Abstract

Foundation models are driving a paradigm shift in robotic intelligence, moving beyond low-level action prediction to goal-directed orchestration of data collection, model training, task planning, and real-world execution. We characterize this emerging paradigm as an **agentic robots**, in which agents reason over human instructions, interact with robotic environments, rectify execution errors autonomously, and dynamically adjust operational strategies. Although this field develops fast, current studies are not well connected. Researchers have no consistent view of how to combine different agentic functions into a complete robot system.

This paper provides a unified overview of existing research on agentic robots. We first trace the development from robot policies to agentic systems, covering vision-language-action models, world models, and world-action models. We then clarify the definition and scope of agentic robots and organize existing research into seventeen categories across four functional domains: **Model**, **Data**, **Environment**, and **Harness**. Across these domains, we examine how agents shape model learning and robot decisions, collect and curate robot data, construct worlds, tasks, and tests for learning and evaluation, and organize capabilities for robot use. Finally, we discuss open challenges and outline directions for future research. Looking forward, we hope this work contributes to a clearer understanding of agentic robots and supports new approaches to making them more capable, reliable, and adaptive.

Correspondence: chonghanqin@link.cuhk.edu.hk, xyyue@ie.cuhk.edu.hk
Repository: https://github.com/chonghanqin/Awesome-Agentic-Robots

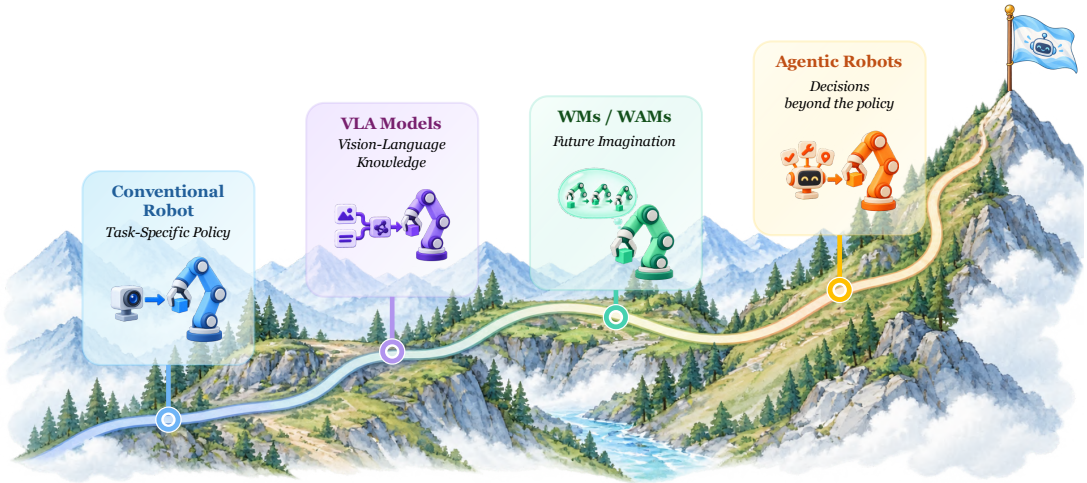


Figure 1. Robot intelligence across four paradigms: task-specific policies for direct control, vision-language-action models grounded in language model knowledge, imagination-centric world models or world-action models, and agentic decision-making that reasons beyond learned policies.

†Core Supervisors

Contents

1 Introduction 3

2 Evolution of Robot Learning 4
- 2.1 Conventional Robot Policies . . . 5
- 2.2 Vision-Language-Action Models . . . 5
- 2.3 World Models and World-Action Models . . . 6
- 2.4 Agentic Robots . . . 7

3 Foundations of Agentic Robots 8
- 3.1 Definition and Scope . . . 8
- 3.2 Taxonomy . . . 9
- 3.3 Comparison with Related Concepts . . . 9

4 Model: What Should the Agent Learn, Understand, Decide, and Control? 11
- 4.1 Training . . . 11
- 4.2 Perception . . . 13
- 4.3 Planning . . . 14
- 4.4 Robot Control . . . 17
- 4.5 Adaptation . . . 19

5 Data: How Should the Agent Collect, Annotate, and Refine Robot Data? 21
- 5.1 Collection . . . 21
- 5.2 Annotation . . . 23
- 5.3 Refinement . . . 25

6 Environment: What Worlds, Tasks, and Tests Should the Agent Build? 26
- 6.1 Reconstruction . . . 26
- 6.2 Task Generation . . . 28
- 6.3 Benchmarking . . . 29

7 Harness: How Should Robot Capabilities Be Orchestrated, Recovered, and Evolved? 31
- 7.1 Orchestration . . . 32
- 7.2 Memory . . . 34
- 7.3 Monitoring and Recovery . . . 36
- 7.4 Skills . . . 38
- 7.5 Infrastructure . . . 41
- 7.6 Layered Systems . . . 44

8 Outlook: What Comes Next for Agentic Robots? 46
- 8.1 Fair Benchmarking . . . 46
- 8.2 Real-robot Agentic Robotics . . . 47
- 8.3 Efficient Robot Use . . . 47
- 8.4 Cross-Embodiment Generalization . . . 48

9 Conclusion 48

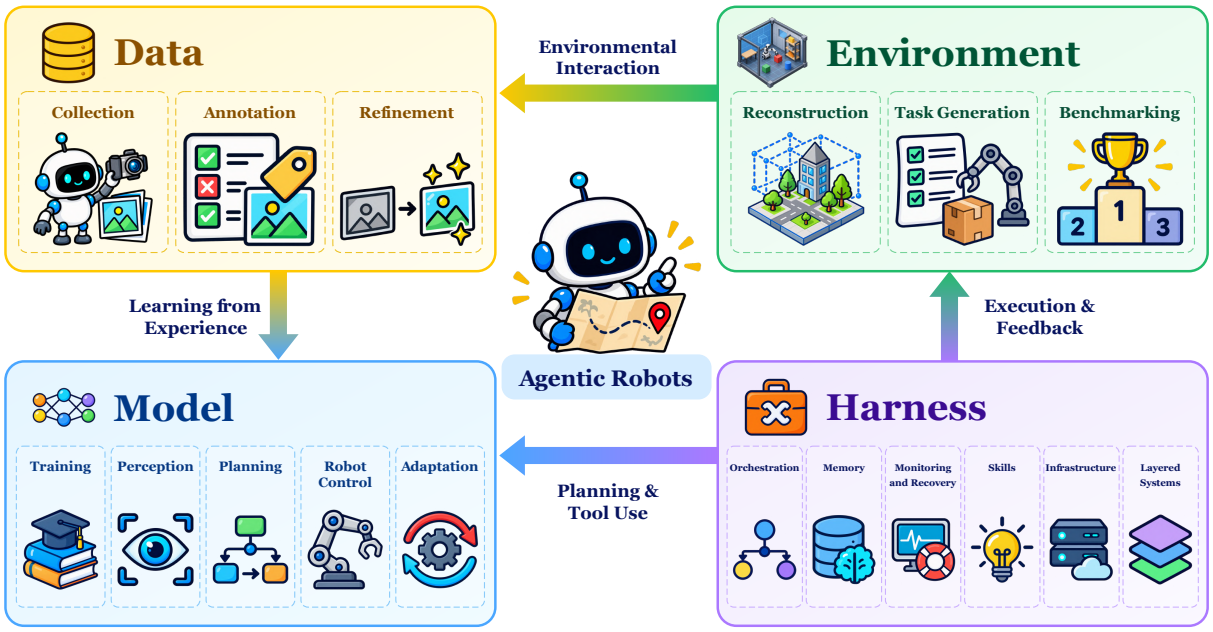


Figure 2. Overview of agentic robots across Model, Data, Environment, and Harness. The four domains connect model decisions, robot execution, environmental interaction, and learning from experience.

1 Introduction

Over the past few years, foundation models including large language models (LLMs) and multimodal large language models (MLLMs) have greatly advanced robotic policy learning. They endow robots with novel capabilities that traditional task-specific models lack, especially in scene and task understanding and cross-scene generalization, by scaling robot datasets and model size [20, 256]. As a dominant technical solution in this field, vision-language-action (VLA) models utilize either discrete action tokens or diffusion- or flow-matching-based continuous action prediction strategies to connect robotic action prediction and the semantic knowledge embedded in pre-trained LLMs [13, 19, 269]. Nevertheless, improved action prediction alone cannot guarantee reliable execution for long-horizon tasks. These tasks demand robots to track task progress, adjust plans amid environmental changes, and recover from failures. VLA policies by themselves cannot consistently deliver these capabilities.

Why Agentic Robots Matter? Recently, agentic robots are attracting surging attention in the research community, which represent a new generation of embodied intelligent systems that integrate foundation model reasoning with embodied agency and environmental interaction capabilities. General-purpose models such as GPT-6 Astra [258] and Gemini 4 Argon [87] are opening new possibilities for such systems. For example, GPT-6 Astra demonstrates promising capabilities as a robot manipulation agent, using visual observations to generate end-effector commands and adjust its actions from execution feedback *without task-specific fine-tuning* [439]. On RoboDojo, it achieves a 22.48% average success rate across 42 simulation tasks, outperforming the trained robot policy DM0.5 (19.34%) [59], GalaxeaVLA G0.5 (14.88%) [76], Xiaomi-Robotics-1 (13.93%) [384], OpenWAM-α (11.92%) [363] and $\pi_{0.5}$ (6.91%) [269]. Despite this progress, inference latency and resource costs highlight the importance of how foundation models are integrated into robotic systems. Beyond optimizing and scaling model parameters, agentic robots use foundation models to plan actions, monitor task progress, and revise decisions based on execution feedback. Their capabilities therefore depend on both the model and the tools, observations, and feedback available in the environment. The agentic robot paradigm addresses the "brain in a vat" limitation of previous robotic solutions, shifting robotic intelligence from static pattern matching toward adaptive autonomous operation.

Agentic Robots Need a Taxonomy. Despite growing interest, research on agentic robots remains fragmented across model learning, data acquisition, environment construction, and runtime system design.

These directions are often studied separately, although their contributions interact within a robotic system. Related terms such as *embodied agents*, *autonomous robots*, and *self-improving robotic systems* further blur the scope of the field by emphasizing different capabilities without consistently clarifying their boundaries. Together, these issues make it difficult to identify where agents make decisions and how those decisions contribute to the overall system. A unified taxonomy should therefore distinguish the roles of individual components while explaining how they work together. To this end, we organize agentic robotics into four functional domains: **Model**, **Data**, **Environment**, and **Harness**, tracing the agent's decisions within each domain and their effects across the system. Figure 2 shows how these four domains work together in agentic robot systems.

Key Questions

❶ **Model:** What should the agent learn, understand, decide, and control?

❷ **Data:** How should the agent collect, annotate, and refine robot data?

❸ **Environment:** What worlds, tasks, and tests should the agent build?

❹ **Harness:** How should robot capabilities be orchestrated, recovered, and evolved?

The four questions above guide our discussion of how agents contribute to robotic systems. Section 4 examines how agents support model learning and robot decision-making, while Section 5 explains how they collect, annotate, and refine robot data. Section 6 considers the worlds, tasks, and tests agents construct for learning and evaluation. Section 7 examines how agents coordinate executable capabilities, monitor execution, recover from failures, and retain useful knowledge for subsequent tasks.

Contributions. This paper makes three main contributions:

- **A Unified Illustration for Agentic Robotics.** We trace the evolution from robot policies to agentic systems and clarify the role of agents in robot task execution and capability development. We introduce a taxonomy spanning **Model**, **Data**, **Environment**, and **Harness**, bringing these research directions into a common framework.
- **A Systematic Analysis of Agentic Methods.** We synthesize the mechanisms through which agents acquire, coordinate, and improve robotic capabilities. By comparing representative approaches across the taxonomy, we explain how agent decisions depend on available tools, robot executors, and interaction feedback, and examine the strengths and limitations of these approaches.
- **A Research Agenda for Practical Agentic Robots.** Drawing on this analysis, we outline research directions for establishing agents' contributions and translating current advances into reliable, efficient robot operation. The agenda connects fair evaluation with the challenges of physical deployment and generalization across robotic platforms.

2 Evolution of Robot Learning

Robot learning spans four connected stages: learning executable behavior, integrating multimodal knowledge, predicting physical outcomes, and coordinating execution and improvement. (1) **Conventional Robot Policies: Learning Executable Behavior.** Reinforcement and imitation learning map observations to actions, while generalist policies share experience across tasks and embodiments. Their capabilities depend largely on the robot experience available during training. (2) **Vision-Language-Action Models: Connecting Knowledge to Action.** VLAs bring the visual and linguistic knowledge of pretrained foundation models into control, connecting the interpretation of scenes, instructions, and semantic relationships to action generation. (3) **World Models and World-Action Models: Anticipating Physical Outcomes.** These models connect action selection and generation to predictions of environmental change, using representations of future states, observations, and their relation to actions. (4) **Agentic Robots:**

Organizing Execution and Improvement. Agentic systems extend model-mediated decisions to selecting and coordinating capabilities, acquiring experience, and using feedback to guide subsequent execution and capability improvement. These stages describe overlapping research traditions and complementary capabilities that can coexist in one system.

2.1 Conventional Robot Policies

Conventional robot policies learn from demonstrations or reward-based experience. We group visual policies by scene representation: image-based policies use image features, while 3D-aware policies introduce geometric structure for action prediction. This distinction concerns scene representation, not action dimensionality.

Image-based Policies. Early policies learn actions from camera images. Guided policy search [168] maps images and robot state to motor torques, while QT-Opt [143] learns an image-conditioned action-value function for grasping. RoboMimic [238] studies visual behavior cloning from offline demonstrations. Visual pretraining supplies reusable policy features: R3M [244] pretrains an image encoder on human videos for downstream imitation learning. Beyond visual encoding, ACT [449] and Diffusion Policy [55] model action sequences with a conditional variational autoencoder and conditional diffusion, respectively. Task conditioning extends these policies to multiple behaviors: BC-Z [128] and RT-1 [20] use task descriptions, while VIMA [138] combines multimodal prompts with object-centered image tokens. Cross-robot datasets broaden training: RT-1-X [260] draws on cross-robot data from Open X-Embodiment, and Octo [256] combines heterogeneous image observations with a transformer and a diffusion action head.

3D-aware Policies. These policies use scene geometry to predict actions. Transporter Networks [427] construct orthographic color-height maps from RGB-D observations to predict spatial pick-and-place actions, using a 2.5D scene representation. PerAct [298] represents observations and action locations with 3D voxels, while RVT [88] renders virtual views of a reconstructed point cloud and combines their predictions in 3D. Act3D [83] lifts image features into a 3D feature field and samples action locations from coarse to fine. These geometric representations also support generative action models: DP3 [426] conditions action diffusion on compact features encoded from sparse point clouds, whereas 3D Diffuser Actor [146] uses spatially located scene tokens and 3D attention to denoise end-effector trajectories. Mamba Policy [25] retains point-cloud conditioning and combines selective state-space models with attention in its denoising network. Other methods learn geometric representations for downstream policies. GSRL [341] pretrains and freezes a generalizable Gaussian predictor, then supplies its predicted 3D Gaussian scene representation to offline reinforcement-learning algorithms. Structural latent point pretraining [137] regularizes point coordinates and features with a point-wise latent variational autoencoder and uses Gaussian rendering supervision; its latent representation is then supplied to an ACT policy. Scene representation, learning objective, and action decoder remain distinct, combinable design choices.

2.2 Vision-Language-Action Models

General VLA Models. VLAs adapt large-scale pretrained vision–language backbones to generate robot actions, as in RT-2 [19], OpenVLA [149], and RoboFlamingo [182]. Earlier policies already used language conditioning and pretrained encoders; VLAs connect broader knowledge of objects, relations, and instructions to action prediction through training on robot trajectories. τ_0-VLA [24] combines a high-level subtask policy with a flow-matching action expert, using execution memory and selective world-model-guided test-time search for subtask selection.

Action Representation. A central choice is how to decode robot actions. RT-2 [19] and OpenVLA [149] discretize continuous actions into tokens for sequence generation. Diffusion- and flow-based approaches, including RDT-1B [217], π_0 [13], $\pi_{0.5}$ [269], CogACT [177], TinyVLA [371], GR00T N1 [254], SmolVLA [299], DexVLA [372], InternVLA-M1 [43], LingBot-VLA [378], and Qwen-VLA [349],

condition continuous action decoders on visual and language features to generate action sequences. FAST [265] compresses action sequences into compact tokens, while OpenVLA-OFT [150] combines parallel decoding and action chunking to predict multiple future actions in one forward pass. Vela [187] represents future motion with cubic B-splines, using a flow-matching expert to jointly predict control points and a motion-dependent temporal span. StarVLA [305] provides a modular open-source framework for training and evaluating combinations of vision–language backbones and action decoders.

VLA with Memory. History and memory provide temporal context when a single observation does not reveal earlier actions or task progress. RoboFlamingo [182] aggregates successive vision–language features with an LSTM policy head. MemoryVLA [295] retrieves perceptual and cognitive features from a memory bank and fuses them with current representations to condition a diffusion action expert. MEM [324] combines a video encoder for short-term observation history in its low-level policy with language summaries of past events in its high-level policy. NativeMEM [365] reuses the pretrained VLA's vision encoder to compress each historical frame from each camera view into a single memory token, allowing the policy to attend to long-term visual history alongside current observations.

VLA with Reasoning. Intermediate reasoning and semantic prediction provide explicit guidance between task instructions and robot actions. RT-H [10] and $\pi_{0.5}$ [269] introduce language descriptions of motions or semantic subtasks that condition subsequent actions. ECoT [425] further combines task reasoning with spatial information, including object locations and end-effector positions, to guide action prediction. OneTwoVLA [203] and Gemini Robotics 1.5 [320] integrate reasoning with action generation, allowing task interpretation to inform the robot's next behavior. Visual subgoals provide a complementary representation: CoT-VLA [447] predicts an intermediate goal image and conditions subsequent actions on the desired scene configuration. Together, these approaches provide explicit intermediate guidance at different levels, from task decomposition and motion descriptions to spatial reasoning and visual goals, connecting the robot's understanding of the task with the actions required to execute it.

VLA with Imagination. Future prediction connects VLA action learning to how scenes evolve during robot interaction. GR-1 [375], GR-2 [32], UP-VLA [434], WorldVLA [29], and RynnVLA-002 [28] combine action learning with future-image prediction, using visual changes as additional supervision for robot control. FLARE [456], VLA-JEPA [309], and World Guidance [307] instead model future observations in latent space, learning predictive features or state transitions without reconstructing images. These objectives incorporate future information into policy representations, although future images need not be generated during execution. BagelVLA [111] further interleaves language reasoning, visual prediction, and action generation, combining semantic task guidance with visual anticipation within the action execution loop.

2.3 World Models and World-Action Models

World models learn how the environment changes in response to robot actions. Predicting future states or observations supports planning over candidate actions and policy learning from imagined interactions [73, 96, 98, 377]. DINO-WM [459] and V-JEPA 2-AC [5] predict action-conditioned visual features and optimize action sequences toward visual goals through model-predictive control.

World Model as Environment. Action-conditioned video models simulate interactions for policy learning and evaluation. UniSim [400] simulates visual outcomes of instructions and low-level controls to train embodied planners and policies. Cosmos [251] provides pretrained diffusion and autoregressive video models for adaptation to robot-specific prediction. Cosmos-Predict2.5 [252] unifies text-, image-, and video-conditioned generation and supports action-conditioned robot rollouts after adaptation. Genie Envisioner [201] and DreamDojo [78] use action-conditioned video prediction for policy evaluation; the latter also learns interaction dynamics from human videos with inferred latent actions before adapting to target robots. Cosmos 3 [253] extends the series toward shared multimodal understanding, world generation, and robot action modeling.

4D World Model. 4D world modeling captures how 3D scene geometry evolves over time, supporting geometric planning and interaction simulation. PointWorld [121] represents robot actions as 3D point flows and predicts the resulting scene-point displacements over a temporal horizon, supporting sampling-based trajectory optimization. TesserAct [454] jointly generates RGB, depth, and surface-normal videos, reconstructs temporally consistent 3D point clouds, and uses a point-cloud inverse dynamics model to infer actions. 4DGen [224] uses cross-view pointmap supervision to generate geometrically consistent multi-view RGB-D futures, from which a pose tracker recovers end-effector trajectories. MVISTA-4D [342] generates synchronized, view-consistent RGB-D futures from a single-view observation. It optimizes a trajectory latent at test time to match the imagined future and refines the decoded actions with residual inverse dynamics. Kinema4D [394] converts robot commands into kinematically computed 4D trajectories and uses their projected pointmaps to condition joint RGB and pointmap generation, simulating the surrounding environment's response.

World Model as Planner. World models supply visual plans. Inverse dynamics maps predicted observations or latents to robot actions. UniPi [62] and RoboDreamer [463] use inverse dynamics to translate language-conditioned visual plans into actions. Seer [323] couples visual foresight and inverse dynamics in an end-to-end model, conditioning action prediction on latent representations of the forecasted visual state. GE-Act 2.0 [2] predicts future visual latents in a single generator pass, then combines them with current visual latents and proprioception in an inverse dynamics model to produce an action chunk. This connection also extends to geometric plans: TesserAct [454] infers actions from reconstructed point-cloud sequences, while MVISTA-4D [342] combines test-time trajectory-latent optimization with residual inverse dynamics to refine actions toward the imagined future. These approaches depend on both the feasibility of the predicted future and the inverse ability to translate it into robot-specific control.

Joint Video–Action Modeling. World-action models (WAMs) combine future visual prediction with robot action learning, linking behavior to its environmental consequences. UVA [179], UWM [468], and Motus [12] jointly learn video prediction and action generation within a shared framework. τ_0-WM [462] unifies video–action generation and action-conditioned simulation. At test time, predicted task progress scores candidate futures; when initial candidates are unreliable, a selected future guides action refinement. DreamZero [408], LingBot-VA [176], Cosmos Policy [151], GigaWorld-Policy [406], and Being-H0.7 [230] connect action learning to future visual information through observation prediction or latent alignment. Fast-WAM [422] learns efficient action generation through video co-training without future imagination at inference. ImageWAM [442] guides action generation with pretrained image-editing features without decoding future images. MaskWAM [418] combines optional initial mask prompts with joint future mask, RGB, and action prediction to focus control on task-relevant regions. A recent direction jointly models language, video, and action: UniWAM [303] connects a physical reasoner, world generator, and action predictor through joint multimodal attention, learning language outputs alongside future visual predictions and actions.

2.4 Agentic Robots

Agentic robots organize robotic execution and capability improvement. Given a goal, they coordinate capabilities, interpret execution feedback, and adjust subsequent operations [119, 125, 196, 297]. Learned policies, including VLAs and world-action models, supply executable capabilities within this process, shifting attention to how behavior is selected, coordinated, and revised during execution.

This perspective also extends to data collection, model training, and experimental deployment. Agentic decisions organize experience acquisition, guide learning experiments, and use evaluation feedback to improve capabilities [233, 383]. These decisions connect current execution with improvements for future tasks. Following section examine four functional domains, **Model**, **Data**, **Environment**, and **Harness**, that describe where these decisions act and how robotic capabilities are acquired, used, and improved.

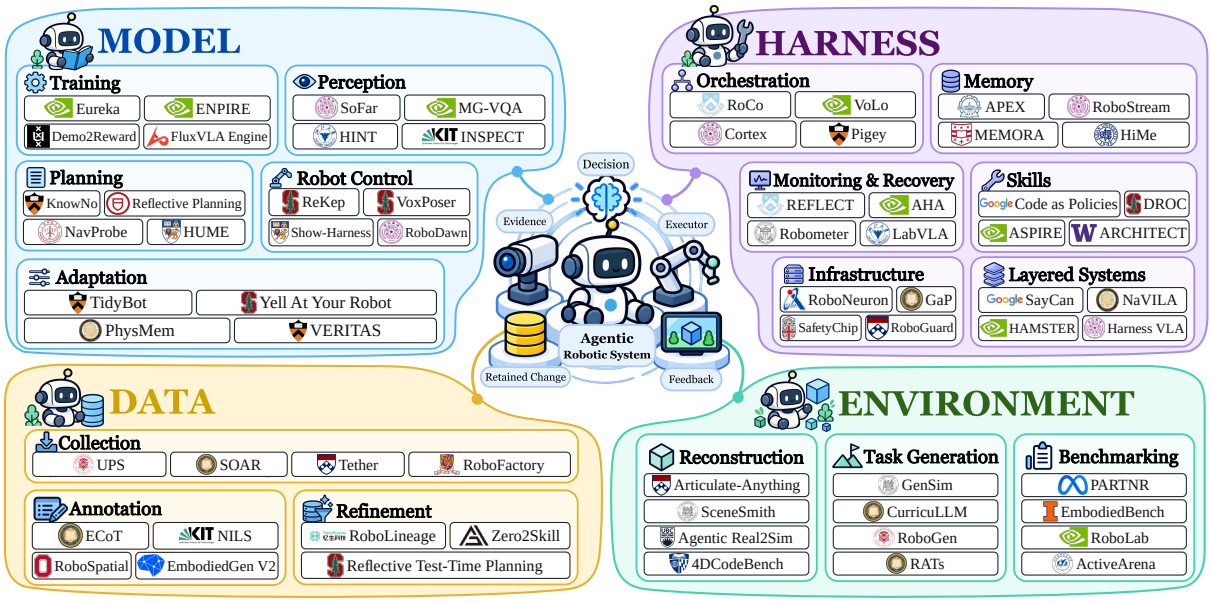


Figure 3. Taxonomy and literature landscape of agentic robotic systems. Four functional domains cover seventeen non-exclusive categories: Model covers learning and robot-facing decisions. Data covers the acquisition and curation of data. Environment covers worlds, tasks, and evaluation. Harness covers how robotic capabilities are coordinated, reused, and improved to support agentic task execution.

3 Foundations of Agentic Robots

The approaches discussed in Section 2 provide capabilities for perception, reasoning, action prediction, and world modeling. Building agentic robots requires coordinating these capabilities through task planning, tool use, and execution feedback, while also supporting the development of new robot capabilities. To characterize these systems, this section first defines agentic robots and their scope, then introduces a taxonomy spanning Model, Data, Environment, and Harness, and finally compares agentic robotics with related concepts.

3.1 Definition and Scope

We define *agentic robots* as robotic systems in which agents autonomously plan and coordinate task execution using environmental observations and execution feedback. Given a goal, these agents determine the steps needed to achieve it, select and coordinate actions, skills, or tools, and monitor progress without requiring a human to specify every step. As execution proceeds, they use new observations and outcomes to decide whether to continue, gather more information, revise the plan, or recover from failure. We consider such systems on both physical robots and in simulation.

These capabilities can be implemented within an integrated model or through a combination of models, policies, programs, and tools. For example, when retrieving an object from a drawer, the agent may select an opening skill, check whether the drawer has opened, and decide whether to retrieve the object or revise the opening attempt. The defining consideration is how the agent organizes and adjusts task execution to achieve the goal. Action prediction or low-level feedback control alone does not establish these capabilities.

Beyond task execution, agents can also contribute to developing robot capabilities. They may collect and refine robot data, construct tasks and environments, revise training procedures, or improve robot skills and execution software. Feedback from interaction and experiments can guide these changes and help improve performance on subsequent tasks. Our paper examines these contributions across Model, Data, Environment, and Harness, covering how agents support robot operation and capability development.

Comparison with Current Works. PAPAV [47] compares computer use and robot use through Perceive, Anticipate, Plan, Act, and Verify, emphasizing the consequences of physical embodiment. [287] examines model–robot integration through protocol interfaces, orchestration, and embedded policies. [129] surveys robot-learning approaches spanning action-predicting weights and executable skills. [250] studies the executable software that organizes agent execution. [331] examines where verification signals come from and whether they are available and trustworthy. Our focus is how these components work together during robot use and capability building.

3.2 Taxonomy

We organize existing work into four domains according to the roles agents play in developing and operating robotic systems: *Model*, *Data*, *Environment*, and *Harness*. *Model* concerns how agents use and improve models for robotic perception, planning, control, and adaptation. *Data* concerns how agents collect, annotate, and refine data for robot learning. *Environment* concerns how agents construct scenes, generate tasks, and design evaluations for robot training and assessment. *Harness* concerns how agents coordinate models, tools, and robot skills to execute tasks, using memory and execution feedback to monitor progress and recover from failures. Together, these domains describe the roles of agents across robot capability development and task execution. Figure 3 presents the overall taxonomy, and Table 1 defines the seventeen categories within these domains.

The taxonomy distinguishes functional contributions rather than assigning each robotic system to a single category. These contributions may span domains: Zero2Skill [334], for example, links agent-directed data collection and preparation to subsequent robot policy training. They may also operate together within a domain, as in HarnessVLA [441], which integrates memory, orchestration, monitoring, and recovery around fixed executors. ENPIRE [383] illustrates how a research agent coordinates experiments to improve robot policies, connecting experiment management with capability development. We therefore discuss each work under the categories corresponding to its relevant functions, allowing a paper to appear in multiple categories when it makes contributions to more than one.

3.3 Comparison with Related Concepts

Agentic robotics is closely related to embodied AI, autonomous robotics, and robot foundation models. These concepts overlap in their methods and applications, but emphasize different aspects of robotic intelligence. We clarify their relationships using the scope adopted in this paper.

Agentic Robotics vs. Embodied AI. Embodied AI studies intelligence through an agent's interaction with its environment, encompassing tasks such as exploration, navigation, manipulation, and embodied question answering. Both embodied AI and agentic robotics connect perception, reasoning, and action in physical or simulated environments [63, 396]. The distinction concerns the emphasis placed on how these capabilities are organized. Embodied AI broadly investigates learning and acting through environmental interaction; our agentic perspective focuses on how agents select capabilities, coordinate task execution, and revise decisions using feedback. This focus is consistent with work examining foundation models as agents that invoke and coordinate robot functionality [287]. It also extends to capability development: AutoRT, for example, uses foundation models to propose and filter tasks for robot data collection [3].

Agentic Robotics vs. Autonomous Robotics. Autonomous robotics concerns robots that carry out tasks without continuous human control. Its established research already integrates planning, acting, monitoring, observation, and learning to support deliberate behavior in changing environments. These functions therefore constitute shared foundations, rather than capabilities unique to agentic robotics [126]. Within this broader field, our paper examines how foundation model agents participate in these functions by interpreting goals, coordinating skills or tools, and incorporating execution feedback into subsequent decisions. Inner Monologue illustrates this connection by returning observations and skill outcomes to a language model for continued planning [119]. Accordingly, "agentic" identifies how decisions are

organized within the system; it does not imply complete independence from human guidance or a higher level of autonomy.

Agentic Robotics vs. Robot Foundation Models. Robot foundation models aim to acquire reusable capabilities for robotic tasks through broad training and adaptation, supporting functions ranging from perception and planning to control [110]. Their capabilities can include reasoning and decision-making: PaLM-E integrates multimodal observations into embodied planning, while RT-2 connects visual and linguistic knowledge to action generation and explores intermediate reasoning before action [19, 61]. The distinction is therefore between the capabilities learned by a model and how those capabilities participate in an ongoing task. Agentic robotics examines how goals, available actions, and execution feedback govern subsequent decisions, whether implemented within one model or across several components. Hi Robot illustrates their compatibility by combining a VLM that updates intermediate instructions with a VLA that executes them [297].

The following four sections review these domains in turn. Within each domain, we group methods by the functions they serve and explain their main approaches through representative work.

Table 1. Operational taxonomy of agentic robots. Four functional domains organize seventeen non-exclusive categories by the part of the robotic system shaped by model-mediated decisions.

Domain	Category	Definition
Model	Training	Designs or revises rewards, objectives, training recipes, or learning experiments.
	Perception	Interprets observations to recover task-relevant concepts, relations, geometry, or beliefs, and decides when additional evidence is needed.
	Planning	Chooses and revises subgoals, action sequences, dependencies, or task solutions as a task progresses.
	Robot Control	Translates selected operations into robot-facing actions, poses, programs, or motion constraints for execution.
	Adaptation	Uses experience to update model parameters or task-specific conditioning and decision rules.
Data	Collection	Acquires or generates new robot experience for learning or later use.
	Annotation	Interprets recorded experience and adds structured supervision, such as text, spatial labels, interaction targets, or outcomes.
	Refinement	Improves existing experience by selecting useful records or segments and revising unreliable supervision.
Environment	Reconstruction	Constructs or revises executable scenes, assets, geometry, and task-relevant physical properties.
	Task Generation	Defines or generates goals, constraints, curricula, or task distributions for practice, learning, or execution.
	Benchmarking	Constructs or adapts tests, perturbations, metrics, evaluators, and evaluation procedures.
Harness	Orchestration	Selects models, policies, skills, or tools and governs their calls and handoffs.
	Memory	Stores, updates, and retrieves interaction history or knowledge for later decisions.
	Monitoring and Recovery	Assesses progress, completion, failure, or constraints and organizes corrective responses.
	Skills	Creates, validates, retains, revises, or retires reusable procedural knowledge and executable behaviors.
	Infrastructure	Exposes, serves, and gates tools and policies through interfaces, services, and runtime checks.
	Layered Systems	Divides information, authority, and operating rates across planners, learned policies, and analytic control.

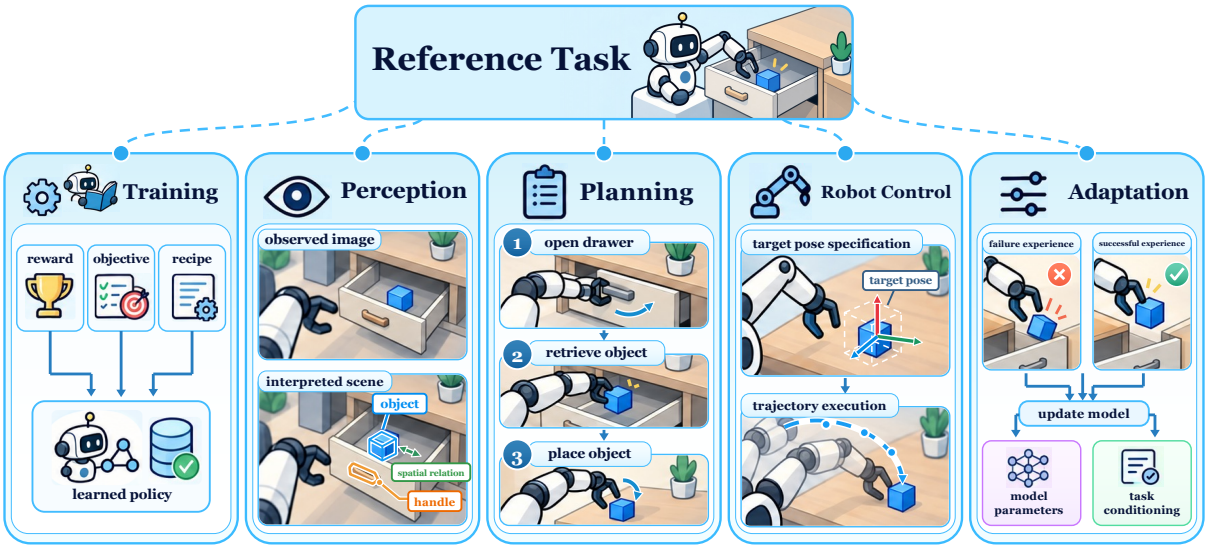


Figure 4. Five model-side interventions illustrated by a drawer-retrieval task. Training shapes the learning process; Perception interprets task-relevant evidence; Planning organizes task structure; Robot Control specifies robot-facing commands; and Adaptation changes model parameters or task-specific conditioning using experience.

4 Model: What Should the Agent Learn, Understand, Decide, and Control?

Figure 4 shows that a drawer task can fail for different reasons, and each calls for a different fix: better evidence about the handle, a revised order of actions, a feasible grasp command, or a policy that improves after repeated failures. These are different model-side interventions. Training asks who chooses the learning process; Perception, Planning, and Robot Control trace how evidence becomes action; Adaptation asks which parameters or task-specific conditioning change with experience. Their effectiveness depends on matching each decision to an executor that can carry it out.

4.1 Training

The agent acts on the learning process rather than in the environment. It specifies rewards, objectives, configurations, or experiments. During this process, an optimizer consumes the resulting artifact, and robot rollouts reveal whether the policy learned the desired behavior. In iterative workflows, the agent uses these results to decide what to revise next. Existing systems differ mainly in the scope of this editable artifact: 1) *reward-design* methods keep the optimizer fixed and revise only the signal it consumes, whereas 2) *program- and experiment-level* methods extend revision to policy code, training pipelines, and 3) the *experimental workflow* itself.

Reward Design. Reward transfers a behavioral objective into a signal an optimizer can use. Eureka [233] established the basic loop: a language model writes reward code and revises it using training feedback. Such feedback, however, can confirm that a terminal predicate is satisfied without revealing whether execution actually followed the instruction. RDA [161] further addresses this gap with semantic analysis of the resulting behavior. It decomposes the instruction into subtasks, generates executable reward candidates, trains policies, and evaluates rollout videos alongside subtask reward logs.

Eureka and RDA evaluate candidate rewards through full policy training, making each evaluation computationally expensive. Later methods reduce this cost in two ways: they validate rewards with cheaper evidence, or they narrow what is searched. EvoNav [453] screens reward programs with rules and short proxy training, assuming that the cheap tests preserve the ranking of useful candidates. Then, CRWM [401] conditions a one-pass reward-code generator on a causal prior derived from offline reward interventions. It pays for the prior up front, and RL outcomes never feed back into the reward. Demo2Reward [92] uses a VLM meta-critic to tune a VLM reward prompt on labelled demonstration prefixes before policy

learning. In ReCoVLA [106], a VLM maps failure observations to descriptors, a deterministic compiler binds them to predefined state-based reward components, and RL trains failure-specific residual policies while the base VLA stays frozen. Generally, each method trades the generality of full-training feedback for a cheaper but more constrained source of reliability: proxy rollouts, offline causal structure, labelled demonstrations, or predefined reward components.

Policy Training. Beyond reward design, agents participate in robot policy development by implementing policies, organizing training experiments, and revising the learning process. ENPIRE [383] first constructs reset, verification, and safety interfaces with human feedback, then exposes these interfaces to coding agents that inspect real rollouts and modify policy or training code. Behavior cloning, reinforcement learning, and heuristic policy synthesis are alternatives inside the research process, not simultaneous properties of every resulting controller. HARBOR [191] organizes simulation reinforcement learning into typed artifacts and gate-checked stages. Failed interface checks return actionable diagnostics; parallel tuning trials retain code, videos, and logs, and recurring lessons are retrieved for later experiments. ENPIRE depends on reliable physical resets; HARBOR depends on agreement among generated task, reward, and algorithm interfaces. Each exposes a different bottleneck to the experimental agent.

This broader search on policy training makes reproducible execution and human control of the research agenda especially consequential. This work [148] uses coding-agent edits, experiment launches, and log-based triage for quadruped policy improvement within human-directed research waves. Nautilus [139] instead standardizes policy, robot, and benchmark contracts and isolates their execution environments, allowing a coding agent to assemble and validate learning workflows. Overall, these works not only expand experiment search under a human agenda, but also reduce the integration work required to implement those experiments.

Workflow. The workflow contains structure and measurements to keep the agent on the right direction. [417] treat each experiment as a durable transaction with an immutable objective, controller-owned measurement, artifact-bound tool quality, and crash recovery. In LWD [359], fleet successes, failures, and human interventions instead update a pretrained generalist VLA through offline-to-online post-training. The two workflows intervene at different levels: policy post-training from fleet experience and management of the experiments used to assess revisions.

That separation of decision and measurement also matters when an agent edits more than the training recipe. Reward, prompt, tool, and data edits can otherwise change both the intervention and its judge. RoboLineage [231] makes this transition explicit by storing rollouts, reviews, dataset decisions, training runs, policy metadata, and deployment recommendations as typed artifacts. From the perspective of foundational infrastructure, FluxVLA Engine [189] supplies configuration-resolved contracts that connect data, training, evaluation, and deployment. EmbodiedSkills [351] then provides separate interfaces through which planning, verification, and execution components can be trained or adapted. The useful design principle is to expose objectives, recipes, and experiments for revision while keeping the measurements used to judge them stable. The learning choices are analyzed here under Training; the execution contracts and measurement interfaces that support those choices are discussed as capability-building infrastructure in Section 7.5. Related works are compared in Table 2.

Table 2. Comparison of works on training (Section 4.1). Training metrics are collected during policy learning; evaluation metrics assess the resulting policies.

Work	Link	Target	Approach	Data / feedback
Eureka [233]		Reward code	Evolutionary search	Training metrics
ADARLR [148]	-	RL code; reward weights	Iterative refinement; hyperparameter search	Training metrics; failure cases
LWD [359]	-	VLA; value models	RL	Rollouts; human feedback
EvoNav [453]	-	Navigation rewards	Evolutionary search; iterative refinement	Offline data; evaluation metrics
Nautilus [139]	-	Policy / benchmark modules	Pipeline composition	Execution checks

Continued on next page

Table 2 (continued)

Work	Link	Target	Approach	Data / feedback
Demo2Reward [92]		Reward prompt	Iterative refinement	Demonstrations
RDA [161]	-	Reward code	Evolutionary search	Rollout videos
HARBOR [191]		RL code; configurations	Iterative refinement; hyperparameter search	Execution checks; training metrics
ReCoVLA [106]	-	Recovery rewards; residual policies	Single-pass generation; RL	Failure cases; simulator state
ENPIRE [383]		Training code	Iterative refinement	Rollouts; evaluation metrics
RoboLineage [231]		Training data; policy records	Data curation; iterative refinement	Rollouts; evaluation metrics
CRWM [401]	-	Reward code	Single-pass generation	Offline data
AgenticRobotics [417]	-	Tool records; reusable workflows	Iterative refinement	Evaluation metrics
EmbodiedSkills [351]		Planner; VLA; scheduler	SFT	Demonstrations
FluxVLA Engine [189]		Training pipelines	Pipeline composition; SFT	Demonstrations

4.2 Perception

Perception translates observations into task-relevant **concepts, relations, geometry, or beliefs**. The agentic intervention may be an interpretation, a request for another observation, or a physical action that reveals missing evidence. The representation must support the next decision: identifying a handle, for example, also requires geometry accurate enough for the intended grasp.

Observations. An observations process chooses what evidence to seek when the current view needs renewal. Situated human-robot conversation systems [160] combine multimodal interaction with task-level attention and tool calls, while HINT [239] updates intent and routes camera views at manipulation-pattern transitions, and re-localizes the same committed target when tracking confidence is low. SCOPE [103] makes camera orientation and zoom explicit actions whose returned observations condition the next perception request, illustrating a camera agent rather than a fixed detector.

Active perception changes the evidence itself. MG-VQA (formerly Probe) [11] lets a VLM invoke perception, grasp-and-remove, and pushing tools to expose occluded information before answering a visual question. [257] uses visual predicate checks and additional viewpoints to correct the state supplied to a Planning Domain Definition Language (PDDL) planner. Manipulation can therefore gather information, while an additional observation can invalidate a symbolic planning premise. The choice to move an object or camera trades the value of new evidence against motion cost and risk.

Decisions. Based on the previous observation, the decision is the final product that the executor can use. ABot-N1 [86] uses a recurrent spatial reasoner to update semantic targets while a distinct action expert supplies continuous navigation behavior; Steerable VLA policies [42] similarly expose semantic-subtask, atomic-motion, and grounded-point or trace steering modes. A distributed conversational manipulation framework [142] grounds pixel predictions through depth and robot coordinates on each command, whereas RoboVista's [386] robot-VQA abstraction evaluates whether a model can answer perception questions across diverse application modules. Whether the output is a renewed view, a geometric target, or a symbolic answer, its quality depends on the downstream decision it must support.

Navigation requires spatial evidence to remain accessible as the robot moves. AgenticNav [188] provides heading-indexed views, selective recall, a compact trajectory map, and geometric tools. The VLM can request evidence and choose a navigation target; depth-based validity checks and movement remain below that choice. SoFar [271] provides an earlier spatial-grounding comparison: object perception constructs a scene graph containing positions and semantic orientation vectors, and the VLM specifies desired object geometry that deterministic fitting and motion planning realize. A textual description such as "upright" becomes useful only when bound to the orientation representation understood by the executor.

Reflections. The robot needs a prompt reflection for the change of environment or behavior. [193] first exposes perception and motion tools through grounded object anchors and execution receipts; the model receives images and receipts while calibrated depth geometry is handled by the backend. An archived identity does not make its old pose a valid motion target: lost or invalid anchors require renewed grounding. Then, [100] use task-conditioned relations and shared instance evidence to request complementary views before accepting a candidate, but their simulation uses oracle instance masks and IDs. INSPECT [370] further evaluates learned view utility and commitment decisions through annotated video replay and a fixed robot-image lattice rather than a closed-loop physical inspection trial.

Navigation summarizes all dynamic and static spatial information. OpenFrontier [261] scores visual frontiers and verifies candidate targets, returning to search when a candidate is rejected; GoalVLM [127] combines frontier scoring with allocation across agents and a shared map. FARE [200] reasons over an evolving belief graph to select global waypoints, while AnchorVLN [330] binds model commands to typed object handles through a ROS server. Frontier scores help choose where to explore, belief graphs preserve longer spatial dependencies, and typed handles constrain command references. Their common vulnerability is stale grounding: a remembered object identity must be re-observed when its location no longer supports the proposed action. Related works are compared in Table 3.

Table 3. Comparison of works on perception (Section 4.2). Input lists observations, excluding task instructions and queries.

Work	Link	Input	Output	Perception process
SoFar [271]		RGB-D	Scene graph	Visual grounding; geometric estimation
FARE [200]	-	LiDAR	Spatial map	Mapping; frontier ranking
MSR [160]	-	Audio; images	Retrieved images	Memory retrieval; view selection
Steerable Policies [42]		Images	Language commands; pixel coordinates	Visual reasoning
OpenFrontier [261]		RGB; depth; camera pose	3D targets	Frontier ranking; visual grounding; verification
GoalVLM [127]	-	RGB-D; robot pose	Spatial map	Mapping; geometric estimation
VAP-TAMP [257]		RGB-D; robot pose	Scene graph; symbolic predicates	View selection; verification
SCOPE [103]		PTZ images	Answers	VQA; view selection
CFHRCM [142]		RGB-D; calibration	3D targets	Visual grounding; geometric estimation
AgenticNav [188]	-	RGB; BEV; depth tool	Distances; retrieved images	Geometric estimation; memory retrieval
RoboVista [386]		Images	Answers	VQA
ABot-N1 [86]		Images; visual history	Pixel coordinates	Visual reasoning; visual grounding
MG-VQA [11]		Top-down RGB	Answers; pixel coordinates	Interactive perception; visual grounding; VQA
HINT [239]		Images; state; torques	Segmentation masks	View selection; tracking; visual grounding
AnchorVLN [330]		Panorama; LiDAR	Object handles; 3D targets	Visual grounding; geometric estimation; verification
INSPECT [370]		RGB; camera poses	Camera viewpoints; answers	View selection; verification
CoRelNav [100]	-	RGB-D; robot pose	Object handles; 3D targets	Frontier ranking; verification
Robo-Harness K1 [193]		RGB; state; geometry tools	Segmentation masks; object handles; 3D targets	Visual grounding; geometric estimation; tracking

4.3 Planning

Based on what it perceives, a robot plans a sequence of actions to complete a task. Before carrying out the plan, it needs to check whether these actions are feasible. The plan may also need to change during execution, for example when an object moves or an action fails. Planning therefore continues as the robot carries out the task and learns whether its decisions work. We discuss this process in three stages: generating a plan, checking it before execution, and updating it during execution.

Generating a Plan. Some planners choose one operation at a time and use its result to decide what to do next. This is useful when later steps depend on information the robot does not yet have. For example, one system provides tools for looking around, moving to a location, grasping objects, and accessing memory [290]. The planner reads the text returned by these tools before choosing its next operation. AgentRob [218] follows a similar approach, with VLM controllers observing the results of action primitives before selecting the next operation. In both systems, the planner chooses the operation, and the robot controller handles the physical movement.

The planner can also specify a subgoal and let a trained policy produce the actions needed to achieve it. Hi Robot [297] uses a high-level VLM to give short language commands to a VLA. These commands are updated periodically or after user intervention. Gemini Robotics-ER 1.5 [320] similarly handles high-level reasoning and gives language instructions to the Gemini Robotics 1.5 action model. LocalNav [9] applies this division to navigation. Its distilled scene-graph reasoner decides whether to move toward an object, explore a room, perform frontier search, or stop. A PointGoal policy then carries out the navigation.

Beyond choosing the next operation or subgoal, a planner can also specify how several operations depend on one another. For example, [145] decomposes multi-robot tasks into PDDL subproblems for a classical planner and uses planning failures to improve the agents' prompts. In an industrial setting, IMR-LLM [306] uses an LLM to help construct a disjunctive graph, which a deterministic solver uses to schedule operations. A process tree then guides the generation of executable robot programs. To connect symbolic planning with execution, EmboTeam [428] combines semantic processing with PDDL-based search and converts the resulting plans into behavior trees containing conditions, fallbacks, and synchronization.

Checking a Plan before Execution. Once actions have been proposed, the planner can check whether they are consistent with its model of the scene and the task requirements. For example, SayPlan [274] identifies the relevant part of a hierarchical 3D scene graph and uses feedback from a scene-graph simulator to revise its plan. Within this process, a classical path planner supplies routes between locations. GAVEL [350] likewise uses a graph world model to verify and repair candidate plans before execution. For constraints drawn from industrial procedures, MaCoPlanner [389] converts manuals into a typed intermediate representation and checks action sequences against procedural and state-transition constraints.

Before planning the actions, the system can also check whether the goal is clear and suitable for the robot. For example, STARS [153] generates candidate goal descriptions and analyzes, repairs, and selects them before the underlying task-learning agent searches for actions to achieve the selected goal. Here, the checks concern whether the proposed goal is understandable and appropriate for the agent and its situation, rather than whether a particular action sequence can be executed.

Alongside checks based on graphs and rules, planners can also use learned models to assess candidate actions. For instance, Reflective Test-Time Planning [105] scores proposed high-level actions with an internal reflection model, while external and retrospective reflection later draw on actual outcomes. Another way to assess actions before execution is to imagine their visual consequences. ReflectVLM [71] predicts future images for a proposed action sequence and considers these predictions together with the current and goal images to select the next action primitive. A scripted controller then executes the selected primitive.

Updating a Plan during Execution. Even after these checks, an action may produce an unexpected result, or a change in the scene may make the remaining plan unsuitable. The planner therefore needs execution feedback to decide whether to continue or revise its next steps. For example, WCM [51] revises its next decision when actual outcomes differ from its predictions, the user changes the instructions, or the scene changes. When necessary, it can also ask the user for clarification.

To support these decisions, the system needs to report execution results and determine when they call for replanning. For example, [202] provides status, event, and reflection tools for this purpose. A failed physical check can serve as a direct trigger. ActionReasoning 2.0 [335] discards the remaining action sequence when an action fails verification and generates a new sequence from the updated scene state, retaining previously verified task progress. Similarly, ATP-VPDSG [232] checks the result of each primitive and uses execution feedback to diagnose failures and revise the remaining actions.

When selecting the next action, the planner can also draw on earlier actions and observations rather than relying only on the latest result. MaP-WAM [448] uses records of completed action segments as planning context and checks execution progress to determine when to update the plan and its context. For visual history, ActiveArena-Plan [186] predicts the current subtask and selects historical frames to retain in memory. A separately trained action policy uses the subtask, selected frames, current observation, and robot state to generate actions.

For embodied question answering, HFLEX-EQA [270] combines past visual observations with a floorplan prior and an incrementally constructed scene graph to choose among room exploration, object inspection, and room discovery. Dedicated low-level planners carry out these operations. Likewise, NavProbe [210] retrieves historical images and geometric records to revise intermediate goals and assess whether they have been satisfied before selecting the next navigation action.

However, revising the action sequence may be insufficient if the planning model does not describe the situation the robot encounters. To address this problem, HUME [382] proposes possible additions to the domain model, treats them as uncertain hypotheses, and specifies conditions for checking them. Perception then confirms or rejects these hypotheses, and rejected hypotheses guide subsequent replanning. In this case, execution feedback changes both the plan and the model used to generate it.

Overall, how far ahead a robot plans should depend on how reliably it can predict the results of its actions. When those results are predictable, longer plans can organize dependencies across several operations. When outcomes are uncertain or the scene changes, planning fewer steps ahead allows new observations to guide subsequent decisions. In either case, checks before execution and feedback during execution help determine whether to continue with the plan, revise its actions, or update the model used to generate it. Related works are compared in Table 4.

Table 4. Comparison of works on planning (Section 4.3). Manipulation includes mobile manipulation; Model lists planning, replanning, or verification backbones.

Work	Link	Task	Plan Output	Model
KnowNo [279]		Manipulation	Action	PaLM-2-L
SayPlan [274]	-	Manipulation	Action	GPT-4
RoCo [237]		Manipulation	Action; Trajectory	GPT-4
ReflectVLM [71]		Manipulation	Action	LLaVA-1.5-13B
HiRobot [297]	-	Manipulation	Goal	PaliGemma-3B
Gemini Robotics 1.5 [320]	-	Manipulation	Goal	Gemini Robotics-ER 1.5
EmboTeam [428]	-	Manipulation	Program	GPT-4o
DV-VLN [194]		Navigation	Action	Llama-2-7B
AARC [202]	-	Manipulation; Navigation	Action	GPT-o3
AgentRob [218]		Human–Robot Interaction	Action	Doubao-Seed-1.8
Reflective Test-Time Planning [105]		Manipulation	Action	LLaVA-3D-7B; Qwen2.5-VL-3B
HLMF [145]	-	Manipulation	Action	GPT-4o
FLA [290]		Manipulation	Action	GPT-4.1; Claude Sonnet 4
IMR-LLM [306]	-	Manipulation	Schedule; Program	GPT-4o; Qwen3-32B
SERP [171]	-	Navigation	Trajectory	Qwen3-VL-Plus; Qwen3.7-Plus
EEAgent [340]	-	Manipulation	Action	GPT-4o
LocalNav [9]	-	Navigation	Goal	Qwen3.5-4B
HUME [382]		Manipulation	Action	GPT-4.1
WCM [51]	-	Manipulation	Action	-
STARS [153]		Manipulation	Goal	GPT-3.5 (text-davinci-002); GPT-4
MaCoPlanner [389]		Manipulation	Action	GPT-4o
NavMCP [167]	-	Navigation; Embodied Question Answering	Goal	Qwen3.6-Plus
ATP-VPDSG [232]	-	Manipulation	Action	Qwen3-VL
ActionReasoning 2.0 [335]	-	Manipulation	Action	Qwen3.7-Plus / Qwen3-7B
SafeMem [180]		Manipulation	Action	GPT-4o
MaP-WAM [448]	-	Manipulation	Goal	Qwen3.5-4B; Wan2.2-5B

Continued on next page

Table 4 (continued)

Work	Link	Task	Plan Output	Model
BTA [235]	-	Navigation; Human–Robot Interaction	Program	GPT-OSS-120B
GAVEL [350]	-	Manipulation	Action; Schedule	Qwen3-8B
ActiveArena (Plan) [186]		Manipulation	Goal	Qwen3-VL-2B
HFLEX-EQA [270]		Embodied Question Answering	Goal	GPT-5.5; Gemini 3.5 Pro
CoRelNav [100]	-	Navigation	Goal	Qwen3.7-Plus
NavProbe [210]		Navigation	Goal; Action	GPT-5.5

4.4 Robot Control

Robot control turns the steps chosen during planning into physical movement. How much detail the model needs to provide depends on what the execution layer can already do. This layer may handle inverse kinematics, collision checks, and low-level control. Below, we discuss three ways for the model to guide the robot's movement: action and language commands, target poses and control programs, and motion constraints and cost functions. We then discuss how action models, language models, and fixed controllers handle execution feedback.

Action and Language Commands. A model can control a robot by choosing from a set of predefined actions. For example, [460] gives navigation agents basic movement commands and evaluates waypoint tools as an alternative or additional interface. A fixed execution interface carries out the selected movements. Show-Harness [46] provides commands for translation, rotation, and gripper control, using fixed rules to convert them into target positions, orientations, and gripper settings. ETA [50] translates commands shared across robot types into actions supported by each robot. It also requires a new observation after every tool call that changes the scene and blocks subsequent actions when the grasp or attachment evidence they require is missing. Demonstrations can also show the model how these commands affect the robot. RoboDawn [93] uses task demonstrations to guide a VLM's choice of translation, rotation, and gripper commands. Even when no task demonstrations are provided, the model still receives examples showing the effects of the basic commands.

The execution layer can also accept language instructions. For example, [130] searches for instruction sequences that improve a frozen VLA's performance and distills them into a language-feedback policy. During deployment, this policy supplies new instructions only when a calibrated predictor judges that they are likely to help; otherwise, the VLA keeps the original instruction. ART [412] instead adds reasoning and tool use to the VLA itself, while retaining a separate output path for continuous robot actions.

Target Poses and Control Programs. When predefined actions or language instructions do not specify enough spatial detail, a model can provide target positions and orientations directly. For instance, VIA [107] lets an agent adjust a virtual gripper, preview its pose, and submit it as a waypoint. Screenshots, coordinates shown when hovering, and pose readouts help the agent choose the target without processing raw arrays of 3D points. GPT-Policy [53] uses a VLM to request robot movements and gripper operations in a structured format. A context compiler assembles task references and interaction history for the VLM, while demonstration actions provide examples for its reasoning. The execution software interpolates between poses, solves inverse kinematics, sets motion timing, and checks execution. These checks determine whether the requested motion is kinematically valid and whether the robot has settled near the target pose. Collision avoidance and task completion require separate checks.

Other systems help the model choose and check motion targets. World Action Agent (WAA) [440] lets an agent preview and edit proposed movements in a visual workspace, using scene geometry and motion-planner feedback before execution. Textual procedures and reference images stored as reusable skills can guide later actions. Robo-Harness K1 [193] provides calibrated spatial measurements and persistent visual anchors to guide motion requests. Its tools support both bounded incremental movements and direct moves toward a target, returning achieved motion and residual error after execution.

Another approach is to generate motion from templates or keyframes prepared in advance. For example, ManiSkillFormer [419] uses human-defined skill schemas to guide agents in generating geometric requirements and motion templates before deployment. During execution, perception supplies the requested geometry, which the templates use to produce robot waypoints. For whole-body motion, KINO [39] uses a VLM to select task-relevant keyframes from a predefined library. A retargeter adjusts the selected keyframes to the current scene, and a keyframe-conditioned policy generates joint-level actions.

A model can also write a program that coordinates several movements and checks their results. MEMENTO [314] evolves executable robot policies using rollout feedback and memory-guided program revision. Its demonstrated policies use finite-state control, grasp and placement checks, and retries to handle execution failures. FAEA [325] generates Python programs that use APIs to take actions, read observations, reset the environment, and check success. It uses states and errors observed during simulator rollouts to revise these programs. Its evaluation provides ground-truth object and gripper states, so applying the same approach to camera images would require perception to supply the state information expected by the APIs. In manufacturing, A-RAM [79] uses an LLM to interpret manufacturing goals and identify which planning variables to explore. Specialized tools then generate G-code and check inverse kinematics, trajectory timing, and joint motion before execution. During execution, a digital shadow records robot and extrusion states and compares them with the selected plan.

Motion Constraints and Cost Functions. Instead of specifying a target pose or a sequence of commands, a model can define requirements for the motion and let a solver find a trajectory. ReKep [120] generates constraints on relationships between keypoints, both at subgoals and along the path. It tracks these keypoints as the scene changes, repeatedly optimizes the motion, and supports recovery when a task stage fails.

VoxPoser [118] writes programs that build spatial value maps for where to move, which regions to avoid, how to orient the gripper, how fast to move, and when to open or close the gripper. These maps are updated from new observations during model-predictive control. In both ReKep and VoxPoser, the motion solver can respond to changes in geometry without calling the language model at every control step.

For manipulation that involves substantial physical contact, CoRAL [469] uses an LLM to generate cost functions and contact strategies for motion optimization, while a VLM provides initial estimates of physical parameters. Interaction feedback supports updates to the physical model and revisions to the cost function or contact strategy. This separates two kinds of adjustment: the solver can find a different motion under the current requirements, or the model can change those requirements when they prove unsuitable.

Execution Feedback. As the robot moves, the system needs to monitor what happens and decide how to respond. Some systems process this feedback within the action model, while others send it to a language model. For example, ARMS [411] adds a visual trigger, robot-state predictions, and a compressed history of past actions to a VLA. These components help the VLA decide when to act and how to respond, while perception and memory continue to update during movement. However, remembering past actions does not guarantee that stored object locations are still correct. If an object moves without being observed, the robot needs a new observation to update its location.

Feedback can also pass from a motion controller to a language model. COMPASS [329] uses this approach in simulated multi-agent flocking, separating slower language-model reasoning from faster motion control. A spatial-transformer controller generates actions and sends learned feedback tokens to the language model. The language model then uses these tokens to update its reasoning.

Other systems give the language model a more limited role before execution. In one such system, the LLM interprets the instruction and resolves ambiguity, then a deterministic finite-state controller carries out the selected operation [104]. The execution layer maps validated tool calls to predefined motion-planning templates. Before moving the robot, it checks kinematic feasibility and possible collisions. Here, the described checks take place before movement, while the fixed controller manages execution.

Overall, the model's output should match what the execution layer can do. For example, if the required skill is already available, the model may only need to name the action or provide a language instruction. When these commands are not enough, it can specify target poses, write control programs, or define

motion constraints and cost functions. In each case, the system can check whether the robot can carry out the requested movement before execution. It can then use feedback during and after execution to check whether the action worked and decide what to do next. Related works are compared in Table 5.

Table 5. Comparison of works on robot control (Section 4.4). Control Output lists the commands or representations passed to the execution module. Executor identifies the module that turns these outputs into robot actions. Feedback lists the information returned during execution.

Work	Link	Control Output	Executor	Feedback
VoxPoser [118]		Cost	Optimizer	Vision; State
ReKep [120]		Constraint	Optimizer	Vision; State
FAEA [325]		Program	Interpreter	State; Execution Status
CoRAL [469]	-	Cost	Optimizer	State
SCOPE [103]		Action	Controller	Text; Execution Status
LWSV [130]	-	Language	Learned Policy	Vision; State
ABot-N1 [86]		Language; Target	Learned Policy	Vision
VIA [107]		Target	Controller	Vision; State; Execution Status
MEMENTO [314]		Program	Interpreter	State; Execution Status
ETA [50]		Action	Controller	Vision; State; Execution Status
ART [412]	-	Action	Controller	Vision; State
BSP [104]	-	Action	Controller	State; Execution Status
Show-Harness [46]		Action	Controller	Vision; State; Execution Status
ManiSkillFormer [419]	-	Target	Controller	State; Execution Status
A-RAM [79]	-	Trajectory	Controller	State
GPT-Policy [53]		Target	Controller	Vision; State; Execution Status
KINO [39]	-	Target	Learned Policy	Vision; State; Execution Status
RoboDawn [93]		Action	Controller	Vision; State; Execution Status
ARMS [411]	-	Feature	Learned Policy	Vision; State
COMPASS [329]	-	Feature	Learned Policy	State; Feature
Robo-Harness K1 [193]		Target	Controller	Vision; State; Execution Status
WAA [440]	-	Target	Controller	Vision; Execution Status

4.5 Adaptation

Adaptation enables robotic systems to improve from experience by modifying inference-time conditioning, planner configurations, or learned model parameters. These mechanisms differ in their scope and persistence, but can be combined to support both immediate recovery and long-term improvement.

Experience-Conditioned Decision Making. Prior experience can guide decisions without updating model parameters. TidyBot [376] infers user preferences from examples, while DROC [430] extracts reusable knowledge from language corrections. CMMR-VLN [173] converts retrieved navigation experience into prioritized rules, whereas PhysMem [172] maintains and revises hypotheses about physical interactions. These methods connect adaptation to memory (Section 7) by conditioning subsequent decisions on stored experience. Similarly, GPT-Policy [53] conditions a frozen VLM on demonstrations, action references, and interaction history to support within-task adaptation. However, improvements that depend on retained context do not necessarily indicate persistent capability acquisition.

Planner Adaptation. Experience can also improve planning without modifying the underlying policy. SERP [171] combines gradient-based optimization of planner parameters with VLM-guided adjustments when optimization stalls. Its MARS extension further retrieves prior recovery experience to inform subsequent configurations. Such methods primarily address failures in planning and search rather than limitations in the policy's learned capabilities.

Policy and Model Updates. Parameter updates enable more persistent behavioral changes. OLAF [209] converts verbal corrections into revised action labels for visuomotor policy training. Yell At Your

Robot [296] instead updates a high-level policy while keeping the low-level controller fixed. Uncertainty-Aware Policy Steering [421] separates instruction clarification from residual policy learning using corrective demonstrations. These approaches target different sources of error, including task interpretation, action selection, and execution.

Autonomous interaction provides an additional learning signal. Self-Improving Embodied Foundation Models [289] uses a frozen supervised checkpoint for progress rewards and success detection while optimizing a separate policy with reinforcement learning. Its physical experiments use LanguageTable, whereas ALOHA self-improvement is evaluated in simulation. Language Model Predictive Control [197] fine-tunes a code-generating LLM through simulated teaching interactions. Reflective Test-Time Planning [105] updates high-level action selection and reflection from retrospective evaluations while keeping the external evaluator fixed. These methods illustrate how learning can target distinct components under different supervision mechanisms.

Learning across Tasks and Deployments. Experience collected during execution can support subsequent training. PAFF [80] relabels exploratory trajectories with language instructions, while RoboCat [18] incorporates data generated by task-adapted policies into successive generalist training iterations. VERITAS [437] selects verified successful executions, whereas HERO [174] combines heuristic execution and exemplar reuse before policy training. VLAW [95] iteratively improves a world model using real interactions and filters generated trajectories for policy updates. At deployment scale, RECAP [268] and Learning While Deploying [359] leverage autonomous experience and human corrections to improve policies. These approaches connect adaptation to data collection and selection (Section 5), with performance depending on supervision quality, experience diversity, and interaction costs.

The choice of adaptation mechanism depends on the source of failure. Perception errors may require additional observations, planning errors may require replanning, and persistent execution failures may motivate policy updates. Evaluation should distinguish within-episode recovery, cross-task generalization, and improvements following retraining, while assessing whether previously acquired capabilities are preserved. Related works are compared in Table 6.

Table 6. Comparison of works on adaptation (Section 4.5). Demonstrations include task examples and action trajectories; Corrections include corrective instructions and interventions. Execution Outcomes include success, failure, and execution feedback.

Work	Link	Adapted Component	Adaptation Method	Adaptation Signal
PAFF [80]		Model Weights	Supervised Learning	Demonstrations; Execution Outcomes
TidyBot [376]		Context	Summarization	Demonstrations
RoboCat [18]	-	Model Weights	Supervised Learning	Demonstrations; Execution Outcomes
OLAF [209]		Model Weights	Supervised Learning	Corrections
DROC [430]		Context; Memory	In-Context Learning; Summarization; Experience Retrieval	Corrections
LMPC [197]	-	Model Weights; Context	Supervised Learning; In-Context Learning	Corrections; Execution Outcomes
Yell At Your Robot [296]		Model Weights	Supervised Learning	Corrections
SIEFM [289]		Model Weights	Reinforcement Learning	Rewards
RECAP [268]	-	Model Weights	Reinforcement Learning	Corrections; Rewards
VLAW [95]		Model Weights	Reinforcement Learning	Rewards; Execution Outcomes
PhysMem [172]		Memory	Hypothesis Testing	Execution Outcomes
Reflective Test-Time Planning [105]		Model Weights; Memory	Supervised Learning; Reinforcement Learning; Reflection	Rewards; Execution Outcomes
UPS [421]		Model Weights	Supervised Learning	Corrections
SERP [171]	-	Planner Settings; Memory	Parameter Tuning; Experience Retrieval	Execution Outcomes
CMMR-VLN [173]	-	Memory	Reflection; Experience Retrieval	Execution Outcomes

Continued on next page

Table 6 (continued)

Work	Link	Adapted Component	Adaptation Method	Adaptation Signal
Learning While Deploying [359]	-	Model Weights	Reinforcement Learning	Corrections; Rewards
VERITAS [437]		Model Weights	Supervised Learning	Demonstrations; Execution Outcomes
HERO [174]	-	Model Weights; Memory	Supervised Learning; Experience Retrieval	Demonstrations; Execution Outcomes
EmbodiedSkills [351]		Model Weights	Supervised Learning	Demonstrations
GPT-Policy [53]		Context	In-Context Learning	Demonstrations; Execution Outcomes
RoboDawn [93]		Memory	In-Context Learning	Execution Outcomes

5 Data: How Should the Agent Collect, Annotate, and Refine Robot Data?

The progress of agentic robots is largely driven by data processing pipelines for robot learning. Modern agent-centric methods actively collect interactive data, interpret robot demonstration behaviors, and refine supervision signals from real-world execution feedback. This section decomposes agent-oriented data operations into three key stages: *collection*, *annotation*, and *refinement*. Data collection obtains agent-explored interactive trajectories and robot self-demonstration datasets from robot task rollouts. Data annotation augments raw robot data with structured supervision for task understanding, spatial grounding, and so on. Data refinement improves data quality by removing noisy samples and correcting inaccurate supervision. Together, these three stages form a complete data optimization workflow for agentic robot learning. The following sections elaborate on each stage and review representative technical approaches.

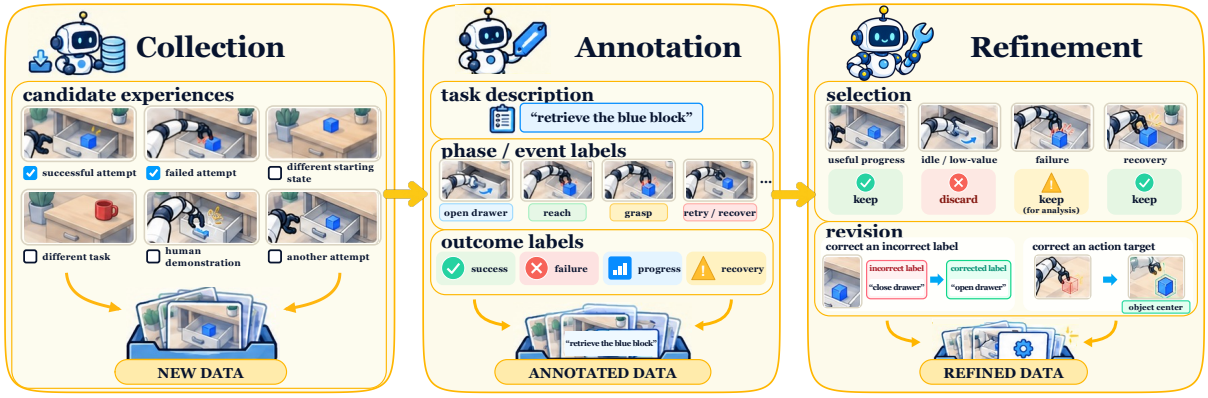


Figure 5. Three data-side interventions over shared robot experience, illustrated by a drawer interaction with failure and recovery. Collection acquires or generates experience; Annotation adds task meaning, temporal structure, and supervision; and Refinement selects useful records or segments and revises unreliable supervision.

5.1 Collection

Agent-based data collection aims to generate new robot trajectories or demonstrations through autonomous agents. We group these methods into three categories. *Physical Data Collection* gathers data from real-robot interaction and human demonstration. *Simulation Data Generation* produces trajectories by running tasks, policies, or programs within a simulator. *Generative Data Synthesis* uses generative models to create artificial observations and motions.

Physical Data Collection. An early example, AutoRT [3], uses language models to propose collection tasks drawing upon existing visual scene descriptions and the capabilities of available collection policies. The selected tasks are then attempted through scripted picking, RT-2 [19], or human teleoperation to

collect real-world trajectories. SOAR [467] uses CogVLM [352] to identify tasks that can be performed in the current scene, then prioritizes less-attempted tasks to collect data across a wider range of tasks.

Sustained agent-based data collection involves key design choices for task execution, scene reset, and human intervention. Zero2Skill [334] leverages Seed1.8 [22] to process images captured after data-collection and scene-reset stages. Based on the resulting trial-outcome evaluation, the data-collection loop either proceeds onward or retries the ongoing trial, with human assistance invoked once retries reach the predefined limit. RoboClaw [178] combines visual observations with task progress in a VLM controller, which invokes a paired reset policy after confirming task completion.

Agents can also direct additional data collection toward underrepresented tasks and weaknesses revealed by robot execution. Tether [199] prioritizes tasks lacking sufficient successful demonstrations and generates scene-adapted preparatory actions. In its early collection experiments, the pipeline used Gemini 2.5 Flash [56] to plan task sequences for scene preparation leading to the target collection task, while GPT-4.1 judged task success. Success evaluation compared camera views before and after execution, with an additional check matching the source demonstration's final keypoint to the gripper's final image position. The paper later adopts Gemini Robotics-ER 1.5 for both task selection and success evaluation, and reports a post-hoc evaluation without the additional correspondence check. RoboLineage [231] reviews recorded video segments and keyframes, together with operator notes and robot logs, to identify where task execution fails. It combines these findings with data-coverage and policy-evaluation summaries to recommend additional collection for weak task phases. FIND [68] also uses a VLM to judge each attempt as successful or unsuccessful from the task instruction and before-and-after scene images. UPS [421] requests human corrections when its verifier determines the current policy cannot complete the task.

Simulation Data Generation. RoboGen [362] is an early example that introduces a robotic agent that proposes tasks, constructs simulation environments, and generates training supervision. Specifically, the agent decomposes the task into subtasks, selects and arranges the required objects in Genesis [82], and uses methods such as motion planning to generate robot behavior in simulation. To generate tasks grounded in everyday human activities, Video2Policy [409] uses internet videos as the source of task specifications. It reconstructs the manipulated objects and estimates their poses, then uses a vision-language model to generate executable task code for Isaac Gym [236]. GenSim2 [113] uses an agent to create manipulation tasks and guide motion planning, then executes the motions in SAPIEN [381] to collect robot trajectories. RoboFactory [273] focuses on collaboration between multiple robots, using a language-model planner to assign subtasks and coordinate their execution in ManiSkill [318].

More recent work broadens the types of simulation data collected and gives agents greater responsibility for the collection process. RoboTalk [84] generates communication messages alongside joint robot actions, validates their sequences, and replays them in RoboCasa365 [246] to collect the corresponding visual observations. ARSTAG [169] and EmbodiedSWE-Gen [294] automate the preparation and expansion of collection programs. Policy failures can also guide what the agent collects next. EmbodiRSI [158] monitors policy execution in Isaac Sim and uses the observed failures to generate recovery trajectories and collect additional data in difficult scene configurations. The policy is then updated and evaluated again, allowing the agent to adjust subsequent collection to the robot's remaining weaknesses.

Generative Data Synthesis. Unlike previous methods like RoboTransfer [214], Real2Edit2Real [451], RoboVIP [333], and RoboDream [407] that synthesize robot data through predefined generation and trajectory-processing pipelines, agent-based methods use large language models or multi-modal large language models to decide what data to generate and coordinate the generation process. RoboCurate [152] leverages Gemini 3 Pro to derive manipulation tasks from visual scene observations and produce prompts for video generation. Additionally, it checks whether simulator replays of the predicted actions are consistent with the motion in the corresponding generated videos. V-Dreamer [101] uses Qwen-Max [4] to turn a scene description into a list of object descriptions, which guides 3D asset generation and scene construction. For long instructions, a foundation model divides the task into subtasks that are processed in sequence. A video model generates a manipulation video from an image of the simulated scene and the current subtask instruction. The pipeline tracks points on the target object across video frames and reconstructs their three-dimensional motion. It combines this motion with a generated grasp pose to

compute a robot trajectory, then executes the trajectory in Genesis [82] to collect demonstrations for policy training. Wh0 [45] uses two agents to generate manipulation instructions: one expands object and attribute vocabulary, while the other selects less-used words based on usage counts. Qwen3-VL describes the required hand and object motions to guide human-hand video generation from edited workspace images. Wrist poses and finger-joint movements recovered from these videos provide action labels, which are combined with real robot demonstrations to train a dexterous policy. These collection routes concern experience records. RATs [435] and Teach and Grow [249] instead primarily retain programs, lessons, or behavior-library entries. Their practice-to-skill workflows are therefore discussed in Section 7.6, which examines the construction, validation, and reuse of procedural artifacts. Related works are compared in Table 7.

Table 7. Comparison of works on data collection (Section 5.1). Type: physical interaction; simulator execution; generative-model synthesis. Counts retain each paper's units and scope.

Work	Link	Type	Collected data	Reported scale
RoboGen [362]			Robot trajectories	**106** generated tasks
SOAR [467]			Robot trajectories	~**30,500** trajectories in the released dataset.
Video2Policy [409]	-		Robot trajectories	**100** trajectories/policy from **100** lifting videos
RoboFactory [273]			Robot trajectories	Up to **150** demos/task across **11** tasks
HumanoidGen [140]			Robot trajectories	**100** demos/task across **20** HGen-Bench tasks
RoboCurate [152]	-		Robot trajectories	**10,000** generated trajectories before filtering
UPS [421]			Teleoperated demonstrations	–
Tether [199]			Robot trajectories	**1,085** successes in **1,946** attempts
V-Dreamer [101]	-		Robot trajectories	Up to **2,500** generated trajectories for lifting
RoboLineage [231]			Robot trajectories	**30** demos/task/round over **3** rounds
Wh0 [45]			Human-hand videos; Hand poses	**50,000** generated video episodes in WM-H
Zero2Skill [334]			Robot trajectories	**50** successful demos for desktop clearing
ARSTAG [169]	-		Robot trajectories	**1,000** successful demos/task for sim-to-real transfer
RoboTalk [84]	-		Inter-robot messages	**7,950** validated trajectories across **53** tasks
FIND [68]	-		Robot trajectories	**456** episodes in six hours
EmbodiRSI [158]	-		Robot trajectories	**400** sim + **10** real trajectories/subtask

5.2 Annotation

While data collection yields videos paired with robot-state sequences, such records are often deficient in textual descriptions, spatial markers, interaction targets, and outcome annotations. To remedy this gap, agent-driven annotation employs intelligent agents to interpret raw records, orchestrate annotation utilities, and produce as well as validate the missing labels. We categorize existing agent-based annotation approaches into four groups: *text annotation*, *spatial annotation*, *interaction annotation*, and *outcome annotation*, based on their respective annotation objectives.

Text Annotation. The most common agent-based text annotation is to produce subtask descriptions and video captions that mark task stages and observable events, respectively. For example, NILS [16] generates task-level language instructions for robot video segments. Specifically, it detects segment boundaries from object movements, state changes, and gripper interactions, then uses Gemini Pro to describe the manipulation stage performed in each segment. IVLR [211] generates subtask-level video captions. It divides a robot trajectory into stages, then queries Qwen3-VL to describe the detailed gripper-object interaction or spatial relation in each stage. EgoTools [322] adopts Gemini 3 Flash to caption consecutive video segments, propagating prior captions to subsequent segments for improved temporal consistency.

Another form of text annotation adopts question-answer (QA) pairs to capture scene properties, action sequences, and fine-grained execution details. For example, RoboAnnotatorX [154] uses GPT-4o to generate QA about the objects present, their spatial relationships, and the order of robot actions, using scene descriptions and timestamped action annotations as the source information. Meanwhile, AnnotateAnything [227] creates robot-oriented QA pairs from simulation trajectory logs. A VLM varies question phrasing for target objects, interacting parts, approach directions and execution outcomes, with answers sourced from simulator records. REAL [241] and RoboTalk [84] use Gemini models to generate dialogue annotations alongside robot actions, covering clarification of ambiguous instructions and communication between cooperating robots. Methods such as SpatialVLM [33], SpaceVista [311], and Exemplar2VQA [415] combine visual models or coding agents with 3D geometry to generate question-answer pairs about object positions, distances, and spatial relationships. RoboSpatial [301] derives spatial answers from 3D geometry, projection, and ray casting, and constructs question-answer pairs with deterministic templates.

Beyond question-answer pairs, reasoning-trace annotations augment recorded behaviors with high-level plans and explanatory rationales. Specifically, ECoT [425] and TIC-VLA [124] use Gemini 1.0 and GPT-5, respectively, to generate reasoning traces that explain the robot's current subtask and next action in manipulation and navigation tasks. Along this line, ERVLA [310] utilizes Qwen3.5-397B to derive shared goals and subtask sequences per episode, alongside fine-grained reasoning for selected timesteps.

Spatial Annotation. Robotic agents perform spatial annotation by interpreting demonstrations, invoking perception tools, and verifying spatial labels. NILS [16] and FSD [424] use language-model-guided pipelines to identify objects for annotation. NILS uses Grounding DINO [216] to obtain object bounding boxes and DEVA [54] to track object masks across video frames. FSD uses Grounded SAM [281] to obtain bounding boxes and segmentation masks. AnnotateAnything [227] and EmbodiedGen V2 [357] combine 3D segmentation with VLM-based interpretation to annotate object parts, with EmbodiedGen V2 additionally checking and refining the resulting labels. SPARC [17] and Keypose Exploration [229] interpret interaction events through language models and align them with visual tracks or robot states to annotate trajectories and keyposes. AutoFocus-IL [85] and Workspace Models [57] identify task-relevant objects and events through VLM reasoning and localize them to generate spatial saliency annotations.

Interaction Annotation. Interaction annotation captures physical engagements between hands, robotic grippers, tools and target objects, covering typical behaviors such as touching, grasping, pushing, and pressing. The annotation pipeline generally consists of reasoning over interaction intent and retrieving contact-related evidence via external tools. For affordance annotation, several representative works, such as AFUN [364], EgoAfford [91], Affordance2Action [213], A4-Agent [444], and Affordance Agent Harness [114] integrate model reasoning with tool-aided localization. In these works, agents or LLM-driven workflows first identify object regions suitable for the desired manipulation, then call detection and segmentation modules to pinpoint these functional areas. For dexterous hands, DextER [162] and OmniDexVLG [436] combine model reasoning with tool-derived contact evidence to annotate finger- and link-level contacts with object parts and interpret their grasp functions.

Outcome Annotation. Outcome annotation labels task achievement in robot executions, including intermediate progress, final success or failure, and safety violations. Agents or model-guided workflows interpret evaluation criteria and inspect execution evidence to produce these labels. For task achievement, RoboReward [163] and Zero2Skill [334] use visual-language judgments against explicit task criteria to assign completion scores or success labels. For intermediate progress, ROVER [288] and ProcVLM [70] organize visual reasoning around subtasks to annotate their completion states and estimate task progress. ARS [109] further uses agents to invoke video-inspection tools, verify proposed task events, and revise frame-level progress annotations. For safety, ShieldVLA [319] uses an agentic workflow to refine safety criteria from misranked examples and generate calibrated frame-level risk labels. Related works are compared in Table 8.

Table 8. Comparison of works on data annotation (Section 5.2). Input: Image; Video; Robot data; Point cloud. Robot data includes demonstration metadata and logs. The Models column lists main semantic or reasoning backbones; the Tools column lists supporting perception models, algorithms, and simulation interfaces. Representative annotation configurations are shown.

Work	Link	Input	Output	Models	Tools
Text Annotation					
SpatialVLM [33]	-		QA pairs	FlexCap	RPN; ShapeMask; ZoeDepth; Geometric QA
ECoT [425]			Reasoning traces	Prismatic-7B; Gemini 1.0	Grounding DINO; OWLv2; SAM
NILS [16]			Task descriptions	Gemini 1.5; Gemini Pro	Grounding DINO; DEVA; Depth Anything; GMFlow; SigLIP
RoboSpatial [301]			QA pairs		Geometric rules; QA templates
FSD [424]			QA pairs	GPT-4o	Grounded SAM; Metric3Dv2; Geometric QA
IVLR [211]	-		Captions	Qwen3-VL	UVD
SPARC [17]			Task descriptions	Qwen3.6-30B	Gripper-event parsing
AnnotateAnything [227]	-		QA pairs	NR	Simulation-log queries
REAL [241]			Dialogue; Reasoning traces	Gemini 3 Pro	Trajectory replay
Exemplar2VQA [415]			QA pairs	Qwen3-Coder-30B-A3B-Instruct	Geometry APIs; AI2-THOR
EgoTools [322]			Captions	Gemini 3 Flash	
Spatial Annotation					
ECoT [425]			Bounding boxes; Gripper positions	Prismatic-7B	Grounding DINO; OWLv2; SAM; RANSAC
NILS [16]			Bounding boxes; Object masks	Gemini 1.5	Grounding DINO; OWLv2; DEVA
FSD [424]			Bounding boxes; Object masks	GPT-4o	Grounded SAM
SPARC [17]			3D object trajectories	Qwen3.6-30B	LLMDet; SAM2.1 Hiera-Small; RobotSeg; AllTracker; MoGe-2
AnnotateAnything [227]	-		3D part masks; Semantic labels	Qwen3.5-VL	P3-SAM; X-Part
EmbodiedGen V2 [357]			3D part masks; Semantic labels	GPT-5.4	P3-SAM
Interaction Annotation					
A4-Agent [444]			Affordance masks	GPT-4o	Qwen-Image-Editing; Rex-Omni; SAM2-Large
DextER [162]			Contact points; Grasp descriptions	Gemma 3	MuJoCo contact queries
Affordance Agent Harness [114]			Affordance masks	GPT-4o	Qwen3-VL-235B-A22B; SAM2-Large; Retrieval; Interaction imagination
Affordance2Action [213]			Affordance masks	Qwen3.5-9B	SAM3; Grounding DINO
Outcome Annotation					
ROVER [288]			Progress scores	Gemini 1.5 Pro	
Zero2Skill [334]			Success labels	Seed1.8	
ARS [109]	-		Progress scores	Qwen3.6-27B	Video inspection; Timeline editing; Progress editing

5.3 Refinement

Data refinement improves the suitability of recorded robot demonstrations through data selection and supervision revision. Agent-based systems combine model reasoning with execution evidence to determine which trajectories or segments to retain and whether earlier annotations need correction.

RoboLineage [231] and Zero2Skill [334] use model-assisted review to determine whether recorded trajectories meet the criteria for training. Selection can also operate within a trajectory: HELP [431] and ARS [109] use estimated task progress to retain useful segments while excluding unproductive actions, allowing parts of unsuccessful attempts to support learning. HELP relies on its VLAC-CUT critic to

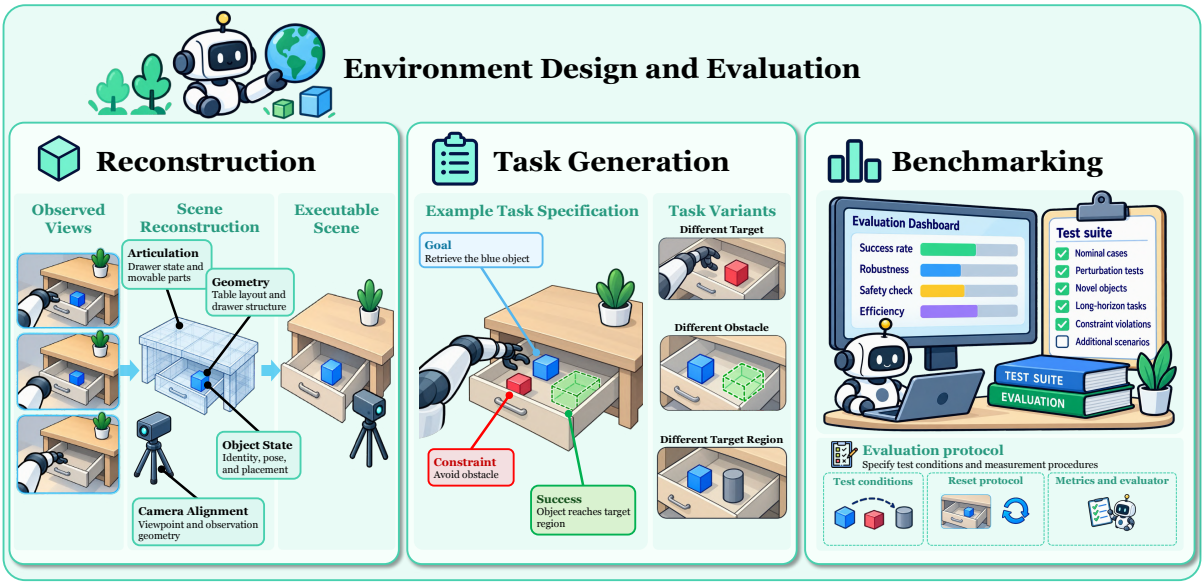


Figure 6. Three ways to build and test an environment for one robot task. Reconstruction recovers an executable scene from observations. Task Generation defines goals, constraints, and success conditions. Benchmarking fixes test conditions, reset protocols, metrics, and evaluators for success, safety, efficiency, and robustness.

distinguish progress and recovery from idle or failure-inducing behavior, while ARS uses agents to call video-inspection tools and verify task events before progress estimates guide segment selection.

Zero2Skill [334] and Reflective Test-Time Planning [105] revisit earlier supervision when feedback changes how an execution should be judged. Zero2Skill applies human-corrected success criteria to previously collected demonstrations and updates their labels. Reflective Test-Time Planning uses later execution outcomes to revise earlier action evaluations, retaining the updated reflections and scores as supervision for subsequent learning. Related works are compared in Table 9.

Table 9. Comparison of works on data refinement (Section 5.3). Model lists backbones used to review data or revise supervision.

Work	Link	Refinement Target	Model
Reflective Test-Time Planning [105]		Action scores and reflections	LLaVA-3D-7B / Qwen2.5-VL-3B
RoboLineage [231]		Robot trajectories	Claude Sonnet 4.6
Zero2Skill [334]		Robot trajectories; Outcome labels	Seed1.8
ARS [109]	-	Action segments; Progress annotations	Qwen3.6-27B

6 Environment: What Worlds, Tasks, and Tests Should the Agent Build?

The Environment branch covers the world, task, and evaluator that shape robot interaction. Reconstruction, Task Generation, and Benchmarking address these three objects (Figure 6). They can fail independently: a scene may be unstable, a task may be infeasible, or an evaluator may say little about the ability under test. In a drawer task, the scene must allow the joint motion, the task must identify the object to retrieve, and the evaluator must check the final relation. The construction loop should link each failed check to the object that needs revision.

6.1 Reconstruction

Reconstruction turns observations, demonstrations, or a requested layout into an executable scene. It must recover the structure that a task uses, the changes caused by interaction, and an interface through which an

agent can inspect and revise the result. Checks should expose errors that matter for execution rather than reward visual similarity alone.

Static Structure. Static structure includes objects, poses, supports, and the properties that make a scene stable. ARSTAG [169] reconstructs instruction-relevant objects and support relations, links identities to segmentation masks, and repairs poses before collecting demonstrations. SimFoundry [275] decomposes video into meshes and articulated parts, stabilizes the scene, and creates digital-cousin variants; Genie Sim 3.0 [414] assembles scenes from an asset library. RoboLab [398] returns displacement errors from stability simulation to the generator, while RoboPlayground [360] settles a goal configuration and repairs task code when its success predicate fails. GIF [393] checks the requested relation after settling objects before generating demonstrations. These checks keep reconstruction tied to support, contact, and task feasibility.

Dynamic and Articulated Structure. A replayable scene must also respond correctly to new actions. Agentic Real2Sim [36] identifies task-relevant objects, reconstructs their meshes, and tracks 6-DoF poses, with optional VLM refinement. Video2World [316] evaluates coding agents that recover a scene and a control program from embodied video, while 4DCodeBench [292] tests code-based dynamic reconstruction with analytic motion, keyframes, or simulation. Matching observed motion does not establish correct response to a new action, so these are separate checks.

Articulated assets require kinematic tests. Articulate-Anything [159] renders generated link and joint code, compares it with the input, and exports a URDF after revision. Articraft [461] specifies parts, joints, and tests in code, using compilation and geometric diagnostics to guide repair. Deformable assets need different representations: CoDimRecon [387] tests curves, surfaces, and volumes through simulation; DiagGen [35] routes diagnostic failures to segmentation, material, or mesh repair; and DexAgent [361] selects reconstruction skills and verifiers for rigid, articulated, or deformable objects before generating trajectories and training data. These procedures establish simulated behavior and task feasibility, not real-world validity.

Construction Interfaces and Revision. The interface determines which scene changes an agent can make. SceneSmith [266] edits structured scene objects through designer, critic, and orchestrator roles with rollback. SAGE [379] exposes placement, motion, and deletion operations and rejects or repairs unstable arrangements. EmbodiedGen V2 [357] provides typed scene-graph operations over a persistent world state. Code interfaces allow more expressive changes: ChronoAgentic [338] generates and repairs PyChrono code; SimWorld Studio's SimCoder [144] uses compilation, collision, physics, and visual feedback; LEGO-Anything [183] rebuilds an image in Blender through repeated code execution; and Code4Scene [410] checks Unreal scene edits for support, penetration, and navigation utility. A constrained API narrows the error space, whereas generated code needs software, geometric, and physical checks.

Revision should match the intended downstream use. RAPID [222] screens perturbed simulator variants with agent-written feasibility filters and keeps the resulting primitive and strategy code fixed at deployment. Awomo-SimDataEngine [6] builds worlds in stages with acceptance checks and dependency-aware repair. DexSim2Real [429] uses a VLM realism critic and an LLM-derived skill curriculum to tune simulation randomization for sim-to-real transfer. Appearance and stability checks cannot replace contact, friction, reachability, or downstream execution tests. Related works are compared in Table 10.

Table 10. Comparison of works on reconstruction (Section 6.1). Selected inputs and representations may co-occur. Scene includes graphs and layouts; Params.: parameters. Engines lists construction or verification software, excluding export-only targets. Input: Text; Image; Video; 3D assets.

Work	Link	Input	Representation	Agent Operations	Engines
Assets					
Articulate-Anything [159]			URDF; Code	Retrieve; Fit; Edit	PyBullet; SAPIEN
Articraft [461]			URDF; Code	Generate; Edit	CadQuery

Continued on next page

Table 10 (continued)

Work	Link	Input	Representation	Agent Operations	Engines
DiagGen [35]			Mesh; Params.	Generate; Edit	Genesis
Scenes					
Video2Policy [409]	-		Mesh; URDF	Generate; Fit; Assemble	Isaac Gym
Genie Sim 3.0 [414]			Scene; Code	Retrieve; Generate; Assemble	Isaac Sim; OpenUSD
SceneSmith [266]			Mesh; Scene	Generate; Assemble; Edit	Drake
SAGE [379]			Mesh; Scene	Generate; Assemble; Edit	Isaac Sim
RoboPlayground [360]	-		Code	Generate; Edit	MuJoCo
RoboLab [398]			Scene	Assemble; Edit	Isaac Sim
DexSim2Real [429]	-		Params.	Optimize	Isaac Sim
SimWorld Studio [144]	-		Scene; Code	Generate; Assemble; Edit	Unreal Engine
SimFoundry [275]			Mesh; URDF; Scene	Generate; Fit; Assemble	PyBullet; CoACD
EmbodiedGen V2 [357]			URDF; Scene	Generate; Assemble	SAPIEN; CoACD
GIF [393]	-		Mesh; Params.	Generate; Fit; Optimize	MuJoCo; Drake
ARSTAG [169]	-		Mesh; Scene	Generate; Fit; Assemble; Edit	Isaac Sim
RAPID [222]	-		Mesh; Scene	Generate; Fit; Assemble	MuJoCo
CoDimRecon [387]	-		Mesh; Scene	Generate; Fit; Edit	Blender; MuJoCo
DexAgent [361]	-		URDF; Scene	Generate; Fit; Edit	MuJoCo; Isaac Sim
LEGO-Anything [183]	-		Code; Scene	Generate; Edit	Blender
Code4Scene [410]	-		Scene	Generate; Assemble; Edit	Unreal Engine
Awomo-SimDataEngine [6]	-		URDF; Scene	Generate; Assemble; Edit	Isaac Sim
Dynamics					
ChronoAgentic [338]	-		Code	Generate; Edit	PyChrono
Agentic Real2Sim [36]			Mesh; Params.	Generate; Fit; Optimize	MuJoCo
4DCodeBench [292]			Code	Generate; Edit	Blender
Video2World [316]			Code; Scene	Generate; Edit	SAPIEN; Isaac Sim; MuJoCo

6.2 Task Generation

Task generation specifies the goal, initial state, constraints, and success conditions that a robot must execute. It covers both construction of executable tasks and adjustment of task distributions as the learner changes.

Executable Task Construction. GenSim [345] generates task libraries as simulation code. RoboGen [362] couples task proposals with scenes and supervision, while Video2Policy [409] derives task code and rewards from internet video. RoboLab [398] composes tasks from competency axes, predicates, and asset constraints, then feeds syntax and asset errors back to the generator. HARBOR [191] checks scene stages, initialization, termination, actions, and observations. Scene2Demo [219] links subtasks through an object–action graph and generates executable initial and goal conditions. PARTNR [31] filters generated instructions and initial states and builds evaluators for propositions, dependencies, and temporal constraints.

Checks can also govern generation of human interaction and physical feasibility. FATE [366] separates static semantic, geometric, and precondition checks from dynamic execution checks and repairs the failing component. GenPHRI [343] uses a text2sim2real pipeline to generate a human, a scene, and an assistive task; collision and kinematic checks constrain the resulting placements and motions. These repairs can change task difficulty or meaning, so feasibility and objective preservation should be checked together.

Generating a solution does not necessarily generate a new task. EmbodiedSmith [272] composes skills into task graphs and revises plans and initial placements while keeping assets, layout, and success criteria fixed. Scene2Demo likewise revises action flows from rollout critiques and restarts the current subtask from a saved state. The resulting demonstrations belong to data generation (Section 5.1), not to a new task definition.

Adaptive Task Distributions. CurricuLLM [286] generates a subtask sequence, candidate reward and goal-distribution code, and passes the selected policy to the next subtask. RATs [435] selects practice

tasks from scene information, a code-skill library, and failure memory, balancing novelty and difficulty. SimWorld Studio [144] instead feeds embodied-agent performance to an environment generator that changes environments and task difficulty. RATs changes the next goal; SimWorld Studio changes the task distribution. Evaluation on separately specified tasks is needed because a curriculum optimized from the learner's own success signal can become easier without improving the target ability. Related works are compared in Table 11.

Table 11. Comparison of works on task generation (Section 6.2). Output lists generated or revised artifacts. Eval.: success/evaluation code; Config.: initialization/randomization settings; Plan: executable solution, which may preserve the task. UE5: Unreal Engine 5; MolmoSp.: MolmoSpaces. Multiple (HARBOR): Isaac Lab, ManiSkill, Genesis, MJLab.

Work	Link	Input	Output	Agent Operations	Tools
Tasks					
GenSim [345]	📄 GH	Goal; Examples	Task; Curriculum	Generate; Select	Ravens
RoboGen [362]	📄 GH	Assets; Examples	Task; Reward; Plan	Generate; Compose	Genesis; OMPL
Video2Policy [409]	📄 -	Demo	Task; Reward; Eval.	Generate; Repair; Select	Isaac Gym
PARTNR [31]	📄 GH	Scene; Examples	Task; Eval.	Generate; Vary	Habitat 3.0
RoboPlayground [360]	📄 -	Goal	Task; Eval.	Generate; Vary; Repair	MuJoCo
RoboLab [398]	📄 GH	Scene; Examples	Task; Eval.	Generate; Compose; Repair	Isaac Sim
FATE [366]	📄 -	Scene; Goal; Policy	Task; Reward	Repair	Isaac Sim
HARBOR [191]	📄 GH	Goal; Code	Task; Reward; Config.	Generate; Repair	Multiple
Curricula					
CurricuLLM [286]	📄 GH	Goal; Scene	Curriculum; Reward	Generate; Schedule; Select	Gymnasium; Isaac Lab
SimWorld Studio [144]	📄 -	Goal; Scene; History	Task; Curriculum	Generate; Vary; Schedule	UE5; Gymnasium
RATs [435]	📄 GH	Scene; Skills; History	Task	Generate; Select	LIBERO-PRO; MolmoSp.
Plans					
ARSTAG [169]	📄 -	Image; Goal	Plan; Config.	Compose; Vary	Isaac Sim
Scene2Demo [219]	📄 -	Image; Goal	Task; Eval.; Plan	Generate; Compose; Repair	OmniGibson
GenPHRI [343]	📄 -	Goal	Config.; Plan	Generate; Repair	Genesis; SceneSmith
EmbodiedSmith [272]	📄 -	Goal; Robot	Plan; Config.	Compose; Repair	Isaac Sim; cuRobo

6.3 Benchmarking

Benchmarking fixes the task, interface, reset protocol, and scoring rule used to measure an ability. Agents may be the systems under test, or they may construct and revise the evaluation itself. The same task can measure different abilities when observation access, execution assistance, feedback timing, or interaction budgets change.

Evaluating Agents in Robot Control. Planning benchmarks test whether an agent converts an instruction into robot operations. EmbodiedBench [399] varies action abstraction and returns environment feedback. Embodied Agent Arena [115] uses multi-round interaction in its Planning and Manipulation domains. MissionBench [247] evaluates an MLLM that selects motion primitives, magnitudes, and stopping decisions for aerial missions. KinDER [122] evaluates open-loop composition of parameterized skills, so its score does not measure feedback-driven replanning.

Tool and programming benchmarks evaluate executable decisions. EmbodiedToolBench [466] separates tool recognition, selection, execution, output use, and tool-chain composition. CaP-X [74] varies state access, API abstraction, examples, and visual grounding while executing generated control code. EmbodiedSWE [294] gives coding agents simulator state, rendering, and execution feedback, then grades the resulting program in a separate environment.

Existing robot suites can host these evaluations. LIBERO [206] provides a standard lifelong-learning suite, and LIBERO-PRO [465] changes the protocol to reduce memorization and improve robustness. CaP-X evaluates CaP-Agent0 on LIBERO-PRO with position and instruction perturbations, while ASPIRE [228] uses LIBERO-PRO, robosuite, and BEHAVIOR-1K with separate protocols for fixed programs and

trace-based code revision. The agent score therefore depends on whether the program is fixed before evaluation or revised from the current trace.

Harness VLA [441] tests agent selection of analytic primitives with a frozen VLA on LIBERO, LIBERO-PRO, RoboCasa365, and RoboTwin C2R. RoboHarness [116] tests heterogeneous policy orchestration, perturbation robustness, and long-horizon execution on LIBERO variants and adds tasks for policy handoffs. In both cases, the task suite alone does not determine the measured ability; the execution and adaptation protocol does as well. RoboDojo [41] combines 42 simulation tasks with 18 real-world tasks and evaluates generalization, memory, precision, long-horizon execution, and open-vocabulary instruction following. RobotWorld [404] extends this setting across manipulation, mobile manipulation, locomotion, driving, and aerial control. Its 84 simulation tasks specify interaction budgets and executable success checks, and its trace analysis links outcomes to the agent's perception, computation, feedback use, and completion decisions.

Active-perception tests evaluate information gathering. VABench [443] asks an MLLM to inspect a scene, issue camera and Cartesian commands, and revise decisions after new observations. ActiveArena-Plan [186] uses a planner to select subtasks and history for a VLA executor. The former tests observation gathering directly; the latter also tests how selected history guides task progress.

Memory tests ask whether earlier observations remain useful. RoboGraph [346] controls dependencies, observation access, and external events to test state maintenance and updating. RoboMemArena [164] uses a VLM planner to select subtasks and retained observations for a low-level executor. EvoNav-Bench [355] introduces hidden scene changes that can make retained evidence stale. MEMOBench [308] adds executable checkpoints for memory storage, update, and compression, separating memory fidelity from final manipulation success. Although its reported subjects are VLA policies, the same process-level checks can be applied to agent-controlled rollouts; memory mechanisms are discussed in Section 7.

Safety tests separate hazard avoidance from task completion. ReactHuman [190] scores an MLLM's motor plan for a sudden event against simulator ground truth. ManiGuard [264] uses specification-grounded runtime monitors and reports safety separately from task success. Its published evaluations use VLA policies, but the monitors can also score agent-controlled rollouts under the same task protocol. ReactHuman pauses simulation during inference, so it measures the hazard-response decision rather than real-time reaction.

A benchmark score therefore depends on the full execution contract: agent decision, executor, feedback path, observation access, controller assistance, and interaction budget.

Agent-Directed Evaluation Construction. An agent that builds an evaluation must specify the ability under test, the task conditions, and the evidence used for scoring. ManipEvalAgent [49] decomposes an evaluation request, generates tasks and evaluation tools, and uses rollout results to propose further tests or summarize the tested policy. Embodied-BenchForge [133] generates task templates, data, and executable verifiers; interactive tasks require restorable states and terminal checks, while offline tasks require grounded evidence and executable scoring. Its dependency graph supports targeted rollback and repair. A2Eval [438] groups embodied-reasoning questions by capability, generates inference and scoring code, and repairs that code from execution feedback.

Adaptive evaluation can reveal weaknesses, but its conclusions depend on test selection, access to prior results, and score validation. Fixing test conditions independently of the evaluated system supports broader claims. Improving the evaluator and improving the robot are separate outcomes.

World construction, task generation, and benchmarking diagnose different failures: unstable geometry requires scene repair, an infeasible goal requires task revision, and an invalid success signal requires evaluator repair. Keeping these checks separate makes failures easier to locate and simulated conditions easier to validate on hardware. Related works are compared in Table 12.

Table 12. Comparison of works on benchmarking (Section 6.3). Interface describes tested-system outputs: Answer (predictions), Action (motor commands), Call (tools/skills), Code (programs); evaluator-generated code is excluded. RobotWorld's Code is optional. Engines lists simulation backends; Multiple denotes mixed suites. NR: not reported; N/A: offline evaluation without simulator execution. Collab.: collaboration; UE5: Unreal Engine 5. Setting: Real; Sim; Offline data.

Work	Link	Setting	Capability	Interface	Engines
Benchmarks					
LIBERO [206]			Transfer	Action	robosuite
PARTNR [31]			Planning; Collab.	Call	Habitat 3.0
EmbodiedBench [399]			Planning; Reasoning	Action; Call	AI2-THOR; Habitat; CoppeliaSim
LIBERO-PRO [465]	-		Robustness	Action	LIBERO
RoboLab [398]			Robustness; Control	Action	Isaac Sim
CaP-X [74]	-		Coding	Code	robosuite; LIBERO-PRO; BEHAVIOR
KinDER [122]			Reasoning	Action; Call	Pymunk; PyBullet; MuJoCo
RoboMemArena [164]			Memory	Action; Call	NR
EmbodiedTool [466]	-		Tools	Answer; Call	N/A
MissionBench [247]			Planning; Perception	Answer; Action	Cosys-AirSim; UE5
RoboGraph [346]	-		Memory	Call	RoboTwin; RoboCasa
EvoNav-Bench [355]	-		Memory; Robustness	Action	AI2-THOR
ReactHuman [190]	-		Reasoning; Safety	Action	Genesis
VABench [443]	-		Perception; Control	Action	NR
ActiveArena [186]			Perception; Memory	Action; Call	RoboTwin 2.0
EmbodiedSWE [294]			Coding; Control	Code	Isaac Lab
Embodied Agent Arena [115]			Reasoning; Planning	Action; Call; Code	Multiple
RoboDojo [41]			Control; Robustness	Action	Isaac Lab; Isaac Sim
ManiGuard [264]	-		Safety	Action	Isaac Sim; OmniGibson
MEMOBench [308]			Memory	Action	LIBERO; robosuite
RobotWorld [404]	-		Planning; Control	Action; Code	Multiple
Systems					
ASPIRE [228]			Coding; Transfer	Code	CaP-X
Harness VLA [441]			Memory; Robustness	Call	robosuite; RoboCasa; RoboTwin
RoboHarness [116]	-		Planning; Robustness	Call	LIBERO; RoboTwin
AGOS-Agent [416]	-		Planning; Collab.	Call	CARLA-Air
Evaluators					
RoboPlayground [360]	-		Robustness	Action	MuJoCo
A2Eval [438]	-		Perception; Reasoning	Answer	N/A
ManipEvalAgent [49]	-		Robustness	Action	LIBERO; RoboTwin
Embodied-BenchForge [133]	-		Reasoning; Planning	Answer; Call	AI2-THOR

7 Harness: How Should Robot Capabilities Be Orchestrated, Recovered, and Evolved?

A robot may have capable models and policies yet still fail if they are poorly organized around execution. We use the *harness* to denote the software that makes agentic capabilities available for robot use, by managing how components are invoked, what information they receive, how execution is checked, and what can be reused later. This section organizes harness functions into six components, as shown in Figure 7. *Orchestration* governs calls and handoffs between components. *Memory* supplies information retained from earlier steps. *Monitoring and Recovery* assess execution and respond to failure. *Skills* package reusable behavior. *Infrastructure* exposes and gates executable capabilities. *Layered Systems* divide authority across planners, learned policies, and analytic control. The first three components act within a single attempt, Skills carry behavior across attempts, and the last two describe the software and system structure beneath them.

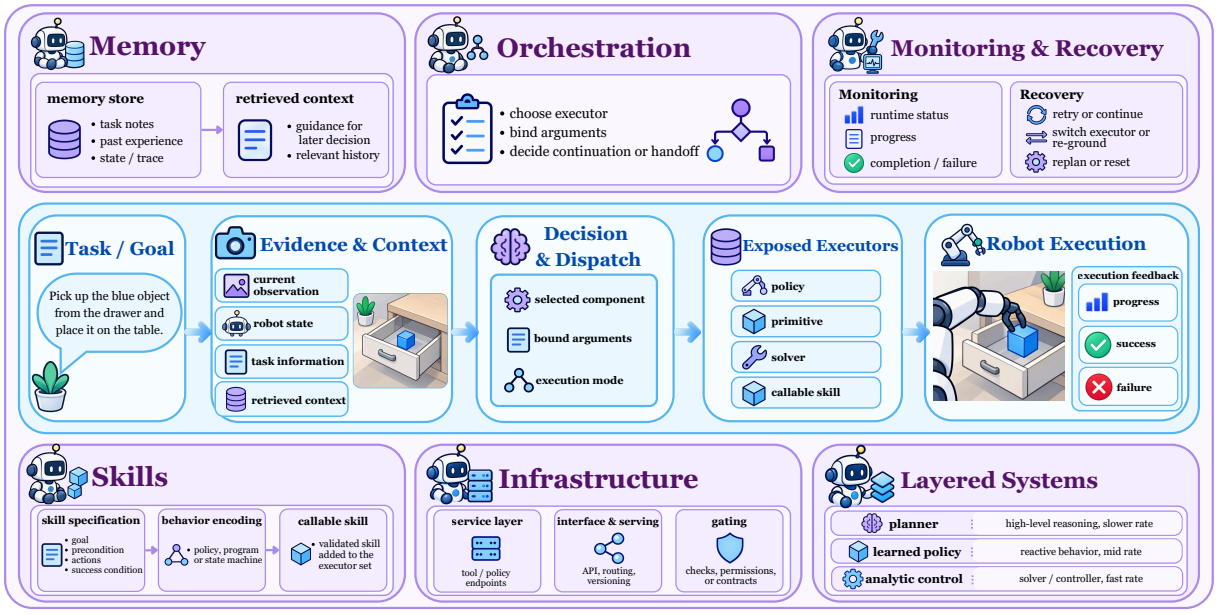


Figure 7. Harness functions that organize agentic capabilities around robot execution. The central row connects task goals and evidence to decision and dispatch, exposed executors, robot execution, and returned feedback. Orchestration governs calls and handoffs; Memory supplies retained information; Monitoring and Recovery assess execution and respond to failure; Skills package reusable behavior; Infrastructure exposes and gates callable capabilities; and Layered Systems divide authority and operating rates across planners, learned policies, and analytic control.

7.1 Orchestration

Orchestration decides which model, policy, skill, or tool runs, how long it keeps control, and how control passes to the next component. We illustrate orchestration by following a single handoff order through four steps, where the harness chooses the next executor, defines what the function calling takes and returns, makes sure the next executor can start from the resulting state, and decides when control comes back.

Routing and Dispatch. In an agentic harness, the planner rarely acts on the robot directly. Instead, it decides which executor should handle the next step, such as a learned policy, an analytic tool, or a human, and passes the task to it. A key design choice is when this decision is made. VoLo [40] makes it during execution. It monitors a running VLA and, when the action goes wrong, halts the VLA and switches to a hand-designed skill guided by perception. MistyPilot [354] makes it in advance. It generates persistent bindings between sensor events and skills for social-robot behavior, so that later events trigger skills without a new planning step. AeroWeaver [226] makes it at the level of the team, assigning body-specific aerial skills to distributed roles in a simulated swarm. M2HRI [99] further includes humans among the parties to route to, adding a coordinator for human participation and an interaction planner conditioned on memory. Together, these systems place the routing decision at different points, from a running action to a standing rule to a team-level role.

Meanwhile, routing is reliable only when the dispatcher knows what each executor can and cannot do. This knowledge can come from a description of the robot's body or from the results of earlier calls. EMOS [37] takes the first approach for teams of different robots. It builds a Robot Resume for each robot from its URDF model and forward-kinematics tools, and each robot's agent uses this resume to assess an assigned subtask and report whether it can perform it before calling its skill library. Harness VLA [441] and Physical Agency [75] take the second approach on a single robot, placing analytic tools and a VLA behind one dispatch interface and choosing the next executor according to the evidence each call returns, such as grasp checks and typed failure messages. Guava [208] applies the same idea across levels of control, choosing between semantic operations, such as grasping and alignment, and numerical motion tools based on visual observations, the robot's state, and earlier tool outcomes.

Function Protocol. Robot capabilities are usually exposed to the agent as functions, which the agent calls with a name and some arguments without seeing how they work inside. A function protocol fills this gap by stating how a function should be called, what it reports back, and when a call counts as successful. The first of these defines what counts as a valid call, and can be written for the agent to follow or enforced by the harness. EmbodiedTool [466] writes it for the agent, giving each tool a typed description of its inputs, outputs, and usage conditions that the agent's call must match. ROSClaw [26] enforces it in the harness, checking each call to a physical TurtleBot3 or Go2 robot against velocity limits and a list of allowed interfaces, so that an invalid call is stopped even if the agent produces it.

Once a call has run, the protocol defines what comes back to the agent. A simple success flag says little about the robot's situation, so some protocols return more. Physical Agentic AI [220] returns each skill outcome with the robot state at that moment, which later dispatch can use without calling the foundation model again. Physical Agency [75] returns typed failures, so that the agent can respond to each kind differently, for example by setting up an operation again or switching to another executor.

Even a detailed result is still the executor's own report. Success conditions add an independent check by stating what should be true after a call succeeds. Thea [348] attaches such conditions to each tool, and a task evaluator separate from the planner checks them against current observations.

State Connection. Each executor works reliably only from certain starting states, so a correctly chosen executor can still fail if the previous step leaves the robot elsewhere. State connection makes sure that the state left by one executor connects to the state the next one needs. Existing systems make this connection in four ways. The first two ways adjust either the state or the executor. RoboHarness [116] adjusts the state to fit the executor. It retrieves past trajectories from memory and uses a motion-planned memory bridge to move the robot to the starting state expected by the next policy. RoboRouter [48] adjusts the executor to fit the state. It selects a policy that suits the current state from several candidates, and uses summaries of past runs to improve later selections.

The other two ways set up the connection before the handoff, which is common when several robots share a task. In this case, the next robot must also know what the others have done and which resources it may use. Some systems check whether the successor is ready. Some systems check this before each handoff. RoCo [237] lets robots agree on a plan through dialogue, then checks whether the plan is feasible before they move. RoboTalk [84] checks each message and joint action against symbolic preconditions and resource ownership, so that a robot acts only when what it was told and what it may use are consistent. Other systems fix the state connection structure in advance. AGOS-Agent [416] divides a task into search for candidates, route planning on the ground, and verification from ground-level views, so that the input each stage receives is set at design time. Together, these approaches either repair a mismatch when it appears or prevent it by checking or fixing the handoff ahead of time.

Handoff Timing. Handoff timing decides when control returns from an executor to the decision process. Returning control too late lets a failing action continue, while returning it too early interrupts an action that would have succeeded. Existing systems differ mainly in the signal that triggers this return, which can be task progress, an execution budget, or a periodic or event-driven schedule. Progress-based triggers return control when the observed state of the task changes. HarnessWAM [89] uses a fast progress estimator to detect such changes, and a VLM task manager then decides whether to move to the next subtask, gather new observations, replan, or reset the robot. Cortex [263] checks visual progress continuously while the robot moves, and keeps a running semantic memory to decide whether the current stage should continue.

Other systems limit each call in time or coordinate components that run at different rates. VLAs-as-Tools [166] runs the VLA as a tool with a limited execution budget, during which the VLA reports progress to an outside monitor that can move on or replan. Retriever [446] runs perception, belief updates, planning, the VLA, and control asynchronously, with separate periodic or event-driven clocks and explicit input-synchronization and commit rules that preserve causality. Generally, these triggers of handoff timing differ in what the harness waits for before taking control back, whether an observed change in progress, the end of a budget, or the next tick of a clock. Related works are compared in Table 13.

Table 13. Comparison of works on orchestration (Section 7.1), grouped by the four categories in the text. The executor columns mark which types of component receive control, namely learned policies such as VLAs and WAMs (Policy), callable utilities such as motion planners and controllers (Tool), reusable behaviors from a skill library (Skill), and other robots or humans (Other Agent). *Selection* shows what decides which executor runs, namely an LLM or VLM agent (Model-based), a model whose calls are checked or triggered by a rule layer (Hybrid), or a preset schedule (Rule-based). Icons link to the paper and, when available, the code.

Work	Link	Policy	Tool	Skill	Other Agent	Selection
Routing and Dispatch						
VoLo [40]		✓	✓	✗	✗	Model-based
MistyPilot [354]		✗	✗	✓	✗	Hybrid
ABot-AgentOS [321]	-	✗	✗	✓	✗	Model-based
AeroWeaver [226]		✗	✗	✓	✓	Model-based
M2HRI [99]		✗	✗	✓	✓	Model-based
EMOS [37]	-	✗	✗	✓	✓	Model-based
Physical Agency [75]		✓	✓	✓	✗	Model-based
Guava [208]	-	✗	✓	✗	✗	Model-based
Function Protocol						
EmbodiedTool [466]	-	✗	✓	✗	✗	Model-based
ROSClaw [26]	-	✗	✓	✗	✗	Hybrid
Physical Agentic AI [220]		✗	✗	✓	✓	Hybrid
EmbodiedSkills [351]		✓	✗	✗	✗	Hybrid
Thea [348]		✗	✓	✗	✗	Model-based
State Connection						
RoboHarness [116]	-	✓	✓	✗	✗	Model-based
RoboRouter [48]	-	✓	✗	✗	✗	Model-based
RoCo [237]		✗	✓	✗	✓	Model-based
RoboTalk [84]	-	✗	✓	✓	✓	Model-based
AGOS-Agent [416]	-	✗	✓	✗	✓	Hybrid
Handoff Timing						
HarnessWAM [89]	-	✓	✗	✗	✗	Model-based
Cortex [263]		✓	✗	✗	✗	Model-based
VLAs-as-Tools [166]	-	✓	✗	✗	✗	Model-based
Retriever [446]		✓	✓	✗	✗	Rule-based

7.2 Memory

Memory supplies information from earlier steps to later decisions without changing model weights. Unlike reusable behavior such as rules or programs, memory records what has happened, including what the robot has tried, what it has observed, and how far the task has progressed. We group existing methods into three categories according to the stage of memory they address. *Memory Writing* decides what is recorded and when. *Memory Updating* keeps records consistent with a changing scene. *Memory Retrieval* selects records for the next decision and reuses earlier solutions.

Memory Writing. Memory writing decides which parts of an episode are worth recording. Common records include failed choices, task progress, and observed experience. Some systems record failures so that the robot can return to them later. HAM-VLN [205] stores each failure together with the place where it happened, which supports backtracking and verified stopping during navigation. Robo-Cortex [30] checks imagined candidate actions against fresh real observations and keeps the results in a memory with two levels of detail. Other systems record task progress. HiMe [131] keeps a subtask list with a status for each item to guide planning, separate from an editable record of the episode, and uses a separate checker to judge recent visual progress. A third group records broader experience. MEMORA [420] stores experience as records of places, objects, activities, and inferred knowledge, which are edited during the task and merged afterward. ABot-AgentOS [321] stores such records as graphs linked to their sources rather than as raw history.

Beyond what to record, a harness must also decide when to write and in what form. Writing after every step

fills memory with redundant records, while writing too rarely misses events that later decisions depend on. MessyMem [8] writes after each physical interaction. A VLM reviews the action, notes outcomes such as a missed grasp or a locked drawer, and updates the matching object entry with these notes and key images. Skills in Weights, Memory in Code [452] instead treats the memory strategy itself as code, developing it from rollout traces and keeping it as a runnable program. These systems thus differ in whether writing is triggered by events or governed by a learned program.

Memory Updating. Memory updating keeps stored records consistent with a changing scene. As objects move and earlier observations become outdated, a memory entry that was correct when written can later be wrong. Existing systems handle this in three ways, by keeping different kinds of records separate, by refreshing records regularly, or by checking a record before relying on it. Mimir [392] keeps task memory separate from world memory, and updates execution progress, failed hypotheses, object relations, and evidence with different operators. RoboStream [123] refreshes its evidence about the robot and the scene at every planning step. OVAL [262] checks each remembered candidate against the goal, rejecting a wrong target and resuming exploration.

Updating also decides how long records are kept. Keeping every record makes memory large and slow to search, while dropping records too early loses information that the robot may still need. MetaTool [235] keeps a plan and tool outcomes in a scratchpad used only within the current episode. Other records compete for a fixed amount of space. EEAgent [340] condenses reflections on successful and failed interactions into a text memory of fixed size, and ActiveArena [186] compares rules for choosing what stays in such a memory. A third kind of record persists beyond what the robot can currently see. SafeMem [180] keeps objects that are out of view in its scene graph, revises their relations when changes are observed, and requests replanning when the predicted risk is high. CoRelNav [100] retains shared evidence about object instances. World Action Agent [440] retains reusable relational skills, procedures, and reference images, which are checked against the current scene; scene-specific advice expires when the associated observation or control state changes.

Memory Retrieval. Memory retrieval decides which stored records reach the next decision. As memory grows, most of what it holds does not bear on the decision at hand, and passing the full history can bury the few details that matter. Existing systems retrieve three kinds of content, evidence for the current decision, the task state needed to continue work, and earlier results that can replace new computation. The first kind is evidence for the current decision. Such evidence is usually selected by its relevance to the current goal, location, or question. NavMCP [167] answers questions from a frozen navigation model with keyframe-based summaries of the route so far, and keeps a list of unresolved goals across calls. HFLEX-EQA [270] and NavProbe [210] select the graph or past-view evidence most relevant to each navigation decision. EmbodiedLGR [282] searches a robot-built scene graph for relevant entries, and CMMR-VLN [173] retrieves stored image and text experience for repeated navigation. HoloAgent-0 [464] retrieves objects, rooms, views, and task history from a 3D spatial memory, and confirms them through visual verification and active exploration. Across these systems, retrieval narrows a long history down to the few records that bear on one decision.

The second kind is the task state needed to continue work. This kind matters when the executor cannot keep track of the task itself, for example a skill that starts fresh at each call or an action model without memory. BATON [391] retrieves this state at skill transitions. Instead of the whole episode, it retrieves the setup and starting conditions recorded for the next skill, and checks them against a live wrist-camera view. 2AM [112] retrieves it during a long task. It condenses its history into the target, the destination, how many times a step has been repeated, and the current stage, and passes this summary as language and optional 2D direction hints to an action model that has no memory of its own. In both systems, the harness keeps the full history on its own side and passes the executor only the state that links the current step to the rest of the task.

The third kind is an earlier result that can replace new computation. When a similar case has been handled before, reusing the earlier result can be faster and cheaper than computing a new one, provided that the result still applies. MeCo [332] retrieves the solution of a similar successful multi-robot task and reuses its waypoint structure, falling back to the LLM from the failed step if validation rejects the transfer.

APEX [432] reuses the spatial evidence produced by earlier VLM updates, so that a drone can keep exploring without waiting for each new update to finish. The two systems handle the question of whether an earlier result still applies in different ways, MeCo by validating the reused solution and APEX by acting on evidence that may be slightly older than the current scene. Related works are compared in Table 14.

Table 14. Comparison of works on memory (Section 7.2). Structured includes records, indices, plans, and code; Persists means memory is retained across episodes.

Work	Link	Stored as	Written at	Retrieved by	Persists
Memory Writing					
HAM-VLN [205]	-	Graph	Step	Query	✗
Robo-Cortex [30]	-	Text	Episode end	Similarity	✓
HiMe [131]		Text	Event	Query	✗
MEMORA [420]		Structured	Step	Query	✓
ARMS [411]	-	Structured	Event	Similarity	✓
MessyMem [8]	-	Graph	Event	Query	✗
HyMeS [452]	-	Structured	Event	Check	✗
Memory Updating					
Mimir [392]	-	Structured	Step	Query	✗
RoboStream [123]		Graph	Step	Direct	✗
OVAL [262]	-	Structured	Step	Query	✗
MetaTool [235]	-	Structured	Event	Direct	✗
EEAgent [340]	-	Text	Event	Direct	✓
ActiveArena [186]		Visual	Event	Direct	✗
SafeMem [180]	-	Graph	Step	Direct	✗
CoRelNav [100]	-	Structured	Step	Query	✗
WAA [440]	-	Structured	Episode end	Query	✓
Robo-Harness K1 [193]		Structured	Event	Check	✗
Memory Retrieval					
NavMCP [167]	-	Structured	Event	Query	✗
HFLEX-EQA [270]		Graph	Step	Query	✗
NavProbe [210]		Structured	Step	Query	✗
EmbodiedLGR [282]	-	Graph	Step	Query	✗
CMMR-VLN [173]	-	Visual	Episode end	Similarity	✓
HoloAgent-0 [464]	-	Graph	Step	Query	✓
BATON [391]	-	Text	Episode end	Check	✓
2AM [112]	-	Structured	Event	Direct	✗
MeCo [332]		Structured	Episode end	Similarity	✓
APEX [432]		Visual	Step	Direct	✗

7.3 Monitoring and Recovery

Monitoring judges whether execution is progressing, has finished, has failed, or has violated a constraint, and recovery changes execution after a problem is found. We group existing methods into three categories that follow this process from detection to repair. *Execution Monitoring* checks execution before, during, or after an action. *Response Selection* turns a verdict into a response and checks that response before it runs. *Recovery Actions* carry out the response, including where recovery starts and when to stop trying.

Execution Monitoring. Execution monitoring can check an action before it runs, while it runs, or after it ends. The timing of a check determines which problems it can catch and what kind of report it produces. Checks before execution act on proposed actions or routes that have not yet had any physical effect, and usually accept, reject, or rank them. SafeHarness [390] tests proposed route against obstacles and returns the rejected reasons so that the route can be revised. DV-VLN [194] answers true or false questions about candidate navigation actions and reorders them accordingly. CoVer [157] prepares rephrasings of the instruction in advance and uses a verifier to score candidate actions and instructions before each action runs. Since these checks only need to choose among options, their outputs are simple decisions or scores.

Checks during execution give signals while the action is still under way. They mainly track constraint

violations and task progress, and some systems check at several levels at once. For constraints, DoReMi [94] uses an LLM to write constraints for the next skill and a VLM to check them at regular intervals, stopping the skill as soon as one is broken. Code-as-Monitor [458] writes visual constraints as code that can detect a failure as it happens or before it happens. Meanwhile, Mission-Level Runtime Assurance [147] checks constraints over mission events and reports ”unknown” when evidence is missing. For progress, OnFly [455] checks the task stage in parallel with flight and reacts when it detects stop or lost conditions, while Robometer [195] tracks progress with a score learned from expert labels and from comparisons between successful and failed runs. ABot-AgentOS [321] combines several levels, checking at runtime, at the skill level, and once more before allowing a task to finish. These checks mostly output signals to stop, adjust, or report progress, since they must act before the motion ends.

Checks after execution can use the full record of what happened, which allows them to judge the outcome and explain its cause. RoboLab [398], after first checking task assets and stability test results, judges the outcome with generated task code that defines when a task has succeeded or should end. REFLECT [223] summarizes an execution at the sensor, event, and subgoal levels to explain a failure to a correction planner. AHA [64] gives failure-specific feedback in language based on sequences of key images. DQAF [245] writes feedback after each episode for the next human operator. Overall, the timing of a check thus shapes its output. Checks before execution choose among candidates, checks during execution signal whether to stop or adjust, and checks after execution explain what happened.

Response Selection. Response selection decides what the system does after a problem is detected. It covers three questions, how the detected problem reaches the agent, how the agent chooses a response, and whether that response is checked before it runs. A detected problem can change behavior only if the agent that chooses the next action can see it. Existing work either builds this link into the agent's tools or tests whether the link affects later decisions. Agentic AI for Robot Control [202] takes the first approach, giving the agent tools to query robot status and events. Diagnosing Semantic Handoff Failures [285] takes the second, checking whether a visual reachability check changes the agent's next skill or route.

Once the agent sees the problem, it must choose a response. Existing systems either pick from a fixed set of responses or let a planner revise the plan. Fixed responses are simple and predictable, while a planner can produce responses that were not listed in advance. Goal2Skill [225] picks among retrying, adjusting, and replanning, and A Closed-Loop Multi-Agent Framework for Robust Multi-Robot Manipulation [102] separates local pose fixes from full replanning. Among planner-based systems, the planner is triggered by an explanation of the failure in REFLECT [223], by a gap between the predicted and actual outcome in WCM [51], and by a lack of progress in Stop Wandering [184].

A chosen response can also be wrong, so systems validate proposed actions and use execution evidence to decide whether to retry. These checks rely on perception, sub-goal evidence, or the validity of tool inputs. ETA [50] refuses to act when it cannot locate the target. ACE [165] uses tracked pick-and-place masks, execution reports, and new observations to assess sub-goal completion and guide retries. HarnessVLN [44] checks tool inputs and evidence before the robot moves or stops.

Recovery Actions. Once a failure is detected, the robot needs a recovery action that fixes it. Generally, they differ in what they change, namely the input to the current policy, the executor, the task flow, or something outside the policy. VLA-Corrector [302] changes the input, feeding stage-specific recovery prompts to a frozen VLA. FLARE [445], VoLo [40], and ReCoVLA [106] change the executor, switching to a trained reset adapter, a hand-designed skill guided by perception, or a simulation-trained correction policy for each known failure type. AgentChord [395] changes the task flow, moving to recovery branches compiled into its task graph without calling the LLM again. SERP [171] and Dream2Fix [170] change something outside the policy, adjusting motion-cost parameters and applying small learned physical corrections. In all cases, recovery is limited to the actions the system provides.

Recovery also needs to choose which state to resume from. A failed attempt often leaves the scene partly changed, with objects moved or a step left half done. The robot can continue from this state and keep the progress already made, or first restore the scene to a known state and start again. PhyAgentOS [221] continues from the current state, diagnosing the failed attempt and resuming with new instructions.

HALTER [134] restores the scene first, using scene-graph histories to find what was completed, planning reset skills, and checking that the reset worked. LIBERO-RECOVER [212] evaluates these two cases separately, comparing recovery from a disturbed scene with an attempt that starts in a restored scene.

Finally, repeated attempts can fail again, so systems need a rule for when to stop. Existing systems stop after a fixed budget, hand the task to a human, retry only under specific conditions, or aim for an end point other than task completion. PEACE [328] lets the planner retry after a drone tool failure only within a fixed budget. LabVLA [280] allows one validated retry during offline workflow-YAML generation before returning the workflow for human review. A physical agentic grasping loop [353] retries only when the gripper is empty, and otherwise stops or asks for clarification. SPINE [97] repairs robot software and settings until the robot is ready for teleoperation, rather than until a task is completed. Related works are compared in Table 15.

Table 15. Comparison of works on monitoring and recovery (Section 7.3). Check Time is relative to the checked step: before, during, or after execution.

Work	Link	Check Time	Checker	Signal	Response
Execution Monitoring					
SafeHarness [390]	-	Before, during	Rules	Constraint	Replan
DV-VLN [194]		Before	VLM	Validity	Select
CoVer [157]		Before	Learned	Validity	Select
DoReMi [94]	-	During	VLM	Constraint	Stop, replan
Code-as-Monitor [458]	-	During	Code	Constraint	Report
MLRA [147]	-	During	Rules	Constraint	Report
OnFly [455]	-	During	VLM	Progress	Replan
Robometer [195]		During	Learned	Progress	Report
RoboLab [398]		After	Code	Progress	Report
REFLECT [223]		After	LLM	Failure	Report, replan
AHA [64]		After	VLM	Failure	Report
DQAF [245]	-	After	VLM + human	Progress	Report
Response Selection					
AARC [202]	-	During	Rules	Progress	Replan
DSHF [285]	-	After	VLM	Success	Retry, replan
Goal2Skill [225]	-	After	VLM	Failure	Retry, replan
CLMAF [102]	-	Before, after	VLM	Validity	Replan
WCM [51]	-	After	VLM	Failure	Replan
MetaNav [184]	-	During	LLM	Progress	Replan
ACE [165]	-	After	VLM + human	Success	Retry
HarnessVLN [44]	-	Before	Rules	Validity	Retry, stop
Recovery Actions					
VLA-Corrector [302]	-	During	Learned	Failure	Adjust
FLARE [445]	-	During	VLM	Failure	Retry, reset
ReCoVLA [106]	-	During	VLM	Failure	Switch
AgentChord [395]		During	Code	Failure	Switch
SERP [171]	-	After	Rules	Failure	Adjust, retry
Dream2Fix [170]	-	After	Learned	Failure	Adjust
PhyAgentOS [221]		After	Rules	Success	Replan
HALTER [134]	-	After	LLM	Success	Reset
LIBERO-RECOVER [212]		–	Benchmark	–	–
PEACE [328]		After	Rules	Failure	Replan
LabVLA [280]		After	Code + human	Success, constraint	Reject
PAL [353]	-	After	Rules + human	Failure	Retry, stop
SPINE [97]	-	During	Rules + human	Failure	Adjust, retry

7.4 Skills

Orchestration, memory, and monitoring decide how a harness handles the current attempt; skills decide what it brings to the next one. A skill is a pluggable package of procedural knowledge that the harness can load or call. Loaded into the context, a textual skill shapes how the language model reasons about the task and uses the harness's tools; called as a function, an executable skill gives the model a reusable operation

on the robot. In the drawer example, the same experience can become a note to approach the handle from below or a callable `open_drawer` program, and either can be swapped in without retraining the model. Skills thus differ from memory, which records what happened, and from model weights, which a skill leaves unchanged. A recent survey ranks code-as-policy systems by whether they use execution feedback, persistent skill memory, and program search [129]; we focus instead on how skills enter, change within, and leave a harness.

From Skills to Code as Policies. Early language-model harnesses treated skills as a fixed set of pretrained primitives, and the model's role was only to choose among them, as in SayCan [125]. Code generation changed that role. In Code as Policies [196], the model composes perception and control APIs into a program and defines missing functions on the fly, so the skill set becomes a vocabulary rather than a menu. ProgPrompt [300] and VoxPoser [118] followed the same idea, expressing plans as programs with assertions or as code that builds value maps for a motion planner. Because these programs are written for one task and then discarded, later work let the program improve while the task is still running: Act-Observe-Rewrite [155] rewrites a controller between trials from keyframes and logs, MEMENTO [314] searches over a single code policy, and Agent as Policy [132] lets a coding agent write and revise programs directly on a physical robot. Generated code thereby became a capable policy within a task, which raised the question of what to keep across tasks.

Skill Libraries and Reuse. Reuse is the natural answer, and it requires organizing skills so that later tasks can find them. Voyager [336] set the pattern in a Minecraft agent: a program joins a library once it passes self-verification, and new tasks retrieve entries by embedding similarity. Skill libraries are now common across other agent domains, where skills are distilled from agent experience and evolved alongside reinforcement learning [380], or built from scientific resources [293] and human-created multimodal material such as tutorials, repositories, and articles [66]. Robot systems adopted this library structure and differ mainly in what they store and how an entry is admitted. RoboCoder [175] stores joint-level action code accepted by a vision–language evaluator, while DROC [430] and ARCHITECT [34] store knowledge distilled from human corrections, so that the same correction need not be given twice. Uni-Skill [385] fills gaps in its library by implementing missing skills from a video-derived repository, and Teach and Grow [249] admits a skill only after it passes a scoped outcome test on held-out states. A library can even be built once and frozen, as CaP-X [74] does with nine helper functions distilled from successful rollouts, or compressed into macros, as in SkillComposer [374]. Since entries in these libraries are checked on admission and rarely revisited, a flawed skill persists until something revises it.

Skill Evolution. Skill evolution supplies that revision by feeding execution results back into the library. Early systems revised skills with simple loops: LRLL [327] periodically abstracts accumulated experience into new functions, and LYRA [240] relies on human corrections while re-testing earlier behavior. Automating the reviser raised a new problem, because an agent that edits its own skills can also make them worse, so recent systems differ chiefly in the gate a revision must pass. ASPIRE [228] lets coding agents debug programs from per-primitive traces and promotes only repairs that validate and generalize; RATs [435] promotes a skill after repeated success and deprecates it after repeated failure; and RoboRSI [373] traces each failure to the earliest failing node of a task–skill hierarchy and keeps a patch once it passes review, functional tests, and a replay of recent runs without regression. SkillOpt [402] makes the analogy to training explicit, treating a skill document as the trainable state of a frozen agent and accepting an edit only if it improves a held-out validation score, while Reconstruct, Practice, Go Real [358] applies a similar rule across robot practice tasks and VASO [403] replaces the score with model-checker counterexamples. The same loop extends from skills to the harness code around them, as in RHO [65], which mutates whole policy repositories, and Zetta [60], which uses agents to diagnose failed rollouts and evolve validated critic–recovery code that monitors execution and triggers local recovery while keeping the base VLA frozen. Without such gates, updates need not accumulate: Explore, Execute, Evolve [388] reports non-monotonic progress across successive updates, and even with replay, paired, and held-out checks, a 123-round study found that accepted changes did not add up to success on its target task [339]. A useful skill library therefore depends as much on how revisions are tested and retired as on how they are generated.

From Skill Evolution to Training. All of these mechanisms leave the model that writes skills unchanged, which raises the question of whether skill experience could instead flow into weights. In practice, most systems that do so train the executor rather than the coder. Agentic Skill Discovery [450] trains a reinforcement-learning policy for each proposed skill, RoboClaw [178] and ENPIRE [383] fine-tune robot policies from collected or agent-designed training runs, and HarnessPAI [356] and RACaP [192] distill harness trajectories into a VLA or a smaller decision model. Training the coder itself is rarer. RoboCodeX [242] and Uni-Skill [385] fine-tune code-generating models on curated or video-derived programs, and CaP-X [74] applies reinforcement learning with environment rewards to a coding model on three simulated tasks before transferring it to a physical arm. Training a multimodal coder continually on its own rollouts, or folding an evolved skill library into its weights, would close the loop between skill evolution and model learning. Related works are compared in Table 16.

Table 16. Comparison of works on skills (Section 7.4), grouped by the four parts of the text. Update distinguishes code/library changes from policy training. NR: not reported.

Work	Link	Skill Form	Built From	Reuse	Update
From Skills to Code as Policies					
Code as Policies [196]		Python functions	Instructions; API examples	Call earlier functions	Generate missing functions
ProgPrompt [300]		Plans with assertions	Instructions; action examples	Compose provided actions	Within-plan recovery; skill revision NR
VoxPoser [118]		Value-map code	Instructions; visual grounding	Rerun code on new observations	Code **fixed** within each subtask
Act-Observe-Rewrite [155]	-	Python controllers	Episode logs; keyframe images	Run controller in the next trial	Rewrite **between episodes**
IMR-LLM [306]	-	Process tree; robot programs	Program examples; human checks	Select branches; reuse code snippets	Tree built once for the reported tests
MEMENTO [314]		Stateful policy programs	Rollout metrics; search history	Run the selected program	Mutate and combine code; retain the elite
AGP [132]	-	Task definitions; scripts	Task materials; robot observations	Reload saved task definitions	Revise scripts during execution
EmbodiedSWE [294]		Executable solutions	Task materials; simulator trials	Use verified solutions as references	Test and revise task programs
RAPID [222]	-	Relational programs	One visual demonstration	Bind a fixed strategy to a new scene	Revise in simulation before deployment
Skill Libraries and Reuse					
Voyager [336]		Action programs	Exploration; execution feedback	Retrieve relevant programs	Add success-verified programs
DROC [430]		Code; correction knowledge	Human corrections	Retrieve task/object knowledge	Revise code; save reusable lessons
RoboCoder [175]	-	Action programs	Basic actions; evaluator feedback	Retrieve similar actions	Repair code; add accepted actions
Uni-Skill [385]	-	On-demand skill code	Video-derived skill examples	Retrieve examples from SkillFolder	Generate missing skill implementations
CaP-X [74]	-	**9** helper functions	Successful benchmark rollouts	Call the synthesized library	Single library-synthesis pass
ARCHITECT [34]		Text rules; code patterns	Human language corrections	Load the full skill library	Append corrections as reusable skills
SkillComposer [374]	-	Callable macros	Accepted robot programs	Expand macros into primitive calls	Relearn macros after accepted prompts
Teach and Grow [249]		Validated behavior blocks	Successful demonstrations	Ground blocks in the current scene	Refine scope or executor; revalidate
LTCLRS [156]	-	Closed-loop software	Demonstrations; simulated trials	Use archived code as examples	Revise code; select on validation
Skill Evolution					
LRLL [327]	-	Python skill functions	Demonstrations; simulated trials	Call library functions	Group similar code; abstract and replay
LYRA [240]		Parameterized functions	Human-guided successful trials	Retrieve skills and examples	Extend functions with human feedback
VASO [403]	-	Labeling code; plan templates	Task; APIs; formal rules	Guide plan generation	Revise using model-checker feedback

Continued on next page

Table 16 (continued)

Work	Link	Skill Form	Built From	Reuse	Update
RHO [65]		Policy repositories	Rollouts; failure feedback	Deploy the selected repository	Mutate files; select by rollout scores
RATs [435]		Manipulation functions	Self-directed play	Retrieve the learned library	Curate during play; **freeze** at test time
ASPIRE [228]		Repair notes; code sketches	Primitive-level execution traces	Load relevant repair guidance	Admit validated, reusable repairs
SHAPER [347]	-	Text; context-building code	Target-environment rollouts	Reuse guidance and context builder	Test edits; retain top candidates
Zetta ζ [60]		Critic, recovery and tool packages	Failure traces; validation rollouts	Deploy versioned harness packages	Patch, validate and consolidate repairs
WSRSI [339]	-	Skill code; model adapters	Recorded failures	Call skills through typed interfaces	Replay failures; test on fresh tasks
RoboSkill [388]		Text; code; supporting records	Exploration and execution records	Select relevant skill packages	Consolidate skills after agent exit
RPG [358]	-	Skill code; system prompts	Simulated practice; failure diagnosis	Deploy the retained system	Test across tasks; merge and retest
From Skill Evolution to Training					
RoboCodeX [242]	-	Object-centered code units	Multimodal code training	Compose motion primitives	Skill-library revision NR
Agentic Skill Discovery [450]		Learned policies	Generated rewards; RL	Compose learned skills	Train missing skills on demand
RoboClaw [178]		Tool-use procedures	Modular skills; trained policies	Compose tool and policy calls	Procedure revision NR
ENPIRE [383]		Policy-learning code	Physical trials; automated rewards	Reuse successful training recipes	Edit code; train and retest policies
AgenticRobotics [417]	-	Skill procedures; tool records	Round logs; measured outcomes	Select skill–tool pairs	Registry promotion: live use **untested**
HarnessPAI [356]	-	Repair notes; linked code	Between-rollout failure analysis	Retrieve repairs by workflow node	Evolve code; save failure–repair pairs
RACaP [192]		Policy API library	Development-task rollouts	Call APIs with runtime arguments	Paired testing; code **frozen** at deployment

7.5 Infrastructure

Beneath skills, every harness function rests on software infrastructure: the interface through which it reaches robot capabilities, the perception and manipulation tools behind that interface, the services that run learned policies, and the checks that stand between a proposed action and the hardware. In the drawer example, `open_drawer` may expand into a segmentation call that finds the handle, a grasp generator that proposes a pose, a motion planner that reaches it, and a VLA served on a remote GPU that performs the pull. Each of these components can fail, add latency, or hide information from the agent. We discuss the interface first, then the toolchains and policy services that dominate practical systems, and finally runtime gating.

Capability Interfaces. The first layer of this infrastructure is the interface, which fixes what an agent can express and what it learns back. EmbodiedTool [466] registers more than a hundred tools with capability cards that state inputs, outputs, and trigger conditions; agents form valid calls almost always but choose the right tool far less often. The return channel matters as much as the call: NavMCP [167] combines structured intent, keyframe-grounded journey summaries, and persistent memory; replacing this complete three-channel protocol with an episodic interface lowers question-answering accuracy from 74.0% to 59.1%. Granularity is a further choice. Show-Harness [46] exposes semantic Cartesian steps that an interpreter projects into safe setpoints, and halving the step size raises zero-shot success from 60% to 82%. The Model Context Protocol is becoming a common transport: RoboNeuron [90] derives typed MCP tools from ROS 2 message schemas, and the ROS MCP Server [284] exposes topics, services, and actions through rosbridge. On the robot side, Cardenas et al.'s ROSClaw [26] checks each call against velocity bounds and an interface allowlist before dispatch, and EMOS [37] derives a capability profile from the robot's URDF and forward kinematics rather than from hand-written text.

Perception and Manipulation Toolchains. Behind the interface, code-as-policy harnesses act through a stack of pretrained tools: open-vocabulary detection and segmentation such as Grounding DINO [216], SAM 2 [276], and SAM 3 [27]; pointing models such as Molmo [58]; pose and depth estimation such as FoundationPose [368] and FoundationStereo [369]; grasp generators such as AnyGrasp [67], ContactGraspNet [313], M2T2 [423], and GraspGen [243]; and GPU motion planners such as cuRobo [312]. CaP-X [74] exposes such a stack as stateless services behind a Python API, and its benchmark separates tiers that receive privileged simulator masks and poses from tiers that must perceive from RGB-D. GaP [38] organizes 51 perception, grasping, motion, and verification skills behind single gRPC methods with shared types for poses, masks, point clouds, and trajectories, and translates them to ROS for physical execution. Tools are also beginning to ship agent-facing endpoints: GraspGen provides a GPU server and an MCP wrapper so that an LLM agent can request grasps without a local model. Physical Agency [75] illustrates a complete physical deployment: detection, segmentation, stereo depth, grasp generation, and task-and-motion planning run as servers on one workstation, while a real-time control computer drives the arm.

These toolchains determine much of what a harness can achieve. Perception is the dominant source of physical failure in several systems: most real-robot errors of VoxPoser [118] come from perception, point tracking is the largest error source in ReKep [120], and OK-Robot [215] shows how errors compound when off-the-shelf detection, segmentation, mapping, and grasping are chained without recovery. Privileged state hides this cost. Kuroki and Tang [156] report 65.0% success with full simulator observations but 22.5% with RGB alone, and CaP-X trains its coding model on the privileged tier to avoid perception noise before transferring to perceived inputs. An agent can repair a toolchain when the toolchain reports its failures. ASPIRE [228] provides an execution engine with primitive-level traces, debugging, and repair validation; adding the engine raises macro-average success on the evaluated LIBERO-Pro perturbations from 14% to 62%. Physical Agency returns typed failures and overrides a reported grasp when the gripper-width sensor shows it is empty. Across systems, coding agents repeatedly add the same hardening around tools, including fallback chains from text-prompted to point-prompted segmentation, filters on grasp orientation, inverse-kinematics checks before grasping, and sensor-checked grasps; RHO [65] and ASPIRE move such logic into the tool wrappers themselves. Tool interfaces that report uncertainty and failure, together with evaluations that separate privileged from perceived inputs, therefore make toolchain results comparable across harnesses.

Policy Inference and Serving. Learned policies form the other half of this stack: when a harness calls a VLA, the policy becomes a service with its own latency and capacity. The openpi remote-inference interface [267] serves a checkpoint over a WebSocket that returns action chunks, keeping robot code free of model dependencies, and LeRobot [23] separates a policy server from a robot client that requests a new chunk when its action queue runs low. Asynchronous execution hides inference delay: SmolVLA [299] uses this design to cut task time by about 30% at a small cost in success, real-time chunking [14, 15] freezes actions committed during inference, and VLASH [315] conditions on the state those actions will reach. Placement is a further design dimension. VLA-Perf [136] models latency across devices and networks and finds that, for π_0, an edge server over WiFi can outperform on-device inference; Gemini Robotics [81] combines a cloud backbone with an on-robot decoder, and RoboECC [457] partitions a VLA between edge and cloud according to predicted bandwidth. For fleets, ROSA [135] schedules VLA and VLM components on a shared GPU pool under latency targets, Robion [1] partitions one GPU between the vision–language and action stages to serve about twenty robots, and Armory [7] schedules batched action-chunk inference across robots running at different rates. Runtime engineering narrows the remaining gap: kernel-level optimization brings π_0 to 27 ms on a consumer GPU [234], OxyGen [181] shares a VLA's cache between action and language heads, vla.cpp [248] runs quantized VLAs on embedded boards, and BitVLA [337] matches a 4-bit baseline with ternary weights. These systems are mostly evaluated on policy throughput; coupling them with agent harnesses will make serving latency part of the planning budget.

Runtime Gating and Versioning. Once tools and policies are reachable, the harness must also decide which proposed commands may reach the hardware, and this check is best placed outside the model.

RoboPAIR [283] motivates this placement by jailbreaking a self-driving language model, a GPT-4o-planned ground robot, and a commercial quadruped in every attempted scenario. Safety Chip [405] compiles expert-verified temporal-logic constraints into an automaton that prunes unsafe actions before execution, and RoboGuard [277] model-checks plans against specifications derived by an isolated model, reducing jailbreak success from 92.3% to 2.3%. SafeGate [255] fails closed by deferring to the user whenever a hazard is not covered by a verified template, and RoboSafe [344] checks each action against generated executable predicates. Other systems gate at the interface: ETA [50] separates what the agent wants to do from what the interface permits, SafeHarness [390] rejects routes that violate obstacle clearance, and the constrained-LLM design of Hong et al. [104] lets the model state only symbolic intent. CORE [278] instead enforces inferred constraints in the controller through control barrier functions. The same gates also apply when the harness itself changes. ETA requires a revised harness to pass contract checks, isolated review, replay, and held-out evaluation, and in its reported runs no candidate passed; Harness Robotic OS [397] proposes versioned, rollback-capable updates without demonstrating a revised executable. Related works are compared in Table 17.

Table 17. Comparison of works on infrastructure (Section 7.5). Includes supporting tools and deployment systems, which need not be autonomous agents.

Work	Link	Purpose	Works on	Main operation
Safety Chip [405]		Action safety	Proposed actions	Check temporal rules; block violations and reprompt
OK-Robot [215]		Mobile manipulation	Navigation; grasping; placement	Connect pretrained tools through a semantic map
RoboPAIR [283]		Adversarial testing	Robot-language interfaces	Refine attack prompts using judges and API checks
Gemini Robotics [81]	-	Safety evaluation	Embodied model responses	Test with ASIMOV; add constitutions and safety training
SmolVLA [299]		Continuous execution	Action chunks	Overlap server inference with queued robot actions
RTC [14]		Continuous execution	Action chunks	Freeze committed actions; fill in the next chunk
Running VLAs at Real-time Speed [234]		Inference acceleration	VLA GPU computation	Fuse operations and optimize GPU kernels
VLASH [315]		Delay compensation	VLA state inputs	Condition on the predicted execution-time robot state
Training-time RTC [15]	-	Delay compensation	Action-chunk policies	Train on committed action prefixes and sampled delays
RoboNeuron [90]		Robot middleware	Tool calls; robot streams	Separate tool orchestration from streaming execution
RoboGuard [277]		Plan safety	LLM-generated plans	Check plans against context-specific temporal rules
BitVLA [337]		Model compression	VLA weights; activations	Quantize weights to ternary values and activations to INT8
RoboSafe [344]	-	Action safety	Proposed actions; recent history	Run generated safety checks; block or request replanning
VLA-Perf [136]		Deployment analysis	Model–hardware choices	Estimate inference latency and throughput analytically
CORE [278]	-	Motion safety	Controller outputs	Ground visual safety regions; filter control commands
LeRobot [23]		Robot-learning workflow	Hardware; data; policies	Unify data collection, training and deployment interfaces
OxyGen [181]		Shared inference	Action and language experts	Reuse observation caches; prioritize action deadlines
RoboECC [457]		Edge–cloud deployment	VLA model partitions	Choose hardware-aware splits; adjust to network changes
SafeGate [255]	-	Task safety	Task requests; execution	Screen commands first; monitor task contracts at runtime
vla.cpp [248]		Portable inference	VLA models; robot clients	Run model adapters through a shared C++ runtime
ROSA [135]	-	Fleet inference	Requests from multiple robots	Pool model serving and batch requests across robots
GaP [38]		Reusable automation	Robot skill graphs	Rehearse and refine graphs; deploy an edge interpreter

Continued on next page

Table 17 (continued)

Work	Link	Purpose	Works on	Main operation
Armory [7]		Fleet scheduling	Queued action chunks	Plan inference batches using robot execution mirrors
ETA [50]		Checked execution	Agent tool calls	Gate commands; return execution records and fresh observations
Bridging Semantics and Physics [104]	-	Checked execution	Pouring tool calls	Validate arguments, then check motion-template feasibility
Show-Harness [46]		Robot action interface	VLM action tokens	Translate semantic actions into robot control deterministically
Harness Robotic OS [397]	-	Inspection runtime	Robot services; agent context	Connect autonomy skills through shared context and interfaces
Robion [1]	-	Fleet inference	Multiple VLA models	Overlap model stages; assign robot traffic across GPUs

7.6 Layered Systems

Layered systems organize the tools and policies above into layers that run at different rates. In the drawer example, a planner decides to open the drawer, a VLA reaches for the handle, and an inverse-kinematics solver tracks the pull, each at its own rate. We describe this stack as three layers: *System 2* planners (VLMs, LLMs, or generated code) that reason over tasks and subgoals, *System 1* learned visuomotor policies, typically VLAs, that react at control rate, and *System 0* analytic controllers such as inverse kinematics, motion planning, model predictive control, and whole-body control. Systems 1 and 2 follow the fast–slow distinction of dual-process theories [141, 304]. System 0 adapts a label that Taniguchi et al. [317] use for morphological computation; here it denotes explicit analytic control, and we avoid System 3 because it already refers both to collective intelligence and to a runtime critic [413]. Existing VLA surveys contrast monolithic with hierarchical models [291] but leave analytic control outside the comparison. The harness is what connects the three layers, and this subsection examines what it passes between them, how often, and which layer holds authority.

Layer Interfaces and Rates. The first design choice is what each layer passes downward and how often. Helix [72] writes a single latent vector from a vision–language model at 7–9 Hz to a 200 Hz policy, and GR00T N1 [254] passes intermediate embeddings from a 10 Hz reasoning module to an action module producing 120 Hz chunks, which a whole-body inverse-kinematics controller tracks. Ground Slow, Move Fast [367] makes all three layers explicit: a 2 Hz planner outputs a pixel goal, a 30 Hz policy produces waypoints, and model predictive control tracks them. Other systems pass interpretable signals. Hi Robot [297] sends language subtasks about once per second or when a user interjects, $\pi_{0.5}$ [269] decodes subtask text before actions within one model, NaVILA [52] sends parameterized language commands to a locomotion policy, and HAMSTER [185] sends a few-point image path once per episode. In a controlled comparison, VLA-OS [77] finds that low-level policies follow visual plans more faithfully than language plans. Because the slow layer's output is stale when used, RoboDual [21] and HiRT [433] train the fast layer on delayed conditioning, and HarnessWAM [89] triggers deliberation on progress events, which outperforms invoking the planner at fixed intervals. Latent interfaces carry more information but are opaque to monitoring; text, paths, and waypoints can be checked at the cost of what they can express.

Agents Calling VLAs as Tools. Beyond integrated dual-system models, a layered system can also be assembled from separate models, with the VLA exposed as one tool among others. Gemini Robotics 1.5 [320] makes its action model available to an orchestrating reasoning model through language instructions, and replacing a general model with the embodied-reasoning orchestrator halves the total failure rate. Harness VLA [441] places a frozen VLA behind a `vla_act` primitive next to analytic motion primitives and verifies each call; the VLA accounts for a minority of calls on its manipulation benchmarks. Physical Agency [75] lets an agent choose between task-and-motion planning and a VLA, and overrides reported successes that contradict grasp sensing. VLAs-as-Tools [166] returns progress signals that let a monitor advance or replan with a few planner calls per episode. A systematic study of hierarchical VLA agents [108] finds that termination rules and VLA steerability matter as much as planner size. Compared with an integrated

dual-system model, this arrangement keeps each layer replaceable and each call inspectable, but every handoff must re-establish state and intent, as discussed under Orchestration.

Decision APIs. Between such orchestrators and their executors, a recent development adds a fast decision layer. Decision models answer bounded questions over a supplied set of options instead of generating text. TypeSafe's Jev [326], described as a System One model, accepts a textual state and a batch of questions that ask it to choose among options, place a case on an ordered scale, or judge whether a condition holds, and returns a probability for every answer without producing output tokens. Liquid AI's d1 [204] offers the same three question types over text and images in a single forward pass, with text decisions in 200–300 ms, and OpenAI has announced a comparable decision interface [259]. In a robot harness, such a model sits beneath a slower planner: code proposes the legal candidates, the decision model selects one with a probability, and the harness acts only above a confidence threshold. EmbodiedJev [69], an open-source simulation workbench, applies this protocol to manipulation, letting Jev choose among skills, phases, or Cartesian steps while inverse kinematics, safety checks, and success judgment remain in code. The idea of restricting a language model to scored candidates predates these products. SayCan [125] combines language-model likelihood with learned affordances over a fixed skill library, MOKA [207] asks a VLM to choose among marked keypoints, and KnowNo [279] and IntroPlan [198] turn option likelihoods into conformal prediction sets that trigger requests for help. Dedicated decision models make this pattern cheap enough to run at every step. Their thresholds are meaningful once calibration is measured on the robot's own decisions, since a candidate's probability expresses preference among the offered options rather than the chance of physical success.

Code as the Upper Layer and Outlook. At the top of the stack, code can itself serve as System 2. The code-as-policies line discussed under Skills composes System 0 primitives in generated programs; ReKep [120] sharpens the division of labor by having a VLM write keypoint-constraint functions once per task with its solver running at about 10 Hz and keypoint tracking and position control at 20 Hz; Instruct2Act [117] composes perception models and motion primitives in generated code. AgentChord [395] compiles monitored task graphs with precompiled recovery branches and constrained solvers, so that detected violations are handled without a new language-model call. Programs that call VLAs alongside analytic primitives, as in Harness VLA, combine both: code remains inspectable and verifiable, while the VLA covers contact-rich segments that are hard to specify. A natural direction is a stack in which VLAs serve as callable lower layers, programs provide the upper layer, and an agent revises both offline through skill evolution. Each choice across the chapter carries a cost in this stack. A bridge between executors broadens successor states but consumes motion time; retrieved history preserves a commitment but can propagate stale evidence; editable code can repair a recurring failure but can also change the monitor that judges it. The strongest comparisons isolate the changed layer or interface, hold the others fixed, and test later behavior beyond the feedback used for optimization. Related works are compared in Table 18.

Table 18. Comparison of works on layered systems (Section 7.6). Update timing identifies the refreshed module; rates refer to the reported configurations. Architecture studies list alternatives. NR: not reported.

Work	Link	Upper layer	Lower layer	Passed down	Update timing
Systems and planning methods					
SayCan [125]		LLM selector	Learned skills	Skill description	Select again after each skill
Instruct2Act [117]		LLM coder	Perception APIs; controller	Python program	Runtime requery schedule NR
KnowNo [279]		LLM planner	Skill executor	Selected action option	Each decision; ask for help if ambiguous
IntroPlan [198]		LLM planner	Executor NR	Calibrated action options	Runtime schedule NR
MOKA [207]		VLM	Motion planner	Keypoints; waypoints	New points at each subtask start
ReKep [120]		VLM	Constraint solver	Keypoint constraints	Solver replans at about **10 Hz**
HiRT [433]	-	VLM	Visual policy	Cached vision-language features	Async; policy uses fresh images each step

Continued on next page

Table 18 (continued)

Work	Link	Upper layer	Lower layer	Passed down	Update timing
RoboDual [21]		Generalist VLA	Diffusion policy	Latent features; action chunk	Async; several policy steps per VLA call
NaVILA [52]		Navigation VLM	Locomotion policy	Movement; distance or angle	Motion duration set by the command
HAMSTER [185]		Path VLM	3D robot policy	2D path; gripper states	Path generated **once** per episode
Hi Robot [297]	-	VLM	VLA	Language subtask	Every **1 s** or on new user feedback
GR00T N1 [254]		VLM	Action transformer	Vision-language features	VLM **10 Hz**; actions **120 Hz** (L40)
$\pi_{0.5}$ [269]		Subtask predictor	Action expert	Language subtask	Actions **50 Hz**; upper-layer rate NR
Gemini Robotics 1.5 [320]	-	VLM orchestrator	VLA tool	Language instruction	Switch subtasks on detected completion
DualVLN [367]		Navigation VLM	Trajectory policy	Latent goal from pixel-waypoint reasoning	Async; fast policy reuses the latest latent
Critic in the Loop [413]	-	VLM planner; critic	Action expert	Language subtask	Replan on completion, anomaly or stagnation
Harness VLA [441]		VLM planner	Analytic tools; frozen VLA	JSON primitive call	New observation after each primitive ends
Architecture studies					
VLA-OS [77]		VLM; planning heads	Alternative action heads	Text / spatial / image plans	Varies across the compared architectures
WMORP [108]	-	VLM planner	VLA	Language subtask	Compare fixed, success-based and VLM-timed switching

8 Outlook: What Comes Next for Agentic Robots?

Agentic robots connect general-purpose reasoning models with robot policies, tools, and physical execution. However, integrating these components does not necessarily make a robotic system more capable. It remains difficult to determine what the agent contributes, whether its decisions can be executed reliably on physical robots, and how well the system performs under limited resources or on unfamiliar platforms. These challenges motivate four research directions.

8.1 Fair Benchmarking

What does a task success rate actually tell us about an agentic robot? Consider two systems evaluated on the same manipulation benchmark. One may rely on a stronger VLA executor, while the other uses an agent to coordinate multiple weaker skills, gather additional observations, and recover from failures. Similar scores can reflect very different capabilities. CaP-X [74] evaluates agents that generate robot-control programs, while Harness VLA [441] and RoboHarness [116] study how agents coordinate robotic policies and skills. Their results depend not only on the agent but also on the underlying executors, available tools, observation access, and allowed retries. Task success alone therefore cannot show which part of the system made the difference.

To compare agents fairly, we need to examine their decisions under comparable conditions. Using the same executors, observation access, and interaction budgets makes it easier to assess planning, tool selection, and recovery. Comparing a system with and without memory or replanning can further show what each component contributes. However, controlled comparisons alone are not enough, since choosing and combining different models and tools is itself an important capability. End-to-end evaluations should therefore report the resources available to each system, including additional observations, computational costs, execution time, and human assistance. Such comparisons can reveal whether the benefits of additional reasoning and tool use justify their costs.

Even with careful comparisons, a benchmark may tell us little about the agent if its tasks can be completed by a single fixed policy. Some robot benchmarks focus on task completion without requiring the agent to handle missing information, unsuitable executors, or failed actions. More informative benchmarks could

introduce unexpected environmental changes, incomplete observations, and tasks that require coordination across different skills. For agents that collect data, update models, or develop new skills, evaluation should also examine whether these changes improve performance on unseen tasks. This would help establish when an agent is useful and whether its improvements generalize beyond the original setting.

8.2 Real-robot Agentic Robotics

A plan that makes sense to an agent may not always be executable by a robot. A reasoning model might request a grasp beyond the robot's reach or select a skill based on an outdated observation. Agentic robots may combine reasoning models, VLAs, world-action models, and specialized robot skills, each with different capabilities and execution requirements. ReKep [120] uses model-generated constraints to guide robot motion, while Harness VLA [441] allows an agent to call a VLA alongside other robot operations. Yet choosing a suitable executor is only part of the problem. A valid command may still fail during physical execution, and the scene may change before the agent receives feedback.

Robot interfaces therefore need to describe more than the names and inputs of available skills. An agent should know when a skill can be used, what conditions it requires, and how to tell whether execution has finished. For example, a failed grasp may require another observation, a revised grasp command, or a different skill. The right response depends on what actually happened: the object may have moved, or the previous command may still be running. Sending a stop request does not guarantee that the robot has stopped moving. Before replanning, the agent needs feedback that confirms the current state and the effects of actions already taken.

These difficulties become harder to address when moving from simulation to physical robots. Simulators can provide accurate object positions and clearly defined success conditions, while real robots must often estimate these from noisy camera images and sensor readings. Small perception errors, calibration differences, and unexpected contacts can cause an otherwise reasonable plan to fail. Future systems should recognize when their observations are insufficient, check important assumptions before execution, and use physical feedback to correct their decisions. Real-robot evaluations should also make clear what information the agent receives, what the lower-level controllers handle, and whether the agent can recover when its assumptions turn out to be wrong.

8.3 Efficient Robot Use

A robot may complete a motion in a few seconds, yet spend much longer waiting for the agent to decide what to do next. Repeated model calls, visual observations, and tool interactions can introduce significant delays, especially when a task requires many small actions. Recent work has begun addressing this problem by reducing unnecessary high-level reasoning. HarnessWAM [89] uses a lightweight progress estimator to monitor execution and invokes its VLM task manager when important events occur. Similarly, VLAs-as-Tools [166] provides execution feedback that allows an agent to replan when needed, rather than continuously checking the robot's progress. Efficiency therefore depends not only on how fast a model runs, but also on when it is called.

The main challenge is deciding when another model call is worth the delay. A robot following a familiar manipulation sequence may continue with lightweight monitoring, while an unexpected obstacle or failed action may require a new decision from the reasoning model. Calling the model too often wastes time, but waiting too long can leave the robot following a plan that no longer works. The timing should depend on task progress, uncertainty, and the possible consequences of a delayed response.

Efficiency also depends on where computation takes place. Large models running in the cloud may offer stronger reasoning but introduce communication delays, while smaller local models can respond more quickly with limited computational resources. Future systems could combine these options, using local components for time-sensitive decisions and stronger models when additional reasoning is needed. Evaluations should therefore consider the full execution process, including reasoning time, tool calls, communication overhead, and physical task completion time. Fewer model calls or faster inference do not necessarily improve efficiency if they lead to more failures, retries, or longer task execution.

8.4 Cross-Embodiment Generalization

An agent that works well with one robot may struggle when given a different platform. Even a simple task, such as reaching a target location, requires different actions from a wheeled robot, a humanoid, or a drone. The high-level goal may remain similar, but the available sensors, control interfaces, and physical constraints are different. RobotWorld [404] evaluates general-purpose agents on 84 simulated tasks spanning manipulation, mobile manipulation, locomotion, driving, and aerial control. What remains unclear is whether the same agent can adapt to a new robot without major changes to its planning and tool-use setup.

One promising direction is to enable agents to understand the capabilities of unfamiliar robots before using them. An agent could identify the robot's available sensors, movement capabilities, and supported skills, then plan around those capabilities. For example, a wheeled robot may need to navigate around obstacles to inspect an area, while a drone could approach it from above. The goal is similar, but the actions and physical constraints are different. The agent must therefore adjust its plan rather than simply translate the same commands into another robot's control interface.

Another important question is how much of an agent's previous experience can be reused. High-level strategies, such as gathering additional observations when information is missing or checking why an action failed, may remain useful across different robots. In contrast, specific motion commands and control parameters often depend on the robot's embodiment. Future systems should retain general task knowledge without assuming that the same execution details apply to every platform. Evaluations could test agents on previously unseen robots, measuring task success, the amount of additional configuration or training required, and whether experience from other robots improves performance.

9 Conclusion

This paper examines how agents support robot execution and learning. We trace the evolution from robot policies to agentic systems and clarify the scope of agentic robots. We organize existing work into seventeen overlapping categories across four functional domains: Model, Data, Environment, and Harness. These categories cover model learning and decisions, data collection and preparation, the construction of worlds, tasks, and tests, and the software that coordinates robot capabilities.

Across these approaches, an agent's decisions depend on what it can observe, what the robot can do, and what feedback it receives. During execution, agents coordinate tools and skills, check progress, and use feedback to decide whether to continue, gather more information, or recover from failure. Beyond the current task, stored experience can guide later decisions and changes to data, training procedures, skills, models, and software. These changes need to be tested on new tasks and conditions, while checking that previously learned capabilities are preserved.

Building on this analysis, we discuss four directions for future research: fair benchmarking, real-robot deployment, efficient robot use, and cross-embodiment generalization. Future work needs to establish when agents improve performance under comparable conditions and how they handle uncertainty and failures on physical robots. It should also examine whether these improvements justify the time and resources used, and how well agents adapt to unfamiliar robots. We hope this work helps researchers connect these questions and develop more capable agentic robots. Progress in agentic robotics could bring us closer to intelligent systems that turn knowledge into action, learn from experience, and adapt to the changing physical world.

References

[1] Dionysios Adamopoulos, Nattapol Chanpaisit, Basel Fakhri, and Christina Giannoula. Efficient Vision-Language-Action Management and Serving for Robot Factories. *arXiv preprint arXiv:2609.12075*, 2026.

[2] AgiBot Research Team, Renhang Liu, Wenzhi Zhao, Zhuo Yang, Liliang Chen, Pengfei Zhou, Shengcong Chen, Guanghui Ren, Youlun Peng, Rongjun Jin, Nan Wang, Sukai Wang, Xindong He, Jinyuan Feng, Ziyu Xiong, Linqing Zhong, Yifei Wei, Feng Han, Long Zhang, Da Huang, Nanshu Zhao, Chenghao Yin, Mo Wu, Zhaodong Yan, Kongtao Hu, Yuxiang Yan, Aogelijiang Niyazi, Yu Fang, Jia Zeng, Lizhu Meng, Daizhen Lv, Haoyu Cao, Zhiwen Hou, Lianjin Ye, Yuehan Niu, Zhikai Cai, Xuan Hu, Hui Min, Xiongfeng Cai, Yue Liao, Jing Wu, Soujanya Poria, Ye Li, Sanping Zhou, and Maoqing Yao. GE-Act 2.0: Pretraining and Scaling a World-Action Model for Robotic Manipulation. *arXiv preprint arXiv:2609.05588*, 2026.

[3] Michael Ahn, Debidatta Dwibedi, Chelsea Finn, et al. AutoRT: Embodied Foundation Models for Large Scale Orchestration of Robotic Agents. *arXiv preprint arXiv:2401.12963*, 2024.

[4] Alibaba Cloud. Qwen-Max, 2026. URL https://www.alibabacloud.com/help/en/model-studio/qwen-max.

[5] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning. *arXiv preprint arXiv:2506.09985*, 2025.

[6] Awomo-PhysicalRSI Team, Danjiao Ma, Enhui Ma, Haohan Liu, Heng Jia, Hui Shan, Jianhua Xu, Jiahuan Zhang, Jiangdi Xu, Kaiwen Guo, Kaicheng Yu, Linwei Zhang, Liyang Jin, Maochun Luo, Pengyao Niu, Shiwen Li, Shuangyu Feng, Tong Zhang, Tianheng Wang, Xin Wang, Xiangru Huang, Yongqiang Huang, Zhaozhi Wang, and Zijian Ma. Awomo-SimDataEngine: Agentic Simulation-Ready World Generation. *arXiv preprint arXiv:2610.02274*, 2026.

[7] Rohan Bansal, David He, Nadun Ranawaka Arachchige, Zhenyang Chen, Soobum Kim, Kexin Rong, and Danfei Xu. Action Chunk Scheduling for Batched Robot Policy Serving. *arXiv preprint arXiv:2608.00337*, 2026.

[8] Anuva Banwasi, William Muckelroy III, Priya Sundaresan, Linfeng Zhao, Jeannette Bohg, and Cherie Ho. MessyMem: Learning-from-Doing Memory for Mobile Manipulation. *arXiv preprint arXiv:2609.15976*, 2026.

[9] Nicolas Baumann, Liam Boyle, Pu Deng, Edoardo Ghignone, Boyang Sun, Marc Pollefeys, Luca Benini, and Michele Magno. LocalNav: Distilling Frontier VLMs and Embodied RL for On-Device Object Goal Navigation. *arXiv preprint arXiv:2606.27871*, 2026.

[10] Suneel Belkhale, Tianli Ding, Ted Xiao, Pierre Sermanet, Quon Vuong, Jonathan Tompson, Yevgen Chebotar, Debidatta Dwibedi, and Dorsa Sadigh. RT-H: Action Hierarchies Using Language. *arXiv preprint arXiv:2403.01823*, 2024.

[11] Vineet Bhat, Mikaela Angelina Uy, Siyi Chen, Alex Zook, Xuning Yang, Stan Birchfield, Valts Blukis, and Jonathan Tremblay. MG-VQA: Manipulation Grounded Visual Question Answering with VLMs. *arXiv preprint arXiv:2608.17129*, 2026.

[12] Hongzhe Bi, Hengkai Tan, Shenghao Xie, Zeyuan Wang, Shuhe Huang, Haitian Liu, Ruowen Zhao, Yao Feng, Chendong Xiang, Yinze Rong, Hongyan Zhao, Hanyu Liu, Zhizhong Su, Lei Ma, Hang Su, and Jun Zhu. Motus: A Unified Latent Action World Model. *arXiv preprint arXiv:2512.13030*, 2025.

[13] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A Vision-Language-Action Flow Model for General Robot Control. *arXiv preprint arXiv:2410.24164*, 2024.

[14] Kevin Black, Manuel Y. Galliker, and Sergey Levine. Real-Time Execution of Action Chunking Flow Policies. *arXiv preprint arXiv:2506.07339*, 2025.

[15] Kevin Black, Allen Z. Ren, Michael Equi, and Sergey Levine. Training-Time Action Conditioning for Efficient Real-Time Chunking. *arXiv preprint arXiv:2512.05964*, 2025.

[16] Nils Blank, Moritz Reuss, Marcel Rühle, Ömer Erdinç Yağmurlu, Fabian Wenzel, Oier Mees, and Rudolf Lioutikov. Scaling Robot Policy Learning via Zero-Shot Labeling with Foundation Models. In *Proceedings of The 8th Conference on Robot Learning*, volume 270, pages 4158–4187. PMLR, 2025.

[17] Nils Blank, Paul Mattes, Maximilian Xiling Li, Jakub Suliga, Thomas Roth, Moritz Reuss, Pankhuri Vanjani, and Rudolf Lioutikov. SPARC: Reliable Spatial Annotations from Robot Demonstrations at Scale. *arXiv preprint arXiv:2606.13497*, 2026.

[18] Konstantinos Bousmalis, Giulia Vezzani, Dushyant Rao, Coline Devin, Alex X. Lee, Maria Bauza, Todor Davchev, Yuxiang Zhou, Agrim Gupta, Akhil Raju, Antoine Laurens, Claudio Fantacci, Valentin Dalibard, Martina Zambelli, Murilo Martins, Rugile Pevceviciute, Michiel Blokzijl, Misha Denil, Nathan Batchelor, Thomas Lampe, Emilio Parisotto, Konrad Żołna, Scott Reed, Sergio Gómez Colmenarejo, Jon Scholz, Abbas Abdolmaleki, Oliver Groth, Jean-Baptiste Regli, Oleg Sushkov, Tom Rothörl, José Enrique Chen, Yusuf Aytar, Dave Barker, Joy Ortiz, Martin Riedmiller, Jost Tobias Springenberg, Raia Hadsell, Francesco Nori, and Nicolas Heess. RoboCat: A Self-Improving Generalist Agent for Robotic Manipulation. *Transactions on Machine Learning Research*, 2023.

[19] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Lisa Lee, Tsang-Wei Edward Lee, Sergey Levine, Yao Lu, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Vuong, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Jialin Wu, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. *arXiv preprint arXiv:2307.15818*, 2023.

[20] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Tomas Jackson, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Kuang-Huei Lee, Sergey Levine, Yao Lu, Utsav Malla, Deeksha Manjunath, Igor Mordatch, Ofir Nachum, Carolina Parada, Jodilyn Peralta, Emily Perez, Karl Pertsch, Jornell Quiambao, Kanishka Rao, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Kevin Sayed, Jaspiar Singh, Sumedh Sontakke, Austin Stone, Clayton Tan, Huong Tran, Vincent Vanhoucke, Steve Vega, Quan Vuong, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. RT-1: Robotics Transformer for Real-World Control at Scale. *arXiv preprint arXiv:2212.06817*, 2023.

[21] Qingwen Bu, Hongyang Li, Li Chen, Jisong Cai, Jia Zeng, Heming Cui, Maoqing Yao, and Yu Qiao. Towards Synergistic, Generalized, and Efficient Dual-System for Robotic Manipulation. *arXiv preprint arXiv:2410.08001*, 2024.

[22] Bytedance Seed. Seed1.8 Model Card: Towards Generalized Real-World Agency. *arXiv preprint arXiv:2603.20633*, 2026.

[23] Remi Cadene, Simon Aliberts, Francesco Capuano, Michel Aractingi, Adil Zouitine, Pepijn Kooijmans, Jade Choghari, Martino Russi, Caroline Pascal, Steven Palma, et al. LeRobot: An Open-Source Library for End-to-End Robot Learning. *arXiv preprint arXiv:2602.22818*, 2026.

[24] Xiaowei Cai, Yunuo Cai, Bingao Chen, Jingxiao Chen, Zhi Chen, Siyuan Feng, Tengyu Hou, Jingshun Huang, Han Jiang, Runkun Ju, Dong Li, Mingxiang Li, Shaowei Li, Xinchen Li, Yifan Li, Yi Liu, Zhongyuan Liu, Jianlan Luo, Junwen Miao, Ruiqi Ni, Buqing Nie, Mingjie Pan, Xinlin Ren, Jianheng Song, Jiaxu Wang, Peiqi Wang, Sen Wang, Xiaoyan Wang, Dafeng Wei, Dongming Wu, Pengwei Xie, Pu Yang, Hangjian Ye, Xiangyu Yue, Jinyu Zhang, Qinglin Zhang, Xueyong Zhao, Pengfei Zhou, and Yue Zhou. τ_0-VLA: a Hierarchical Robot Foundation Model with World-Model-Guided Test-Time Computation. *arXiv preprint arXiv:2608.16885*, 2026.

[25] Jiahang Cao, Qiang Zhang, Jingkai Sun, Jiaxu Wang, Hao Cheng, Yulin Li, Jun Ma, Kun Wu, Zhiyuan Xu, Yecheng Shao, Wen Zhao, Gang Han, Yijie Guo, and Renjing Xu. Mamba Policy: Towards Efficient 3D Diffusion Policy with Hybrid Selective State Models. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 11359–11366. IEEE, 2025. doi: 10.1109/IROS60139.2025.11247625.

[26] Irvin Steve Cardenas, Marcus Anthony Arnett, Natalie Catherine Yeo, Lucky Sah, and Jong-Hoon Kim. ROSClaw: An OpenClaw ROS 2 Framework for Agentic Robot Control and Interaction. *arXiv preprint arXiv:2603.26997*, 2026.

[27] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, et al. SAM 3: Segment Anything with Concepts. *arXiv preprint arXiv:2511.16719*, 2025.

[28] Jun Cen, Siteng Huang, Yuqian Yuan, Kehan Li, Hangjie Yuan, Chaohui Yu, Yuming Jiang, Jiayan Guo, Xin Li, Hao Luo, Fan Wang, Deli Zhao, and Hao Chen. RynnVLA-002: A Unified Vision-Language-Action and World Model. *arXiv preprint arXiv:2511.17502*, 2025.

[29] Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, Xin Li, Yibing Song, Hao Luo, Fan Wang, Deli Zhao, and Hao Chen. WorldVLA: Towards Autoregressive Action World Model. *arXiv preprint arXiv:2506.21539*, 2025.

[30] Nga Teng Chan, Yi Zhang, Yechi Liu, Renwen Cui, Fanhu Zeng, Zeyuan Ding, Xiancong Ren, Zhang Zhang, Qifeng Chen, Jian Liu, Yong Dai, and Xiaozhu Ju. Robo-Cortex: A Self-Evolving Embodied Agent via Dual-Grain Cognitive Memory and Autonomous Knowledge Induction. *arXiv preprint arXiv:2605.18729*, 2026.

[31] Matthew Chang, Gunjan Chhablani, Alexander Clegg, Mikael Dallaire Cote, Ruta Desai, Michal Hlavac, Vladimir Karashchuk, Jacob Krantz, Roozbeh Mottaghi, Priyam Parashar, Siddharth Patki, Ishita Prasad, Xavier Puig, Akshara Rai, Ram Ramrakhya, Daniel Tran, Joanne Truong, John M. Turner, Eric Undersander, and Tsung-Yen Yang. PARTNR: A Benchmark for Planning and Reasoning in Embodied Multi-agent Tasks. In *International Conference on Learning Representations*, 2025.

[32] Chi-Lam Cheang, Guangzeng Chen, Ya Jing, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Hongtao Wu, Jiafeng Xu, Yichu Yang, Hanbo Zhang, and Minzhao Zhu. GR-2: A Generative Video-Language-Action Model with Web-Scale Knowledge for Robot Manipulation. *arXiv preprint arXiv:2410.06158*, 2024.

[33] Boyuan Chen, Zhuo Xu, Sean Kirmani, Brian Ichter, Dorsa Sadigh, Leonidas Guibas, and Fei Xia. SpatialVLM: Endowing Vision-Language Models with Spatial Reasoning Capabilities. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14455–14465, 2024.

[34] Daphne Chen, Archit Ritesh Jain, Eric Goossen, Emma Romig, Michael Murray, Nick Walker, and Maya Cakmak. A Few Words Go a Long Way: Language Guided Robot Policy Synthesis. *arXiv preprint arXiv:2607.23784*, 2026.

[35] Guanxiong Chen, Yiduo Qu, Qianjun Xia, Pengyu Jing, Yixian Cheng, Bole Ma, Pengzhi Yang, Bingyang Zhou, Ziming Li, Shashwat Suri, Gongbo Sun, Chao Liu, Peter Yichen Chen, Ziqiu Zeng, and Fan Shi. DiagGen: Agentic Generation of Deformable Assets with Sim-based Diagnostics for Robotic Simulation. *arXiv preprint arXiv:2609.23103*, 2026.

[36] Guanxiong Chen, Qianjun Xia, Jiawei Peng, Heng Zhang, Pengyu Jing, Bole Ma, Justin Qian, Yixian Cheng, Ziyi Jiao, Bingyang Zhou, Yiduo Qu, Luoxin Ye, Kaifeng Zhang, Kunyi Wang, Weijia Zeng, Yunuo Chen, Pengzhi Yang, Ziqiu Zeng, Siyuan Luo, Huamin Wang, Chao Liu, Alan Yuille, Fan Shi, Changxi Zheng, Yunzhu Li, Chenfanfu Jiang, and Peter Yichen Chen. Agentic Real2Sim: Physics-based World Modeling with Vision-Language Agents. *arXiv preprint arXiv:2607.19190*, 2026.

[37] Junting Chen, Checheng Yu, Xunzhe Zhou, Tianqi Xu, Yao Mu, Mengkang Hu, Wenqi Shao, Yikai Wang, Guohao Li, and Lin Shao. EMOS: Embodiment-aware Heterogeneous Multi-robot Operating System with LLM Agents. In *International Conference on Learning Representations*, 2025.

[38] Kaiyuan Chen, Shuangyu Xie, Letian Fu, Justin Yu, William Pacini, Sandeep Bajamahal, Hudson Kim, Jaimyn Drake, Daehwa Kim, Haoru Xue, Jonathan Francis, Christian Juette, Peter Schaldenbrand, Muhammet Yunus Seker, Ruwan Wickramarachchi, Uksang Yoo, Guanzhi Wang, Adithyavairavan Murali, Balakumar Sundaralingam, S. Shankar Sastry, Spencer Huang, Yuke Zhu, Linxi "Jim" Fan, and Ken Goldberg. GaP: A Graph-as-Policy Multi-Agent Self-Learning Harness For Variational Automation Tasks. *arXiv preprint arXiv:2607.05369*, 2026.

[39] Sitong Chen, Fatemeh Zargarbashi, Jin Cheng, Tianxu An, and Stelian Coros. KINO: A Keyframe Interface for VLM Planning and Whole-Body Control in Humanoid Loco-Manipulation. *arXiv preprint arXiv:2609.18869*, 2026.

[40] Siyi Chen, Hugo Hadfield, Alex Zook, Mikaela Angelina Uy, Chan Hee Song, Erwin Coumans, Xuning Yang, Faisal Ladhak, Qing Qu, Stan Birchfield, Jonathan Tremblay, and Valts Blukis. VoLo: A Physical Orchestrator for Open-Vocabulary Long-Horizon Manipulation. *arXiv preprint arXiv:2606.07723*, 2026.

[41] Tianxing Chen, Yue Chen, Zixuan Li, Junyuan Tang, Kailun Su, Haoran Lu, Weijie Wan, Baijun Chen, Songling Liu, Haowen Yan, Honghao Su, Zhiyang Dou, Kaixuan Wang, Dandan Zhang, Yunze Liu, Yan Qin, Qiwei Liang, Qiwei Wu, Zijian Lin, Wenwei Lin, Yuran Wang, Minghua He, Tianshu Wu, Ruihai Wu, Jingquan Zhou, Kai-Chong Lei, Haibao Yu, Yuanfeng Ji, Weiyang Jin, Guanyu Lin, Xiaofan Li, Qi Xiong, Renjing Xu, Zhongyu Li, Wenhao Chai, Enze Xie, Ziwei Wang, Yao Mu, Hao Dong, Wojciech Matusik, Mingyu Ding, Wenbo Ding, Ping Luo, and Masayoshi Tomizuka. RoboDojo: A Unified Sim-and-Real Benchmark for Comprehensive Evaluation of Generalist Robot Manipulation Policies. *arXiv preprint arXiv:2607.04434*, 2026.

[42] William Chen, Jagdeep Singh Bhatia, Catherine Glossop, Nikhil Mathihalli, Ria Doshi, Andy Tang, Danny Driess, Karl Pertsch, and Sergey Levine. Steerable Vision-Language-Action Policies for Embodied Reasoning and Hierarchical Control. *arXiv preprint arXiv:2602.13193*, 2026.

[43] Xinyi Chen, Yilun Chen, Yanwei Fu, Ning Gao, Jiaya Jia, Weiyang Jin, Hao Li, Yao Mu, Jiangmiao Pang, Yu Qiao, Yang Tian, Bin Wang, Bolun Wang, Fangjing Wang, Hanqing Wang, Tai Wang, Ziqin Wang, Xueyuan Wei, Chao Wu, Shuai Yang, Jinhui Ye, Junqiu Yu, Jia Zeng, Jingjing Zhang, Jinyu Zhang, Shi Zhang, Feng Zheng, Bowen Zhou, and Yangkun Zhu. InternVLA-M1: A Spatially Guided Vision-Language-Action Framework for Generalist Robot Policy. *arXiv preprint arXiv:2510.13778*, 2025.

[44] Yang Chen, Lirong Che, Zhenyu Huang, Wenbo Fu, Chuang Wang, Xu Cao, Daqi Liu, Yuzhe Yang, Jian Su, and Lan-Zhe Guo. HarnessVLN: Unifying Training-Free Embodied Navigation through an Agent Harness. *arXiv preprint arXiv:2609.15195*, 2026.

[45] Yangtao Chen, Zixuan Chen, Peiyang Wang, Yong-Lu Li, Jing Huo, Jieqi Shi, and Yang Gao. Wh0: Generative World Models as Scalable Sources of Egocentric Human Hand Manipulation Data. *arXiv preprint arXiv:2606.22136*, 2026.

[46] Yanzhe Chen, Zechen Bai, Zhijun Cao, Wenzheng Zeng, Kevin Qinghong Lin, Yiqi Lin, Guoqiang Liang, Kevin Yuchen Ma, Qiming Huang, and Mike Zheng Shou. Show-Harness: Just a VLM Agent Can Play Robots. *arXiv preprint arXiv:2609.10522*, 2026.

[47] Yanzhe Chen, Ziyi Yang, Jifeng Zhu, Qiming Huang, Ruihe An, Peiyao Xu, Hesen Yang, Runda Liu, Chang Gong, Zhijun Cao, Zechen Bai, Wenzheng Zeng, Yiqi Lin, Guoqiang Liang, Kevin Yuchen Ma, Kevin Qinghong Lin, and Mike Zheng Shou. Survey on Multimodal Embodied Agents: A Unified Capability-centric Perspective from Computer-Use to Robot-Use, 2026.

[48] Yiteng Chen, Zhe Cao, Hongjia Ren, Chenjie Yang, Wenbo Li, Shiyi Wang, Yemin Wang, Li Zhang, Yanming Shao, Zhenjun Zhao, Huiping Zhuang, and Qingyao Wu. RoboRouter: Training-Free Policy Routing for Robotic Manipulation. *arXiv preprint arXiv:2603.07892*, 2026.

[49] Yiteng Chen, Huiping Zhuang, Wenbo Li, Shiyi Wang, Xiangyu Zhao, and Qingyao Wu. ManipEvalAgent: Promptable and Efficient Evaluation Framework for Robotic Manipulation Policies. In *International Conference on Learning Representations*, volume 2026, pages 84574–84598, 2026.

[50] Yitong Chen, Zezheng Huai, Sixian Li, Yubang Wang, Haozhe Zhang, Yifei Zhang, Hechang Chen, Jingjing Gong, Yu-Gang Jiang, and Xipeng Qiu. ETA: A New Agentic Paradigm for Embodied Tasks. *arXiv preprint arXiv:2608.03924*, 2026.

[51] Yuzhen Chen and KC Zhou. WCM: World-Cognition Model for Generalizable Human-Robot Interaction. *arXiv preprint arXiv:2607.22999*, 2026.

[52] An-Chieh Cheng, Yandong Ji, Zhaojing Yang, Zaitian Gongye, Xueyan Zou, Jan Kautz, Erdem Bıyık, Hongxu Yin, Sifei Liu, and Xiaolong Wang. NaVILA: Legged Robot Vision-Language-Action Model for Navigation. *arXiv preprint arXiv:2412.04453*, 2024.

[53] Dongzhou Cheng, Taoran Yi, Ye Fang, Xingwu Zhang, Fan Feng, Yixuan Li, Gengxiong Zhuang, Rongze Wang, Shuai Yang, Wei Song, Weizhi Xue, Minyan Wu, Jie Gui, Jiaqi Wang, and Tong Wu. In-Context Robot Learning with VLM Agents. *arXiv preprint arXiv:2609.19138*, 2026.

[54] Ho Kei Cheng, Seoung Wug Oh, Brian Price, Alexander Schwing, and Joon-Young Lee. Tracking Anything with Decoupled Video Segmentation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1316–1326, 2023.

[55] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. *arXiv preprint arXiv:2303.04137*, 2024.

[56] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, et al. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. *arXiv preprint arXiv:2507.06261*, 2025.

[57] Nitish Dashora, Douglas Chen, Idan Shenfeld, John Marangola, Pulkit Agrawal, and Max Simchowitz. Workspace Models: Lightweight Robotic Memory via Saliency-Driven Supervision. *arXiv preprint arXiv:2609.20820*, 2026.

[58] Matt Deitke, Christopher Clark, Sangho Lee, Rohun Tripathi, Yue Yang, Jae Sung Park, Mohammadreza Salehi, Niklas Muennighoff, Kyle Lo, Luca Soldaini, et al. Molmo and PixMo: Open Weights and Open Data for State-of-the-Art Vision-Language Models. *arXiv preprint arXiv:2409.17146*, 2024.

[59] Dexmal. OpenDM: DM0.5 for Open-World Robot Control, 2026. URL https://github.com/dexmal/opendm.

[60] Xin Ding, Liang Mi, Mingzhe Huang, Zixuan Wang, Chao Zhang, Zixu Hao, Fu Chen, Xiangyu Li, Yikai Zheng, Yaoyu Guo, Weijun Wang, Kun Li, Hao Wu, Yunxin Liu, and Ting Cao. Zetta ζ: An Efficient Closed-Loop Embodied Harness for Self-Evolving Physical Intelligence. *arXiv preprint arXiv:2608.16590*, 2026.

[61] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. PaLM-E: An Embodied Multimodal Language Model. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 8469–8488. PMLR, 2023.

[62] Yilun Du, Mengjiao Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Joshua B. Tenenbaum, Dale Schuurmans, and Pieter Abbeel. Learning Universal Policies via Text-Guided Video Generation. *arXiv preprint arXiv:2302.00111*, 2023.

[63] Jiafei Duan, Samson Yu, Hui Li Tan, Hongyuan Zhu, and Cheston Tan. A Survey of Embodied AI: From Simulators to Research Tasks. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 6(2): 230–244, 2022. doi: 10.1109/TETCI.2022.3141105.

[64] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. AHA: A Vision-Language-Model for Detecting and Reasoning Over Failures in Robotic Manipulation. In *International Conference on Learning Representations*, 2025.

[65] Karim Elmaaroufi, Justin Svegliato, Sarunas Kalade, Graham Schelle, Sanjit A. Seshia, and Matei Zaharia. RHO: Your Coding Agent is Secretly a Roboticist. *arXiv preprint arXiv:2606.16458*, 2026.

[66] Yijia Fan, Zonglin Di, Zimo Wen, Yifan Yang, Mingxi Cheng, Qi Dai, Bei Liu, Kai Qiu, Yue Dong, Ji Li, and Chong Luo. Resource2Skill: Distilling Executable Agent Skills from Human-Created Multimodal Resources. *arXiv preprint arXiv:2606.29538*, 2026.

[67] Hao-Shu Fang, Chenxi Wang, Hongjie Fang, Minghao Gou, Jirong Liu, Hengxu Yan, Wenhai Liu, Yichen Xie, and Cewu Lu. AnyGrasp: Robust and Efficient Grasp Perception in Spatial and Temporal Domains. *arXiv preprint arXiv:2212.08333*, 2022.

[68] Yuan Fang, Zechu Li, Haolei Tong, Puze Liu, and Georgia Chalvatzaki. Find Something You Can't Do: Agentic Real-World Reinforcement Learning for Self-Improving VLA Models. *arXiv preprint arXiv:2609.32069*, 2026.

[69] FBddcz. EmbodiedJev: A Browser-Based Workbench for Language-Model Decisions in Robot Manipulation, 2026. URL https://github.com/FBddcz/embodied-jev.

[70] Youhe Feng, Hansen Shi, Haoyang Li, Xinlei Guo, Yang Wang, Chengyang Zhang, Jinkai Zhang, Xiaohan Zhang, Jie Tang, and Jing Zhang. ProcVLM: Learning Procedure-Grounded Progress Rewards for Robotic Manipulation. *arXiv preprint arXiv:2605.08774*, 2026.

[71] Yunhai Feng, Jiaming Han, Zhuoran Yang, Xiangyu Yue, Sergey Levine, and Jianlan Luo. Reflective Planning: Vision-Language Models for Multi-Stage Long-Horizon Robotic Manipulation. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 2038–2062. PMLR, 2025.

[72] Figure AI. Helix: A Vision-Language-Action Model for Generalist Humanoid Control, 2025. URL https://www.figure.ai/news/helix.

[73] Chelsea Finn and Sergey Levine. Deep Visual Foresight for Planning Robot Motion. *arXiv preprint arXiv:1610.00696*, 2017.

[74] Letian Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Guanzhi Wang, Dantong Niu, Fei-Fei Li, Guanya Shi, Jiajun Wu, Shankar Sastry, Yuke Zhu, Ken Goldberg, and Linxi "Jim" Fan. CaP-X: A Framework for Benchmarking and Improving Coding Agents for Robot Manipulation. *arXiv preprint arXiv:2603.22435*, 2026.

[75] Liane Galanti, Dhruv Shah, and Tri Dao. Addressing the Orchestration Gap in Generalist Robots via Physical Agency. *arXiv preprint arXiv:2607.21725*, 2026.

[76] Galaxea Team. Galaxea G0.5: One Autoregressive Stream for Robot Reasoning and Action, 2026.

[77] Chongkai Gao, Zixuan Liu, Zhenghao Chi, Junshan Huang, Xin Fei, Yiwen Hou, Yuxuan Zhang, Yudi Lin, Zhirui Fang, Zeyu Jiang, et al. VLA-OS: Structuring and Dissecting Planning Representations and Paradigms in Vision-Language-Action Models. *arXiv preprint arXiv:2506.17561*, 2025.

[78] Shenyuan Gao, William Liang, Kaiyuan Zheng, Ayaan Malik, Seonghyeon Ye, Sihyun Yu, Wei-Cheng Tseng, Yuzhu Dong, Kaichun Mo, Chen-Hsuan Lin, Qianli Ma, Seungjun Nah, Loic Magne, Jiannan Xiang, Yuqi Xie, Ruijie Zheng, Dantong Niu, You Liang Tan, K. R. Zentner, George Kurian, Suneel Indupuru, Pooya Jannaty, Jinwei Gu, Jun Zhang, Jitendra Malik, Pieter Abbeel, Ming-Yu Liu, Yuke Zhu, Joel Jang, and Linxi "Jim" Fan. DreamDojo: A Generalist Robot World Model from Large-Scale Human Videos. *arXiv preprint arXiv:2602.06949*, 2026.

[79] Jingzhan Ge, Ruimin Chen, Azadeh Haghighi, Jiong Tang, and Farhad Imani. Kinematics-Grounded Agentic AI for Robotic Additive Manufacturing Process Planning. *arXiv preprint arXiv:2609.19347*, 2026.

[80] Yuying Ge, Annabella Macaluso, Li Erran Li, Ping Luo, and Xiaolong Wang. Policy Adaptation from Foundation Model Feedback. *arXiv preprint arXiv:2212.07398*, 2023.

[81] Gemini Robotics Team et al. Gemini Robotics: Bringing AI into the Physical World. *arXiv preprint arXiv:2503.20020*, 2025.

[82] Genesis Authors. Genesis: A Generative and Universal Physics Engine for Robotics and Beyond, December 2024. URL https://github.com/Genesis-Embodied-AI/genesis-world.

[83] Theophile Gervet, Zhou Xian, Nikolaos Gkanatsios, and Katerina Fragkiadaki. Act3D: 3D Feature Field Transformers for Multi-Task Robotic Manipulation. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 3949–3965. PMLR, 2023.

[84] Dorian Benhamou Goldfajn, Mason Nakamura, Saaduddin Mahmud, Justin Svegliato, Kyle H. Wray, and Shlomo Zilberstein. RoboTalk: Learning Multi-Robot Communication and Coordination from Multimodal Demonstrations. *arXiv preprint arXiv:2609.23997*, 2026.

[85] Litian Gong, Fatemeh Bahrani, Yutai Zhou, Amin Banayeeanzade, Jiachen Li, and Erdem Bıyık. AutoFocus-IL: VLM-based Saliency Maps for Data-Efficient Visual Imitation Learning without Extra Human Annotations. *arXiv preprint arXiv:2511.18617*, 2025.

[86] Ruiyan Gong, Yingnan Guo, Junjun Hu, Jintao Kong, Xiaoxu Leng, Tianlun Li, Weize Li, Fei Liu, Zhicheng Liu, Jia Lu, Minghua Luo, Chenlin Ming, Yanfen Shen, Jiyue Tao, Zhengbo Wang, Mingyang Yin, Minqi Gu, Zihao Guan, Wei Guo, Guoqing Liu, Huachong Pang, Menglin Yang, Zeqian Ye, Xiaoxiao Geng, Zhining Gu, Honglin Han, Di Jing, Hongyu Pan, Mingchao Sun, Kuan Yang, Jianfang Zhang, Yanghong Chen, Ye He, Wei Mei, Jiahao Shi, Xiangpo Yang, Yanqing Zhu, Yang Cai, Jingjing Ma, Shihui Su, Zixiao Tang, Linbo

Zheng, Zedong Chu, Xiaolong Wu, Wenbin Tang, and Mu Xu. ABot-N1: Toward a General Visual Language Navigation Foundation Model. *arXiv preprint arXiv:2607.10383*, 2026.

[87] Google DeepMind. Gemini 4 Argon: Model Card, 2026. URL https://deepmind.google/models/model-cards/gemini-4-argon/. Accessed October 14, 2026.

[88] Ankit Goyal, Jie Xu, Yijie Guo, Valts Blukis, Yu-Wei Chao, and Dieter Fox. RVT: Robotic View Transformer for 3D Object Manipulation. *arXiv preprint arXiv:2306.14896*, 2023.

[89] Zhaopeng Gu, Bingke Zhu, Tianxi Lin, Guibo Zhu, Yingying Chen, Kai Wang, Tingyu Yuan, Chaoyang Zhao, Zhaowen Li, Peng Su, and Jinqiao Wang. HarnessWAM: Bridging Prediction and Deliberation in World Action Models. *arXiv preprint arXiv:2608.09516*, 2026.

[90] Weifan Guan, Qinghao Hu, Huasen Xi, Chenxiao Zhang, Aosheng Li, and Jian Cheng. RoboNeuron: A Middle-Layer Infrastructure for Agent-Driven Orchestration in Embodied AI. *arXiv preprint arXiv:2512.10394*, 2025.

[91] Xinyuan Guan, Feifan Chen, Xinyu Zhan, Fu-Cheng Zhang, Cewu Lu, and Lixin Yang. EgoAfford: Task-Oriented Affordance Grounding via Egocentric Referring Segmentation. *arXiv preprint arXiv:2608.04533*, 2026.

[92] Christian Gumbsch, Leonardo Barcellona, Lennard Schünemann, Platon Karageorgis, Andrii Zadaianchuk, Zehao Wang, Sergey Zakharov, Fabien Despinoy, Rahaf Aljundi, and Efstratios Gavves. From Demonstrations to Rewards: Test-Time Prompt Optimization for VLM Reward Models. *arXiv preprint arXiv:2606.00083*, 2026.

[93] Meng-Hao Guo, Zhe-Han Mo, Jia-Jun Wang, Yi Zhang, Kejin Wang, Yi-Xuan Deng, Jia-Peng Zhang, Yongming Rao, and Shi-Min Hu. Transferring the Intelligence of VLMs to Robotic Control. *arXiv preprint arXiv:2609.22966*, 2026.

[94] Yanjiang Guo, Yen-Jen Wang, Lihan Zha, and Jianyu Chen. DoReMi: Grounding Language Model by Detecting and Recovering from Plan-Execution Misalignment. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.

[95] Yanjiang Guo, Tony Lee, Lucy Xiaoyang Shi, Jianyu Chen, Percy Liang, and Chelsea Finn. VLAW: Iterative Co-Improvement of Vision-Language-Action Policy and World Model. *arXiv preprint arXiv:2602.12063*, 2026.

[96] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning Latent Dynamics for Planning from Pixels. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2555–2565. PMLR, 2019.

[97] Minkyu Ham, Dongho Kim, Chan Lee, Min Jun Kim, Yixi Zhang, Jiayi Wang, and Han Liu. SPINE: Bridging the Cyber-Physical Gap with Agentic AI. *arXiv preprint arXiv:2607.13049*, 2026.

[98] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, Robust World Models for Continuous Control. *arXiv preprint arXiv:2310.16828*, 2023.

[99] Shaid Hasan, Breenice Lee, Sujan Sarker, and Tariq Iqbal. M2HRI: An LLM-Driven Multimodal Multi-Agent Framework for Personalized Human-Robot Interaction. *arXiv preprint arXiv:2604.11975*, 2026.

[100] Jinyu He, Zihao Mao, Haonan Jin, Mengyin Fu, and Wenjie Song. CoRelNav: Collaborative Relational Navigation for Multi-Robot Spatially Constrained Semantic Navigation. *arXiv preprint arXiv:2609.27720*, 2026.

[101] Songjia He, Zixuan Chen, Hongyu Ding, Dian Shao, Jieqi Shi, Chenxu Li, Jing Huo, and Yang Gao. V-Dreamer: Automating Robotic Simulation and Trajectory Synthesis via Video Generation Priors. *arXiv preprint arXiv:2603.18811*, 2026.

[102] Yi-Xiang He, Lan Wei, Haoming Cen, Jian-Jian Jiang, Zhuohao Li, Guanxing Lu, Yihan Yang, Dandan Zhang, and Wei-Shi Zheng. A Closed-Loop Multi-Agent Framework for Robust Multi-Robot Manipulation. In *Proceedings of Robotics: Science and Systems*, 2026. doi: 10.15607/RSS.2026.XXII.036.

[103] Nikolaj Hindsbo, Sina Ehsani, and Pragyana Mishra. SCOPE: Real-Time Natural Language Camera Agent at the Edge. In *Proceedings of the 21st ACM/IEEE International Conference on Human-Robot Interaction*, pages 553–561. ACM, 2026. doi: 10.1145/3757279.3785641.

[104] Wenhao Hong, Lan Wei, and Dandan Zhang. Bridging Semantics and Physics with Constrained LLMs for Safe and Trustworthy Robotic Manipulation. *arXiv preprint arXiv:2608.29379*, 2026.

[105] Yining Hong, Huang Huang, Manling Li, Li Fei-Fei, Leonidas Guibas, Jiajun Wu, and Yejin Choi. Learning from Trials and Errors: Reflective Test-Time Planning for Embodied LLMs. *arXiv preprint arXiv:2602.21198*, 2026.

[106] Haodi Hu, Chung-Ta Huang, Jing Liu, Ye Wang, Kei Suzuki, Matthew Brand, and Toshiaki Koike-Akino. ReCoVLA: VLM-Guided Reward Compilation for Failure Recovery in Vision-Language-Action Policies. *arXiv preprint arXiv:2606.09630*, 2026.

[107] Hengyuan Hu, Jensen Gao, Priya Sundaresan, Satvik Sharma, Jeannette Bohg, and Dorsa Sadigh. VIA: Visual Interface Agent for Robot Control. *arXiv preprint arXiv:2607.11119*, 2026.

[108] Jiaheng Hu, Mohit Shridhar, Caden Lu, Dhruv Shah, Hao-Tien Lewis Chiang, Jie Tan, and Annie Xie. What Matters in Orchestrating Robot Policies: A Systematic Study of Hierarchical VLA Agents. *arXiv preprint arXiv:2606.10267*, 2026.

[109] Sheng Hu, Weiyi Lu, Lingbing Zeng, Gan Weng, Weiwei Zhang, Kai Xie, Xiaofeng Mou, and Yi Xu. ARS: Agentic Reward System for Robot Learning. *arXiv preprint arXiv:2609.34484*, 2026.

[110] Yafei Hu, Quanting Xie, Vidhi Jain, Jonathan Francis, Jay Patrikar, Nikhil Keetha, Seungchan Kim, Yaqi Xie, Tianyi Zhang, Hao-Shu Fang, Shibo Zhao, Shayegan Omidshafiei, Dong-Ki Kim, Ali-akbar Aghamohammadi, Katia Sycara, Matthew Johnson-Roberson, Dhruv Batra, Xiaolong Wang, Sebastian Scherer, Chen Wang, Zsolt Kira, Fei Xia, and Yonatan Bisk. Toward General-Purpose Robots via Foundation Models: A Survey and Meta-Analysis. *arXiv preprint arXiv:2312.08782*, 2024.

[111] Yucheng Hu, Jianke Zhang, Yuanfei Luo, Yanjiang Guo, Xiaoyu Chen, Xinshu Sun, Kun Feng, Qingzhou Lu, Sheng Chen, Yangang Zhang, Wei Li, and Jianyu Chen. BagelVLA: Enhancing Long-Horizon Manipulation via Interleaved Vision-Language-Action Generation. *arXiv preprint arXiv:2602.09849*, 2026.

[112] Yutong Hu, Fengjiao Chen, Xuezhi Cao, and Renaud Detry. 2AM: Grounding Agent-Side Memory as Guidance for Steerable Action Models in Long-Horizon Manipulation. *arXiv preprint arXiv:2609.11308*, 2026.

[113] Pu Hua, Minghuan Liu, Annabella Macaluso, Yunfeng Lin, Weinan Zhang, Huazhe Xu, and Lirui Wang. GenSim2: Scaling Robot Data Generation with Multi-modal and Reasoning LLMs. In *Proceedings of The 8th Conference on Robot Learning*, volume 270, pages 5030–5066. PMLR, 2025.

[114] Haojian Huang, Jiahao Shi, Yinchuan Li, and Yingcong Chen. Affordance Agent Harness: Verification-Gated Skill Orchestration. *arXiv preprint arXiv:2605.00663*, 2026.

[115] Haojian Huang, Pukun Zhao, Zexi Li, Yehang Zhang, Yangkai Wei, Wenqian Li, Han Yang, Kaiwen Zhou, Ying-Cong Chen, and Yinchuan Li. Are Frontier VLM Agents Ready to Be Robot Generalists? An Empirical Study with the Embodied Agent Arena. *arXiv preprint arXiv:2610.00854*, 2026.

[116] Jinbang Huang, Zhiyuan Li, Yuanzhao Hu, Ran Qi, Yixin Xiao, Mark Coates, Tongtong Cao, Zhanguang Zhang, and Yingxue Zhang. RoboHarness: Memory-Driven Orchestration of Heterogeneous Robot Policies for Long-Horizon Planning. *arXiv preprint arXiv:2607.18060*, 2026.

[117] Siyuan Huang, Zhengkai Jiang, Hao Dong, Yu Qiao, Peng Gao, and Hongsheng Li. Instruct2Act: Mapping Multi-modality Instructions to Robotic Actions with Large Language Model. *arXiv preprint arXiv:2305.11176*, 2023.

[118] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 540–562. PMLR, 2023.

[119] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Tomas Jackson, Noah Brown, Linda Luu, Sergey Levine, Karol Hausman, and Brian Ichter. Inner Monologue: Embodied Reasoning through Planning with Language Models. In *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 1769–1782. PMLR, 2023.

[120] Wenlong Huang, Chen Wang, Yunzhu Li, Ruohan Zhang, and Li Fei-Fei. ReKep: Spatio-Temporal Reasoning of Relational Keypoint Constraints for Robotic Manipulation. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 4573–4602. PMLR, 2025.

[121] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Fei-Fei Li. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20765–20779, 2026.

[122] Yixuan Huang, Bowen Li, Vaibhav Saxena, Yichao Liang, Utkarsh Aashu Mishra, Liang Ji, Lihan Zha, Jimmy Wu, Nishanth Kumar, Sebastian Scherer, Danfei Xu, and Tom Silver. KinDER: A Physical Reasoning Benchmark for Robot Learning and Planning. In *Proceedings of Robotics: Science and Systems*, 2026. doi: 10.15607/RSS.2026.XXII.184.

[123] Yuzhi Huang, Jie Wu, Weijue Bu, Ziyi Xiong, Gaoyang Jiang, Ye Li, Kangye Ji, Shuzhao Xie, Yue Huang, Chenglei Wu, Jingyan Jiang, and Zhi Wang. RoboStream: Weaving Spatio-Temporal Reasoning with Memory in Vision-Language Models for Robotics. *arXiv preprint arXiv:2603.12939*, 2026.

[124] Zhiyu Huang, Yun Zhang, Johnson Liu, Rui Song, Chen Tang, and Jiaqi Ma. TIC-VLA: A Think-in-Control Vision-Language-Action Model for Robot Navigation in Dynamic Environments. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306, pages 48344–48367. PMLR, 2026.

[125] Brian Ichter, Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, Ryan Julian, Dmitry Kalashnikov, Sergey Levine, Yao Lu, Carolina Parada, Kanishka Rao, Pierre Sermanet, Alexander T Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Mengyuan Yan, Noah Brown, Michael Ahn, Omar Cortes, Nicolas Sievers, Clayton Tan, Sichun Xu, Diego Reyes, Jarek Rettinghouse, Jornell Quiambao, Peter Pastor, Linda Luu, Kuang-Huei Lee, Yuheng Kuang, Sally Jesmonth, Nikhil J. Joshi, Kyle Jeffrey, Rosario Jauregui Ruano, Jasmine Hsu, Keerthana Gopalakrishnan, Byron David, Andy Zeng, and Chuyuan Kelly Fu. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. In *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318. PMLR, 2023.

[126] Félix Ingrand and Malik Ghallab. Deliberation for Autonomous Robots: A Survey. *Artificial Intelligence*, 247:10–44, 2017. doi: 10.1016/j.artint.2014.11.003.

[127] MoniJesu James, Amir Atef Habel, Aleksey Fedoseev, and Dzmitry Tsetserokou. GoalVLM: VLM-driven Object Goal Navigation for Multi-Agent System. *arXiv preprint arXiv:2603.18210*, 2026.

[128] Eric Jang, Alex Irpan, Mohi Khansari, Daniel Kappler, Frederik Ebert, Corey Lynch, Sergey Levine, and Chelsea Finn. BC-Z: Zero-Shot Task Generalization with Robotic Imitation Learning. *arXiv preprint arXiv:2202.02005*, 2022.

[129] Gaytri Jena, Kapil Wanaskar, Vinija Jain, Aman Chadha, Vasu Sharma, and Amitava Das. Weights or Skills? A Survey of Robot-Learning Techniques: from Action-Predicting Weights to Robots that Write their Own Skills. *arXiv preprint arXiv:2608.01851*, 2026.

[130] Hyun Joe Jeong, Gokul Swamy, and Andrea Bajcsy. Learning What to Say to Your VLA: Mostly Harmless Vision Language Action Model Steering. *arXiv preprint arXiv:2606.12299*, 2026.

[131] Li Ji, Siyin Wang, Pengfang Qian, Xiaopeng Yu, Yihai Tian, Zhaoye Fei, Jingjing Gong, and Xipeng Qiu. HiMe: Hierarchical Embodied Memory for Long-Horizon Vision-Language-Action Control. *arXiv preprint arXiv:2607.03449*, 2026.

[132] Mengzhao Jia, Yang Lin, Xixin Zhang, Zhihan Zhang, Xiaobai Liu, and Meng Jiang. Agent as Policy for Robotic Manipulation. *arXiv preprint arXiv:2609.12541*, 2026.

[133] Baoyang Jiang, Fengchun Zhang, Leyuan Wang, Haotian Li, Yida Wang, Zhe Ji, Jinshan Lai, Xi Ren, Danyang Li, Zheng Yang, Jianwei Hu, and Qiang Ma. Embodied-BenchForge: A Closed-Loop Agentic Workflow for Embodied Benchmark Construction. *arXiv preprint arXiv:2609.13082*, 2026.

[134] Jing Jiang, Yue Yang, Xinkai Jiang, Gedas Bertasius, Daniel J. Szafir, and Rudolf Lioutikov. From Rollout to Reset: A Graph-Based Harness for Autonomous Long-Horizon Manipulation Evaluation. *arXiv preprint arXiv:2609.19413*, 2026.

[135] Wenqi Jiang, Jason Clemons, Rowland O'Flaherty, Hugo Hadfield, Alperen Degirmenci, Shuran Song, Yashraj Narang, and Christos Kozyrakis. ROSA: A Robotics Foundation Model Serving System for Robot Factories. *arXiv preprint arXiv:2607.01088*, 2026.

[136] Wenqi Jiang, Jason Clemons, Karu Sankaralingam, and Christos Kozyrakis. How Fast Can I Run My VLA? Demystifying VLA Inference Performance with VLA-Perf. *arXiv preprint arXiv:2602.18397*, 2026.

[137] Yicheng Jiang, Jiaxu Wang, Junhao He, Zesen Gan, Junhao Li, Qiang Zhang, Jingkai Sun, Jiahang Cao, Mingyuan Sun, Xiangyu Yue, and Qiming Shao. Learning Structural Latent Points for Efficient Visual Representations in Robotic Manipulation. In *2026 IEEE International Conference on Robotics and Automation (ICRA)*, pages 10457–10464. IEEE, 2026. doi: 10.1109/ICRA57385.2026.11696212.

[138] Yunfan Jiang, Agrim Gupta, Zichen Zhang, Guanzhi Wang, Yongqiang Dou, Yanjun Chen, Li Fei-Fei, Anima Anandkumar, Yuke Zhu, and Linxi Fan. VIMA: General Robot Manipulation with Multimodal Prompts. *arXiv preprint arXiv:2210.03094*, 2022.

[139] Yufeng Jin, Jianfei Guo, Xiaogang Jia, Yu Deng, Zechu Li, Han Liu, Weiran Liao, Vignesh Prasad, Mathias Franzius, Gerhard Neumann, and Georgia Chalvatzaki. Nautilus: From One Prompt to Plug-and-Play Robot Learning. *arXiv preprint arXiv:2605.11665*, 2026.

[140] Zhi Jing, Siyuan Yang, Jicong Ao, Ting Xiao, Yu-Gang Jiang, and Chenjia Bai. HumanoidGen: Data Generation for Bimanual Dexterous Manipulation via LLM Reasoning. In *Advances in Neural Information Processing Systems*, volume 38, pages 173413–173459, 2025. doi: 10.52202/085713-5220.

[141] Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

[142] Arash Ghasemzadeh Kakroudi and Roel Pieters. A Conversational Framework for Human-Robot Collaborative Manipulation with Distributed Generative AI models. *arXiv preprint arXiv:2606.06061*, 2026.

[143] Dmitry Kalashnikov, Alex Irpan, Peter Pastor, Julian Ibarz, Alexander Herzog, Eric Jang, Deirdre Quillen, Ethan Holly, Mrinal Kalakrishnan, Vincent Vanhoucke, and Sergey Levine. QT-Opt: Scalable Deep Reinforcement Learning for Vision-Based Robotic Manipulation. *arXiv preprint arXiv:1806.10293*, 2018.

[144] Haoqiang Kang, Xiaokang Ye, Yuhan Liu, Siddhant Hitesh Mantri, Lingjun Mao, James Fleming, Drishti Regmi, and Lianhui Qin. SimWorld Studio: Automatic Environment Generation with Evolving Coding Agent for Embodied Agent Learning. *arXiv preprint arXiv:2605.09423*, 2026.

[145] Tomoya Kawabe and Rin Takano. Hierarchical LLM-Based Multi-Agent Framework with Prompt Optimization for Multi-Robot Task Planning. *arXiv preprint arXiv:2602.21670*, 2026.

[146] Tsung-Wei Ke, Nikolaos Gkanatsios, and Katerina Fragkiadaki. 3D Diffuser Actor: Policy Diffusion with 3D Scene Representations. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 1949–1974. PMLR, 2025. CoRL 2024.

[147] Nikolaos Kekatos, Panagiotis Katsaros, Alexios Lekidis, Theodoros Nestoridis, and Tom Nianios. Mission-Level Runtime Assurance for LLM-Assisted ISR Swarms over a Verification-Aware Fabric. *arXiv preprint arXiv:2607.23532*, 2026.

[148] Nimesh Khandelwal and Shakti S. Gupta. Agent-Driven Autonomous Reinforcement Learning Research: Iterative Policy Improvement for Quadruped Locomotion. *arXiv preprint arXiv:2603.27416*, 2026.

[149] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An Open-Source Vision-Language-Action Model. *arXiv preprint arXiv:2406.09246*, 2024.

[150] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success. *arXiv preprint arXiv:2502.19645*, 2025.

[151] Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, and Jinwei Gu. Cosmos Policy: Fine-Tuning Video Models for Visuomotor Control and Planning. *arXiv preprint arXiv:2601.16163*, 2026.

[152] Seungku Kim, Suhyeok Jang, Byungjun Yoon, Dongyoung Kim, John Won, and Jinwoo Shin. RoboCurate: Harnessing Diversity with Action-Verified Neural Trajectory for Robot Learning. *arXiv preprint arXiv:2602.18742*, 2026.

[153] James R. Kirk, Robert E. Wray, and John E. Laird. STARS: extending interactive task learning with large language models. *Frontiers in Computer Science*, 8:1848830, 2026. doi: 10.3389/fcomp.2026.1848830.

[154] Longxin Kou, Fei Ni, Yan Zheng, Peilong Han, Jinyi Liu, Haiqin Cui, Rui Liu, and Jianye Hao. RoboAnnotatorX: A Comprehensive and Universal Annotation Framework for Accurate Understanding of Long-horizon Robot Demonstration. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 10353–10363, October 2025.

[155] Vaishak Kumar. Act-Observe-Rewrite: Multimodal Coding Agents as In-Context Policy Learners for Robot Manipulation. *arXiv preprint arXiv:2603.04466*, 2026.

[156] So Kuroki and Yujin Tang. Learning and Transferring Closed-Loop Robot Software. *arXiv preprint arXiv:2609.19906*, 2026.

[157] Jacky Kwok, Xilun Zhang, Mengdi Xu, Yuejiang Liu, Azalia Mirhoseini, Chelsea Finn, and Marco Pavone. Scaling Verification Can Be More Effective than Scaling Policy Learning for Vision-Language-Action Alignment. *arXiv preprint arXiv:2602.12281*, 2026.

[158] Haoran Lang, Haotao Lu, Shiyu Sang, Haoyang Luo, Guo Chen, Qun Li, Jingyi Yu, Ye Shi, and Jingya Wang. EmbodiRSI: Recursive Self-Improvement for Data-Efficient Robot Adaptation. *arXiv preprint arXiv:2609.38905*, 2026.

[159] Long Le, Jason Xie, William Liang, Hung-Ju Wang, Yue Yang, Yecheng Jason Ma, Kyle Vedder, Arjun Krishna, Dinesh Jayaraman, and Eric Eaton. Articulate-Anything: Automatic Modeling of Articulated Objects via a Vision-Language Foundation Model. *arXiv preprint arXiv:2410.13882*, 2024.

[160] Dong Won Lee, Sarah Gillet, Louis-Philippe Morency, Cynthia Breazeal, and Hae Won Park. A Modern System Recipe for Situated Embodied Human-Robot Conversation with Real-Time Multimodal LLMs and Tool-Calling. *arXiv preprint arXiv:2602.04157*, 2026.

[161] Hojoon Lee, Ajay Subramanian, Ben Abbatematteo, Vijay Veerabadran, Pedro Matias, Karl Ridgeway, and Nitin Kamra. Reward Design Agent for Reinforcement Learning. *Reinforcement Learning Journal*, 7, 2026.

[162] Junha Lee, Eunha Park, and Minsu Cho. DextER: Language-driven Dexterous Grasp Generation with Embodied Reasoning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1067–1077, 2026.

[163] Tony Lee, Andrew Wagenmaker, Karl Pertsch, Percy Liang, Sergey Levine, and Chelsea Finn. RoboReward: General-Purpose Vision-Language Reward Models for Robotics. *arXiv preprint arXiv:2601.00675*, 2026.

[164] Huashuo Lei, Wenxuan Song, Huarui Zhang, Jieyuan Pei, Jiayi Chen, Haodong Yan, Han Zhao, Pengxiang Ding, Zhipeng Zhang, Lida Huang, Donglin Wang, Yan Wang, and Haoang Li. RoboMemArena: A Comprehensive and Challenging Robotic Memory Benchmark. *arXiv preprint arXiv:2605.10921*, 2026.

[165] Iok Tong Lei, QianZhi Li, Ying Jie Yap, Yujie Zhang, Rui Zhong, Haichao Gui, Xiaolong Liu, and Zhidong Deng. ACE: Agentic Control for Embodied Manipulation via Zero-shot Workflow Reasoning. *arXiv preprint arXiv:2607.04162*, 2026.

[166] Zixing Lei, Changxing Liu, Yichen Xiong, Minhao Xiong, Yuanzhuo Ding, Zhipeng Zhang, Weixin Li, and Siheng Chen. Towards Long-horizon Embodied Agents with Tool-Aligned Vision-Language-Action Models. *arXiv preprint arXiv:2605.13119*, 2026.

[167] Zixing Lei, Gengze Zhou, Xiong-Hui Chen, Jiazhao Zhang, Yiyang Huang, Hang Yin, Haoqi Yuan, Qi Wu, Weixin Li, and Siheng Chen. Scaffolding Foundation Models into Physical-World Agents Pushes the Frontier of Long-Horizon Navigation. *arXiv preprint arXiv:2608.30396*, 2026.

[168] Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. End-to-End Training of Deep Visuomotor Policies. *Journal of Machine Learning Research*, 17(39):1–40, 2016.

[169] Bowei Li, Yuner Zhang, and Changliu Liu. ARSTAG: An Agentic Real2Sim2Real System for Task-Specific Robot Data Generation. *arXiv preprint arXiv:2609.24563*, 2026.

[170] Dayou Li, Jiuzhou Lei, Hao Wang, Lulin Liu, Yunhao Yang, Zihan Wang, Bangya Liu, Minghui Zheng, and Zhiwen Fan. Learning Actionable Manipulation Recovery via Counterfactual Failure Synthesis. *arXiv preprint arXiv:2603.13528*, 2026.

[171] Guoliang Li, Ruihua Han, Chengyang Li, He Li, Shuai Wang, Wenchao Ding, Hong Zhang, and Chengzhong Xu. Self-Evolutionary Replanning for Failure-Aware Motion Planning. *arXiv preprint arXiv:2603.02772*, 2026.

[172] Haoyang Li, Yang You, Hao Su, and Leonidas Guibas. PhysMem: Scaling Test-Time Memory for Embodied Physical Reasoning. *arXiv preprint arXiv:2602.20323*, 2026.

[173] Haozhou Li, Xiangyu Dong, Huiyan Jiang, Yaoming Zhou, and Xiaoguang Ma. CMMR-VLN: Vision-and-Language Navigation via Continual Multimodal Memory Retrieval. *arXiv preprint arXiv:2603.07997*, 2026.

[174] Jialiang Li, Yuhan Wang, Haojun Li, Gaojing Zhang, Yangtian Ye, Qipeng Liu, Haotian Liang, and Wenzhao Lian. Practice Makes Policies: Bootstrapping and Consolidating Robotic Capabilities from Zero Human Demonstrations. *arXiv preprint arXiv:2607.26809*, 2026.

[175] Jingyao Li, Pengguang Chen, Sitong Wu, Chuanyang Zheng, Hong Xu, and Jiaya Jia. RoboCoder: Robotic Learning from Basic Skills to General Tasks with Large Language Models. *arXiv preprint arXiv:2406.03757*, 2024.

[176] Lin Li, Qihang Zhang, Yiming Luo, Shuai Yang, Ruilin Wang, Fei Han, Mingrui Yu, Zelin Gao, Nan Xue, Xing Zhu, Yujun Shen, and Yinghao Xu. Causal World Modeling for Robot Control. *arXiv preprint arXiv:2601.21998*, 2026.

[177] Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, Xiaofan Wang, Bei Liu, Jianlong Fu, Jianmin Bao, Dong Chen, Yuanchun Shi, Jiaolong Yang, and Baining Guo. CogACT: A Foundational Vision-Language-Action Model for Synergizing Cognition and Action in Robotic Manipulation. *arXiv preprint arXiv:2411.19650*, 2024.

[178] Ruiying Li, Yunlang Zhou, YuYao Zhu, Kylin Chen, Jingyuan Wang, Sukai Wang, Kongtao Hu, Minhui Yu, Bowen Jiang, Zhan Su, Jiayao Ma, Xin He, Yongjian Shen, Yang Yang, Guanghui Ren, Maoqing Yao, Wenhao Wang, and Yao Mu. RoboClaw: An Agentic Framework for Scalable Long-Horizon Robotic Tasks. *arXiv preprint arXiv:2603.11558*, 2026.

[179] Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified Video Action Model. *arXiv preprint arXiv:2503.00200*, 2025.

[180] Siyuan Li, Taiyan Lang, Aoqi Yan, Jia Yu, Feifan Liu, Yihan Du, Yu Zheng, Xun Wang, and Peng Liu. Safe Task Planning with Long-Term Graph Memory for Embodied Agents. *arXiv preprint arXiv:2609.08444*, 2026.

[181] Xiangyu Li, Huaizhi Tang, Xin Ding, Weijun Wang, Ting Cao, and Yunxin Liu. OxyGen: Unified KV Cache Management for VLA Inference under Multi-Task Parallelism. *arXiv preprint arXiv:2603.14371*, 2026.

[182] Xinghang Li, Minghuan Liu, Hanbo Zhang, Cunjun Yu, Jie Xu, Hongtao Wu, Chilam Cheang, Ya Jing, Weinan Zhang, Huaping Liu, Hang Li, and Tao Kong. Vision-Language Foundation Models as Effective Robot Imitators. *arXiv preprint arXiv:2311.01378*, 2023.

[183] Xirui Li, Peng Shi, Mingwen Dong, Sheng Zhang, Zhuoyan Xu, Dongkyu Lee, Shuaichen Chang, Yi Xiang, Lin Pan, and Jiarong Jiang. LEGO-Anything: Coding Agents for 3D Scene Reconstruction. *arXiv preprint arXiv:2609.36380*, 2026.

[184] Xueying Li, Feng Lyu, Hao Wu, Mingliu Liu, Jia-Nan Liu, and Guozi Liu. Stop Wandering: Efficient Vision-Language Navigation via Metacognitive Reasoning. *arXiv preprint arXiv:2604.02318*, 2026.

[185] Yi Li, Yuquan Deng, Jesse Zhang, Joel Jang, Marius Memmel, Raymond Yu, Caelan Reed Garrett, Fabio Ramos, Dieter Fox, Anqi Li, et al. HAMSTER: Hierarchical Action Models For Open-World Robot Manipulation. *arXiv preprint arXiv:2502.05485*, 2025.

[186] Yibo Li, Enshen Zhou, Rui Chen, Yanjun Ding, Mengzhen Liu, Yi Han, Jiabo Zhan, Lipeng Wang, Shanghang Zhang, and Lu Sheng. ActiveArena: Benchmarking and Understanding Active Perception in Robotic Manipulation. *arXiv preprint arXiv:2609.24124*, 2026.

[187] Yifan Li, Jiaxu Wang, Dongming Wu, Yicheng Jiang, Ryan Ji, Xiangyu Yue, and Yanwei Fu. Vela: Scaling Vision-Language-Action Models with Adaptive Action Curve Parametrization. *arXiv preprint arXiv:2610.05230*, 2026.

[188] Yijian Li, Changze Li, Han Zheng, Jiyuan Cai, Tong Qin, and Ming Yang. AgenticNav: Zero-Shot Vision-and-Language Navigation as a Tool-Calling Harness. *arXiv preprint arXiv:2606.10577*, 2026.

[189] Yinhao Li, Weixin Mao, Zihan Lan, Jikun Rong, Qirui Hu, Yiming Zhang, Weipeng Deng, Bowen Shen, Minzhao Zhu, Yiming Mao, Yan Yang, Chenguang Cui, Hongyuan Chen, Xu Huang, Zheyi Zhao, Pinxi Shen, Bozhen He, Zhen Fu, Yifan Wang, Zexin Zhang, Ang Gao, Haoyu Chen, Chengqi Shi, and Hua Chen. FluxVLA Engine: A One-Stop VLA Engineering Platform for Embodied Intelligence. *arXiv preprint arXiv:2609.17210*, 2026.

[190] Yizhan Li, Jianxin You, Mengyang Xiong, Yinhuan Chen, Zicheng Zhao, Dekun Wu, Dongqing Zhang, and Bang Liu. ReactHuman: A Physics-Grounded Benchmark for Human-Like Reactive Decision-Making in Embodied Multimodal LLMs. *arXiv preprint arXiv:2609.10895*, 2026.

[191] Zechu Li, Yufeng Jin, Xiaoyang Liu, Puze Liu, Vignesh Prasad, Carlo D'Eramo, and Georgia Chalvatzaki. HARBOR: A Harness Framework for Agentic Robot Reinforcement Learning. *arXiv preprint arXiv:2606.08610*, 2026.

[192] Zexi Li, Yehang Zhang, Haojian Huang, Bohan Zhou, Wenqian Li, Chenxu Wang, Yifan Chang, Yangkai Wei, Tianyi Zhang, Ying-Cong Chen, Kaiwen Zhou, Yinchuan Li, and James Cheng. RACaP: Agentic Reasoning, Acting, and Coding as Policies for Evolvable Robot Learning. *arXiv preprint arXiv:2609.29394*, 2026.

[193] Zexi Li, Yehang Zhang, Wenqian Li, Haojian Huang, Chenxu Wang, Shiyuan Deng, Yangkai Wei, Tianyi Zhang, Binghui Xie, Bohan Zhou, Yifan Chang, Kaiwen Zhou, Ying-Cong Chen, James Cheng, and Yinchuan Li. Robo-Harness K1: Harnessing Robot-Use Agents via Perception Augmentation. *arXiv preprint arXiv:2609.29389*, 2026.

[194] Zijun Li, Shijie Li, Zhenxi Zhang, Bin Li, and Shoujun Zhou. DV-VLN: Dual Verification for Reliable LLM-Based Vision-and-Language Navigation. *arXiv preprint arXiv:2601.18492*, 2026.

[195] Anthony Liang, Yigit Korkmaz, Jiahui Zhang, Minyoung Hwang, Abrar Anwar, Sidhant Kaushik, Aditya Shah, Alex S. Huang, Luke Zettlemoyer, Dieter Fox, Yu Xiang, Anqi Li, Andreea Bobu, Abhishek Gupta, Stephen Tu, Erdem Biyik, and Jesse Zhang. Robometer: Scaling General-Purpose Robotic Reward Models via Trajectory Comparisons. In *Proceedings of Robotics: Science and Systems*, Sydney, Australia, July 2026. doi: 10.15607/RSS.2026.XXII.140.

[196] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as Policies: Language Model Programs for Embodied Control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500. IEEE, 2023. doi: 10.1109/icra48891.2023.10160591.

[197] Jacky Liang, Fei Xia, Wenhao Yu, Andy Zeng, Montserrat Gonzalez Arenas, Maria Attarian, Maria Bauza, Matthew Bennice, Alex Bewley, Adil Dostmohamed, Chuyuan Kelly Fu, Nimrod Gileadi, Marissa Giustina, Keerthana Gopalakrishnan, Leonard Hasenclever, Jan Humplik, Jasmine Hsu, Nikhil Joshi, Ben Jyenis, Chase Kew, Sean Kirmani, Tsang-Wei Edward Lee, Kuang-Huei Lee, Assaf Hurwitz Michaely, Joss Moore, Ken Oslund, Dushyant Rao, Allen Ren, Baruch Tabanpour, Quan Vuong, Ayzaan Wahid, Ted Xiao, Ying Xu, Vincent Zhuang, Peng Xu, Erik Frey, Ken Caluwaerts, Tingnan Zhang, Brian Ichter, Jonathan Tompson, Leila Takayama, Vincent Vanhoucke, Izhak Shafran, Maja Mataric, Dorsa Sadigh, Nicolas Heess, Kanishka Rao, Nik Stewart, Jie Tan, and Carolina Parada. Learning to Learn Faster from Human Feedback with Language Model Predictive Control. *arXiv preprint arXiv:2402.11450*, 2024.

[198] Kaiqu Liang, Zixu Zhang, and Jaime Fernández Fisac. Introspective Planning: Aligning Robots' Uncertainty with Inherent Task Ambiguity. *arXiv preprint arXiv:2402.06529*, 2024.

[199] William Liang, Sam Wang, Hung-Ju Wang, Osbert Bastani, Yecheng Jason Ma, and Dinesh Jayaraman. Tether: Autonomous Functional Play with Correspondence-Driven Trajectory Warping. In *International Conference on Learning Representations*, 2026.

[200] Shuhao Liao, Xuxin Lv, Jeric Lew, Shizhe Zhang, Jingsong Liang, Peizhuo Li, Yuhong Cao, Wenjun Wu, and Guillaume Sartoretti. FARE: Fast-Slow Agentic Robotic Exploration. *arXiv preprint arXiv:2601.14681*, 2026.

[201] Yue Liao, Pengfei Zhou, Siyuan Huang, Donglin Yang, Shengcong Chen, Yuxin Jiang, Yue Hu, Jingbin Cai, Si Liu, Jianlan Luo, Liliang Chen, Shuicheng Yan, Maoqing Yao, and Guanghui Ren. Genie Envisioner: A Unified World Foundation Platform for Robotic Manipulation. *arXiv preprint arXiv:2508.05635*, 2025.

[202] Oscar Lima, Marc Vinci, Martin Günther, Marian Renz, Alexander Sung, Sebastian Stock, Johannes Brust, Lennart Niecksch, Zongyao Yi, Felix Igelbrink, Benjamin Kisliuk, Martin Atzmueller, and Joachim Hertzberg. Agentic AI for Robot Control: Flexible but still Fragile. *Proceedings of the AAAI Symposium Series*, 8(1): 465–473, 2026. doi: 10.1609/aaaiss.v8i1.42578.

[203] Fanqi Lin, Ruiqian Nai, Yingdong Hu, Jiacheng You, Junming Zhao, and Yang Gao. OneTwoVLA: A Unified Vision-Language-Action Model with Adaptive Reasoning. *arXiv preprint arXiv:2505.11917*, 2025.

[204] Liquid AI. Introducing d1: The most capable decision model, now with vision, 2026. URL https://www.liquid.ai/blog/d1-decision-model.

[205] An Liu, Bingxi Liu, Hongyu Ding, Yixuan Jiang, Yaran Chen, Fulin Tang, Cong Leng, Hong Zhang, and Jian Cheng. HAM-VLN: Harnessing Hierarchical Agentic Memory for Zero-Shot Vision-and-Language Navigation. *arXiv preprint arXiv:2607.29600*, 2026.

[206] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 44776–44791. Curran Associates, Inc., 2023. doi: 10.52202/075280-1939.

[207] Fangchen Liu, Kuan Fang, Pieter Abbeel, and Sergey Levine. MOKA: Open-World Robotic Manipulation through Mark-Based Visual Prompting. *arXiv preprint arXiv:2403.03174*, 2024.

[208] Haowen Liu, Xirui Li, Shaoxiong Yao, Peng Shi, Tianyi Zhou, Jia-Bin Huang, Furong Huang, and Jiayuan Mao. Guava: Distilling Frontier VLMs into a Compact Agent through a Robotic Manipulation Harness. *arXiv preprint arXiv:2606.18363*, 2026.

[209] Huihan Liu, Alice Chen, Yuke Zhu, Adith Swaminathan, Andrey Kolobov, and Ching-An Cheng. Interactive Robot Learning from Verbal Correction. *arXiv preprint arXiv:2310.17555*, 2023.

[210] Jingyang Liu, Sujia Yao, Jiayuan Gu, and Lan Xu. NavProbe: Evidence-Grounded Reasoning with Active Memory Retrieval for Zero-Shot Navigation. *arXiv preprint arXiv:2609.27526*, 2026.

[211] Jinkun Liu, Haohan Chi, Lingfeng Zhang, Yifan Xie, YuAn Wang, Long Chen, Hangjun Ye, Xiaoshuai Hao, and Wenbo Ding. Thinking in Text and Images: Interleaved Vision–Language Reasoning Traces for Long-Horizon Robot Manipulation. *arXiv preprint arXiv:2605.00438*, 2026.

[212] Lin Liu, Zhicheng Bao, Lu Zhang, Ziying Song, Wu Yang, Yuzheng Zhuang, Shuai Tao, Wulong Liu, Caiyan Jia, and Huchuan Lu. LIBERO-RECOVER: Beyond Task Success Towards Failure Recovery in Robotic Manipulation Models. *arXiv preprint arXiv:2609.05178*, 2026.

[213] Litao Liu, Yifan Han, Pengfei Yi, Wenbo Yu, Hanqing Wang, Haoran Du, Enze Yuan, Zilin Yuan, Ruiding Feng, Michael Liu, Qi Zhang, and Jingjin Yu. Affordance2Action: Task-Conditioned Scene-level Affordance Grounding for Real-Time Manipulation. *arXiv preprint arXiv:2606.04172*, 2026.

[214] Liu Liu, Xiaofeng Wang, Guosheng Zhao, Keyu Li, Wenkang Qin, Jiagang Zhu, Jiaxiong Qiu, Zheng Zhu, Guan Huang, and Zhizhong Su. RoboTransfer: Controllable Geometry-Consistent Video Diffusion for Manipulation Policy Transfer. *arXiv preprint arXiv:2505.23171*, 2025.

[215] Peiqi Liu, Yaswanth Orru, Jay Vakil, Chris Paxton, Nur Muhammad Mahi Shafiullah, and Lerrel Pinto. OK-Robot: What Really Matters in Integrating Open-Knowledge Models for Robotics. *arXiv preprint arXiv:2401.12202*, 2024.

[216] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection. *arXiv preprint arXiv:2303.05499*, 2023.

[217] Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. RDT-1B: a Diffusion Foundation Model for Bimanual Manipulation. *arXiv preprint arXiv:2410.07864*, 2024.

[218] Wenrui Liu, Yaxuan Wang, Xun Zhang, Yanshu Wang, Jiashen Wei, Yifan Xiang, Yuhang Wang, Mingshen Ye, Elsie Dai, Zhiqi Liu, Yingjie Xu, Xinyang Chen, Hengzhe Sun, Jiyu Shen, Jingjing He, and Tong Yang. AgentRob: From Virtual Forum Agents to Hijacked Physical Robots. *arXiv preprint arXiv:2602.13591*, 2026.

[219] Xiang Liu, Sen Cui, Guocai Yao, Zhong Cao, Jingheng Ma, Min Zhang, and Changshui Zhang. Scene2Demo: Self-Evolving Embodied Data Generation via Object-Action Graph. *arXiv preprint arXiv:2602.12065*, 2026.

[220] Xinyuan Liu, Eren Sadikoglu, Riana Chatterjee, and Ransalu Senanayake. Physical Agentic AI: An Architecture for Orchestrating a Robot Crew with LLMs. *arXiv preprint arXiv:2608.22657*, 2026.

[221] Yang Liu, Weixing Chen, Xinshuai Song, Tao Pu, Siwen Mo, Yongjie Bai, Zihao Chen, Qianran Sun, Liruo Zhong, Ying Shen, and Liang Lin. PhyAgentOS: A Self-Evolving Operating System for Embodied Agents with Decoupled Cognitive Planning and Physical Execution. *arXiv preprint arXiv:2607.16636*, 2026.

[222] Yuyao Liu, Jiayuan Mao, David Hsu, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. RAPID: Robot Agentic Programming from Demonstrations. *arXiv preprint arXiv:2609.30249*, 2026.

[223] Zeyi Liu, Arpit Bahety, and Shuran Song. REFLECT: Summarizing Robot Experiences for Failure Explanation and Correction. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 3468–3484. PMLR, 2023.

[224] Zeyi Liu, Shuang Li, Eric Cousineau, Siyuan Feng, Benjamin Burchfiel, and Shuran Song. Geometry-aware 4D Video Generation for Robot Manipulation. In *International Conference on Learning Representations*, 2026.

[225] Zhen Liu, Xinyu Ning, Zhe Hu, Xinxin Xie, Weize Li, Zhipeng Tang, Chongyu Wang, Zejun Yang, Hanlin Wang, Yitong Liu, and Zhongzhu Pu. Goal2Skill: Long-Horizon Manipulation with Adaptive Planning and Reflection. *arXiv preprint arXiv:2604.13942*, 2026.

[226] Jiabin Lou, Yirong Yang, Haopeng Wang, Xuxin Lv, Xinyu Liu, Diyuan Hou, Xuehong Liu, Rongye Shi, and Wenjun Wu. AeroWeaver: An Embodied-Agent Harness for Weaving Aerial Skills into Distributed, Adaptive Swarm Execution. *arXiv preprint arXiv:2609.18520*, 2026.

[227] Haoran Lu, Mutian Shen, Shuyang Yu, Yu Xiao, Songling Liu, Jianshu Zhang, Shang Wu, Yue Chen, Guo Ye, Jiayi Wang, Zhaoran Wang, and Han Liu. AnnotateAnything: Automatic Annotation of 3D Assets for Robot Manipulation. *arXiv preprint arXiv:2606.17446*, 2026.

[228] Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, Ken Goldberg, Ang Chen, Mosharaf Chowdhury, Yuke Zhu, Linxi "Jim" Fan, and Guanzhi Wang. ASPIRE: Agentic /Skills Discovery for Robotics. *arXiv preprint arXiv:2607.00272*, 2026.

[229] Yupu Lu, Hang Xu, Yizhou Chen, and Jia Pan. Keypose Exploration: Efficient Automatic Trajectory Labelling and Cross-Embodiment Policy Transfer. *arXiv preprint arXiv:2606.29028*, 2026.

[230] Hao Luo, Wanpeng Zhang, Yicheng Feng, Sipeng Zheng, Haiweng Xu, Chaoyi Xu, Ziheng Xi, Yuhui Fu, and Zongqing Lu. Being-H0.7: A Latent World-Action Model from Egocentric Videos. *arXiv preprint arXiv:2605.00078*, 2026.

[231] Qian Luo, Wentao Guo, Zhennan Qin, Nanchun Guo, Yunhan Zhao, Yi Ma, and Yanchao Yang. RoboLineage: Agent-Native Data Lifecycle Governance Across Robot Policy Iterations. *arXiv preprint arXiv:2606.22142*, 2026.

[232] Guanghui Ma, Jiahui Guo, Xinhua Tang, Huaidong Zhou, and Yongfeng Rong. Adaptive Task Planning for Long-Horizon Robotic Manipulation Based on Video Priors and Dynamic Scene Graphs. *Sensors*, 26(17): 5595, 2026. doi: 10.3390/s26175595.

[233] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-Level Reward Design via Coding Large Language Models. In *International Conference on Learning Representations*, volume 2024, pages 26516–26560, 2024.

[234] Yunchao Ma, Yizhuang Zhou, Yunhuan Yang, Tiancai Wang, and Haoqiang Fan. Running VLAs at Real-time Speed. *arXiv preprint arXiv:2510.26742*, 2025.

[235] Kazi Abrar Mahmud, Nilotpaul Kundu Dhurubo, Tamal Kirttonia, Sabbir Hossain Ujjal, and Mohammad Ariful Haque. Bridging Thought and Action: Taming Long-Horizon Instability in Open-Source LLM Agents with a MetaTool-Enhanced ROS Framework. *arXiv preprint arXiv:2609.13335*, 2026.

[236] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and Gavriel State. Isaac Gym: High Performance GPU-Based Physics Simulation For Robot Learning. *arXiv preprint arXiv:2108.10470*, 2021.

[237] Zhao Mandi, Shreeya Jain, and Shuran Song. RoCo: Dialectic Multi-Robot Collaboration with Large Language Models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024. doi: 10.1109/ICRA57147.2024.10610855.

[238] Ajay Mandlekar, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Li Fei-Fei, Silvio Savarese, Yuke Zhu, and Roberto Martín-Martín. What Matters in Learning from Offline Human Demonstrations for Robot Manipulation. *arXiv preprint arXiv:2108.03298*, 2021.

[239] Mingyu Mei, Haojie Xu, Shihao Jin, Zibo Dai, Qihao Cheng, Zhengrui Lv, Hongjie Fang, Shirun Tang, Guang Chen, Xinyue Zhao, Huiliang Shen, and Zaixing He. HINT: Human-Intent Inception for Long-Horizon Robot Manipulation. *arXiv preprint arXiv:2609.02653*, 2026.

[240] Yuan Meng, Zhenguo Sun, Max Fest, Xiangtong Yao, Xukun Li, Zhenshan Bing, and Alois Knoll. Robotic Ultra-Long-Horizon Manipulation Skills via Human-guided Lifelong Code Generation. *arXiv preprint arXiv:2509.18597*, 2025.

[241] Boyu Mi, Mengchen Ma, Yifei Yao, Xing Gao, Junting Chen, Yangzi Li, Zihou Zhu, Guohao Li, Zhenfei Yin, Tai Wang, Yao Mu, Jiangmiao Pang, and Hanqing Wang. Exploratory, Communicative, and Deployable: Vision-Driven Embodied Agents for Open-World Mobile Manipulation. In *Computer Vision – ECCV 2026*, volume 17042 of *Lecture Notes in Computer Science*, pages 627–645. Springer Nature Switzerland, 2026. doi: 10.1007/978-3-032-37422-6_36.

[242] Yao Mu, Junting Chen, Qinglong Zhang, Shoufa Chen, Qiaojun Yu, Chongjian Ge, Runjian Chen, Zhixuan Liang, Mengkang Hu, Chaofan Tao, et al. RoboCodeX: Multimodal Code Generation for Robotic Behavior Synthesis. *arXiv preprint arXiv:2402.16117*, 2024.

[243] Adithyavairavan Murali, Balakumar Sundaralingam, Yu-Wei Chao, Wentao Yuan, Jun Yamada, Mark Carlson, Fabio Ramos, Stan Birchfield, Dieter Fox, and Clemens Eppner. GraspGen: A Diffusion-based Framework for 6-DOF Grasping with On-Generator Training. *arXiv preprint arXiv:2507.13097*, 2025.

[244] Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3M: A Universal Visual Representation for Robot Manipulation. *arXiv preprint arXiv:2203.12601*, 2022.

[245] Gokul Narayanan, Yash Shahapurkar, Melih Erdogan, Brian Zhu, and Eugen Solowjow. Closing the Loop in Teleoperation: Episode-Level Data Quality Assessment and Feedback for High-Quality Demonstration Collection. *arXiv preprint arXiv:2605.26349*, 2026.

[246] Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, and Yuke Zhu. RoboCasa365: A Large-Scale Simulation Framework for Training and Benchmarking Generalist Robots. *arXiv preprint arXiv:2603.04356*, 2026.

[247] Suman Navaratnarajah, Taehyoung Kim, Jona Ruthardt, Ishaan Bhimwal, Ryousuke Yamada, Yannik Blei, Wolfram Burgard, and Yuki M Asano. Zero-Shot Mission-Level Evaluation for Aerial MLLM Agents. *arXiv preprint arXiv:2607.22014*, 2026.

[248] Khanh D. Nguyen, Hung T. Ho, Chinh T. Nguyen, Thanh Q. Duong, Linh D. Le, Duy M. H. Nguyen, Vien A. Ngo, and An T. Le. vla.cpp: A Unified Inference Runtime for Vision-Language-Action Models. *arXiv preprint arXiv:2606.08094*, 2026.

[249] Chang Nie, Zhe Liu, and Hesheng Wang. Teach and Grow: An Agent-Centered Architecture for General Robot Learning. *arXiv preprint arXiv:2608.17209*, 2026.

[250] Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, Mengting Ai, Zhining Liu, Ting-Wei Li, Lingjie Chen, Yanjun Zhao, Ke Yang, Bingxuan Li, Cheng Qian, Gaotang Li, Xiao Lin, Zhichen Zeng, Ruizhong Qiu, Sirui Chen, Yifan Sun, Xiyuan Yang, Ruida Wang, Rui Pan, Chenyuan Yang, Dylan Zhang, Liri Fang, Zikun Cui, Yang Cao, Pan Chen, Dorothy Sun, Ren Chen, Mahesh Srinivasan, Nipun Mathur, Yinglong Xia, Hong Li, Hong Yan, Pan Lu, Lingming Zhang, Tong Zhang, Hanghang Tong, and Jingrui He. Code as Agent Harness. *arXiv preprint arXiv:2605.18747*, 2026.

[251] NVIDIA. Cosmos World Foundation Model Platform for Physical AI. *arXiv preprint arXiv:2501.03575*, 2025.

[252] NVIDIA. World Simulation with Video Foundation Models for Physical AI. *arXiv preprint arXiv:2511.00062*, 2025.

[253] NVIDIA. Cosmos 3: Omnimodal World Models for Physical AI. *arXiv preprint arXiv:2606.02800*, 2026.

[254] NVIDIA et al. GR00T N1: An Open Foundation Model for Generalist Humanoid Robots. *arXiv preprint arXiv:2503.14734*, 2025.

[255] Ike Obi, Vishnunandan L. N. Venkatesh, Weizheng Wang, Ruiqi Wang, Dayoon Suh, Temitope I. Amosa, Wonse Jo, and Byung-Cheol Min. Pre-Execution Safety Gate & Task Safety Contracts for LLM-Controlled Robot Systems. *arXiv preprint arXiv:2604.05427*, 2026.

[256] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An Open-Source Generalist Robot Policy. *arXiv preprint arXiv:2405.12213*, 2024.

[257] Austine Oloo, Zainab Altaweel, Yohei Hayamizu, Peiqi Liu, Yan Ding, Saeid Amiri, Hao Yang, Andy Kaminski, Chad Esselink, Chris Paxton, Xiaohan Zhang, and Shiqi Zhang. Robot Planning and Situation Handling with Active Perception. *arXiv preprint arXiv:2604.26988*, 2026.

[258] OpenAI. GPT-6 Astra: The Next Generation in Intelligence for Work, 2026. URL https://openai.com/index/gpt-6-astra-next-generation-work/. Accessed October 14, 2026.

[259] OpenAI. Decisions, 2026. URL https://developers.openai.com/api/docs/guides/decisions.

[260] Abhishek Padalkar, Acorn Pooley, Ajinkya Jain, Alex Bewley, Alex Herzog, Alex Irpan, Alexander Khazatsky, Anant Rai, Anikait Singh, Anthony Brohan, Antonin Raffin, Ayzaan Wahid, Ben Burgess-Limerick, Beomjoon Kim, Bernhard Schölkopf, Brian Ichter, Cewu Lu, Charles Xu, Chelsea Finn, Chenfeng Xu, Cheng Chi, Chenguang Huang, Christine Chan, Chuer Pan, Chuyuan Fu, Coline Devin, Danny Driess, Deepak Pathak, Dhruv Shah, Dieter Büchler, Dmitry Kalashnikov, Dorsa Sadigh, Edward Johns, Federico Ceola, Fei Xia, Freek Stulp, Gaoyue Zhou, Gaurav S. Sukhatme, Gautam Salhotra, Ge Yan, Giulio Schiavi, Gregory Kahn, Hao Su, Hao-Shu Fang, Haochen Shi, Heni Ben Amor, Henrik I Christensen, Hiroki Furuta, Homer Walke, Hongjie Fang, Igor Mordatch, Ilija Radosavovic, Isabel Leal, Jacky Liang, Jad Abou-Chakra, Jaehyung Kim, Jan Peters, Jan Schneider, Jasmine Hsu, Jeannette Bohg, Jeffrey Bingham, Jiajun Wu, Jialin Wu, Jianlan Luo, Jiayuan Gu, Jie Tan, Jihoon Oh, Jitendra Malik, Jonathan Tompson, Jonathan Yang, Joseph J. Lim, João Silvério, Junhyek Han, Kanishka Rao, Karl Pertsch, Karol Hausman, Keegan Go, Keerthana Gopalakrishnan, Ken Goldberg, Kendra Byrne, Kenneth Oslund, Kento Kawaharazuka, Kevin Zhang, Krishan Rana, Krishnan Srinivasan, Lawrence Yunliang Chen, Lerrel Pinto, Liam Tan, Lionel Ott, Lisa Lee, Masayoshi Tomizuka, Maximilian Du, Michael Ahn, Mingtong Zhang, Mingyu Ding, Mohan Kumar Srirama, Mohit Sharma, Moo Jin Kim, Naoaki Kanazawa, Nicklas Hansen, Nicolas Heess, Nikhil J Joshi, Niko Suenderhauf, Norman Di Palo, Nur Muhammad Mahi Shafiullah, Oier Mees, Oliver Kroemer, Pannag R Sanketi, Paul Wohlhart, Peng Xu, Pierre Sermanet, Priya Sundaresan, Quan Vuong, Rafael Rafailov, Ran Tian, Ria Doshi, Roberto Martín-Martín, Russell Mendonca, Rutav Shah, Ryan Hoque, Ryan Julian, Samuel Bustamante, Sean Kirmani, Sergey Levine, Sherry Moore, Shikhar Bahl, Shivin Dass, Shubham Sonawani, Shuran Song, Sichun Xu, Siddhant Haldar, Simeon Adebola, Simon Guist, Soroush Nasiriany, Stefan Schaal, Stefan Welker, Stephen Tian, Sudeep Dasari, Suneel Belkhale, Takayuki Osa, Tatsuya Harada, Tatsuya Matsushima, Ted Xiao, Tianhe Yu, Tianli Ding, Todor Davchev, Tony Z. Zhao, Travis Armstrong, Trevor Darrell, Vidhi Jain, Vincent Vanhoucke, Wei Zhan, Wenxuan Zhou, Wolfram Burgard, Xi Chen, Xiaolong Wang, Xinghao Zhu, Xuanlin Li, Yao Lu, Yevgen Chebotar, Yifan Zhou, Yifeng Zhu, Ying Xu, Yixuan Wang, Yonatan Bisk, Yoonyoung Cho, Youngwoon Lee, Yuchen Cui, Yueh-Hua Wu, Yujin Tang, Yuke Zhu, Yunzhu Li, Yusuke Iwasawa, Yutaka Matsuo, Zhuo Xu, and Zichen Jeff Cui. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. *arXiv preprint arXiv:2310.08864*, 2023.

[261] Esteban Padilla-Cerdio, Boyang Sun, Marc Pollefeys, and Hermann Blum. OpenFrontier: General Navigation with Visual-Language Grounded Frontiers. In *Proceedings of Robotics: Science and Systems*, 2026. doi: 10.15607/RSS.2026.XXII.067.

[262] Jiahua Pei, Yi Liu, Guoping Pan, Yuanhao Jiang, Houde Liu, and Xueqian Wang. OVAL: Open-Vocabulary Augmented Memory Model for Lifelong Object Goal Navigation. *arXiv preprint arXiv:2604.12872*, 2026.

[263] Jiaqi Peng, Xiqian Yu, Delin Feng, Yuqiang Yang, Wenzhe Cai, Jing Xiong, Ganlin Yang, Jinliang Zheng, Jiafei Cao, Xueyuan Wei, Jiangmiao Pang, Yuan Shen, and Tai Wang. Cortex: A Bidirectionally Aligned Embodied Agent Framework for Long-horizon Manipulation. *arXiv preprint arXiv:2607.05377*, 2026.

[264] Yiyan Peng, Philip Wang, Simon Sinong Zhan, Yiqi Lyu, Zhenyang Ni, Jixin Yan, Fiorelli Wong, Ruochen Jiao, Hang Yin, Xinyu Cao, Huajie Shao, Manling Li, Ruohan Zhang, and Qi Zhu. ManiGuard: A Benchmark and Data Suite for Specification-Grounded Safety Evaluation and Improvement of Robotic Manipulation. *arXiv preprint arXiv:2608.17386*, 2026.

[265] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. FAST: Efficient Action Tokenization for Vision-Language-Action Models. *arXiv preprint arXiv:2501.09747*, 2025.

[266] Nicholas Pfaff, Thomas Cohn, Sergey Zakharov, Rick Cory, and Russ Tedrake. SceneSmith: Agentic Generation of Simulation-Ready Indoor Scenes. *arXiv preprint arXiv:2602.09153*, 2026.

[267] Physical Intelligence. openpi: Remote Inference, 2025. URL https://github.com/Physical-Intelligence/openpi/blob/main/docs/remote_inference.md.

[268] Physical Intelligence, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Jared DiCarlo, Danny Driess, Michael Equi, Adnan Esmail, Yunhao Fang, Chelsea Finn, Catherine Glossop, Thomas Godden, Ivan Goryachev, Lachy Groom, Hunter Hancock, Karol Hausman, Gashon Hussein, Brian Ichter, Szymon Jakubczak, Rowan Jen, Tim Jones, Ben Katz, Liyiming Ke, Chandra Kuchi, Marinda Lamb, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Yao Lu, Vishnu Mano, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Charvi Sharma, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, Will Stoeckle, Alex Swerdlow, James Tanner, Marcel Torne, Quan Vuong, Anna Walling, Haohuan Wang, Blake Williams, Sukwon Yoo, Lili Yu, Ury Zhilinsky, and Zhiyuan Zhou. $\pi^*_{0.6}$: A VLA That Learns From Experience. *arXiv preprint arXiv:2511.14759*, 2025.

[269] Physical Intelligence et al. $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[270] Albert Gassol Puigjaner and Kostas Alexis. Hierarchical Floorplan-Guided Vision-Language Exploration for Embodied Question Answering. *arXiv preprint arXiv:2609.26360*, 2026.

[271] Zekun Qi, Wenyao Zhang, Yufei Ding, Runpei Dong, Xinqiang Yu, Jingwen Li, Lingyun Xu, Baoyu Li, Xialin He, Guofan Fan, Jiazhao Zhang, Jiawei He, Jiayuan Gu, Xin Jin, Kaisheng Ma, Zhizheng Zhang, He Wang, and Li Yi. SoFar: Language-Grounded Orientation Bridges Spatial Reasoning and Object Manipulation. In *Advances in Neural Information Processing Systems*, volume 38, pages 76367–76412. Curran Associates, Inc., 2025. doi: 10.52202/085713-2565.

[272] Yikai Qin, Yifei Deng, Mingjian Liang, Wenxuan Song, Zepeng Lin, Zhiyi Jiang, Jiajun Fu, Qiao Sun, Huashuo Lei, Xicheng Gong, Jiayi Chen, Han Zhao, Shuanghao Bai, Pengxiang Ding, Pengwei Wang, and Haoang Li. EmbodiedSmith: Scaling Embodied Data through Recursive Self-Improvement Flywheel in Simulation. *arXiv preprint arXiv:2610.07969*, 2026.

[273] Yiran Qin, Li Kang, Xiufeng Song, Zhenfei Yin, Xiaohong Liu, Xihui Liu, Ruimao Zhang, and Lei Bai. RoboFactory: Exploring Embodied Agent Collaboration with Compositional Constraints. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 10075–10085, 2025.

[274] Krishan Rana, Jesse Haviland, Sourav Garg, Jad Abou-Chakra, Ian Reid, and Niko Suenderhauf. SayPlan: Grounding Large Language Models using 3D Scene Graphs for Scalable Robot Task Planning. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 23–72. PMLR, 2023.

[275] Nadun Ranawaka, Josiah Wong, Wei-Lin Pai, Wei-Teng Chu, Tianyuan Dai, Masoud Moghani, Hang Yin, Yunfan Jiang, Wesley Durbano, Brandon Huynh, Yu Fang, Danfei Xu, Ruohan Zhang, Li Fei-Fei, Linxi Fan, Bowen Wen, Ajay Mandlekar, and Yuke Zhu. SimFoundry: Modular and Automated Scene Generation for Policy Learning and Evaluation. *arXiv preprint arXiv:2606.28276*, 2026.

[276] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, et al. SAM 2: Segment Anything in Images and Videos. *arXiv preprint arXiv:2408.00714*, 2024.

[277] Zachary Ravichandran, Alexander Robey, Vijay Kumar, George J. Pappas, and Hamed Hassani. Safety Guardrails for LLM-Enabled Robots. *arXiv preprint arXiv:2503.07885*, 2025.

[278] Zachary Ravichandran, David Snyder, Alexander Robey, Hamed Hassani, Vijay Kumar, and George J. Pappas. Contextual Safety Reasoning and Grounding for Open-World Robots. *arXiv preprint arXiv:2602.19983*, 2026.

[279] Allen Z. Ren, Anushri Dixit, Alexandra Bodrova, Sumeet Singh, Stephen Tu, Noah Brown, Peng Xu, Leila Takayama, Fei Xia, Jake Varley, Zhenjia Xu, Dorsa Sadigh, Andy Zeng, and Anirudha Majumdar. Robots That Ask For Help: Uncertainty Alignment for Large Language Model Planners. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 661–682. PMLR, 2023.

[280] Baochang Ren, Xinjie Liu, Xi Chen, Yanshuo Liu, Chenxi Li, Daqi Gao, Zeqin Su, Jintao Xing, Zirui Xue, Rui Li, Xiangyu Zhao, Shuofei Qiao, Minting Pan, Wangmeng Zuo, Lei Bai, Dongzhan Zhou, Ningyu Zhang, and Huajun Chen. LabVLA: Grounding Vision-Language-Action Models in Scientific Laboratories. *arXiv preprint arXiv:2606.13578*, 2026.

[281] Tianhe Ren, Shilong Liu, Ailing Zeng, Jing Lin, Kunchang Li, He Cao, Jiayu Chen, Xinyu Huang, Yukang Chen, Feng Yan, Zhaoyang Zeng, Hao Zhang, Feng Li, Jie Yang, Hongyang Li, Qing Jiang, and Lei Zhang. Grounded SAM: Assembling Open-World Models for Diverse Visual Tasks. *arXiv preprint arXiv:2401.14159*, 2024.

[282] Paolo Riva, Leonardo Gargani, Matteo Frosi, and Matteo Matteucci. EmbodiedLGR: Integrating Lightweight Graph Representation and Retrieval for Semantic-Spatial Memory in Robotic Agents. *arXiv preprint arXiv:2604.18271*, 2026.

[283] Alexander Robey, Zachary Ravichandran, Vijay Kumar, Hamed Hassani, and George J. Pappas. Jailbreaking LLM-Controlled Robots. *arXiv preprint arXiv:2410.13691*, 2024.

[284] robotmcp. ROS MCP Server, 2025. URL https://github.com/robotmcp/ros-mcp-server.

[285] Ke Rui, Yushen Zuo, Jiawei Wang, Haoran Jia, Jinming Ma, Weitao Zhou, and Minglei Li. Diagnosing Semantic Handoff Failures in Agent-Orchestrated Vision-Language-Action Skill Composition. *arXiv preprint arXiv:2607.06256*, 2026.

[286] Kanghyun Ryu, Qiayuan Liao, Zhongyu Li, Payam Delgosha, Koushil Sreenath, and Negar Mehr. CurricuLLM: Automatic Task Curricula Design for Learning Complex Robot Skills using Large Language Models. *arXiv preprint arXiv:2409.18382*, 2024.

[287] Sahar Salimpour, Lei Fu, Kajetan Rachwał, Pascal Bertrand, Kevin O'Sullivan, Robert Jakob, Farhad Keramat, Leonardo Militano, Giovanni Toffetti, Harry Edelman, and Jorge Peña Queralta. Towards Embodied Agentic AI: Review and Classification of LLM- and VLM-Driven Robot Autonomy and Interaction. *arXiv preprint arXiv:2508.05294*, 2025.

[288] Philip Schroeder, Ondrej Biza, Thomas Weng, Hongyin Luo, and James Glass. ROVER: Recursive Reasoning Over Videos with Vision-Language Models for Embodied Tasks. In *Advances in Neural Information Processing Systems*, 2025.

[289] Seyed Kamyar Seyed Ghasemipour, Ayzaan Wahid, Jonathan Tompson, Pannag Sanketi, and Igor Mordatch. Self-Improving Embodied Foundation Models. In *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 112651–112686. Curran Associates, Inc., 2025. doi: 10.52202/085713-3762.

[290] Shinas Shaji, Fabian Huppertz, Alex Mitrevski, and Sebastian Houben. From Language to Action: Can LLM-Based Agents Be Used for Embodied Robot Cognition? *arXiv preprint arXiv:2603.03148*, 2026.

[291] Rui Shao, Wei Li, Lingsen Zhang, Renshan Zhang, Zhiyang Liu, Ran Chen, and Liqiang Nie. Large VLM-based Vision-Language-Action Models for Robotic Manipulation: A Survey. *arXiv preprint arXiv:2508.13073*, 2025.

[292] Ruihong Shen, Žiga Kovačič, Peter Kulits, Xingrui Wang, Zizhang Li, Joshua B. Tenenbaum, Alan Yuille, Jieneng Chen, and Jiajun Wu. 4DCodeBench: Benchmarking Agents on Inverse Graphics of Dynamic Scenes. *arXiv preprint arXiv:2610.03715*, 2026.

[293] Shuaike Shen, Wenduo Cheng, Mingqian Ma, Alistair Turcan, Martin Jinye Zhang, and Jian Ma. SkillFoundry: Building Self-Evolving Agent Skill Libraries from Heterogeneous Scientific Resources. *arXiv preprint arXiv:2604.03964*, 2026.

[294] Zeyu Shen, Haoxiang You, Yilang Liu, Zhicheng Zheng, Lihan Zha, Kashu Yamazaki, Mingtong Zhang, Suning Huang, Jiankai Sun, Qianzhong Chen, Lucy He, Kaiyuan Liu, Haoran Chang, Katerina Fragkiadaki, Dhruv Shah, Mac Schwager, Peter Henderson, Ian Abraham, and Canwen Xu. EmbodiedSWE: Coding Agents for Long Horizon Dexterous Robotics. *arXiv preprint arXiv:2609.27308*, 2026.

[295] Hao Shi, Bin Xie, Yingfei Liu, Lin Sun, Fengrong Liu, Tiancai Wang, Erjin Zhou, Haoqiang Fan, Xiangyu Zhang, and Gao Huang. MemoryVLA: Perceptual-Cognitive Memory in Vision-Language-Action Models for Robotic Manipulation. In *International Conference on Learning Representations*, 2026.

[296] Lucy Xiaoyang Shi, Zheyuan Hu, Tony Z. Zhao, Archit Sharma, Karl Pertsch, Jianlan Luo, Sergey Levine, and Chelsea Finn. Yell At Your Robot: Improving On-the-Fly from Language Corrections. *arXiv preprint arXiv:2403.12910*, 2024.

[297] Lucy Xiaoyang Shi, Brian Ichter, Michael Robert Equi, Liyiming Ke, Karl Pertsch, Quan Vuong, James Tanner, Anna Walling, Haohuan Wang, Niccolo Fusai, Adrian Li-Bell, Danny Driess, Lachy Groom, Sergey Levine, and Chelsea Finn. Hi Robot: Open-Ended Instruction Following with Hierarchical Vision-Language-Action Models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 54919–54933. PMLR, 2025.

[298] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Perceiver-Actor: A Multi-Task Transformer for Robotic Manipulation. *arXiv preprint arXiv:2209.05451*, 2022.

[299] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics. *arXiv preprint arXiv:2506.01844*, 2025.

[300] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. ProgPrompt: Generating Situated Robot Task Plans using Large Language Models. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11523–11530. IEEE, 2023. doi: 10.1109/icra48891.2023.10161317.

[301] Chan Hee Song, Valts Blukis, Jonathan Tremblay, Stephen Tyree, Yu Su, and Stan Birchfield. RoboSpatial: Teaching Spatial Understanding to 2D and 3D Vision-Language Models for Robotics. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15768–15780, 2025.

[302] Chang Song, Bin Qian, Yan Feng, and Zhijie Song. VLA-Corrector: Stage-Aware Observable State Understanding for Prompt-Based Closed-Loop Recovery of Vision-Language-Action Policies. *arXiv preprint arXiv:2609.06508*, 2026.

[303] Wenxuan Song, Jiayi Chen, Jingbo Wang, Shuai Zhou, Xicheng Gong, Zehua Fan, Ziyang Zhou, Junwu E, Haodong Yan, Fuhao Li, Qize Yu, Xu Huang, Pengwei Wang, Wen Chen, Shunbo Zhou, and Haoang Li. UniWAM: Unified World-Action Model. *arXiv preprint arXiv:2610.02054*, 2026.

[304] Keith E. Stanovich and Richard F. West. Individual Differences in Reasoning: Implications for the Rationality Debate? *Behavioral and Brain Sciences*, 23(5):645–665, 2000.

[305] StarVLA Community. StarVLA: A lego-like codebase for vision-language-action model developing. *arXiv preprint arXiv:2604.05014*, 2026.

[306] Xiangyu Su, Juzhan Xu, Oliver van Kaick, Kai Xu, and Ruizhen Hu. IMR-LLM: Industrial Multi-Robot Task Planning and Program Generation using Large Language Models. *arXiv preprint arXiv:2603.02669*, 2026.

[307] Yue Su, Sijin Chen, Haixin Shi, Mingyu Liu, Zhengshen Zhang, Ningyuan Huang, Weiheng Zhong, Zhengbang Zhu, Yuxiao Liu, and Xihui Liu. World Guidance: World Modeling in Condition Space for Action Generation. *arXiv preprint arXiv:2602.22010*, 2026.

[308] Haiyang Sun, Haoxiao Wang, Junming Chen, Weicheng Fang, Zihao Su, Jingkun Yi, Wenyou Yi, Hao Chen, and Zhou Zhao. MEMOBench: A Process Level Memory Benchmark for Robotic Manipulation. *arXiv preprint arXiv:2609.07047*, 2026.

[309] Jingwen Sun, Wenyao Zhang, Zekun Qi, Shaojie Ren, Zezhi Liu, Hanxin Zhu, Guangzhong Sun, Xin Jin, and Zhibo Chen. VLA-JEPA: Enhancing Vision-Language-Action Model with Latent World Model. *arXiv preprint arXiv:2602.10098*, 2026.

[310] Nan Sun, Yuan Zhang, Yongkun Yang, Wentao Zhao, Peiyan Li, Jun Guo, Wenxuan Song, Pengxiang Ding, Runze Suo, Yifei Su, Xin Xiao, Xinghang Li, and Huaping Liu. Revisiting Embodied Chain-of-Thought for Generalizable Robot Manipulation. *arXiv preprint arXiv:2606.03784*, 2026.

[311] Peiwen Sun, Shiqiang Lang, Dongming Wu, Yi Ding, Kaituo Feng, Huadai Liu, Zhen Ye, Rui Liu, Yun-Hui Liu, Jianan Wang, and Xiangyu Yue. SpaceVista: All-Scale Visual Spatial Reasoning from mm to km. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306, pages 116715–116752. PMLR, 2026.

[312] Balakumar Sundaralingam, Siva Kumar Sastry Hari, Adam Fishman, Caelan Garrett, Karl Van Wyk, Valts Blukis, Alexander Millane, Helen Oleynikova, Ankur Handa, Fabio Ramos, et al. cuRobo: Parallelized Collision-Free Minimum-Jerk Robot Motion Generation. *arXiv preprint arXiv:2310.17274*, 2023.

[313] Martin Sundermeyer, Arsalan Mousavian, Rudolph Triebel, and Dieter Fox. Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes. *arXiv preprint arXiv:2103.14127*, 2021.

[314] Alkis Sygkounas, Victor Aregbede, Amy Loutfi, and Andreas Persson. MEMENTO: Memory-Guided Memetic Code-as-Policy Evolution. *arXiv preprint arXiv:2607.22832*, 2026.

[315] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhijian Liu, and Song Han. VLASH: Real-Time VLAs via Future-State-Aware Asynchronous Inference. *arXiv preprint arXiv:2512.01031*, 2025.

[316] Jinzhou Tang, Zijun Zhang, Jing Yang, Yuchen Yan, Kun Zhou, Lingjun Mao, Ruobing Han, Jinglin Cao, Wenpeng Xu, Lukun He, Minghao Fu, Fan Feng, and Biwei Huang. Video2World: Benchmarking Coding Agents for Interactive World Modeling from Embodied Videos. *arXiv preprint arXiv:2610.04432*, 2026.

[317] Tadahiro Taniguchi, Yasushi Hirai, Masahiro Suzuki, Shingo Murata, Takato Horii, and Kazutoshi Tanaka. System 0/1/2/3: Quad-process theory for multi-timescale embodied collective cognitive systems. *arXiv preprint arXiv:2503.06138*, 2025.

[318] Stone Tao, Fanbo Xiang, Arth Shukla, Yuzhe Qin, Xander Hinrichsen, Xiaodi Yuan, Chen Bao, Xinsong Lin, Yulin Liu, Tse kai Chan, Yuan Gao, Xuanlin Li, Tongzhou Mu, Nan Xiao, Arnav Gurha, Viswesh Nagaswamy Rajesh, Yong Woo Choi, Yen-Ru Chen, Zhiao Huang, Roberto Calandra, Rui Chen, Shan Luo, and Hao Su. ManiSkill3: GPU Parallelized Robotics Simulation and Rendering for Generalizable Embodied AI. *arXiv preprint arXiv:2410.00425*, 2024.

[319] Manan Tayal and Akshay Nambi. ShieldVLA: Feasibility-Aware Safety Alignment for Vision-Language-Action Models. *arXiv preprint arXiv:2609.13231*, 2026.

[320] Gemini Robotics Team, Abbas Abdolmaleki, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, Montserrat Gonzalez Arenas, Ashwin Balakrishna, Nathan Batchelor, Alex Bewley, Jeff Bingham, Michael Bloesch, Konstantinos Bousmalis, Philemon Brakel, Anthony Brohan, Thomas Buschmann, Arunkumar Byravan, Serkan Cabi, Ken Caluwaerts, Federico Casarini, Christine Chan, Oscar Chang, London Chappellet-Volpini, Jose Enrique Chen, Xi Chen, Hao-Tien Lewis Chiang, Krzysztof Choromanski, Adrian Collister, David B. D'Ambrosio, Sudeep Dasari, Todor Davchev, Meet Kirankumar Dave, Coline Devin, Norman Di Palo, Tianli Ding, Carl Doersch, Adil Dostmohamed, Yilun Du, Debidatta Dwibedi, Sathish Thoppay Egambaram, Michael Elabd, Tom Erez, Xiaolin Fang, Claudio Fantacci, Cody Fong, Erik Frey, Chuyuan Fu, Ruiqi Gao, Marissa Giustina, Keerthana Gopalakrishnan, Laura Graesser, Oliver Groth, Agrim Gupta, Roland Hafner, Steven Hansen, Leonard Hasenclever, Sam Haves, Nicolas Heess, Brandon Hernaez, Alex Hofer, Jasmine Hsu, Lu Huang, Sandy H. Huang, Atil Iscen, Mithun George Jacob, Deepali Jain, Sally Jesmonth, Abhishek Jindal, Ryan Julian, Dmitry Kalashnikov, M. Emre Karagozler, Stefani Karp, Matija Kecman, J. Chase Kew, Donnie Kim, Frank Kim, Junkyung Kim, Thomas Kipf, Sean Kirmani, Ksenia Konyushkova, Li Yang Ku, Yuheng Kuang, Thomas Lampe, Antoine Laurens, Tuan Anh Le, Isabel Leal, Alex X. Lee, Tsang-Wei Edward Lee, Guy Lever, Jacky Liang, Li-Heng Lin, Fangchen Liu, Shangbang Long, Caden Lu, Sharath Maddineni, Anirudha Majumdar, Kevis-Kokitsi Maninis, Andrew Marmon, Sergio Martinez, Assaf Hurwitz Michaely, Niko Milonopoulos, Joss Moore, Robert Moreno, Michael Neunert, Francesco Nori, Joy Ortiz, Kenneth Oslund, Carolina Parada, Emilio Parisotto, Amaris Paryag, Acorn Pooley, Thomas Power, Alessio Quaglino, Haroon Qureshi, Rajkumar Vasudeva Raju, Helen Ran, Dushyant Rao, Kanishka Rao, Isaac Reid, David Rendleman, Krista Reymann, Miguel Rivas, Francesco Romano, Yulia Rubanova, Peter Pastor Sampedro, Pannag R Sanketi, Dhruv Shah, Mohit Sharma, Kathryn Shea, Mohit Shridhar, Charles Shu, Vikas Sindhwani, Sumeet Singh, Radu Soricut, Rachel Sterneck, Ian Storz, Razvan Surdulescu, Jie Tan, Jonathan Tompson, Saran Tunyasuvunakool, Jake Varley, Grace Vesom, Giulia Vezzani,

Maria Bauza Villalonga, Oriol Vinyals, René Wagner, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Chengda Wu, Markus Wulfmeier, Fei Xia, Ted Xiao, Annie Xie, Jinyu Xie, Peng Xu, Sichun Xu, Ying Xu, Zhuo Xu, Jimmy Yan, Sherry Yang, Skye Yang, Yuxiang Yang, Hiu Hong Yu, Wenhao Yu, Wentao Yuan, Yuan Yuan, Jingwei Zhang, Tingnan Zhang, Zhiyuan Zhang, Allan Zhou, Guangyao Zhou, and Yuxiang Zhou. Gemini Robotics 1.5: Pushing the Frontier of Generalist Robots with Advanced Embodied Reasoning, Thinking, and Motion Transfer. *arXiv preprint arXiv:2510.03342*, 2025.

[321] Jiayi Tian, Shiao Liu, Yuting Xu, Jia Lu, Zihao Guan, Honglin Han, Di Yang, Minqi Gu, Yifei Qian, Tianlin Zhang, Yanqing Zhu, Zeqian Ye, Menglin Yang, Fei Wang, Xu Hu, Xiuxian Li, Wei Zhang, Shihui Su, Yiyan Ji, Jingbo Wang, Ziteng Feng, Jiaheng Liu, Zhaoxiang Zhang, Xiaolong Wu, Zixiao Tang, Zhining Gu, Yang Cai, Linbo Zheng, Jingjing Ma, Mingyang Yin, Zedong Chu, Wenbin Tang, and Mu Xu. ABot-AgentOS: A General Robotic Agent OS with Lifelong Multi-modal Memory. *arXiv preprint arXiv:2607.10350*, 2026.

[322] Shulin Tian, Junsu Kim, Shuai Liu, Hao Li, Yujiao Shen, Sihan Li, Zhe Yang, Yeongon Kim, Feiyu Li, Jialin Wu, Yichi Zhang, Wenhui Wang, Runmao Yao, Yuhao Dong, Zhaoxi Chen, Fangzhou Hong, Antonino Furnari, Jingkang Yang, Hongyuan Zhu, and Ziwei Liu. EgoTools: Towards Tool-Centric Reasoning in Real-World Egocentric Videos. *arXiv preprint arXiv:2609.39378*, 2026.

[323] Yang Tian, Sizhe Yang, Jia Zeng, Ping Wang, Dahua Lin, Hao Dong, and Jiangmiao Pang. Predictive Inverse Dynamics Models are Scalable Learners for Robotic Manipulation. In *International Conference on Learning Representations*, 2025.

[324] Marcel Torne, Karl Pertsch, Homer Walke, Kyle Vedder, Suraj Nair, Brian Ichter, Allen Z. Ren, Haohuan Wang, Jiaming Tang, Kyle Stachowicz, Karan Dhabalia, Michael Equi, Quan Vuong, Jost Tobias Springenberg, Sergey Levine, Chelsea Finn, and Danny Driess. MEM: Multi-Scale Embodied Memory for Vision Language Action Models. *arXiv preprint arXiv:2603.03596*, 2026.

[325] Brian Y. Tsui, Alan Y. Fang, and Tiffany J. Hwu. Demonstration-Free Robotic Control via LLM Agents. *arXiv preprint arXiv:2601.20334*, 2026.

[326] TypeSafe AI. Jev: A System One Decision Model, 2026. URL https://openrouter.ai/docs/guides/community/jev.

[327] Georgios Tziafas and Hamidreza Kasaei. Lifelong Robot Library Learning: Bootstrapping Composable and Generalizable Skills for Embodied Control with Language Models. *arXiv preprint arXiv:2406.18746*, 2024.

[328] Erdem Uysal, Timo Kehrer, and Sebastiano Panichella. PEACE: A Planner-Executor Agent with Constraint Enforcement for UAVs. *arXiv preprint arXiv:2606.00104*, 2026.

[329] Frederic Vatnsdal, Roshan Gopal, Romina Garcia Camargo, Vijay Kumar, and Alejandro Ribeiro. Controlling Collectives of AI Agents in Reasoning Space with Spatial Transformers. *arXiv preprint arXiv:2609.28247*, 2026.

[330] Long Giang Vu, Chengkai Yao, Yuxin Liu, FNU Aryan, and Rajath Chandrashekar Aralikatti. AnchorVLN: Geometry-Anchored Vision-Language Grounding Reasoning for Open-Vocabulary Navigation. *arXiv preprint arXiv:2609.12285*, 2026.

[331] Yang Wan, Xihang Yue, Zhirui Liu, Ziyuan Chu, Shuxun Wang, Yuhan Chen, Xiaonan Jiang, Xukun Zhu, Yubo Dong, and Linchao Zhu. No Free Checker: A Survey of Verifiers for Robot Policies. *arXiv preprint arXiv:2609.09250*, 2026.

[332] Baiqing Wang, Helei Cui, Bo Zhang, Xiaolong Zheng, Bin Guo, and Zhiwen Yu. MeCo: Enhancing LLM-Empowered Multi-Robot Collaboration via Similar Task Memoization. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 2026. doi: 10.65109/qpuc1624.

[333] Boyang Wang, Haoran Zhang, Shujie Zhang, Jinkun Hao, Mingda Jia, Qi Lv, Yucheng Mao, Zhaoyang Lyu, Jia Zeng, Xudong Xu, and Jiangmiao Pang. RoboVIP: Multi-View Video Generation with Visual Identity Prompting Augments Robot Manipulation. *arXiv preprint arXiv:2601.05241*, 2026.

[334] Boyuan Wang, Zhenyuan Zhang, Zhiqin Yang, Peijun Gu, Shuya Wang, Xiaofeng Wang, Xianghui Ze, Yifan Chang, Guosheng Zhao, Jiangnan Shao, Guan Huang, Hengyu Liu, Yonggang Zhang, Wei Xue, Chunyuan Guan, Chenglin Pu, Yike Guo, Xingang Wang, and Zheng Zhu. Zero2Skill: Bootstrapping Robot Skills through Autonomous Data Collection, Training, and Deployment. *arXiv preprint arXiv:2607.14047*, 2026.

[335] Guangming Wang, Pengfei Ye, Qizhen Ying, Yixiong Jing, Yuxiang Ma, Haonan Chen, Haibing Wu, Olaf Wysocki, Molong Duan, and Brian Sheil. A Brain-inspired Hierarchical Framework for Zero-Shot Robot Task Reasoning and Execution. *arXiv preprint arXiv:2609.05985*, 2026.

[336] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291*, 2023.

[337] Hongyu Wang, Chuyan Xiong, Ruiping Wang, and Xilin Chen. BitVLA: 1-bit Vision-Language-Action Models for Robotics Manipulation. *arXiv preprint arXiv:2506.07530*, 2025.

[338] Hongyu Wang, Jingquan Wang, Ashvin Anilkumar, Bocheng Zou, Radu Serban, and Dan Negrut. ChronoAgentic: A Code-based Multi-Agent World Simulator for Physically Grounded Simulation Construction. *arXiv preprint arXiv:2605.14398*, 2026.

[339] Jiaming Wang. What Stops Recursive Self-Improvement in Robotics? Lessons from 123 Rounds of Agentic Skill Discovery. *arXiv preprint arXiv:2609.31760*, 2026.

[340] Jianzong Wang, Botao Zhao, Yayun He, Junqing Peng, and Xulong Zhang. Evolvable Embodied Agent for Robotic Manipulation via Long Short-Term Reflection and Optimization. *arXiv preprint arXiv:2604.13533*, 2026.

[341] Jiaxu Wang, Qiang Zhang, Jingkai Sun, Jiahang Cao, Gang Han, Wen Zhao, Weining Zhang, Yecheng Shao, Yijie Guo, and Renjing Xu. Reinforcement Learning with Generalizable Gaussian Splatting. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 435–441. IEEE, 2024. doi: 10.1109/IROS58592.2024.10801348.

[342] Jiaxu Wang, Yicheng Jiang, Tianlun He, Jingkai Sun, Qiang Zhang, Jiahang Cao, Zesen Gan, Mingyuan Sun, Qiming Shao, and Xiangyu Yue. MVISTA-4D: View-consistent 4D world model with test-time action inference for robotic manipulation. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 131846–131866. PMLR, 2026.

[343] Junxiang Wang, Nir Pechuk, Xinwen Xu, Tiancheng Wu, Nayon Kim, Julian Millan, and Zackory Erickson. GenPHRI: Agentic Generative Simulation for Physical Human-Robot Interaction. *arXiv preprint arXiv:2604.08664*, 2026.

[344] Le Wang, Zonghao Ying, Xiao Yang, Quanchen Zou, Zhenfei Yin, Tianlin Li, Jian Yang, Yaodong Yang, Aishan Liu, and Xianglong Liu. RoboSafe: Safeguarding Embodied Agents via Executable Safety Logic. *arXiv preprint arXiv:2512.21220*, 2025.

[345] Lirui Wang, Yiyang Ling, Zhecheng Yuan, Mohit Shridhar, Chen Bao, Yuzhe Qin, Bailin Wang, Huazhe Xu, and Xiaolong Wang. GenSim: Generating Robotic Simulation Tasks via Large Language Models. *arXiv preprint arXiv:2310.01361*, 2023.

[346] Meiqi Wang and Shichao Li. Compiling and Benchmarking Task-State Horizons for Embodied Agents. *arXiv preprint arXiv:2608.08036*, 2026.

[347] Peidong Wang, Zhiming Ma, Ying Chang, Xufang Luo, Yiqun Zhang, Zihan Wang, Xiaocui Yang, Shi Feng, Yuqing Yang, and Dongsheng Li. Self-Evolving Embodied Agents via Skill-Harness Evolution. *arXiv preprint arXiv:2608.11350*, 2026.

[348] Qi Wang, Tianyi Wang, Chengyang Li, Shikun Ban, Yurun Chen, Yizhong Ge, Jason Qin, Chengtai Li, and Wentao Zhu. Towards the Harness of Embodied Agents. *arXiv preprint arXiv:2608.11246*, 2026.

[349] Qiuyue Wang, Mingsheng Li, Jian Guan, Jinhui Ye, Sicheng Xie, Yitao Liu, Junhao Chen, Zhixuan Liang, Jie Zhang, Xintong Hu, Xuhong Huang, Pei Lin, Junyang Lin, Dayiheng Liu, Shuai Bai, Jingren Zhou, Jiazhao Zhang, Haoqi Yuan, Gengze Zhou, Hang Yin, Ye Wang, Yiyang Huang, Zixing Lei, Wujian Peng, Delin Chen, Yingming Zheng, Jingyang Fan, Xianwei Zhuang, Xin Zhou, Haoyang Li, Anzhe Chen, Tong Zhang, Xuejing Liu, Yuchong Sun, Ruizhe Chen, Zhaohai Li, Chenxu Lü, Zhibo Yang, Tao Yu, and Xionghui Chen. Qwen-VLA: Unifying Vision-Language-Action Modeling across Tasks, Environments, and Robot Embodiments. *arXiv preprint arXiv:2605.30280*, 2026.

[350] Ruiyang Wang, Hao-Lun Hsu, Swarajh Mehta, Jiwoo Kim, Zhihao Dou, and Miroslav Pajic. GAVEL: Graph World Models for Verified and Efficient Long-Horizon LLM Task Planning. *arXiv preprint arXiv:2609.19315*, 2026.

[351] Wei Wang, Wenqiao Zhang, Yutong Lin, Yuqian Yuan, Tianwei Lin, Jinhao Mao, Zhenxuan Fan, Mingjian Gao, Yang Dai, Wentong Li, Zheqi Lv, Zheng Dong, Yingjie Niu, Jiaqi Zhu, Jun Xiao, Chao Li, and Yueting Zhuang. EmbodiedSkills: A Unified Framework for Orchestrating, Training, and Deploying VLA Agents. *arXiv preprint arXiv:2609.01281*, 2026.

[352] Weihan Wang, Qingsong Lv, Wenmeng Yu, Wenyi Hong, Ji Qi, Yan Wang, Junhui Ji, Zhuoyi Yang, Lei Zhao, Xixuan Song, Jiazheng Xu, Bin Xu, Juanzi Li, Yuxiao Dong, Ming Ding, and Jie Tang. CogVLM: Visual Expert for Pretrained Language Models. *arXiv preprint arXiv:2311.03079*, 2023.

[353] Wenze Wang, Mehdi Hosseinzadeh, and Feras Dayoub. A Physical Agentic Loop for Language-Guided Grasping with Execution-State Monitoring. *arXiv preprint arXiv:2604.07395*, 2026.

[354] Xiao Wang, Lu Dong, Ifeoma Nwogu, Srirangaraj Setlur, and Venu Govindaraju. MistyPilot: Enabling Social-Robot Control through Multi-Agent LLM Skill Orchestration. *arXiv preprint arXiv:2608.15549*, 2026.

[355] Xilin Wang, Guoxi Zhang, Hongming Xu, Zhuofan Zhang, Tianxu Wang, and Lifeng Fan. EvoNav-Bench: Benchmarking Lifelong Navigation in Evolving Environments. *arXiv preprint arXiv:2609.08292*, 2026.

[356] Xin Wang, Wenhao Wu, Menghao Zhang, Zhi Wang, Kun Shao, Jian Luan, Yang Li, Qing Li, Shangding Gu, Huichi Zhou, Shuqing Shi, Fei Ni, Shuo Lu, Weicheng Meng, Kang Li, Jin Wu, Kang Zhao, Shangmin Guo, Gen Li, Yongqiang Tang, Zhizhong Zhang, Yuan Xie, and Heng Qu. HarnessPAI: An Evolving Harness for Physical AI. *arXiv preprint arXiv:2609.29166*, 2026.

[357] Xinjie Wang, Liu Liu, Taojun Ding, Andrew Choi, Chaodong Huang, Mengao Zhao, Ziang Li, Jackson Jiang, Chunlei Yu, Shengxiang Liu, Wei Xu, and Zhizhong Su. EmbodiedGen V2: An Agentic, Simulation-Ready 3D World Engine for Embodied AI. *arXiv preprint arXiv:2607.07459*, 2026.

[358] Yen-Jen Wang, Haozhe Jiang, Shuying Deng, Haoru Xue, Weirui Ye, Rocky Duan, Nika Haghtalab, S. Shankar Sastry, Pieter Abbeel, and Haozhi Qi. Reconstruct, Practice, Go Real: Guided Self-Improvement for Embodied Agents. *arXiv preprint arXiv:2610.02204*, 2026.

[359] Yi Wang, Xinchen Li, Pengwei Xie, Pu Yang, Buqing Nie, Yunuo Cai, Qinglin Zhang, Chendi Qu, Jeffrey Wu, Jianheng Song, Xinlin Ren, Jingshun Huang, Mingjie Pan, Siyuan Feng, Zhi Chen, and Jianlan Luo. Learning While Deploying: Fleet-Scale Reinforcement Learning for Generalist Robot Policies. *arXiv preprint arXiv:2605.00416*, 2026.

[360] Yi Ru Wang, Carter Ung, Evan Gubarev, Christopher Tan, Siddhartha Srinivasa, and Dieter Fox. RoboPlayground: Democratizing Robotic Evaluation through Structured Physical Domains. *arXiv preprint arXiv:2604.05226*, 2026.

[361] Youhui Wang, Yunzhu Li, Li Fei-Fei, Jiajun Wu, and Huang Huang. DexAgent: An Agentic Human2Sim2Robot Framework for Dexterous Manipulation with Self-Evolving Tool Library. *arXiv preprint arXiv:2609.35318*, 2026.

[362] Yufei Wang, Zhou Xian, Feng Chen, Tsun-Hsuan Wang, Yian Wang, Katerina Fragkiadaki, Zackory Erickson, David Held, and Chuang Gan. RoboGen: Towards Unleashing Infinite Data for Automated Robot Learning via Generative Simulation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 51936–51983. PMLR, 2024.

[363] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An Open, Modular Exploration Towards Systematic World-Action Model Pretraining. *arXiv preprint arXiv:2609.07398*, 2026.

[364] Zhaoning Wang, Yi Zhong, Jiawei Fu, Henrik I. Christensen, and Jun Gao. AFUN: Towards an Affordance Foundation Model for Functionality Understanding. *arXiv preprint arXiv:2606.02551*, 2026.

[365] Ziye Wang, Modi Shi, Chaojun Ni, Jiazhi Yang, Mengdi Li, Zhizhong Su, Tianwei Lin, and Hongyang Li. NativeMEM: Native memory compression for long-horizon robotic manipulation. *arXiv preprint arXiv:2607.06678*, 2026.

[366] Bingchuan Wei, Bingqi Huang, Jingheng Ma, Zeyu Zhang, and Sen Cui. FATE: Closed-Loop Feasibility-Aware Task Generation with Active Repair for Physically Grounded Robotic Curricula. *arXiv preprint arXiv:2603.01505*, 2026.

[367] Meng Wei, Chenyang Wan, Jiaqi Peng, Xiqian Yu, Yuqiang Yang, Delin Feng, Wenzhe Cai, Chenming Zhu, Tai Wang, Jiangmiao Pang, and Xihui Liu. Ground Slow, Move Fast: A Dual-System Foundation Model for Generalizable Vision-and-Language Navigation. *arXiv preprint arXiv:2512.08186*, 2025.

[368] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. FoundationPose: Unified 6D Pose Estimation and Tracking of Novel Objects. *arXiv preprint arXiv:2312.08344*, 2023.

[369] Bowen Wen, Matthew Trepte, Joseph Aribido, Jan Kautz, Orazio Gallo, and Stan Birchfield. FoundationStereo: Zero-Shot Stereo Matching. *arXiv preprint arXiv:2501.09898*, 2025.

[370] Di Wen, Kailun Yang, Wenhao Guo, Yitian Shi, Junwei Zheng, Yufan Chen, Ruiping Liu, Jiale Wei, Rania Rayyes, and Kunyu Peng. INSPECT: Learning Robot View Selection from Assistant Use. *arXiv preprint arXiv:2609.20615*, 2026.

[371] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, Yaxin Peng, Feifei Feng, and Jian Tang. TinyVLA: Towards Fast, Data-Efficient Vision-Language-Action Models for Robotic Manipulation. *arXiv preprint arXiv:2409.12514*, 2024.

[372] Junjie Wen, Yichen Zhu, Jinming Li, Zhibin Tang, Chaomin Shen, and Feifei Feng. DexVLA: Vision-Language Model with Plug-In Diffusion Expert for General Robot Control. *arXiv preprint arXiv:2502.05855*, 2025.

[373] Zimo Wen, Yijin Chen, Yuxuan Cao, Wendi Chen, Yanwen Zou, Wenye Yu, Fuhang Kuang, Han Xue, Jun Lv, Chuan Wen, and Cewu Lu. RoboRSI: Stable, Efficient, and Reusable Robot Self-Evolution in Complex Real-World Environments. *arXiv preprint arXiv:2610.12424*, 2026.

[374] John Woods and Hasti Seifi. SkillComposer: Learning Reusable Skills for Natural-Language Robot Programming. *arXiv preprint arXiv:2608.14944*, 2026.

[375] Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing Large-Scale Video Generative Pre-training for Visual Robot Manipulation. *arXiv preprint arXiv:2312.13139*, 2023.

[376] Jimmy Wu, Rika Antonova, Adam Kan, Marion Lepert, Andy Zeng, Shuran Song, Jeannette Bohg, Szymon Rusinkiewicz, and Thomas Funkhouser. TidyBot: Personalized Robot Assistance with Large Language Models. *arXiv preprint arXiv:2305.05658*, 2023.

[377] Philipp Wu, Alejandro Escontrela, Danijar Hafner, Ken Goldberg, and Pieter Abbeel. DayDreamer: World Models for Physical Robot Learning. *arXiv preprint arXiv:2206.14176*, 2022.

[378] Wei Wu, Fan Lu, Yunnan Wang, Shuai Yang, Shi Liu, Fangjing Wang, Qian Zhu, He Sun, Yong Wang, Shuailei Ma, Yiyu Ren, Kejia Zhang, Hui Yu, Jingmei Zhao, Shuai Zhou, Zhenqi Qiu, Houlong Xiong, Ziyu Wang, Zechen Wang, Ran Cheng, Yong-Lu Li, Yongtao Huang, Xing Zhu, Yujun Shen, and Kecheng Zheng. A Pragmatic VLA Foundation Model. *arXiv preprint arXiv:2601.18692*, 2026.

[379] Hongchi Xia, Xuan Li, Zhaoshuo Li, Qianli Ma, Jiashu Xu, Ming-Yu Liu, Yin Cui, Tsung-Yi Lin, Wei-Chiu Ma, Shenlong Wang, Shuran Song, and Fangyin Wei. SAGE: Scalable Agentic 3D Scene Generation for Embodied AI. *arXiv preprint arXiv:2602.10116*, 2026.

[380] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. *arXiv preprint arXiv:2602.08234*, 2026.

[381] Fanbo Xiang, Yuzhe Qin, Kaichun Mo, Yikuan Xia, Hao Zhu, Fangchen Liu, Minghua Liu, Hanxiao Jiang, Yifu Yuan, He Wang, Li Yi, Angel X. Chang, Leonidas J. Guibas, and Hao Su. SAPIEN: A SimulAted Part-based Interactive ENvironment. *arXiv preprint arXiv:2003.08515*, 2020.

[382] Anxing Xiao, Hanbo Zhang, Tianrun Hu, and David Hsu. Hypothesis-driven Model Expansion under Uncertainty for Open-World Robot Planning. In *Proceedings of Robotics: Science and Systems*, 2026. doi: 10.15607/RSS.2026.XXII.181.

[383] Wenli Xiao, Jia Xie, Tonghe Zhang, Haotian Lin, Letian "Max" Fu, Haoru Xue, Jalen Lu, Yi Yang, Cunxi Dai, Zi Wang, Jimmy Wu, Guanzhi Wang, S. Shankar Sastry, Ken Goldberg, Linxi "Jim" Fan, Yuke Zhu, and Guanya Shi. ENPIRE: Agentic Robot Policy Self-Improvement in the Real World. *arXiv preprint arXiv:2606.19980*, 2026.

[384] Xiaomi Robotics Team, Jun Guo, Piaopiao Jin, Jason Li, Peiyan Li, Yingyan Li, Futeng Liu, Wanli Peng, Optimus Qin, Yifei Su, Nan Sun, Qiao Sun, Runze Suo, Heyun Wang, Yunhong Wang, Rujie Wu, Caoyu Xia, Lina Zhang, Jack Zhao, Guoliang Chen, Wenlong Chen, Xinze He, Bin Li, Qing Li, Zhuorong Li, Heng Qu, Wenxuan Song, Diyun Xiang, Yifan Xie, Peiran Xu, Hangjun Ye, Wen Ye, Han Zhao, and Quanyun Zhou. Xiaomi-Robotics-1: Scaling Vision-Language-Action Models with over 100K Hours of Real-World Trajectories. *arXiv preprint arXiv:2607.15330*, 2026.

[385] Senwei Xie, Yuntian Zhang, Ruiping Wang, and Xilin Chen. Uni-Skill: Building Self-Evolving Skill Repository for Generalizable Robotic Manipulation. *arXiv preprint arXiv:2603.02623*, 2026.

[386] Shuangyu Xie, Kaiyuan Chen, Ziyang Chen, Simeon Adebola, Yixuan Huang, Zehan Ma, Tianshuang Qiu, Wentao Yuan, Dhruv Shah, Pannag R. Sanketi, and Ken Goldberg. RoboVista: Evaluating Vision Language Models for Diverse Robot Applications. In *Proceedings of Robotics: Science and Systems*, 2026. doi: 10.15607/RSS.2026.XXII.095.

[387] Shuzhao Xie, Lelin Wang, Guying Lin, Zhi Wang, and Minchen Li. CoDimRecon: Agentic Reconstruction of Sim-Ready 3D Scenes with Deformable Curves, Surfaces, and Volumes. *arXiv preprint arXiv:2609.36024*, 2026.

[388] Sicheng Xie, Yitong Chen, Haidong Cao, Shunlin Lu, Zuxuan Wu, and Yu-Gang Jiang. Explore, Execute, Evolve: A Skill Acquisition and Reuse Loop for Embodied Agents. *arXiv preprint arXiv:2609.37810*, 2026.

[389] Guipeng Xin, Jiahe Xu, Mohammad Deghat, Chenhui Wan, Jie Liu, Youmin Hu, and Zhongxu Hu. MaCoPlanner: LLM-Assisted Manual-Compiled Task Planning with Proactive Safety Verification for Robotic Industrial Panel Operation. *arXiv preprint arXiv:2608.28300*, 2026.

[390] Bingxin Xu, Yuzhang Shang, Zhen Dong, and Emilio Ferrara. Coding Agents with an Obstacle-Aware Harness for Safe Robot Manipulation. *arXiv preprint arXiv:2609.20822*, 2026.

[391] Bingxin Xu, Yuzhang Shang, and Emilio Ferrara. Don't Drop the BATON: Long-Horizon Robot Manipulation via Agentic Subtask Exploration and Transition-aware Memory. *arXiv preprint arXiv:2608.16889*, 2026.

[392] Haoming Xu, Zhenlin He, Hengyi Wang, Jiafeng Xu, and Hao Dong. Mimir: A Neuro-Symbolic Memory System with Dynamic Grounding for Embodied Agents in Interactive Environments. *arXiv preprint arXiv:2608.04933*, 2026.

[393] Long Xu, Zhiqi Zhang, Mi Yan, Shengliang Deng, Chong Xia, Mingyu Dong, Jiayi Chen, Jiangran Lyu, Fei Gao, Zhizheng Zhang, and He Wang. GIF: Agentic Generation of Interactive and Functional Object Compositions for Robot Learning. *arXiv preprint arXiv:2609.05927*, 2026.

[394] Mutian Xu, Tianbao Zhang, Tianqi Liu, Zhaoxi Chen, Xiaoguang Han, and Ziwei Liu. Kinema4D: Kinematic 4D World Modeling for Spatiotemporal Embodied Simulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 4495–4500, 2026.

[395] Sheng Xu, Ruixing Jin, Huayi Zhou, Bo Yue, Guanren Qiao, Yunxin Tai, Yueci Deng, Kui Jia, and Guiliang Liu. From Reaction to Anticipation: Proactive Failure Recovery through Agentic Task Graph for Robotic Manipulation. In *Proceedings of Robotics: Science and Systems*, 2026. doi: 10.15607/RSS.2026.XXII.180.

[396] Zhiyuan Xu, Kun Wu, Junjie Wen, Jinming Li, Ning Liu, Zhengping Che, and Jian Tang. A Survey on Robotics with Foundation Models: toward Embodied AI. *arXiv preprint arXiv:2402.02385*, 2024.

[397] Yaoyuan Yan, Zhiyou Heng, Haoxiang Jie, Gang Liu, Hongjie Yan, and Wei Zhou. Harness Robotic OS: A Unified Embodied-Agent Runtime for Closed-Loop Quadruped Inspection. *arXiv preprint arXiv:2609.11225*, 2026.

[398] Jenai Xuning Yang, Rishit Dagli, Alex Zook, Hugo Hadfield, Ankit Goyal, Stan Birchfield, Fabio Ramos, and Jonathan Tremblay. RoboLab: A High-Fidelity Simulation Benchmark for Analysis of Task Generalist Policies. In *Proceedings of Robotics: Science and Systems*, 2026. doi: 10.15607/RSS.2026.XXII.096.

[399] Rui Yang, Hanyang Chen, Junyu Zhang, Mark Zhao, Cheng Qian, Kangrui Wang, Qineng Wang, Teja Venkat Koripella, Marziyeh Movahedi, Manling Li, Heng Ji, Huan Zhang, and Tong Zhang. EmbodiedBench: Comprehensive Benchmarking Multi-modal Large Language Models for Vision-Driven Embodied Agents. *arXiv preprint arXiv:2502.09560*, 2025.

[400] Sherry Yang, Yilun Du, Kamyar Ghasemipour, Jonathan Tompson, Leslie Kaelbling, Dale Schuurmans, and Pieter Abbeel. Learning Interactive Real-World Simulators. *arXiv preprint arXiv:2310.06114*, 2023.

[401] Yang Yang, Yuchuang Tong, Zhengtao Zhang, Xu Ding, Ning Yang, Yifan Zhang, Haipeng Li, Kehu Yang, and Miao Xin. Causal Reward World Models: Zero-shot Reward Design for Automated Skill Generation. *arXiv preprint arXiv:2606.23280*, 2026.

[402] Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, et al. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. *arXiv preprint arXiv:2605.23904*, 2026.

[403] Yunhao Yang, Neel P. Bhatt, Kevin Wang, Samuel Tetteh, Zhangyang Wang, and Ufuk Topcu. VASO: Formally Verifiable Self-Evolving Skills for Physical AI Agents. *arXiv preprint arXiv:2606.05395*, 2026.

[404] Zhiqin Yang, Chenxin Li, Xiaomeng Hu, Yibin Liu, Weidong Huang, Jiankai Sun, Haitao Li, Zijian Wu, Yuzhi Huang, Fanding Huang, Hanwen Sun, Jiashun Liu, Jingqi Tong, Mingxin Huang, Shaoli Hu, Shijue Huang, Tianyi Bai, Xinyuan Wang, Yunlong Lin, Zhengyang Tang, Zhexin Zhang, Zhuo Chen, Xierui Song, Juntao Dai, Boyuan Chen, Jiaming Ji, Fangneng Zhan, Mengkang Hu, Wei Xue, Yonggang Zhang, Han Hu, Tsung-Yi Ho, and Yike Guo. RobotWorld: Benchmarking Multimodal Agents for Robot Use Across Diverse Tasks and Embodiments. *arXiv preprint arXiv:2610.10409*, 2026.

[405] Ziyi Yang, Shreyas S. Raman, Ankit Shah, and Stefanie Tellex. Plug in the Safety Chip: Enforcing Constraints for LLM-driven Robot Agents. *arXiv preprint arXiv:2309.09919*, 2023.

[406] Angen Ye, Boyuan Wang, Chaojun Ni, Guan Huang, Guosheng Zhao, Hao Li, Hengtao Li, Jie Li, Jindi Lv, Jingyu Liu, Min Cao, Peng Li, Qiuping Deng, Wenjun Mei, Xiaofeng Wang, Xinze Chen, Xinyu Zhou, Yang Wang, Yifan Chang, Yifan Li, Yukun Zhou, Yun Ye, Zhichao Liu, and Zheng Zhu. GigaWorld-Policy: An Efficient Action-Centered World–Action Model. *arXiv preprint arXiv:2603.17240*, 2026.

[407] Junjie Ye, Rong Xue, Basile Van Hoorick, Runhao Li, Harshitha Rajaprakash, Pavel Tokmakov, Muhammad Zubair Irshad, Vitor Guizilini, and Yue Wang. RoboDream: Compositional World Models for Scalable Robot Data Synthesis. *arXiv preprint arXiv:2606.02577*, 2026.

[408] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, Ayaan Malik, Kyungmin Lee, William Liang, Nadun Ranawaka, Jiasheng Gu, Yinzhen Xu, Guanzhi Wang, Fengyuan Hu, Avnish Narayan, Johan Bjorck, Jing Wang, Gwanghyun Kim, Dantong Niu, Ruijie Zheng, Yuqi Xie, Jimmy Wu, Qi Wang, Ryan Julian, Danfei Xu, Yilun Du, Yevgen Chebotar, Scott Reed, Jan Kautz, Yuke Zhu, Linxi "Jim" Fan, and Joel Jang. World Action Models are Zero-shot Policies. *arXiv preprint arXiv:2602.15922*, 2026.

[409] Weirui Ye, Fangchen Liu, Zheng Ding, Yang Gao, Oleh Rybkin, and Pieter Abbeel. Video2Policy: Scaling up Manipulation Tasks in Simulation through Internet Videos. *arXiv preprint arXiv:2502.09886*, 2025.

[410] Xiaokang Ye, Siddhant Hitesh Mantri, Zimeng Chen, Edward Zhang, Zhaoxu Zheng, Yuanheng Li, Yizhao Chen, Tianyang Huang, and Lianhui Qin. Code4Scene: Benchmarking Coding Agents for Constructing and Editing 3D Scenes. *arXiv preprint arXiv:2609.36777*, 2026.

[411] Ding Yi, Peiwen Sun, Chenchu Rong, Jianan Wang, Xili Dai, Xiangyu Yue, and Xi Lin. Watch, Recall, Act: Always-On Robots in Concurrent Embodied Streams. *arXiv preprint arXiv:2609.28429*, 2026.

[412] Ding Yi, Yanzhao Yu, Xili Dai, Xianbiao Qi, Peiwen Sun, Xueqian Wang, Xiangyu Yue, and Jianan Wang. Evolve Vision-Language-Action Model into an Agent with On-the-fly Tool-use. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Findings*, pages 1346–1357, 2026.

[413] Pengfei Yi, Yingjie Ma, Wenjiang Xu, Yanan Hao, Shuai Gan, Wanting Li, and Shanlin Zhong. Critic in the Loop: A Tri-System VLA Framework for Robust Long-Horizon Manipulation. *arXiv preprint arXiv:2603.05185*, 2026.

[414] Chenghao Yin, Da Huang, Di Yang, Jichao Wang, Nanshu Zhao, Chen Xu, Wenjun Sun, Linjie Hou, Zhijun Li, Junhui Wu, Zhaobo Liu, Zhen Xiao, Sheng Zhang, Lei Bao, Rui Feng, Zhenquan Pang, Jiayu Li, Qian Wang, and Maoqing Yao. Genie Sim 3.0 : A High-Fidelity Comprehensive Simulation Platform for Humanoid Robot. *arXiv preprint arXiv:2601.02078*, 2026.

[415] Jiayu Ying, Qijian Tian, Ruijie Xu, Xinnan Zhu, Daoguo Dong, Jiachen Xu, and Xin Tan. Exemplar2VQA: A Scalable Exemplar-Driven Visual Question Answering Generation Framework via Multi-Agent Coding. *arXiv preprint arXiv:2609.37655*, 2026.

[416] Boao Yu, Zimo Chen, Junreng Rao, Yue Hu, Zhengqiu Zhu, Yong Zhao, and Rusheng Ju. Towards Embodied Air-Ground Cooperative Object Search: Benchmark, Dataset and Agentic Method. *arXiv preprint arXiv:2609.08402*, 2026.

[417] Hang Yu. You Don't Need To Stay in The Loop: An Agentic Robotics Loop for Robot-Policy Improvement. *arXiv preprint arXiv:2608.07555*, 2026.

[418] Hanyang Yu, Haitao Lin, Jingbo Zhang, Wenyao Zhang, Chenghao Gu, Heng Li, and Ping Tan. MaskWAM: Unifying Mask Prompting and Prediction for World-Action Models. *arXiv preprint arXiv:2606.13515*, 2026.

[419] Peiqi Yu, Mosam Dabhi, Shangtao Li, Bowei Li, Laszlo Jeni, and Changliu Liu. ManiSkillFormer: Demonstration-Free Compositional Manipulation via Geometric Contracts and Agentic Skill Graph. *arXiv preprint arXiv:2609.16331*, 2026.

[420] Zihao Yu, Xiu Yuan, and Chongjie Zhang. MEMORA: Embodied Action Memory from Egocentric Videos for Reasoning and Planning. *arXiv preprint arXiv:2607.14252*, 2026.

[421] Jessie Yuan, Yilin Wu, and Andrea Bajcsy. When to Act, Ask, or Learn: Uncertainty-Aware Policy Steering. In *Proceedings of Robotics: Science and Systems*, 2026. doi: 10.15607/RSS.2026.XXII.142.

[422] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do World Action Models Need Test-time Future Imagination? *arXiv preprint arXiv:2603.16666*, 2026.

[423] Wentao Yuan, Adithyavairavan Murali, Arsalan Mousavian, and Dieter Fox. M2T2: Multi-Task Masked Transformer for Object-centric Pick and Place. *arXiv preprint arXiv:2311.00926*, 2023.

[424] Yifu Yuan, Haiqin Cui, Yibin Chen, Zibin Dong, Fei Ni, Longxin Kou, Jinyi Liu, Pengyi Li, Yan Zheng, and Jianye Hao. From Seeing to Doing: Bridging Reasoning and Decision for Robotic Manipulation. In *International Conference on Learning Representations*, 2026.

[425] Michał Zawalski, William Chen, Karl Pertsch, Oier Mees, Chelsea Finn, and Sergey Levine. Robotic Control via Embodied Chain-of-Thought Reasoning. In *Proceedings of The 8th Conference on Robot Learning*, volume 270, pages 3157–3181. PMLR, 2025.

[426] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3D Diffusion Policy: Generalizable Visuomotor Policy Learning via Simple 3D Representations. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, July 2024. doi: 10.15607/RSS.2024.XX.067.

[427] Andy Zeng, Pete Florence, Jonathan Tompson, Stefan Welker, Jonathan Chien, Maria Attarian, Travis Armstrong, Ivan Krasin, Dan Duong, Vikas Sindhwani, and Johnny Lee. Transporter Networks: Rearranging the Visual World for Robotic Manipulation. In *Proceedings of the 2020 Conference on Robot Learning*, volume 155 of *Proceedings of Machine Learning Research*, pages 726–747. PMLR, 2021.

[428] Haishan Zeng, Mengna Wang, and Peng Li. EmboTeam: Grounding LLM Reasoning into Reactive Behavior Trees via PDDL for Embodied Multi-Robot Collaboration. *arXiv preprint arXiv:2601.11063*, 2026.

[429] Zijian Zeng, Fei Ding, Huiming Yang, Xianwei Li, and Yuhao Liao. DexSim2Real: Foundation Model-Guided Sim-to-Real Transfer for Generalizable Dexterous Manipulation. *arXiv preprint arXiv:2605.05241*, 2026.

[430] Lihan Zha, Yuchen Cui, Li-Heng Lin, Minae Kwon, Montserrat Gonzalez Arenas, Andy Zeng, Fei Xia, and Dorsa Sadigh. Distilling and Retrieving Generalizable Knowledge for Robot Manipulation via Language Corrections. *arXiv preprint arXiv:2311.10678*, 2023.

[431] Shaopeng Zhai, Qi Zhang, Tianyi Zhang, Haoran Zhang, Fuxian Huang, Zhanhui Lin, Zijun Xu, and Weinan Zhang. HELP: Human-Efficient Large-Scale Robot Post-Training with Rollout Segmentation. *arXiv preprint arXiv:2607.09776*, 2026.

[432] Daoxuan Zhang, Ping Chen, Xiaobo Xia, Xiu Su, Ruichen Zhen, Jianqiang Xiao, and Shuo Yang. APEX: A Decoupled Memory-based Explorer for Asynchronous Aerial Object Goal Navigation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15232–15242, 2026.

[433] Jianke Zhang, Yanjiang Guo, Xiaoyu Chen, Yen-Jen Wang, Yucheng Hu, Chengming Shi, and Jianyu Chen. HiRT: Enhancing Robotic Control with Hierarchical Robot Transformers. *arXiv preprint arXiv:2410.05273*, 2024.

[434] Jianke Zhang, Yanjiang Guo, Yucheng Hu, Xiaoyu Chen, Xiang Zhu, and Jianyu Chen. UP-VLA: A Unified Understanding and Prediction Model for Embodied Agent. *arXiv preprint arXiv:2501.18867*, 2025.

[435] Junyi Zhang, Jiaxin Ge, Hanjun Yoo, Letian Fu, Zihan Yang, Yaowei Liu, Raj Saravanan, Shaofeng Yin, Justin Yu, Dantong Niu, Zirui Wang, Roei Herzig, Ken Goldberg, Yutong Bai, David M. Chan, Ion Stoica, Angjoo Kanazawa, Jiahui Lei, Haiwen Feng, and Trevor Darrell. Playful Agentic Robot Learning. *arXiv preprint arXiv:2606.19419*, 2026.

[436] Lei Zhang, Diwen Zheng, Kaixin Bai, Zhenshan Bing, Zoltan-Csaba Marton, Zhaopeng Chen, Alois Christian Knoll, and Jianwei Zhang. OmniDexVLG: Learning Dexterous Grasp Generation from Vision Language Model-Guided Grasp Semantics, Taxonomy and Functional Affordance. *arXiv preprint arXiv:2512.03874*, 2025.

[437] Mingtong Zhang and Dhruv Shah. Visual Verification Enables Inference-time Steering and Autonomous Policy Improvement. In *Proceedings of Robotics: Science and Systems*, 2026. doi: 10.15607/RSS.2026.XXII.079.

[438] Shuai Zhang, Jiayu Hu, Zijie Chen, Zeyuan Ding, Yi Zhang, Yingji Zhang, Ziyi Zhou, Junwei Liao, Shengjie Zhou, Yong Dai, Zhenzhong Lan, and Xiaozhu Ju. A2Eval: Agentic and Automated Evaluation for Embodied Brain. *arXiv preprint arXiv:2602.01640*, 2026.

[439] Wenbo Zhang, Kaixuan Wang, Yutao Ouyang, Xiaoyu Huang, Liyang Li, Kailun Su, Weiyang Jin, Wenhao Chai, Haotian Liang, Zhiyang Dou, Yue Chen, and Tianxing Chen. An Unexpected Robot Policy: Early Evaluations of GPT-6 Astra on RoboDojo and Beyond. *arXiv preprint arXiv:2609.24170*, 2026.

[440] Yehang Zhang, Haojian Huang, Yifan Chang, Jianchong Su, Bohan Zhou, Yingjie Xu, Wosong Chen, Tianhao Zhou, Chenxu Wang, Tianyi Zhang, Yangkai Wei, Wenqian Li, Shiyuan Deng, Yinchuan Li, Ying-Cong Chen, and Zexi Li. World Action Agent: Harnessing VLMs for Robot Manipulation via World Action Rehearsal. *arXiv preprint arXiv:2609.29964*, 2026.

[441] Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Yi Nie, Chunyang Zhu, Jiaxing Qiu, Yuchen Yan, Jiyuan Liu, Wenhao Tang, Jiaji Rao, Zhengru Fang, Changxu Wei, Yu Wang, Wenbo Ding, and Chao Yu. Harness VLA: Steering Frozen VLAs into Reliable Manipulation Primitives via Memory-Guided Agents. *arXiv preprint arXiv:2607.08448*, 2026.

[442] Yuyang Zhang, Wenyao Zhang, Zekun Qi, He Zhang, Haitao Lin, Jingbo Zhang, Yao Mu, Xiaokang Yang, Wenjun Zeng, and Xin Jin. ImageWAM: Do World Action Models Really Need Video Generation, or Just Image Editing? *arXiv preprint arXiv:2606.19531*, 2026.

[443] Zhongbo Zhang, Jiayi Jin, Yifan Wang, Zaibin Zhang, Haiwen Diao, Lijun Wang, and Huchuan Lu. VABench: Measuring Embodied Spatial Intelligence through Visual Demonstrations, Active Perception, and Metric Control. *arXiv preprint arXiv:2609.19554*, 2026.

[444] Zixin Zhang, Kanghao Chen, Hanqing Wang, Hongfei Zhang, Harold Haodong Chen, Chenfei Liao, Litao Guo, and Ying-Cong Chen. A4-Agent: An Agentic Framework for Zero-Shot Affordance Reasoning. In *Computer Vision – ECCV 2026*, pages 153–172, 2026.

[445] Ganlong Zhao, Zijia Tang, Xingping Chen, Zhanghui Kuang, Ye Tian, and Guanbin Li. FLARE: A Failure-Aware Framework for Autonomous Correction and Recovery in Visual-Language Robotic Manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22391–22401, 2026.

[446] Linfeng Zhao, Haojie Huang, Jiayuan Mao, Weiyu Liu, Mykel Kochenderfer, and Lawson L.S. Wong. Retriever: Composing the Perception-Reasoning-Action Loop for Long-Horizon Manipulation. *arXiv preprint arXiv:2607.17213*, 2026.

[447] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, Ankur Handa, Ming-Yu Liu, Donglai Xiang, Gordon Wetzstein, and Tsung-Yi Lin. CoT-VLA: Visual Chain-of-Thought Reasoning for Vision-Language-Action Models. *arXiv preprint arXiv:2503.22020*, 2025.

[448] Sizhe Zhao, Haozhe Xie, Weiyu Zhao, Chenchu Zhang, Huan Wang, Chenyang Wang, Qinglin Liu, and Shengping Zhang. Memory as Plans: World-Action Modeling with Memory-Grounded Planning. *arXiv preprint arXiv:2609.11561*, 2026.

[449] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. *arXiv preprint arXiv:2304.13705*, 2023.

[450] Xufeng Zhao, Cornelius Weber, and Stefan Wermter. Agentic Skill Discovery. *arXiv preprint arXiv:2405.15019*, 2024.

[451] Yujie Zhao, Hongwei Fan, Di Chen, Shengcong Chen, Liliang Chen, Xiaoqi Li, Guanghui Ren, and Hao Dong. Real2Edit2Real: Generating Robotic Demonstrations via a 3D Control Interface. *arXiv preprint arXiv:2512.19402*, 2025.

[452] Yunhao Zhao, Zhenyang Ni, Haoyang Chen, Ruohan Zhang, and Qi Zhu. Skills in Weights, Memory in Code: Hybrid Learning for Memory-Dependent Robot Manipulation. *arXiv preprint arXiv:2608.09410*, 2026.

[453] Zhikai Zhao, Chuanbo Hua, Federico Berto, Zihan Ma, Kanghoon Lee, Jiachen Li, and Jinkyoo Park. EvoNav: Evolutionary Reward Function Design for Robot Navigation with Large Language Models. *arXiv preprint arXiv:2605.11859*, 2026.

[454] Haoyu Zhen, Qiao Sun, Hongxin Zhang, Junyan Li, Siyuan Zhou, Yilun Du, and Chuang Gan. Learning 4D embodied world models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5337–5347, 2025.

[455] Guiyong Zheng, Yueting Ban, Mingjie Zhang, Juepeng Zheng, and Boyu Zhou. OnFly: Onboard Zero-Shot Aerial Vision-Language Navigation toward Safety and Efficiency. *arXiv preprint arXiv:2603.10682*, 2026.

[456] Ruijie Zheng, Jing Wang, Scott Reed, Johan Bjorck, Yu Fang, Fengyuan Hu, Joel Jang, Kaushil Kundalia, Zongyu Lin, Loic Magne, Avnish Narayan, You Liang Tan, Guanzhi Wang, Qi Wang, Jiannan Xiang, Yinzhen Xu, Seonghyeon Ye, Jan Kautz, Furong Huang, Yuke Zhu, and Linxi Fan. FLARE: Robot Learning with Implicit World Modeling. *arXiv preprint arXiv:2505.15659*, 2025.

[457] Zihao Zheng, Hangyu Cao, Jiayu Chen, Sicheng Tian, Chenyue Li, Maoliang Li, Xinhao Sun, Guojie Luo, and Xiang Chen. RoboECC: Multi-Factor-Aware Edge-Cloud Collaborative Deployment for VLA Models. *arXiv preprint arXiv:2603.20711*, 2026.

[458] Enshen Zhou, Qi Su, Cheng Chi, Zhizheng Zhang, Zhongyuan Wang, Tiejun Huang, Lu Sheng, and He Wang. Code-as-Monitor: Constraint-aware Visual Programming for Reactive and Proactive Robotic Failure Detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6919–6929, 2025.

[459] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning. *arXiv preprint arXiv:2411.04983*, 2024.

[460] Jian Zhou, Xunyi Zhao, Gengze Zhou, Zerui Li, Sihao Lin, Jiajun Liu, and Qi Wu. Embodied Agents Take Control: Minimal-Interface Zero-Shot Agents Rival Industrial-Scale Policies in Vision-and-Language Navigation. *arXiv preprint arXiv:2607.26148*, 2026.

[461] Matt Zhou, Ruining Li, Xiaoyang Lyu, Zhaomou Song, Zhening Huang, Chuanxia Zheng, Christian Rupprecht, Andrea Vedaldi, and Shangzhe Wu. Articraft: An Agentic System for Scalable Articulated 3D Asset Generation. *arXiv preprint arXiv:2605.15187*, 2026.

[462] Pengfei Zhou, Shengcong Chen, Di Chen, Jiaxu Wang, Rongjun Jin, Bingwen Zhu, Yike Pan, Songen Gu, Kuanning Wang, Shufeng Nan, Xingyu Qiu, Chenhao Qiu, Pu Yang, Yunuo Cai, Jianxiong Gao, Yifan Li, Yanwei Fu, Xiangyu Yue, Zhi Chen, and Jianlan Luo. τ_0-WM: A unified video-action world model for robotic manipulation. In *Conference on Robot Learning (CoRL)*, 2026.

[463] Siyuan Zhou, Yilun Du, Jiaben Chen, Yandong Li, Dit-Yan Yeung, and Chuang Gan. RoboDreamer: Learning Compositional World Models for Robot Imagination. *arXiv preprint arXiv:2404.12377*, 2024.

[464] Xiaolin Zhou, Liu Liu, Tingyang Xiao, Wei Feng, Fa Fu, Xinrui Meng, Xinjie Wang, Jialiang Han, Boyang Yu, Yun Du, Wei Sui, and Zhizhong Su. HoloAgent-0: A Unified Embodied Agent Framework with 3D Spatial Memory. *arXiv preprint arXiv:2606.23565*, 2026.

[465] Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. LIBERO-PRO: Towards Robust and Fair Evaluation of Vision-Language-Action Models Beyond Memorization. *arXiv preprint arXiv:2510.03827*, 2025.

[466] Xueyang Zhou, Zijia Wang, Qianjiang Li, Yibo Hu, Guiyao Tie, Li Wan, Yidan Liu, Pan Zhou, Lichao Sun, and Yongchao Chen. Enabling Extensible Embodied Capabilities with Tools. *arXiv preprint arXiv:2605.26637*, 2026.

[467] Zhiyuan Zhou, Pranav Atreya, Abraham Lee, Homer Walke, Oier Mees, and Sergey Levine. Autonomous Improvement of Instruction Following Skills via Foundation Models. In *Proceedings of The 8th Conference on Robot Learning*, volume 270, pages 4805–4825. PMLR, 2025.

[468] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified World Models: Coupling Video and Action Diffusion for Pretraining on Large Robotic Datasets. *arXiv preprint arXiv:2504.02792*, 2025.

[469] Berk Çiçek, Mert K. Er, and Ozgur S. Oguz. CoRAL: Contact-Rich Adaptive LLM-based Control for Robotic Manipulation. In *Proceedings of Robotics: Science and Systems*, 2026. doi: 10.15607/RSS.2026.XXII.054.